

Panda3D 引擎技术专题 整合文档

Panda3D Engine Technical Topics — Integrated Reference

渲染管线

场景图

变换系统

光照阴影

透明混合

动画系统

物理碰撞

纹理内存

Shader

多线程

后处理

LOD流式

跨平台

精度稳定性

共 14 个专题 · 涵盖 Panda3D 引擎核心子系统

每个专题包含：问题背景 · 数学原理 · 工程实践 · 源码解析 · 代码演示 · 性能分析

目录

专题 01 渲染管线与状态管理

专题 02 场景图与空间加速结构

专题 03 变换系统与坐标空间

专题 04 光照与阴影

专题 05 透明度与混合

专题 06 动画系统

专题 07 物理与碰撞检测

专题 08 纹理与内存管理

专题 09 Shader 系统

专题 10 多线程架构

专题 11 后处理与离屏渲染

专题 12 LOD 与流式加载

专题 13 跨平台图形 API

专题 14 精度与数值稳定性

专题01：渲染管线与状态管理

对应源码： `panda/src/display/`、`panda/src/pgraph/`、`panda/src/glstuff/`

目录

1. 问题背景
 2. 数学原理
 3. 工程实践
 4. Panda3D 源码解析
 5. 代码演示
 6. 性能分析与调优
-

1. 问题背景

1.1 什么是渲染状态

GPU 是一个有状态的状态机。在发出绘制命令（`glDrawElements`）之前，必须先配置好所有渲染参数：

```
当前绑定的 Shader Program
当前绑定的纹理 (Texture Unit 0~N)
深度测试函数 (GL_LESS / GL_LEQUAL / ...)
深度写入开关 (glDepthMask)
面剔除模式 (GL_BACK / GL_FRONT / GL_NONE)
混合函数 (glBlendFunc)
模板测试参数
裁剪矩形 (glScissor)
视口 (glViewport)
... (共数十个状态)
```

每次状态切换都有 **CPU→GPU 同步开销**，在极端情况下（如每个 Draw Call 都切换 Shader）会导致 GPU 流水线停顿（Pipeline Stall）。

1.2 核心矛盾

- **正确性**：每个物体必须用正确的状态渲染
- **性能**：状态切换越少越好
- **灵活性**：场景图中任意节点可以设置任意状态

这三者之间存在根本性张力，是渲染引擎设计的核心难题之一。

2. 数学原理

2.1 状态的代数结构

渲染状态可以被建模为一个幺半群 (Monoid) :

设 S 为所有可能的渲染状态集合
操作 $\circ : S \times S \rightarrow S$ (状态组合/继承)
单位元 $e \in S$ (空状态, 不改变任何设置)

满足:

结合律: $(a \circ b) \circ c = a \circ (b \circ c)$
单位元: $e \circ a = a \circ e = a$

场景图中的状态传播就是沿树路径的幺半群乘法:

$\text{root_state} \circ \text{node1_state} \circ \text{node2_state} \circ \dots \circ \text{leaf_state} = \text{最终渲染状态}$

2.2 状态组合规则 (优先级语义)

每个 `RenderAttrib` 有一个优先级 (priority), 组合时高优先级覆盖低优先级:

```
compose(parent_attrib, child_attrib):  
    if child_attrib.priority >= parent_attrib.priority:  
        return child_attrib // 子节点覆盖父节点  
    else:  
        return parent_attrib // 父节点保持
```

这实现了 CSS 类似的层叠 (Cascade) 语义。

2.3 状态缓存的数学基础

`RenderState` 的全局缓存利用了结构共享 (Structural Sharing) :

若 $\text{state_A} == \text{state_B}$ (所有 attrib 相同)
则 state_A 和 state_B 是同一个对象 (指针相同)

推论:

$\text{state_A} == \text{state_B} \iff \&\text{state_A} == \&\text{state_B}$
状态比较 $O(N) \rightarrow$ 指针比较 $O(1)$

`compose()` 的结果也被缓存（二维哈希表）：

```
cache[state_A][state_B] = state_A ◦ state_B
```

命中率分析：

场景图中相同路径的节点会产生相同的 `compose` 调用

缓存命中率通常 > 90%

2.4 增量更新的信息论视角

设当前 GPU 状态为 `S_current`，目标状态为 `S_target`。

朴素方法：每帧全量设置所有状态，代价 $O(N)$ (N = 状态数量)

增量方法：只设置 `S_target - S_current`（集合差），代价 $O(|diff|)$

用位图（BitMask）表示“哪些状态已同步”：

`_state_mask`: 每个 bit 对应一个 `attrib slot`

bit = 1 → 该 `attrib` 已同步到 GPU

bit = 0 → 该 `attrib` 需要重新设置

更新条件：

需要更新 slot $i \iff (target[i] \neq current[i]) \text{ OR } (mask[i] == 0)$

3. 工程实践

3.1 状态排序 (State Sorting)

核心思想：将渲染顺序从"场景图顺序"改为"状态相似度顺序"，最小化状态切换次数。

朴素顺序 (场景图 DFS) :

obj1(shader=A, tex=X) → obj2(shader=B, tex=Y) → obj3(shader=A, tex=Z)

状态切换: A→B→A (2次 Shader 切换)

排序后:

obj1(shader=A, tex=X) → obj3(shader=A, tex=Z) → obj2(shader=B, tex=Y)

状态切换: A→B (1次 Shader 切换)

排序键设计 (从重到轻) :

1. Shader Program (切换代价最高, ~1ms)
2. 纹理绑定 (~0.1ms)
3. 混合状态 (~0.01ms)
4. 深度状态 (~0.01ms)
5. 其他轻量状态

Panda3D 的 `CullBinStateSorted` 实现了按 `RenderState` 指针排序 (等价于按状态内容排序, 因为相同状态共享指针)。

3.2 状态分组 (State Batching)

Draw Call 合并：将使用相同状态的多个物体合并为一个 Draw Call :

条件:

- 相同 Shader
- 相同纹理集合
- 相同混合/深度状态
- 顶点格式兼容

合并方式:

- 静态合并: 预处理阶段合并 (`SceneGraphReducer`)
- 动态合并: 运行时合并 (`Instancing`)

Panda3D 的 `SceneGraphReducer` 提供静态合并:


```
reducer = SceneGraphReducer()
reducer.apply_attribs(scene) # 将状态下推到叶节点
reducer.flatten(scene, SceneGraphReducer.FT_full) # 合并几何体
```

3.3 状态失效 (State Invalidation)

某些操作会导致 `_state_mask` 部分位被清零（强制重新设置）：

触发事件	失效的状态
Shader 切换	纹理 slot（需重新绑定到新 Shader 的 uniform）
FBO 切换	视口、裁剪矩形
上下文丢失（窗口最小化）	全部状态
多线程上下文切换	全部状态

3.4 Uber Shader 策略

问题：频繁切换 Shader 代价极高。

解法：将多个功能合并到一个“超级 Shader”，用 `#define` 或 uniform 开关控制：

```
// Uber Shader 示例
uniform int u_has_normal_map;
uniform int u_has_shadow;
uniform int u_num_lights;

void main() {
    vec3 normal = vertex_normal;

    #ifdef HAS_NORMAL_MAP
    if (u_has_normal_map > 0) {
        normal = texture(normal_map, uv).rgb * 2.0 - 1.0;
    }
    #endif

    // 光照计算
    for (int i = 0; i < u_num_lights; i++) {
        // ...
    }
}
```

权衡： - 优点：减少 Shader 切换 - 缺点：单个 Shader 更复杂，可能影响 GPU 寄存器分配

4. Panda3D 源码解析

4.1 RenderState：不可变状态集合

源码位置：`renderState.cxx`

```
class RenderState : public NodeCachedReferenceCount {
    // 每个 slot 存储一个 RenderAttrib
    Attribute _attributes[RenderAttribRegistry::_max_slots];

    // 哪些 slot 有值 (位图)
    SlotMask _filled_slots;

    // compose() 结果缓存 (二维哈希表)
    CompositionCache _composition_cache;

    // 全局状态池 (所有存活的 RenderState 对象)
    static States _states; // pset<const RenderState *>
    static LightReMutex *_states_lock;
};
```

`compose()` 的缓存机制:

```
// renderState.cxx:339
CPT(RenderState) RenderState::compose(const RenderState *other) const {
    // 快速路径: 空状态
    if (is_empty()) return other;
    if (other->is_empty()) return this;

    // 查缓存
    int index = _composition_cache.find(other);
    if (index != -1) {
        Composition &comp = _composition_cache.modify_data(index);
        if (comp._result != nullptr) {
            _cache_stats.inc_hits();
            return comp._result; // 缓存命中!
        }
    }

    // 缓存未命中, 计算新结果
    _cache_stats.inc_misses();
    CPT(RenderState) result = do_compose(other);

    // 双向记录缓存条目 (便于析构时清理)
    _composition_cache[other]._result = result;
    other->_composition_cache[this]._result = nullptr; // 反向只记录 key

    result->cache_ref(); // 防止被 GC
    return result;
}
```

do_compose() 的实际合并逻辑:

```
CPT(RenderState) RenderState::do_compose(const RenderState *other) const {
    // 遍历两个状态的所有 slot
    SlotMask mask = _filled_slots | other->_filled_slots;

    RenderState *new_state = new RenderState;
    int slot = mask.get_lowest_on_bit();
    while (slot >= 0) {
        const RenderAttrib *a = _attributes[slot]._attrib; // 父状态的 attrib
        const RenderAttrib *b = other->_attributes[slot]._attrib; // 子状态的 attrib

        if (b == nullptr) {
            // 子状态没有这个 attrib, 继承父状态
            new_state->_attributes[slot] = _attributes[slot];
        } else if (a == nullptr) {
            // 父状态没有这个 attrib, 使用子状态
            new_state->_attributes[slot] = other->_attributes[slot];
        } else {
            // 两者都有, 调用 attrib 自身的 compose 逻辑 (考虑优先级)
            int result = a->compose(b);
            new_state->_attributes[slot]._attrib = result;
        }

        mask.clear_bit(slot);
        slot = mask.get_lowest_on_bit();
    }

    // 在全局池中查找或注册 (保证唯一性)
    return return_new(new_state);
}
```

4.2 GSG 的增量状态更新

源码位置: `glGraphicsStateGuardian_src.cxx:12760`

```

void CLP(GraphicsStateGuardian)::set_state_and_transform(
    const RenderState *target, const TransformState *transform) {

    // — 变换矩阵 —————
    if (transform != _internal_transform) {
        _internal_transform = transform;
        do_issue_transform(); // 上传 ModelView 矩阵到 Shader
    }

    // — 快速路径：状态完全相同 —————
    if (target == _state_rs && (_state_mask | _inv_state_mask).is_all_on()) {
        // 只需更新 Shader 的 transform uniform (已在上面处理)
        return;
    }

    _target_rs = target;

    // — Shader 切换 (代价最高, 优先处理) —————
    determine_target_shader();
    if (_target_shader != _state_shader) {
        do_issue_shader(); // glUseProgram()
        _state_shader = _target_shader;
        // Shader 切换后纹理 slot 需要重新绑定
        _state_mask.clear_bit(TextureAttrib::get_class_slot());
    }

    // 更新 Shader Uniforms (矩阵、光照参数等)
    if (_current_shader_context) {
        _current_shader_context->set_state_and_transform(
            target, transform,
            _scene_setup->get_camera_transform(),
            _projection_mat);
    }

    // — 逐属性增量更新 —————
    // 模式：只有当 attrib 指针变化 OR 该 slot 未同步时才调用 do_issue_*( )

    // 深度测试
    int depth_test_slot = DepthTestAttrib::get_class_slot();
    if (_target_rs->get_attrib(depth_test_slot) != _state_rs->get_attrib(depth_test_slot)
        || !_state_mask.get_bit(depth_test_slot)) {
        do_issue_depth_test(); // glDepthFunc() + glEnable/Disable(GL_DEPTH_TEST)
        _state_mask.set_bit(depth_test_slot);
    }

    // 深度写入
    int depth_write_slot = DepthWriteAttrib::get_class_slot();
    if (_target_rs->get_attrib(depth_write_slot) != _state_rs->get_attrib(depth_write_slot)
        || !_state_mask.get_bit(depth_write_slot)) {
        do_issue_depth_write(); // glDepthMask()
        _state_mask.set_bit(depth_write_slot);
    }

    // 面剔除
    int cull_face_slot = CullFaceAttrib::get_class_slot();
    if (_target_rs->get_attrib(cull_face_slot) != _state_rs->get_attrib(cull_face_slot)
        || !_state_mask.get_bit(cull_face_slot)) {
        do_issue_cull_face(); // glCullFace() + glEnable/Disable(GL_CULL_FACE)
        _state_mask.set_bit(cull_face_slot);
    }
}

```

```

// 纹理（代价较高，需要绑定多个纹理单元）
int texture_slot = TextureAttrib::get_class_slot();
if (_target_rs->get_attrib(texture_slot) != _state_rs->get_attrib(texture_slot)
    || !_state_mask.get_bit(texture_slot)) {
    do_issue_texture(); // glActiveTexture() + glBindTexture() × N
    _state_mask.set_bit(texture_slot);
}

// 混合
int transparency_slot = TransparencyAttrib::get_class_slot();
if (_target_rs->get_attrib(transparency_slot) != _state_rs->get_attrib(transparency_slot)
    || !_state_mask.get_bit(transparency_slot)) {
    do_issue_blending(); // glBlendFunc() + glEnable/Disable(GL_BLEND)
    _state_mask.set_bit(transparency_slot);
}

// ... 其他属性（光照、雾、模板、裁剪等）

_state_rs = target; // 记录当前已同步状态
}

```

4.3 RenderAttrib 的优先级系统

源码位置： `renderAttrib.h`

```

class RenderAttrib {
public:
    // 每个 attrib 有一个优先级（默认 0）
    // 高优先级的 attrib 在 compose 时覆盖低优先级
    virtual int get_override() const { return _override; }

    // 子类实现具体的 compose 逻辑
    virtual CPT(RenderAttrib) compose(const RenderAttrib *other) const;
};

// 以 DepthTestAttrib 为例
CPT(RenderAttrib) DepthTestAttrib::compose(const RenderAttrib *other) const {
    const DepthTestAttrib *ta = (const DepthTestAttrib *)other;
    // 高优先级覆盖低优先级
    if (ta->get_override() >= get_override()) {
        return other; // 子节点（other）优先级更高，使用子节点的值
    }
    return this; // 父节点优先级更高，保持父节点的值
}

```

5. 代码演示

5.1 Python：基本状态设置

```

from panda3d.core import (
    NodePath, GeomNode, RenderState,
    DepthTestAttrib, DepthWriteAttrib, CullFaceAttrib,
    TransparencyAttrib, ColorBlendAttrib,
    LVecBase4f
)

## — 场景图状态继承 —————
root = render # 场景根节点

## 父节点设置深度测试
root.set_depth_test(True)
root.set_depth_write(True)

## 子节点覆盖：关闭深度写入（透明物体常用）
transparent_node = root.attach_new_node("transparent")
transparent_node.set_depth_write(False)
transparent_node.set_transparency(TransparencyAttrib.M_alpha)

## 孙节点继承父链的所有状态（深度测试=True, 深度写入=False, 透明=alpha）
leaf = transparent_node.attach_new_node("leaf")
## leaf 自动继承上面的状态，无需重复设置

## — 手动构建 RenderState —————
state = RenderState.make(
    DepthTestAttrib.make(DepthTestAttrib.M_less),
    DepthWriteAttrib.make(DepthWriteAttrib.M_off),
    CullFaceAttrib.make(CullFaceAttrib.M_cull_none),
    TransparencyAttrib.make(TransparencyAttrib.M_alpha),
)

## 应用到节点
node.set_state(state)

## — 状态组合（compose）—————
parent_state = RenderState.make(
    DepthTestAttrib.make(DepthTestAttrib.M_less),
)
child_state = RenderState.make(
    DepthWriteAttrib.make(DepthWriteAttrib.M_off),
)

## compose：子状态叠加到父状态上
combined = parent_state.compose(child_state)
## combined 包含：depth_test=less, depth_write=off

## — 优先级覆盖 —————
## 父节点强制设置（高优先级），子节点无法覆盖
root.set_depth_test(True, 100) # priority=100

child = root.attach_new_node("child")
child.set_depth_test(False, 50) # priority=50, 低于父节点，无效！
## child 实际使用 depth_test=True（父节点的高优先级设置）

```

5.2 Python：状态排序优化

```
from panda3d.core import CullBinManager, CullBinAttrib

## — 配置 Bin 排序 —————
bin_mgr = CullBinManager.get_global_ptr()

## 查看默认 Bin
for i in range(bin_mgr.get_num_bins()):
    name = bin_mgr.get_bin_name(i)
    sort = bin_mgr.get_bin_sort(i)
    btype = bin_mgr.get_bin_type(i)
    print(f"Bin[{i}]: name={name}, sort={sort}, type={btype}")

## 输出示例：
## Bin[0]: name=background, sort=-100, type=BT_fixed
## Bin[1]: name=opaque,      sort=0,      type=BT_state_sorted ← 按状态排序
## Bin[2]: name=transparent,sort=10,     type=BT_back_to_front ← 从后往前
## Bin[3]: name=fixed,      sort=20,     type=BT_fixed
## Bin[4]: name=unsorted,   sort=30,     type=BT_unsorted

## — 将物体放入特定 Bin —————
## 不透明物体（默认在 opaque bin, 按状态排序）
opaque_obj.set_bin("opaque", 0)

## 透明物体（在 transparent bin, 从后往前排序）
transparent_obj.set_bin("transparent", 0)

## GUI 元素（固定顺序，不受深度影响）
gui_element.set_bin("fixed", 10) # sort=10 决定 GUI 内部顺序

## — 自定义 Bin —————
## 创建一个新的 state_sorted bin, 用于特殊效果
bin_mgr.add_bin("effects", CullBinManager.BT_state_sorted, 5)
effect_obj.set_bin("effects", 0)
```


5.3 Python：监控状态切换次数

```
from panda3d.core import PStatClient, RenderState, TransformState

## 连接 PStats 性能分析器
PStatClient.connect()

## 在游戏循环中监控
def monitor_state_changes(task):
    # 查看 RenderState 缓存统计
    num_states = RenderState.get_num_states()
    num_unused = RenderState.get_num_unused_states()
    print(f"RenderStates: {num_states} total, {num_unused} unused")

    # 查看 TransformState 缓存统计
    num_transforms = TransformState.get_num_states()
    print(f"TransformStates: {num_transforms} total")

    return task.cont

taskMgr.add(monitor_state_changes, "monitor", sort=50)

## 手动触发垃圾回收（清理未使用的缓存状态）
RenderState.clear_cache()
TransformState.clear_cache()
```

5.4 Python：强制状态刷新

```
## 某些情况下需要强制 GSG 重新设置所有状态
## （例如：外部 OpenGL 代码修改了状态后）

from panda3d.core import GraphicsEngine

engine = base.graphicsEngine
for win in engine.windows:
    gsg = win.gsg
    if gsg:
        gsg.clear_state_and_transform() # 清空 _state_mask, 强制全量更新
```

5.5 GLSL : Shader 内的状态感知

```
// vertex.glsl
#version 330 core

// Panda3D 自动注入的矩阵 uniform
uniform mat4 p3d_ModelViewProjectionMatrix; // MVP
uniform mat4 p3d_ModelViewMatrix;          // MV
uniform mat3 p3d_NormalMatrix;              // 法线矩阵 (MV的逆转置)

// 自定义 uniform (通过 set_shader_input 设置)
uniform vec4 u_color_tint;
uniform float u_time;

in vec4 p3d_Vertex;
in vec3 p3d_Normal;
in vec2 p3d_MultiTexCoord0;

out vec3 v_normal_ws;
out vec2 v_uv;

void main() {
    gl_Position = p3d_ModelViewProjectionMatrix * p3d_Vertex;
    v_normal_ws = normalize(p3d_NormalMatrix * p3d_Normal);
    v_uv = p3d_MultiTexCoord0;
}
```

```
// fragment.glsl
#version 330 core

uniform sampler2D p3d_Texture0;
uniform vec4 u_color_tint;

in vec3 v_normal_ws;
in vec2 v_uv;

out vec4 frag_color;

void main() {
    vec4 tex_color = texture(p3d_Texture0, v_uv);
    frag_color = tex_color * u_color_tint;
}
```

```

## Python 端设置 Shader
from panda3d.core import Shader, ShaderAttrib

shader = Shader.load(Shader.SL_GLSL,
    vertex="vertex.glsl",
    fragment="fragment.glsl")

node.set_shader(shader)
node.set_shader_input("u_color_tint", LVecBase4f(1, 0.5, 0.5, 1))
node.set_shader_input("u_time", 0.0)

## 每帧更新 time
def update_shader(task):
    node.set_shader_input("u_time", task.time)
    return task.cont
taskMgr.add(update_shader, "update_shader")

```

5.6 C++：自定义 RenderAttrib

```

// 自定义一个 WireframeAttrib (示例)
class WireframeAttrib : public RenderAttrib {
PUBLISHED:
    static CPT(RenderAttrib) make(bool enabled) {
        WireframeAttrib *attrib = new WireframeAttrib(enabled);
        return return_new(attrib);
    }

    bool get_enabled() const { return _enabled; }

public:
    // compose : 子节点的设置覆盖父节点
    virtual CPT(RenderAttrib) compose(const RenderAttrib *other) const override {
        const WireframeAttrib *ta = (const WireframeAttrib *)other;
        if (ta->get_override() >= get_override()) {
            return other;
        }
        return this;
    }

    // 注册到 GSG (在 do_issue_* 中调用 glPolygonMode)
    static int get_class_slot() { return _attrib_slot; }

private:
    WireframeAttrib(bool enabled) : _enabled(enabled) {}
    bool _enabled;
    static int _attrib_slot;
};

```

6. 性能分析与调优

6.1 Draw Call 数量 vs 状态切换次数

性能公式 (近似) :

$$\text{帧时间} \approx N_{\text{drawcall}} \times T_{\text{drawcall}} + N_{\text{statechange}} \times T_{\text{statechange}} + N_{\text{vertex}} \times T_{\text{vertex}}$$

典型值 (现代 GPU, CPU 端) :

$T_{\text{drawcall}} \approx 5\sim50 \mu\text{s}$ (CPU 提交开销)

$T_{\text{statechange}} \approx 0.1\sim10 \mu\text{s}$ (取决于状态类型)

$T_{\text{vertex}} \approx \text{极小}$ (GPU 并行处理)

优化目标:

减少 N_{drawcall} (合并几何体、Instancing)

减少 $N_{\text{statechange}}$ (状态排序、Uber Shader)

6.2 各状态切换代价排行

状态	切换代价	优化策略
Shader Program	★★★★★	Uber Shader、减少 Shader 种类
Render Target (FBO)	★★★★★	合并 Pass、Render Graph 自动调度
纹理绑定	★★★★☆	Texture Array、Bindless Texture
Uniform Buffer	★★★☆☆	UBO 批量更新
深度/混合状态	★★☆☆☆	Pipeline State Object (Vulkan/DX12)
顶点格式	★★☆☆☆	统一顶点格式

6.3 RenderState 缓存命中率优化

```
## 避免每帧创建新的 RenderState (会导致缓存污染)

## ❌ 错误: 每帧创建新状态
def bad_update(task):
    state = RenderState.make(
        ColorAttrib.make_flat(LVecBase4f(task.time % 1, 0, 0, 1)) # 每帧不同!
    )
    node.set_state(state)
    return task.cont

## ✅ 正确: 缓存状态对象, 只在需要时更新
cached_states = {}
def good_update(task):
    # 离散化颜色值, 减少唯一状态数量
    r = int(task.time * 10) % 10 / 10.0 # 只有 10 种值
    if r not in cached_states:
        cached_states[r] = RenderState.make(
            ColorAttrib.make_flat(LVecBase4f(r, 0, 0, 1))
        )
    node.set_state(cached_states[r])
    return task.cont
```

6.4 状态排序的实际效果

场景: 1000 个物体, 10 种 Shader, 每种 Shader 100 个物体

未排序 (场景图顺序):

平均每个物体切换 Shader 概率 $\approx 90\%$

Shader 切换次数 ≈ 900 次

按 Shader 排序后:

Shader 切换次数 = 9 次 (10 种 Shader 之间的边界)

性能提升: 100x (Shader 切换)

实测 (典型场景):

排序前: 帧时间 16ms (60fps 边缘)

排序后: 帧时间 8ms (120fps)

小结

概念	核心理想	Panda3D 实现
RenderState 不可变缓存	相同状态共享指针，O(1) 比较	<code>renderState.cxx</code> 全局 <code>_states</code> 池
<code>compose()</code> 缓存	状态组合结果缓存，避免重复计算	<code>_composition_cache</code> 二维哈希表
差量更新	<code>_state_mask</code> 位图，只更新变化的属性	<code>glGraphicsStateGuardian_src.cxx:12760</code>
Bin 排序	按状态相似度重排绘制顺序	<code>CullBinStateSorted</code>
优先级继承	高优先级父节点状态不被子节点覆盖	<code>RenderAttrib::get_override()</code>

专题02：场景图与空间加速结构

目录

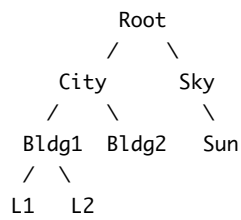
1. 问题背景
 2. 数学原理 - 2.1 树形场景图的代数结构 - 2.2 包围体层次 (BVH) 理论 - 2.3 视锥体裁剪的几何数学 - 2.4 空间查询的复杂度分析
 3. 工程实践 - 3.1 脏标记 (Dirty Flag) 传播 - 3.2 懒惰求值 (Lazy Evaluation) - 3.3 DrawMask 位掩码系统 - 3.4 Portal 遮挡剔除
 4. Panda3D 源码解析 - 4.1 PandaNode 包围体系统 - 4.2 update_cached() 递归更新 - 4.3 CullTraverserData 视锥变换
 5. 代码演示
 6. 性能分析
-

1. 问题背景

3D 场景中可能有数万个体，每帧都对所有物体调用 GPU 绘制命令是不可接受的。场景图 (Scene Graph) 解决两个核心问题：

1. **层次变换**：子节点自动继承父节点的位置/旋转/缩放，无需手动计算世界坐标
2. **空间剔除**：利用层次包围体，快速排除不在视野内的整个子树，避免无效绘制

场景图示例：



当 **City** 节点的包围球不与视锥体相交时，**Bldg1**、**Bldg2** 及其所有子节点都被一次性剔除，无需逐一检测。

2. 数学原理

2.1 树形场景图的代数结构

场景图是一棵有向无环图（DAG），节点 n 的世界变换矩阵定义为从根到该节点路径上所有局部变换的复合：

$$M_{\text{world}}(n) = M_{\text{local}}(n_0) \cdot M_{\text{local}}(n_1) \cdot \dots \cdot M_{\text{local}}(n)$$

其中 $n_0, n_1, \dots, n$ 是从根到 n 的路径。

关键性质： - 矩阵乘法满足结合律： $(A \cdot B) \cdot C = A \cdot (B \cdot C)$ - 但不满足交换律： $A \cdot B \neq B \cdot A$ （旋转顺序敏感）

增量更新： 若节点 n 的局部变换改变，只需重新计算 n 及其所有后代的世界变换，而非整棵树。

2.2 包围体层次（BVH）理论

包围体（Bounding Volume, BV） 是包含几何体的简单形状，用于快速相交测试。

常见类型及其权衡：

类型	存储	相交测试	紧密度
包围球（Sphere）	4 floats	$O(1)$	低
AABB（轴对齐包围盒）	6 floats	$O(1)$	中
OBB（有向包围盒）	15 floats	$O(n)$	高
k-DOP	2k floats	$O(k)$	可调

BVH 构建： Panda3D 使用自底向上的方式构建 BVH： - 叶节点：包含几何体的精确包围体 - 内部节点：所有子节点包围体的并集（`around()`）

BVH 查询复杂度： - 最坏情况： $O(n)$ （所有节点都在视野内） - 平均情况： $O(\log n)$ （大部分子树被早期剔除） - 最优情况： $O(1)$ （根节点即被剔除）

2.3 视锥体裁剪的几何数学

视锥体（View Frustum） 由 6 个平面定义：近平面、远平面、左、右、上、下。

每个平面用法向量 $\mathbf{n}$ 和距离 d 表示： $\mathbf{n} \cdot \mathbf{x} + d = 0$

球体-平面相交测试：

给定球心 $\mathbf{c}$, 半径 r , 平面 $(\mathbf{n}, d)$:

$$\text{dist} = \mathbf{n} \cdot \mathbf{c} + d$$

- $\text{dist} > r$: 球完全在平面外侧 → **剔除**
- $\text{dist} < -r$: 球完全在平面内侧 → **完全包含**
- 否则: 球与平面相交 → **部分包含, 继续检测子节点**

视锥变换到局部空间:

Panda3D 的关键优化——不是将包围体变换到世界空间, 而是将视锥体变换到节点的**局部空间**:

$$F_{\text{local}} = F_{\text{world}} \cdot M_{\text{world}}^{-1}$$

这样每个节点的包围体保持在其局部坐标系中, 无需每帧重新计算。

2.4 空间查询的复杂度分析

设场景有 N 个节点, 树高为 h , 每层平均分支因子为 b :

$$N = b^h \Rightarrow h = \log_b N$$

视锥剔除效率: - 若视锥体覆盖场景的比例为 p , 则期望访问节点数为 $O(p \cdot N + h)$ - 当 $p \ll 1$ (如室内场景) 时, 效率极高

包围体更新代价: - 单节点变换: 需更新从该节点到根的路径, 代价 $O(h) = O(\log N)$ - 使用脏标记延迟更新: 实际代价摊销为 $O(1)$ (仅在查询时才重算)

3. 工程实践

3.1 脏标记 (Dirty Flag) 传播

问题：每次节点变换都立即重算包围体代价高昂，且同一帧内可能多次修改。

解决方案：脏标记 (Dirty Flag) 模式

```
节点变换改变
↓
mark_bounds_stale() ← 向上传播脏标记
↓
父节点也标记为脏
↓
... 直到根节点
↓
下一帧 CullTraverser 遍历时
↓
get_bounds() 检测到脏标记
↓
update_cached() 重新计算
```

关键数据结构 (`pandaNode.cxx`) :

```
UpdateSeq _last_update;          // 上次更新的序列号
UpdateSeq _next_update;          // 期望的最新序列号
UpdateSeq _last_bounds_update;   // 上次包围体更新序列号
```

当 `_last_bounds_update != _next_update` 时，包围体被认为是脏的。

3.2 懒惰求值 (Lazy Evaluation)

Panda3D 的包围体系统是**完全懒惰**的： - 修改节点时只标记脏，不立即计算 - 只有在 `get_bounds()` 被调用时才触发重算 - 重算时递归检查子节点，若子节点也脏则先更新子节点

这种策略的优势： 1. 同一帧内多次修改只触发一次重算 2. 从未被查询的节点永远不会重算 3. 与多线程 Pipeline Cyclers 天然兼容

3.3 DrawMask 位掩码系统

每个节点有两个 32 位掩码：

```
DrawMask _draw_control_mask; // 控制位: 1 = 该位被此节点控制
DrawMask _draw_show_mask;    // 显示位: 1 = 该位显示, 0 = 隐藏
```

组合语义：

control	show	含义
0	0	非渲染节点
0	1	正常可见
1	0	隐藏
1	1	强制显示 (show-through)

向上传播规则：父节点的净掩码是所有子节点掩码的并集，但有一个例外：若一个子节点隐藏（10）而另一个正常可见（01），结果为正常可见（01），因为只有**所有**渲染节点都隐藏时才向上传播隐藏状态。

3.4 Portal 遮挡剔除

对于室内场景，视锥剔除不够高效（大量物体在视锥内但被墙壁遮挡）。Portal 剔除通过"门洞"限制可见区域：

[房间A] --[门洞]--> [房间B]

遍历时，每经过一个 Portal，视锥体被裁剪为 Portal 开口的形状，从而排除门洞外的所有物体。

Panda3D 通过 OccluderEffect 和 CullPlanes 实现类似功能。

4. Panda3D 源码解析

4.1 PandaNode 包围体系统

文件: `panda/src/pgraph/pandaNode.cxx`

`get_bounds()` — 懒惰求值入口

```
// pandaNode.cxx:1875
CPT(BoundingBox) PandaNode::
get_bounds(Thread *current_thread) const {
    int pipeline_stage = current_thread->get_pipeline_stage();
    CDLockedStageReader cdata(_cycler, pipeline_stage, current_thread);

    // 检查脏标记: last_bounds_update != next_update 表示包围体已过期
    if (cdata->_last_bounds_update != cdata->_next_update) {
        // 缓存失效, 需要重建
        CPT(BoundingBox) result;
        {
            PStatTimer timer(_update_bounds_pcollector);
            // update_cached() 会递归更新整个子树
            CDStageWriter cdataw =
                ((PandaNode *)this)->update_cached(true, pipeline_stage, cdata);
            result = cdataw->_external_bounds;
        }
        return result;
    }
    // 缓存有效, 直接返回
    return cdata->_external_bounds;
}
```

关键点: - `CDLockedStageReader` 是线程安全的读锁 - `_last_bounds_update != _next_update` 是脏标记检测
- `update_cached(true, ...)` 的 `true` 参数表示同时更新包围体

`mark_bounds_stale()` — 脏标记向上传播

```
// pandaNode.cxx:1964
void PandaNode::
mark_bounds_stale(Thread *current_thread) const {
    // 遍历当前及上游所有 pipeline stage
    OPEN_ITERATE_CURRENT_AND_UPSTREAM_NOLOCK(_cycler, current_thread) {
        mark_bounds_stale(pipeline_stage, current_thread);
    }
    CLOSE_ITERATE_CURRENT_AND_UPSTREAM_NOLOCK(_cycler);
}
```

`mark_bounds_stale(pipeline_stage, ...)` 的内部实现会： 1. 递增 `_next_update` 序号（使 `_last_bounds_update != _next_update`） 2. 对每个父节点递归调用 `mark_bounds_stale()`

`compute_internal_bounds()` — 叶节点包围体

```
// pandaNode.cxx:2249
void PandaNode::
compute_internal_bounds(CPT(BoundingVolume) &internal_bounds,
                        int &internal_vertices,
                        int pipeline_stage,
                        Thread *current_thread) const {
    // 基类默认返回空包围球
    // GeomNode 等子类会重写此方法，计算几何体的精确包围体
    internal_bounds = new BoundingSphere;
    internal_vertices = 0;
}
```

compute_external_bounds() — 合并子节点包围体

```
// pandaNode.cxx:2263
void PandaNode::
compute_external_bounds(CPT(BoundingVolume) &external_bounds,
                        BoundingVolume::BoundsType btype,
                        const BoundingVolume **volumes, size_t num_volumes,
                        int pipeline_stage, Thread *current_thread) const {

    CPT(TransformState) transform = get_transform(current_thread);
    PT(GeometricBoundingVolume) gbv;

    // 根据类型选择包围体形状
    if (btype == BoundingVolume::BT_box) {
        gbv = new BoundingBox;
    }
    else if (btype == BoundingVolume::BT_sphere || !transform->is_identity()) {
        // 有变换时必须用球（球在变换下保持球形）
        gbv = new BoundingSphere;
    }
    else {
        // 若所有子节点都是 AABB 且无变换，结果也是 AABB
        bool all_box = true;
        for (size_t i = 0; i < num_volumes; ++i) {
            if (volumes[i]->as_bounding_box() == nullptr) {
                all_box = false;
            }
        }
        gbv = all_box ? (PT(GeometricBoundingVolume))new BoundingBox
            : (PT(GeometricBoundingVolume))new BoundingSphere;
    }

    if (num_volumes > 0) {
        // around() 计算所有子包围体的最小外接包围体
        const BoundingVolume **child_begin = &volumes[0];
        const BoundingVolume **child_end = child_begin + num_volumes;
        ((BoundingVolume *)gbv)->around(child_begin, child_end);

        // 应用节点自身的变换
        if (!transform->is_identity()) {
            gbv->xform(transform->get_mat());
        }
    }

    external_bounds = gbv;
}
```


4.2 update_cached() 递归更新

```

// pandaNode.cxx:3231
PandaNode::CDStageWriter PandaNode::
update_cached(bool update_bounds, int pipeline_stage,
              PandaNode::CDLockedStageReader &cdata) {
do {
    UpdateSeq last_update = cdata->_last_update;
    UpdateSeq next_update = cdata->_next_update;

    // 收集子节点列表
    PT(Down) down;
    {
        CDStageWriter cdataw(_cycler, pipeline_stage, cdata);
        down = cdataw->modify_down();
    }

    int num_children = down->size();
    const BoundingBoxVolume **child_volumes = nullptr;
    int child_volumes_i = 0;

    if (update_bounds) {
        // 分配栈上数组存放子包围体指针
        child_volumes = (const BoundingBoxVolume **)
            alloca(sizeof(BoundingBoxVolume *) * (num_children + 1));

        // 先获取自身内部包围体 (叶节点几何体)
        CPT(BoundingBoxVolume) internal_bounds =
            get_internal_bounds(pipeline_stage, current_thread);

        if (!internal_bounds->is_empty()) {
            child_volumes[child_volumes_i++] = internal_bounds;
        }
    }

    // 遍历所有子节点
    for (int i = 0; i < num_children; ++i) {
        PandaNode *child = (*down)[i].get_child();
        CDLockedStageReader child_cdata(child->_cycler, pipeline_stage, current_thread);

        // 若子节点也是脏的, 先递归更新子节点
        if (child_cdata->_last_bounds_update != child_cdata->_next_update) {
            CDStageWriter child_cdataw =
                child->update_cached(update_bounds, pipeline_stage, child_cdata);

            // 收集子节点的包围体
            if (update_bounds && !child_cdataw->external_bounds->is_empty()) {
                child_volumes[child_volumes_i++] = child_cdataw->external_bounds;
            }
        }
    }

    // 计算本节点的外部包围体 (包含自身 + 所有子节点)
    if (update_bounds) {
        CPT(BoundingBoxVolume) external_bounds;
        compute_external_bounds(external_bounds, btype,
                                child_volumes, child_volumes_i,
                                pipeline_stage, current_thread);
        cdataw->external_bounds = external_bounds;
        cdataw->_last_bounds_update = next_update; // 清除脏标记
    }
}

```

```
} while (...); // 处理并发竞争的重试逻辑  
}
```

递归结构：

```
update_cached(Root)  
├─ update_cached(City)      ─ 若 City 是脏的  
│   └─ update_cached(Bldg1) ─ 若 Bldg1 是脏的  
│       └─ get_internal_bounds(L1)  
│           └─ get_internal_bounds(L2)  
│               └─ compute_external_bounds([L1_bv, L2_bv])  
│   └─ get_bounds(Bldg2)      ─ 若 Bldg2 不脏，直接返回缓存  
│       └─ compute_external_bounds([Bldg1_bv, Bldg2_bv])  
└─ get_bounds(Sky)           ─ 若 Sky 不脏，直接返回缓存  
    └─ compute_external_bounds([City_bv, Sky_bv])
```

4.3 CullTraverserData 视锥变换

文件： `panda/src/pgraph/cullTraverserData.cxx`

apply_transform() — 将视锥变换到局部空间

```
// cullTraverserData.cxx:96
void CullTraverserData::
apply_transform(const TransformState *node_transform) {
    if (!node_transform->is_identity()) {
        // 处理实例化 (GPU Instancing) 的特殊情况
        if (_instances != nullptr) {
            Instancelist *instances = new Instancelist(*_instances);
            for (Instancelist::Instance &instance : *instances) {
                instance.set_transform(
                    instance.get_transform()->compose(node_transform));
            }
            _instances = std::move(instances);
            return;
        }

        // 累积世界变换: net_transform = parent_net_transform * node_local_transform
        _net_transform = _net_transform->compose(node_transform);

        if (_view_frustum != nullptr || _cull_planes != nullptr) {
            // 关键优化: 将视锥体变换到节点的局部空间
            // 而不是将包围体变换到世界空间
            const LMatrix4 *inverse_mat = node_transform->get_inverse_mat();

            if (inverse_mat != nullptr) {
                if (_view_frustum != nullptr) {
                    // 复制视锥体 (避免修改父节点的视锥体)
                    _view_frustum = _view_frustum->make_copy()
                        ->as_geometric_bounding_volume();

                    // 用逆矩阵将视锥体变换到局部空间
                    // F_local = F_world * M_world_to_local
                    _view_frustum->xform(*inverse_mat);
                }

                if (_cull_planes != nullptr) {
                    _cull_planes = _cull_planes->xform(*inverse_mat);
                }
            }
            else {
                // 奇异矩阵 (如零缩放), 无法求逆
                // 放弃视锥剔除, 保守地认为所有子节点都可见
                pgraph_cat.warning()
                    << "Singular transformation detected on node: "
                    << get_node_path() << "\n";
                _view_frustum = nullptr;
                _cull_planes = nullptr;
            }
        }
    }
}
```

apply_transform_and_state() — 完整的节点处理

```
// cullTraverserData.cxx:31
void CullTraverserData::
apply_transform_and_state(CullTraverser *trav) {
    CPT(RenderState) node_state = _node_reader.get_state();

    // 处理 Camera Tag State (相机特定的状态覆盖)
    if (trav->has_tag_state_key() &&
        _node_reader.has_tag(trav->get_tag_state_key())) {
        const Camera *camera = trav->get_scene()->get_camera_node();
        std::string tag_state = _node_reader.get_tag(trav->get_tag_state_key());
        node_state = node_state->compose(camera->get_tag_state(tag_state));
    }

    // 更新 DrawMask
    _node_reader.compose_draw_mask(_draw_mask);

    // 处理 RenderEffects (Billboard、Compass 等特殊效果)
    const RenderEffects *node_effects = _node_reader.get_effects();
    if (!node_effects->has_cull_callback()) {
        apply_transform(_node_reader.get_transform());
    } else {
        // Billboard 等效果可能修改变换
        CPT(TransformState) node_transform = _node_reader.get_transform();
        node_effects->cull_callback(trav, *this, node_transform, node_state);
        apply_transform(node_transform);
        _node_reader.check_cached(false);
    }

    // 累积渲染状态
    if (!node_state->is_empty()) {
        _state = _state->compose(node_state);
    }

    // 处理裁剪平面 (Clip Planes)
    if (clip_plane_cull) {
        const ClipPlaneAttrib *cpa = (const ClipPlaneAttrib *)
            node_state->get_attrib(ClipPlaneAttrib::get_class_slot());
        // ...
    }

    // 处理雾效 (Fog)
    const FogAttrib *fog_attr;
    if (node_state->get_attrib(fog_attr)) {
        Fog *fog = fog_attr->get_fog();
        if (fog != nullptr) {
            fog->adjust_to_camera(trav->get_camera_transform());
        }
    }
}
```

5. 代码演示

5.1 Python：场景图基本操作

```
from panda3d.core import NodePath, PandaNode, GeomNode, BoundingSphere, Point3

## — 构建场景图 —————
root = NodePath("root")
city = root.attach_new_node("city")
bldg1 = city.attach_new_node("building_1")
bldg2 = city.attach_new_node("building_2")

## 设置局部变换
city.set_pos(100, 0, 0)
bldg1.set_pos(0, 0, 0) # 相对于 city
bldg2.set_pos(50, 0, 0) # 相对于 city

## 获取世界坐标（自动计算 net_transform）
print(bldg1.get_pos(render)) # (100, 0, 0) - city + bldg1 的累积
print(bldg2.get_pos(render)) # (150, 0, 0)

## — 包围体操作 —————
## 获取节点的包围体（懒惰计算）
bounds = city.node().get_bounds()
print(f"City bounds: {bounds}") # BoundingSphere 或 BoundingBox

## 手动设置自定义包围体（覆盖自动计算）
custom_sphere = BoundingSphere(Point3(0, 0, 5), 20.0)
bldg1.node().set_bounds(custom_sphere)
bldg1.node().set_final(True) # 不再向下递归计算子节点包围体

## 强制重新计算包围体
bldg1.node().mark_bounds_stale()

## — 可见性控制 —————
## 隐藏节点（设置 DrawMask）
bldg2.hide()
bldg2.show()

## 按相机掩码控制可见性
from panda3d.core import BitMask32
MAIN_CAM_MASK = BitMask32.bit(0)
SHADOW_CAM_MASK = BitMask32.bit(1)

bldg1.show(MAIN_CAM_MASK) # 主相机可见
bldg1.hide(SHADOW_CAM_MASK) # 阴影相机不可见（不投射阴影）
```

5.2 Python：空间查询

```
from panda3d.core import NodePath, CollisionTraverser, CollisionNode
from panda3d.core import CollisionSphere, CollisionHandlerQueue, Point3

## — 射线拾取 (Ray Picking) —————

from panda3d.core import CollisionRay, CollisionHandlerQueue

picker = CollisionTraverser("picker")
pick_queue = CollisionHandlerQueue()

pick_node = CollisionNode("mouse_ray")
pick_node.set_from_collide_mask(GeomNode.get_default_collide_mask())
pick_ray = CollisionRay()
pick_node.add_solid(pick_ray)

pick_np = base.camera.attach_new_node(pick_node)
picker.add_collider(pick_np, pick_queue)

def pick_object(mouse_x, mouse_y):
    # 从鼠标位置发射射线
    pick_ray.set_from_lens(base.camNode, mouse_x, mouse_y)
    picker.traverse(render)

    if pick_queue.get_num_entries() > 0:
        pick_queue.sort_entries()
        hit = pick_queue.get_entry(0)
        return hit.get_into_node_path()
    return None

## — 范围查询 (Sphere Query) —————

def find_nodes_in_sphere(center: Point3, radius: float, root: NodePath):
    """查找在球形范围内的所有节点"""
    results = []
    query_sphere = BoundingSphere(center, radius)

    def check_node(np: NodePath):
        bounds = np.node().get_bounds()
        if bounds.contains(query_sphere) != BoundingVolume.IF_no_intersection:
            results.append(np)
            for child in np.get_children():
                check_node(child)

    check_node(root)
    return results
```

5.3 Python：LOD 节点（空间加速的应用）

```
from panda3d.core import LODNode, NodePath

## LODNode 根据距离自动切换细节级别
lod = LODNode("my_lod")
lod_np = NodePath(lod)
lod_np.reparent_to(render)

## 加载不同精度的模型
high_detail = loader.load_model("building_high.egg")
med_detail = loader.load_model("building_med.egg")
low_detail = loader.load_model("building_low.egg")
billboard = loader.load_model("building_billboard.egg")

## 添加 LOD 级别: (switch_in_distance, switch_out_distance)
lod.add_switch(100, 0)    # 0-100 单位: 高精度
lod.add_switch(300, 100) # 100-300 单位: 中精度
lod.add_switch(800, 300) # 300-800 单位: 低精度
lod.add_switch(2000, 800) # 800-2000 单位: 广告牌

high_detail.reparent_to(lod_np)
med_detail.reparent_to(lod_np)
low_detail.reparent_to(lod_np)
billboard.reparent_to(lod_np)
```

5.4 Python：自定义包围体优化剔除

```
from panda3d.core import BoundingBox, Point3

## 对于建筑物（轴对齐长方体），AABB 比球体更紧密
building_np = loader.load_model("skyscraper.egg")

## 手动设置精确的 AABB
tight_box = BoundingBox(
    Point3(-10, -10, 0),    # 最小角
    Point3(10, 10, 100)    # 最大角 (100 单位高)
)
building_np.node().set_bounds(tight_box)

## 对于复杂不规则形状，用多个子节点分别设置包围体
complex_obj = NodePath("complex")
part1 = complex_obj.attach_new_node("part1")
part2 = complex_obj.attach_new_node("part2")
## part1 和 part2 各自有紧密的包围体
## complex_obj 的包围体自动合并两者
```


5.5 C++：自定义节点重写包围体计算

```
// 自定义节点，重写 compute_internal_bounds()
class ProceduralTerrainNode : public PandaNode {
public:
    ProceduralTerrainNode(const std::string &name)
        : PandaNode(name), _width(1000.f), _height(1000.f), _max_elev(200.f) {}

    virtual void compute_internal_bounds(
        CPT(BoundingVolume) &internal_bounds,
        int &internal_vertices,
        int pipeline_stage,
        Thread *current_thread) const override {

        // 地形天然适合 AABB
        internal_bounds = new BoundingBox(
            LPoint3(-_width/2, -_height/2, 0),
            LPoint3(_width/2, _height/2, _max_elev)
        );
        internal_vertices = (int)(_width/10) * (int)(_height/10);
    }

    void set_max_elevation(float elev) {
        _max_elev = elev;
        mark_internal_bounds_stale(); // 触发重新计算
    }

private:
    float _width, _height, _max_elev;
};
```

6. 性能分析

6.1 包围体类型选择

场景类型	推荐包围体	原因
角色/生物	BoundingSphere	旋转不变，更新代价低
建筑/家具	BoundingBox	轴对齐，更紧密
地形	BoundingBox	天然轴对齐
粒子系统	BoundingSphere	动态更新频繁，球更快

6.2 场景图深度 vs 宽度

结构	优点	缺点
深树（深度大）	剔除粒度细	遍历开销大，缓存不友好
宽树（宽度大）	遍历快	剔除效率低（大包围体）
平衡树	最优剔除效率	需要手动组织

实践建议： - 将空间上相邻的物体组织在同一父节点下 - 避免将相距很远的物体放在同一父节点（导致包围体过大） - 深度控制在 5-10 层为宜

6.3 常见性能陷阱

```
## ❌ 错误：将整个城市放在一个节点下
city = NodePath("city")
for i in range(10000):
    building = loader.load_model("building.egg")
    building.reparent_to(city) # 所有建筑共享一个父节点
## 问题：city 的包围体包含所有建筑，视锥剔除几乎无效

## ✅ 正确：按区域分组
city = NodePath("city")
for block_x in range(10):
    for block_y in range(10):
        block = city.attach_new_node(f"block_{block_x}_{block_y}")
        block.set_pos(block_x * 100, block_y * 100, 0)
        for i in range(100):
            building = loader.load_model("building.egg")
            building.reparent_to(block)
## 优势：每个 block 有紧密的包围体，可以整块剔除

## ❌ 错误：频繁调用 get_bounds() 触发重算
for node in all_nodes:
    node.set_pos(new_pos[node])
    bounds = node.get_bounds() # 每次 set_pos 后立即查询，触发重算

## ✅ 正确：批量修改后统一查询
for node in all_nodes:
    node.set_pos(new_pos[node]) # 只标记脏，不重算
## 统一查询（此时才触发一次批量重算）
for node in all_nodes:
    bounds = node.get_bounds()
```

小结

概念	核心思想	Panda3D 实现
场景图	层次变换，子节点继承父节点	<code>NodePath + PandaNode</code>
BVH	层次包围体，快速剔除子树	<code>get_bounds() + compute_external_bounds()</code>
脏标记	延迟计算，只在需要时更新	<code>_last_bounds_update != _next_update</code>
懒惰求值	修改时只标记，查询时才计算	<code>mark_bounds_stale() + update_cached()</code>
视锥变换	将视锥变换到局部空间	<code>apply_transform() + xform(inverse_mat)</code>
DrawMask	位掩码控制多相机可见性	<code>_draw_control_mask + _draw_show_mask</code>

核心设计哲学： 1. **空间局部性：** 相邻物体组织在同一子树，包围体紧密 2. **时间局部性：** 脏标记 + 懒惰求值，避免重复计算 3. **保守正确性：** 奇异矩阵时放弃剔除，宁可多画不可漏画 4. **线程安全：** Pipeline Cycller 保证多线程下的数据一致性

专题03：变换系统与坐标空间

目录

1. 问题背景
 2. 数学原理 - 2.1 仿射变换与齐次坐标 - 2.2 TRS 分解：位置、旋转、缩放 - 2.3 四元数旋转 - 2.4 坐标系约定与转换 - 2.5 变换复合的代数性质
 3. 工程实践 - 3.1 双重表示：组件式 vs 矩阵式 - 3.2 懒惰计算与标志位系统 - 3.3 全局缓存（Flyweight 模式） - 3.4 坐标系统一
 4. Panda3D 源码解析 - 4.1 TransformState 类结构 - 4.2 do_compose() — 变换复合 - 4.3 do_calc_mat() — 组件转矩阵 - 4.4 do_calc_components() — 矩阵分解
 5. 代码演示
 6. 性能分析
-

1. 问题背景

3D 引擎中的变换系统需要解决以下核心问题：

1. **表示问题**：如何存储一个物体的位置、旋转、缩放？
2. **复合问题**：父子节点的变换如何合并为世界变换？
3. **坐标系问题**：不同系统（物理引擎、动画系统、渲染器）使用不同坐标系约定，如何统一？
4. **性能问题**：矩阵乘法代价高，如何避免重复计算？

Panda3D 的 `TransformState` 类是这些问题的核心解决方案。

2. 数学原理

2.1 仿射变换与齐次坐标

3D 变换（平移、旋转、缩放）统一用 **4x4 齐次矩阵** 表示：

$$M = \begin{pmatrix} R_{3 \times 3} & \mathbf{t} \\ \mathbf{0}^T & 1 \end{pmatrix}$$

其中 $R_{3 \times 3}$ 是旋转+缩放矩阵， $\mathbf{t}$ 是平移向量。

齐次坐标：3D 点 (x, y, z) 表示为 $(x, y, z, 1)$ ，方向向量表示为 $(x, y, z, 0)$ 。

$$\begin{pmatrix} x' \\ y' \\ z' \\ 1 \end{pmatrix} = M \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$

$$\begin{pmatrix} x' \\ y' \\ z' \\ 0 \end{pmatrix} = M \begin{pmatrix} x \\ y \\ z \\ 0 \end{pmatrix}$$

2.2 TRS 分解：位置、旋转、缩放

标准 TRS 矩阵分解：

$$M_{\text{TRS}} = T \cdot R \cdot S$$

其中： - T ：平移矩阵 $\begin{pmatrix} I & \mathbf{t} \\ 0 & 1 \end{pmatrix}$ - R ：旋转矩阵（正交矩阵， $R^T = R^{-1}$ ） - S ：缩放矩阵 $\text{diag}(s_x, s_y, s_z, 1)$

展开形式：

$$M_{\text{TRS}} = \begin{pmatrix} s_x r_{00} & s_y r_{01} & s_z r_{02} & t_x \\ s_x r_{10} & s_y r_{11} & s_z r_{12} & t_y \\ s_x r_{20} & s_y r_{21} & s_z r_{22} & t_z \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

矩阵分解 (Decompose)：给定矩阵 M ，提取 TRS 分量：

- 提取缩放： $s_i = |\text{column}_i(M)|$
- 提取旋转： $R = M_{3 \times 3} \cdot \text{diag}(1/s_x, 1/s_y, 1/s_z)$
- 提取平移：直接读取第4列

注意：非均匀缩放 ($s_x \neq s_y \neq s_z$) 与旋转的复合会产生剪切 (Shear)，使分解变得复杂。

2.3 四元数旋转

四元数 $q = w + xi + yj + zk$ ，其中 $i^2 = j^2 = k^2 = ijk = -1$ 。

单位四元数 ($|q| = 1$) 表示旋转：绕轴 $\hat{n}$ 旋转角度 θ ：

$$\mathbf{q} = \cos\frac{\theta}{2} + \sin\frac{\theta}{2}(n_x i + n_y j + n_z k)$$

四元数复合（旋转叠加）：

$$\mathbf{q}_{AB} = \mathbf{q}_B \cdot \mathbf{q}_A$$

注意顺序：先应用 $\mathbf{q}_A$ ，再应用 $\mathbf{q}_B$ ，结果是 $\mathbf{q}_B \cdot \mathbf{q}_A$ （右乘）。

四元数 vs 欧拉角（HPR）：

特性	四元数	欧拉角（HPR）
存储	4 floats	3 floats
插值	SLERP（平滑）	线性（可能万向锁）
万向锁	无	有
直觉性	低	高
复合代价	O(1)	需转换为矩阵

Panda3D 内部优先使用四元数，但提供 HPR 接口供用户使用。

HPR 约定（Panda3D 特有）： - **H**（Heading）：绕 Z 轴旋转（偏航，Yaw） - **P**（Pitch）：绕 Y 轴旋转（俯仰） - **R**（Roll）：绕 X 轴旋转（翻滚） - 应用顺序：H → P → R（ZYX 欧拉角）

2.4 坐标系约定与转换

不同系统使用不同的坐标系约定：

系统	右手/左手	上方向	前方向
Panda3D	右手	Z 轴向上	Y 轴向前
OpenGL	右手	Y 轴向上	-Z 轴向前
DirectX	左手	Y 轴向上	Z 轴向前
Blender	右手	Z 轴向上	-Y 轴向前
Unity	左手	Y 轴向上	Z 轴向前

坐标系转换矩阵（Panda3D → OpenGL）：

$$M_{P3D \rightarrow GL} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

即： $x_{GL} = x_{P3D}$, $y_{GL} = -z_{P3D}$, $z_{GL} = y_{P3D}$

Panda3D 在 GSG 层自动处理这个转换，用户无需手动处理。

2.5 变换复合的代数性质

变换复合（父变换 P ，子变换 C ）：

$$M_{\text{world}} = M_P \cdot M_C$$

均匀缩放时的快速复合（Panda3D 优化）：

当父变换有均匀缩放 s_P ，子变换有位置 $\mathbf{p}_C$ 、四元数 q_C 、缩放 s_C 时：

$$\begin{aligned} \mathbf{p}_{\text{world}} &= \mathbf{p}_P + q_P \cdot (\mathbf{p}_C \cdot s_P) \\ q &= q_C \cdot q_P \\ s_{\text{world}} &= s_C \cdot s_P \end{aligned}$$

这比完整的矩阵乘法（64次乘法+48次加法）快得多。

逆变换（用于视图矩阵）：

$$M^{-1}_{\text{TRS}} = S^{-1} \cdot R^{-1} \cdot T^{-1} = S^{-1} \cdot R^T \cdot T^{-1}$$

3. 工程实践

3.1 双重表示：组件式 vs 矩阵式

`TransformState` 同时支持两种表示方式：

```
创建方式 1：组件式（用户友好）
make_pos_hpr_scale(pos, hpr, scale)
→ 存储 _pos, _hpr/_quat, _scale
→ 矩阵按需计算（懒惰）

创建方式 2：矩阵式（精确控制）
make_mat(matrix)
→ 存储 _mat
→ 组件按需分解（懒惰）
```

优势： - 组件式：避免矩阵分解的精度损失，支持快速组件复合 - 矩阵式：支持任意仿射变换（包括剪切）

3.2 懒惰计算与标志位系统

`TransformState` 使用位标志（`_flags`）跟踪哪些表示已计算：

```
// transformState.h 中的标志位（简化）
enum Flags {
    F_is_identity      = 0x000001, // 是否为单位变换
    F_is_invalid       = 0x000002, // 是否无效（奇异矩阵）
    F_components_given = 0x000004, // 是否以组件方式创建
    F_components_known = 0x000008, // 组件是否已计算
    F_has_components   = 0x000010, // 是否有有效组件
    F_mat_known        = 0x000020, // 矩阵是否已计算
    F_hpr_given        = 0x000040, // 是否以 HPR 方式创建
    F_hpr_known        = 0x000080, // HPR 是否已计算
    F_quat_given       = 0x000100, // 是否以四元数方式创建
    F_quat_known       = 0x000200, // 四元数是否已计算
    F_singular_known   = 0x000400, // 是否已检测奇异性
    F_is_singular      = 0x000800, // 是否为奇异矩阵
    F_uniform_scale    = 0x01000,  // 是否均匀缩放
    F_identity_scale    = 0x02000,  // 缩放是否为 1
    F_is_2d            = 0x04000,  // 是否为 2D 变换
};
```

懒惰计算流程：

```

get_mat() 被调用
↓
检查 F_mat_known 标志
↓ 未设置
do_calc_mat()
↓
检查 F_hpr_known
↓ 未设置
do_calc_hpr() ← 从四元数转换
↓
compose_matrix(_mat, _scale, _shear, _hpr, _pos)
↓
设置 F_mat_known 标志

```

3.3 全局缓存（Flyweight 模式）

与 `RenderState` 相同，`TransformState` 也使用全局哈希表缓存所有实例：

```

// 全局缓存：相同变换只存一份
static States _states; // unordered_set<TransformState*>

static CPT(TransformState) return_new(TransformState *state) {
    // 在全局缓存中查找等价的已有状态
    auto si = _states.find(state);
    if (si != _states.end()) {
        delete state;
        return *si; // 返回已有实例
    }
    _states.insert(state);
    return state;
}

```

复合缓存：每个 `TransformState` 还维护一个局部复合缓存：

```

// 记录 this->compose(other) 的结果
typedef SimpleHashMap<const TransformState*, const TransformState*> CompositionCache;
CompositionCache _composition_cache;
CompositionCache _invert_composition_cache;

```

3.4 坐标系统一

Panda3D 在不同层次处理坐标系转换：

用户代码 (Panda3D 坐标系: Z-up, Y-forward)



NodePath.set_pos() / set_hpr()



TransformState (Panda3D 坐标系)



GSG.set_state_and_transform()



cs_transform (坐标系转换矩阵)



OpenGL/DirectX (各自的坐标系)

4. Panda3D 源码解析

4.1 TransformState 类结构

文件: `panda/src/pgraph/transformState.h`

```

// transformState.h:54
class EXPCL_PANDA_PGRAPH TransformState final
    : public NodeCachedReferenceCount {

    // — 工厂方法（静态，返回全局唯一实例） —————
    static CPT(TransformState) make_identity();
    static CPT(TransformState) make_pos(const LVecBase3 &pos);
    static CPT(TransformState) make_hpr(const LVecBase3 &hpr);
    static CPT(TransformState) make_quat(const LQuaternion &quat);
    static CPT(TransformState) make_pos_hpr_scale(
        const LVecBase3 &pos, const LVecBase3 &hpr, const LVecBase3 &scale);
    static CPT(TransformState) make_mat(const LMatrix4 &mat);

    // — 查询方法（懒惰计算） —————
    bool is_identity() const;
    bool has_components() const;    // 是否有有效的 TRS 分量
    bool has_uniform_scale() const;
    const LPoint3 &get_pos() const;
    const LVecBase3 &get_hpr() const;
    const LQuaternion &get_quat() const;
    const LVecBase3 &get_scale() const;
    const LMatrix4 &get_mat() const;
    const LMatrix4 *get_inverse_mat() const;    // 可能返回 nullptr（奇异矩阵）

    // — 变换操作（返回新实例，不修改自身） —————
    CPT(TransformState) compose(const TransformState *other) const;
    CPT(TransformState) invert_compose(const TransformState *other) const;
    CPT(TransformState) get_inverse() const;

    // — 修改方法（返回新实例） —————
    CPT(TransformState) set_pos(const LVecBase3 &pos) const;
    CPT(TransformState) set_hpr(const LVecBase3 &hpr) const;
    CPT(TransformState) set_scale(const LVecBase3 &scale) const;

private:
    // 内部存储（双重表示）
    mutable LPoint3 _pos;
    mutable LVecBase3 _hpr;
    mutable LQuaternion _quat, _norm_quat;
    mutable LVecBase3 _scale, _shear;
    mutable LMatrix4 _mat;
    mutable LMatrix4 *_inv_mat;    // 按需计算的逆矩阵

    mutable int _flags;    // 标志位，跟踪哪些表示已计算
    mutable LightMutex _lock;    // 线程安全锁

    // 复合缓存
    CompositionCache _composition_cache;
    CompositionCache _invert_composition_cache;
};

```

4.2 do_compose() — 变换复合

文件：panda/src/pgraph/transformState.cxx:1513

```

CPT(TransformState) TransformState::
do_compose(const TransformState *other) const {
    PStatTimer timer(_transform_compose_pcollector);

    // — 快速路径: 组件式复合 (均匀缩放, 无剪切) —————
    if (compose_componentwise &&
        has_uniform_scale() &&
        !has_nonzero_shear() && !other->has_nonzero_shear() &&
        ((components_given() && other->has_components()) ||
         (other->components_given() && has_components())) {

        // 3D 组件式复合
        LVecBase3 pos = get_pos();
        LQuaternion quat = get_norm_quat();
        PN_stdfloat scale = get_uniform_scale();

        // 关键公式: 子节点位置需要经过父节点的旋转和缩放变换
        // p_world = p_parent + q_parent.rotate(p_child * scale_parent)
        pos += quat.xform(other->get_pos()) * scale;

        // 旋转复合: q_world = q_child * q_parent (注意顺序)
        quat = other->get_norm_quat() * quat;

        // 缩放复合: s_world = s_child * s_parent
        LVecBase3 new_scale = other->get_scale() * scale;

        return make_pos_quat_scale(pos, quat, new_scale);
    }

    // — 慢速路径: 矩阵乘法 —————
    // new_mat = other->mat * this->mat
    // (Panda3D 使用行向量, 所以是右乘)
    LMatrix4 new_mat;
    new_mat.multiply(other->get_mat(), get_mat());
    return make_mat(new_mat);
}

```

关键设计: - 优先使用组件式复合 (避免矩阵分解的精度损失) - 条件: 均匀缩放 + 无剪切 + 至少一方以组件方式创建 - 退化到矩阵乘法处理一般情况

4.3 do_calc_mat() — 组件转矩阵

```
// transformState.cxx:2216
void TransformState::
do_calc_mat() {
    if ((_flags & F_mat_known) != 0) {
        return; // 已计算, 直接返回
    }

    // 确保 HPR 已知 (可能需要从四元数转换)
    if ((_flags & F_hpr_known) == 0) {
        do_calc_hpr();
    }

    // 调用数学库函数: TRS → 矩阵
    // compose_matrix(mat, scale, shear, hpr, pos)
    compose_matrix(_mat, _scale, _shear, get_hpr(), _pos);

    _flags |= F_mat_known;
}
```

`compose_matrix()` 的数学实现:

$$M = T(pos) * R(hpr) * S(scale) * Sh(shear)$$

其中 $R(hpr) = R_H * R_P * R_R$

R_H = 绕 Z 轴旋转 H 度

R_P = 绕 Y 轴旋转 P 度

R_R = 绕 X 轴旋转 R 度

4.4 do_calc_components() — 矩阵分解

```
// transformState.cxx:2111
void TransformState::
do_calc_components() {
    if ((_flags & F_components_known) != 0) {
        return; // 已计算
    }

    // 必须已有矩阵
    nassertv((_flags & F_mat_known) != 0);

    // 调用矩阵分解函数
    // decompose_matrix(mat, scale, shear, hpr, pos)
    bool possible = decompose_matrix(_mat, _scale, _shear, _hpr, _pos);

    if (!possible) {
        // 矩阵无法分解 (如零缩放)
        // 存储近似值, 但不设置 F_has_components
        _flags |= F_components_known | F_hpr_known;
    } else {
        // 分解成功
        _flags |= F_has_components | F_components_known | F_hpr_known;
        check_uniform_scale(); // 检查是否均匀缩放
    }
}
```

矩阵分解算法 (`decompose_matrix`) :

输入: 4x4 矩阵 M
输出: pos, scale, shear, hpr

1. 提取平移: `pos = M[3][0..2]`
2. 提取上3x3子矩阵 R' (包含旋转和缩放)
3. 提取缩放 (列向量的模):
`scale.x = |R'[col0]|`
`scale.y = |R'[col1]|`
`scale.z = |R'[col2]|`
4. 归一化得到纯旋转矩阵 R:
`R = R' * diag(1/scale.x, 1/scale.y, 1/scale.z)`
5. 提取剪切 (非对角元素)
6. 从旋转矩阵提取 HPR (欧拉角):
`P = asin(-R[2][0])`
`H = atan2(R[1][0], R[0][0])`
`R_angle = atan2(R[2][1], R[2][2])`

5. 代码演示

5.1 Python：基本变换操作

```
from panda3d.core import NodePath, TransformState, LPoint3, LVecBase3, LQuaternion
```

```
## — NodePath 高层接口
```

```
node = NodePath("my_node")
```

```
## 设置位置 (局部坐标)
```

```
node.set_pos(10, 20, 5)
```

```
node.set_x(10) # 只设置 X 分量
```

```
## 设置旋转 (HPR: Heading, Pitch, Roll, 单位: 度)
```

```
node.set_hpr(45, 0, 0) # 绕 Z 轴旋转 45 度
```

```
node.set_h(45) # 只设置 Heading
```

```
## 设置缩放
```

```
node.set_scale(2.0) # 均匀缩放
```

```
node.set_scale(2, 1, 0.5) # 非均匀缩放
```

```
## 组合设置
```

```
node.set_pos_hpr_scale(
```

```
    LPoint3(10, 20, 5),
```

```
    LVecBase3(45, 0, 0),
```

```
    LVecBase3(1, 1, 1)
```

```
)
```

```
## — 世界坐标 vs 局部坐标
```

```
parent = NodePath("parent")
```

```
parent.set_pos(100, 0, 0)
```

```
child = parent.attach_new_node("child")
```

```
child.set_pos(10, 0, 0) # 局部坐标
```

```
## 获取世界坐标
```

```
world_pos = child.get_pos(render) # (110, 0, 0)
```

```
print(f"World pos: {world_pos}")
```

```
## 在不同坐标系间转换
```

```
local_pos = child.get_pos(parent) # (10, 0, 0)
```

```
print(f"Local pos: {local_pos}")
```

```
## — 直接操作 TransformState
```

```
## 创建变换状态
```

```
ts1 = TransformState.make_pos(LPoint3(10, 0, 0))
```

```
ts2 = TransformState.make_hpr(LVecBase3(45, 0, 0))
```

```
ts3 = TransformState.make_scale(2.0)
```

```
## 复合变换 (先缩放, 再旋转, 再平移)
```

```
combined = ts1.compose(ts2.compose(ts3))
```

```
print(f"Combined matrix:\n{combined.get_mat()}")
```

```
## 获取逆变换
```

```
inverse = combined.get_inverse()
```

```
print(f"Is identity after compose with inverse: "
```

```
    f"{combined.compose(inverse).is_identity()}")
```

5.2 Python：四元数操作

```
from panda3d.core import LQuaternion, LVecBase3, LPoint3, TransformState
import math

## — 四元数基本操作 —————
## 绕 Z 轴旋转 90 度
q1 = LQuaternion()
q1.set_from_axis_angle(90, LVecBase3(0, 0, 1))

## 绕 X 轴旋转 45 度
q2 = LQuaternion()
q2.set_from_axis_angle(45, LVecBase3(1, 0, 0))

## 复合旋转 (先 q1, 再 q2)
q_combined = q2 * q1 # 注意: 右乘表示先应用 q1

## 旋转一个向量
v = LVecBase3(0, 1, 0) # Y 轴方向
v_rotated = q1.xform(v)
print(f"Rotated: {v_rotated}") # 应该接近 (-1, 0, 0)

## — SLERP 插值 (动画中常用) —————
q_start = LQuaternion()
q_start.set_from_axis_angle(0, LVecBase3(0, 0, 1))

q_end = LQuaternion()
q_end.set_from_axis_angle(180, LVecBase3(0, 0, 1))

## 在 0 到 1 之间插值
t = 0.5
q_mid = LQuaternion()
q_mid.slerp(q_start, q_end, t)
print(f"SLERP at t=0.5: {q_mid.get_hpr()}") # 应该接近 (90, 0, 0)

## — HPR 与四元数转换 —————
hpr = LVecBase3(45, 30, 0)
q = LQuaternion()
q.set_hpr(hpr)
print(f"HPR to Quat: {q}")

## 反向转换
hpr_back = q.get_hpr()
print(f"Quat to HPR: {hpr_back}") # 应该接近原始 HPR
```

5.3 Python：坐标系转换

```
from panda3d.core import NodePath, LPoint3, LVecBase3

## — 在不同节点坐标系间转换 —————
## 场景结构：
## world
##   └─ room (pos=100,0,0, hpr=45,0,0)
##       └─ table (pos=0,5,0)
##           └─ cup (pos=0,0,1)

world = render
room = world.attach_new_node("room")
room.set_pos(100, 0, 0)
room.set_h(45) # 旋转 45 度

table = room.attach_new_node("table")
table.set_pos(0, 5, 0)

cup = table.attach_new_node("cup")
cup.set_pos(0, 0, 1)

## 获取 cup 在不同坐标系中的位置
cup_world = cup.get_pos(world) # 世界坐标
cup_room = cup.get_pos(room) # 相对于 room 的坐标
cup_table = cup.get_pos(table) # 相对于 table 的坐标 (即局部坐标)

print(f"Cup in world: {cup_world}")
print(f"Cup in room: {cup_room}")
print(f"Cup in table: {cup_table}")

## — 将世界坐标转换为局部坐标 —————
world_point = LPoint3(110, 5, 1)
local_point = room.get_relative_point(world, world_point)
print(f"World {world_point} → Room local: {local_point}")

## — 对齐到另一个节点 —————
## 将 cup 的变换设置为与另一个节点相同
target = NodePath("target")
target.set_pos_hpr(50, 50, 0, 90, 0, 0)
cup.set_transform(target.get_transform(world))
```

5.4 Python：变换插值（动画过渡）

```
from panda3d.core import LPoint3, LVecBase3, LQuaternion, TransformState
from direct.interval.LerpInterval import LerpPosHprInterval

## — 使用 Interval 系统进行平滑过渡 —————
node = NodePath("animated_node")
node.reparent_to(render)

## 位置插值（线性）
pos_interval = node.posInterval(
    duration=2.0,
    pos=LPoint3(100, 0, 0),
    startPos=LPoint3(0, 0, 0)
)

## HPR 插值（线性，可能有万向锁问题）
hpr_interval = node.hprInterval(
    duration=2.0,
    hpr=LVecBase3(180, 0, 0),
    startHpr=LVecBase3(0, 0, 0)
)

## 四元数插值（SLERP, 更平滑）
quat_interval = node.quatInterval(
    duration=2.0,
    quat=LQuaternion(), # 目标四元数
    startQuat=LQuaternion() # 起始四元数
)

## — 手动 SLERP 插值 —————
def lerp_transform(ts_start: TransformState, ts_end: TransformState, t: float):
    """在两个变换状态之间插值"""
    # 位置线性插值
    pos = ts_start.get_pos() + (ts_end.get_pos() - ts_start.get_pos()) * t

    # 旋转 SLERP 插值
    q_start = ts_start.get_norm_quat()
    q_end = ts_end.get_norm_quat()
    q_interp = LQuaternion()
    q_interp.slerp(q_start, q_end, t)

    # 缩放线性插值
    scale = ts_start.get_scale() + (ts_end.get_scale() - ts_start.get_scale()) * t

    return TransformState.make_pos_quat_scale(pos, q_interp, scale)
```

5.5 C++：自定义变换操作

```
#include "transformState.h"
#include "nodePath.h"

// 计算两个节点之间的相对变换
CPT(TransformState) get_relative_transform(
    const NodePath &from, const NodePath &to) {

    // from 的世界变换
    CPT(TransformState) from_world = from.get_net_transform();
    // to 的世界变换
    CPT(TransformState) to_world = to.get_net_transform();

    // 相对变换 = from_world^(-1) * to_world
    // 即：先撤销 from 的变换，再应用 to 的变换
    return from_world->invert_compose(to_world);
}

// 在局部空间中旋转节点（绕世界轴旋转）
void rotate_around_world_axis(NodePath &node,
    const LVecBase3 &world_axis,
    float angle_deg) {

    // 将世界轴转换到节点的局部空间
    NodePath parent = node.get_parent();
    LVecBase3 local_axis = parent.get_relative_vector(render, world_axis);

    // 在局部空间中旋转
    LQuaternion q;
    q.set_from_axis_angle(angle_deg, local_axis);

    // 复合到当前旋转
    LQuaternion current_q = node.get_quat();
    node.set_quat(q * current_q);
}
```


6. 性能分析

6.1 变换操作代价对比

操作	代价	说明
<code>make_identity()</code>	$O(1)$	返回全局单例
<code>make_pos(pos)</code>	$O(1)$	哈希查找
<code>compose()</code> — 组件式	~20 ops	四元数乘法 + 向量运算
<code>compose()</code> — 矩阵式	~64 muls	4×4 矩阵乘法
<code>get_mat()</code> — 已缓存	$O(1)$	直接返回
<code>get_mat()</code> — 未缓存	~50 ops	<code>compose_matrix()</code>
<code>get_hpr()</code> — 未缓存	~100 ops	矩阵分解
<code>get_inverse_mat()</code>	~100 ops	高斯消元

6.2 常见性能陷阱

```
## ❌ 错误：频繁调用 get_hpr() (触发矩阵分解)
for i in range(1000):
    node.set_mat(some_matrix) # 矩阵方式设置
    hpr = node.get_hpr()      # 每次都触发矩阵分解!

## ✅ 正确：如果需要 HPR, 直接用 HPR 方式设置
for i in range(1000):
    node.set_hpr(some_hpr)    # 组件方式设置, HPR 直接可用
    hpr = node.get_hpr()      # 直接返回, 无需分解

## ❌ 错误：非均匀缩放 + 旋转 (强制走矩阵路径)
node.set_scale(2, 1, 0.5)    # 非均匀缩放
node.set_hpr(45, 0, 0)       # 旋转
## compose() 无法走快速路径, 必须用矩阵乘法

## ✅ 正确：尽量使用均匀缩放
node.set_scale(2.0)          # 均匀缩放
node.set_hpr(45, 0, 0)       # compose() 可以走快速路径

## ❌ 错误：每帧重新创建 TransformState
def update(dt):
    ts = TransformState.make_pos(LPoint3(x, y, z)) # 每帧创建新对象
    node.set_transform(ts)

## ✅ 正确：直接使用 NodePath 接口 (内部优化)
def update(dt):
    node.set_pos(x, y, z) # 内部高效处理
```

6.3 坐标系转换代价

```
## get_pos(other_node) 的内部实现:
## 1. 计算 self 的世界变换 (可能触发 update_cached)
## 2. 计算 other_node 的世界变换 (可能触发 update_cached)
## 3. 计算相对变换: world_self * inverse(world_other)
## 代价: O(depth_self + depth_other)

## 优化：缓存频繁使用的相对变换
class CachedRelativePos:
    def __init__(self, node, reference):
        self.node = node
        self.reference = reference
        self._cached_pos = None
        self._last_transform_seq = -1

    def get(self):
        # 检查变换是否改变 (通过序列号)
        current_seq = self.node.get_transform().get_hash()
        if current_seq != self._last_transform_seq:
            self._cached_pos = self.node.get_pos(self.reference)
            self._last_transform_seq = current_seq
        return self._cached_pos
```


小结

概念	核心思想	Panda3D 实现
齐次坐标	统一平移/旋转/缩放为矩阵乘法	<code>LMatrix4</code> (4x4)
TRS 分解	位置+旋转+缩放的直觉表示	<code>_pos</code> , <code>_quat</code> , <code>_scale</code>
四元数	无万向锁的旋转表示, 支持 SLERP	<code>LQuaternion</code>
双重表示	组件式和矩阵式互相转换	<code>_flags</code> 标志位 + 懒惰计算
全局缓存	相同变换只存一份	<code>_states</code> 哈希表
快速复合	均匀缩放时用四元数复合代替矩阵乘法	<code>do_compose()</code> 快速路径
坐标系	Z-up, Y-forward, GSG 层自动转换	<code>cs_transform</code>

核心设计哲学：

- 1. **不可变性：** `TransformState` 创建后不可修改，修改返回新实例
- 2. **懒惰计算：** 矩阵和组件按需互相转换，避免不必要的计算
- 3. **全局唯一：** 相同变换全局只有一份，节省内存和比较代价
- 4. **快速路径：** 均匀缩放的常见情况走组件式复合，避免矩阵分解

专题04：光照与阴影

目录

1. 问题背景
 2. 数学原理 - 2.1 光照模型：Phong/Blinn-Phong - 2.2 Shadow Map 原理 - 2.3 PCF 软阴影 - 2.4 级联阴影贴图 (CSM)
 3. 工程实践 - 3.1 光照类型与继承体系 - 3.2 LightAttrib 与光照绑定 - 3.3 阴影贴图渲染流程 - 3.4 ShaderGenerator 自动生成光照 Shader
 4. Panda3D 源码解析 - 4.1 Light 基类与 LightLensNode - 4.2 DirectionalLight 阴影设置 - 4.3 ShaderGenerator::ShaderKey 光照信息
 5. 代码演示
 6. 性能分析
-

1. 问题背景

光照与阴影是 3D 渲染中最复杂的问题之一：

1. **光照计算**：每个像素需要对所有光源求和，代价随光源数量线性增长
2. **阴影**：判断一个点是否被遮挡，需要从光源视角渲染整个场景
3. **软阴影**：真实阴影边缘是模糊的（半影区），需要多次采样
4. **全局光照**：光线在场景中多次反弹，实时计算极其昂贵

Panda3D 提供了从固定管线光照到自动 Shader 生成的完整解决方案。

2. 数学原理

2.1 光照模型：Phong/Blinn-Phong

Phong 光照模型将光照分为三个分量：

$$L = L_{\text{ambient}} + L_{\text{diffuse}} + L_{\text{specular}}$$

环境光 (Ambient) : $L_{\text{ambient}} = k_a \cdot I_a$

漫反射 (Diffuse) : 基于 Lambert 余弦定律 $L_{\text{diffuse}} = k_d \cdot I_d \cdot \max(\mathbf{n} \cdot \mathbf{l}, 0)$

其中： - $\mathbf{n}$: 表面法向量 (单位向量) - $\mathbf{l}$: 指向光源的方向向量 (单位向量) - k_d : 漫反射系数 (材质颜色) - I_d : 光源漫反射强度

镜面反射 (Specular) : - Phong 模型: $L_{\text{spec}} = k_s \cdot I_s \cdot \max(\mathbf{r} \cdot \mathbf{v}, 0)^n$ - $\mathbf{r} = 2(\mathbf{n} \cdot \mathbf{l})\mathbf{n} - \mathbf{l}$: 反射向量 - $\mathbf{v}$: 指向观察者的方向 - Blinn-Phong 模型 (更高效) : $L_{\text{spec}} = k_s \cdot I_s \cdot \max(\mathbf{n} \cdot \mathbf{h}, 0)^n$ - $\mathbf{h} = \frac{\mathbf{l} + \mathbf{v}}{|\mathbf{l} + \mathbf{v}|}$: 半程向量 (Half Vector)

点光源衰减 : $\text{attenuation} = \frac{1}{k_c + k_l \cdot d + k_q \cdot d^2}$

其中 d 是到光源的距离, k_c, k_l, k_q 分别是常数、线性、二次衰减系数。

聚光灯 (Spotlight) : $\text{spot_factor} = \max(\mathbf{l}_{\text{dir}} \cdot (-\mathbf{l}), 0)^e$

其中 $\mathbf{l}_{\text{dir}}$ 是聚光灯方向, e 是聚光指数 (控制光锥锐度)。

2.2 Shadow Map 原理

Shadow Map 是最常用的实时阴影技术：

第一步：从光源视角渲染深度图

将相机放在光源位置，渲染场景，只记录每个像素的深度值（距离光源的距离）。

$$\text{shadow_map}[u, v] = \min_{(x,y,z) \in (u,v)} \frac{z - z_{\text{near}}}{z_{\text{far}} - z_{\text{near}}}$$

第二步：在主渲染中判断遮挡

对于每个片段 P ，将其变换到光源的裁剪空间：

$$P_{\text{light}} = M_{\text{proj_light}} \cdot M_{\text{view_light}} \cdot P_{\text{world}}$$

归一化到 $[0,1]^3$ ：

$$\begin{aligned} u &= \frac{P_{\text{light}}.x}{P_{\text{light}}.w} \cdot 0.5 + 0.5 \\ v &= \frac{P_{\text{light}}.y}{P_{\text{light}}.w} \cdot 0.5 + 0.5 \\ z_{\text{current}} &= \frac{P_{\text{light}}.z}{P_{\text{light}}.w} \cdot 0.5 + 0.5 \end{aligned}$$

遮挡判断：

$$\text{in_shadow} = (z_{\text{current}} > \text{shadow_map}[u, v] + \text{bias})$$

深度偏移 (Depth Bias)：防止自遮挡 (Shadow Acne)：

$$\text{bias} = \max(0.05 \cdot (1 - \mathbf{n} \cdot \mathbf{l}), 0.005)$$

2.3 PCF 软阴影

PCF (Percentage Closer Filtering) 通过对 Shadow Map 周围多个采样点取平均，产生软阴影效果：

$$\text{shadow} = \frac{1}{N} \sum_{i=1}^N \text{compare}(z_{\text{current}}, \text{shadow_map}[u + \delta u_i, v + \delta v_i])$$

常用采样模式：
2x2 PCF：4 次采样，轻微软化
3x3 PCF：9 次采样，中等软化
Poisson Disk PCF：随机分布采样，更自然

PCSS (Percentage Closer Soft Shadows)：根据遮挡物距离动态调整 PCF 核大小，模拟真实半影。

2.4 级联阴影贴图 (CSM)

问题：单张 Shadow Map 无法同时覆盖近处 (需要高精度) 和远处 (需要大范围)。

CSM 解决方案：将视锥体分割为多个子视锥体，每个子视锥体使用独立的 Shadow Map：

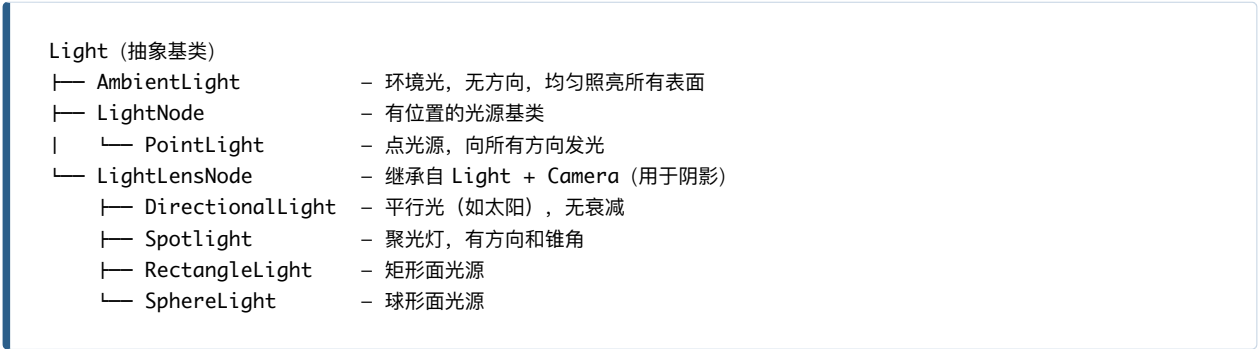
$$\text{split}_i = \lambda \cdot z_{\text{right}} \cdot \left(\frac{z_{\text{far}}}{z_{\text{near}}} \right)^{i/N} + (1 - \lambda) \cdot \left(z_{\text{near}} + \frac{i}{N} (z_{\text{far}} - z_{\text{near}}) \right)$$

其中 λ 控制对数分割和均匀分割的混合比例。

典型配置：
 - 级联 0：0-10m, Shadow Map 1024×1024
 - 级联 1：10-50m, Shadow Map 1024×1024
 - 级联 2：50-200m, Shadow Map 1024×1024
 - 级联 3：200-1000m, Shadow Map 512×512

3. 工程实践

3.1 光照类型与继承体系



LightLensNode 的双重身份： - 作为 **Light**：提供光照参数（颜色、强度、衰减） - 作为 **Camera**：从光源视角渲染 Shadow Map

3.2 LightAttrib 与光照绑定

光照通过 **LightAttrib** 附加到节点，影响该节点及其子树：



3.3 阴影贴图渲染流程

1. 调用 `light.set_shadow_caster(True, 1024, 1024)`
↓
2. `LightLensNode` 创建离屏缓冲区 (`GraphicsOutput`)
↓
3. 每帧, `GraphicsEngine` 先渲染阴影缓冲区:
 - 使用光源的 `Camera` (`LightLensNode` 继承自 `Camera`)
 - 只渲染深度, 不渲染颜色↓
4. 阴影贴图 (`Texture`) 绑定到主渲染的 `Shader`
↓
5. 主渲染中, `Shader` 采样阴影贴图判断遮挡

3.4 ShaderGenerator 自动生成光照 Shader

Panda3D 的 `ShaderGenerator` 根据 `RenderState` 自动生成 GLSL/Cg Shader :

```
render.set_shader_auto()
↓
ShaderGenerator.synthesize_shader(render_state, anim_spec)
↓
analyze_renderstate() 分析:
- 有哪些光源类型 (Directional/Point/Spot)
- 是否有阴影 (LF_has_shadows)
- 是否有法线贴图 (TF_map_normal)
- 是否有高光贴图 (TF_map_gloss)
↓
根据 ShaderKey 生成对应的 GLSL 代码
↓
缓存到 _generated_shaders (相同 Key 复用)
```

4. Panda3D 源码解析

4.1 Light 基类与 LightLensNode

文件: panda/src/pgraph/light.h

```
// light.h:38
class EXPCL_PANDA_PGRAPH Light {
PUBLISHED:
    // 光源颜色 (RGBA)
    INLINE const LColor &get_color() const;
    INLINE void set_color(const LColor &color);

    // 色温 (开尔文), 自动转换为 RGB 颜色
    INLINE bool has_color_temperature() const;
    void set_color_temperature(PN_stdfloat temperature);

    // 镜面反射颜色 (可与漫反射颜色不同)
    virtual const LColor &get_specular_color() const;

    // 衰减系数 (常数、线性、二次)
    virtual const LVecBase3 &get_attenuation() const;

    // 优先级 (当光源数量超过 GPU 限制时, 低优先级光源被丢弃)
    INLINE void set_priority(int priority);
    INLINE int get_priority() const;

    // 光源类型优先级 (内部使用)
    enum ClassPriority {
        CP_ambient_priority,      // 最低
        CP_point_priority,
        CP_directional_priority,
        CP_spot_priority,
        CP_area_priority,         // 最高
    };

public:
    // 将光源参数绑定到 GSG (纯虚函数, 各子类实现)
    virtual void bind(GraphicsStateGuardianBase *gsg,
                     const NodePath &light, int light_id) = 0;
};
```

文件: panda/src/pgraphnodes/lightLensNode.h

```

// lightLensNode.h:33
// LightLensNode 同时继承 Light 和 Camera
// Camera 部分用于从光源视角渲染 Shadow Map
class EXPCL_PANDA_PGRAPHNODES LightLensNode : public Light, public Camera {
PUBLISHED:
    // 启用/禁用阴影投射
    INLINE bool is_shadow_caster() const;
    void set_shadow_caster(bool caster);
    void set_shadow_caster(bool caster, int buffer_xsize, int buffer_ysize,
                           int sort = -10);

    // Shadow Map 缓冲区大小
    INLINE LVecBase2i get_shadow_buffer_size() const;
    INLINE void set_shadow_buffer_size(const LVecBase2i &size);

    // 获取对应 GSG 的 Shadow Map 缓冲区
    INLINE GraphicsOutputBase *get_shadow_buffer(GraphicsStateGuardianBase *gsg);

protected:
    LVecBase2i _sb_size;          // Shadow Map 分辨率
    bool _shadow_caster;         // 是否投射阴影
    PT(Texture) _shadow_map;     // 阴影贴图纹理

    // 每个 GSG 对应一个离屏缓冲区（支持多窗口）
    typedef pmap<PT(GraphicsStateGuardianBase),
                PT(GraphicsOutputBase)> ShadowBuffers;
    ShadowBuffers _sbuffers;
};

```

4.2 DirectionalLight 阴影设置

文件：panda/src/pgraphnodes/directionalLight.h

```

// directionalLight.h:25
class EXPCL_PANDA_PGRAPHNODES DirectionalLight : public LightLensNode {
PUBLISHED:
    // 镜面反射颜色（可独立于漫反射颜色）
    INLINE const LColor &get_specular_color() const final;
    INLINE void set_specular_color(const LColor &color);

    // 光源方向（在光源局部坐标系中）
    INLINE const LVector3 &get_direction() const;
    INLINE void set_direction(const LVector3 &direction);

public:
    // 将光源方向向量变换到物体局部空间
    virtual bool get_vector_to_light(LVector3 &result,
                                     const LPoint3 &from_object_point,
                                     const LMatrix4 &to_object_space);

    // 绑定到 GSG（设置 OpenGL 光源参数）
    virtual void bind(GraphicsStateGuardianBase *gsg,
                     const NodePath &light, int light_id);
};

```

4.3 ShaderGenerator::ShaderKey 光照信息

文件: panda/src/pgraphnodes/shaderGenerator.h

```
// shaderGenerator.h:96
struct ShaderKey {
    // 光照标志
    enum LightFlags {
        LF_has_shadows = 1,          // 该光源投射阴影
        LF_has_specular_color = 2,    // 该光源有独立镜面反射颜色
    };

    // 每个光源的信息
    struct LightInfo {
        TypeHandle _type;    // 光源类型 (DirectionalLight, PointLight, etc.)
        int _flags;          // LightFlags 组合
    };
    pvector<LightInfo> _lights; // 所有激活的光源列表

    bool _lighting;          // 是否启用光照
    bool _have_separate_ambient; // 是否有独立的环境光

    // 纹理贴图标志
    enum TextureFlags {
        TF_map_normal    = 0x020, // 法线贴图
        TF_map_height     = 0x040, // 高度贴图 (视差映射)
        TF_map_glow       = 0x080, // 自发光贴图
        TF_map_gloss      = 0x100, // 高光贴图
        TF_map_emission   = 0x001000000, // 自发光贴图 (PBR)
        TF_map_occlusion   = 0x002000000, // 遮蔽贴图 (AO)
    };
};

// synthesize_shader() 根据 ShaderKey 生成对应的 GLSL 代码
// 相同 ShaderKey 的 Shader 会被缓存复用
typedef phash_map<ShaderKey, CPT(ShaderAttrib)> GeneratedShaders;
GeneratedShaders _generated_shaders;
```

5. 代码演示

5.1 Python：基本光照设置

```

from panda3d.core import (
    AmbientLight, DirectionalLight, PointLight, Spotlight,
    LColor, LVector3, LPoint3, NodePath
)

## — 环境光 —————
ambient = AmbientLight("ambient")
ambient.set_color(LColor(0.2, 0.2, 0.2, 1)) # 低强度白色环境光
ambient_np = render.attach_new_node(ambient)
render.set_light(ambient_np)

## — 平行光（太阳光） —————
sun = DirectionalLight("sun")
sun.set_color(LColor(1.0, 0.95, 0.8, 1)) # 暖白色
sun.set_direction(LVector3(-1, -1, -2)) # 从右上方照射
sun_np = render.attach_new_node(sun)
render.set_light(sun_np)

## — 点光源 —————
point = PointLight("point_light")
point.set_color(LColor(1, 0.5, 0, 1)) # 橙色
## 设置衰减: (常数, 线性, 二次)
point.set_attenuation(LVector3(1, 0.1, 0.01))
point_np = render.attach_new_node(point)
point_np.set_pos(0, 0, 5) # 放置在场景中
render.set_light(point_np)

## — 聚光灯 —————
spot = Spotlight("spotlight")
spot.set_color(LColor(1, 1, 1, 1))
spot.set_exponent(40) # 聚光指数 (越大越集中)
## 设置聚光角度
from panda3d.core import PerspectiveLens
lens = PerspectiveLens()
lens.set_fov(30) # 30 度视角
spot.set_lens(lens)
spot_np = render.attach_new_node(spot)
spot_np.set_pos(0, -10, 10)
spot_np.look_at(0, 0, 0) # 指向原点
render.set_light(spot_np)

## — 局部光照（只影响特定节点） —————
## 只让 room 节点受到 room_light 的影响
room = render.attach_new_node("room")
room_light = PointLight("room_light")
room_light_np = room.attach_new_node(room_light)
room.set_light(room_light_np) # 只影响 room 及其子节点

## 关闭某个节点的光照
some_node.set_light_off() # 关闭所有光照
some_node.set_light_off(sun_np) # 只关闭太阳光

```


5.2 Python：阴影设置

```
from panda3d.core import DirectionalLight, LColor, LVector3
```

```
## — 启用阴影 —————
```

```
sun = DirectionalLight("sun")
sun.set_color(LColor(1, 1, 1, 1))
```

```
## 启用阴影投射，设置 Shadow Map 分辨率
sun.set_shadow_caster(True, 2048, 2048)
```

```
## 调整阴影相机的视锥体（覆盖整个场景）
sun.get_lens().set_near_far(1, 500)
sun.get_lens().set_film_size(200, 200) # 正交投影范围
```

```
sun_np = render.attach_new_node(sun)
sun_np.set_hpr(45, -45, 0) # 设置光源方向
render.set_light(sun_np)
```

```
## 启用自动 Shader（包含阴影计算）
render.set_shader_auto()
```

```
## — 调整阴影质量 —————
```

```
## 更大的 Shadow Map = 更清晰的阴影（但更耗内存）
sun.set_shadow_caster(True, 4096, 4096)
```

```
## 调整深度偏移（防止 Shadow Acne）
## 通过 Shader 输入设置
render.set_shader_input("shadow_bias", 0.005)
```

```
## — 聚光灯阴影 —————
```

```
spot = Spotlight("spot")
spot.set_shadow_caster(True, 1024, 1024)
spot_np = render.attach_new_node(spot)
spot_np.set_pos(0, -10, 10)
spot_np.look_at(0, 0, 0)
render.set_light(spot_np)
render.set_shader_auto()
```

5.3 Python：自定义光照 Shader

```

from panda3d.core import Shader, NodePath

## — 自定义 Blinn-Phong 光照 Shader
vert_shader = """
#version 330

uniform mat4 p3d_ModelViewProjectionMatrix;
uniform mat4 p3d_ModelMatrix;
uniform mat3 p3d_NormalMatrix;

in vec4 p3d_Vertex;
in vec3 p3d_Normal;
in vec2 p3d_MultiTexCoord0;

out vec3 v_world_pos;
out vec3 v_normal;
out vec2 v_texcoord;

void main() {
    gl_Position = p3d_ModelViewProjectionMatrix * p3d_Vertex;
    v_world_pos = (p3d_ModelMatrix * p3d_Vertex).xyz;
    v_normal = normalize(p3d_NormalMatrix * p3d_Normal);
    v_texcoord = p3d_MultiTexCoord0;
}
"""

frag_shader = """
#version 330

// Panda3D 自动提供的光照 Uniform
struct p3d_LightSourceParameters {
    vec4 color;
    vec4 position;          // w=0 表示平行光, w=1 表示点光源
    vec3 attenuation;      // (常数, 线性, 二次)
    vec3 spotDirection;
    float spotExponent;
    float spotCosCutoff;
};

uniform p3d_LightSourceParameters p3d_LightSource[4];
uniform vec4 p3d_LightModel.ambient;

uniform sampler2D p3d_Texture0;
uniform vec3 camera_pos;

in vec3 v_world_pos;
in vec3 v_normal;
in vec2 v_texcoord;

out vec4 frag_color;

void main() {
    vec3 N = normalize(v_normal);
    vec3 V = normalize(camera_pos - v_world_pos);
    vec4 base_color = texture(p3d_Texture0, v_texcoord);

    // 环境光
    vec3 ambient = p3d_LightModel.ambient.rgb * base_color.rgb;
    vec3 diffuse = vec3(0.0);
    vec3 specular = vec3(0.0);

```

```

for (int i = 0; i < 4; i++) {
    vec3 L;
    float attenuation = 1.0;

    if (p3d_LightSource[i].position.w == 0.0) {
        // 平行光
        L = normalize(-p3d_LightSource[i].position.xyz);
    } else {
        // 点光源
        vec3 light_vec = p3d_LightSource[i].position.xyz - v_world_pos;
        float dist = length(light_vec);
        L = normalize(light_vec);
        vec3 att = p3d_LightSource[i].attenuation;
        attenuation = 1.0 / (att.x + att.y * dist + att.z * dist * dist);
    }

    // Blinn-Phong 漫反射
    float NdotL = max(dot(N, L), 0.0);
    diffuse += p3d_LightSource[i].color.rgb * base_color.rgb
               * NdotL * attenuation;

    // Blinn-Phong 镜面反射
    if (NdotL > 0.0) {
        vec3 H = normalize(L + V); // 半程向量
        float NdotH = max(dot(N, H), 0.0);
        specular += p3d_LightSource[i].color.rgb
                   * pow(NdotH, 64.0) * attenuation;
    }
}

frag_color = vec4(ambient + diffuse + specular * 0.5, base_color.a);
}
"""

shader = Shader.make(Shader.SL_GLSL, vert_shader, frag_shader)
render.set_shader(shader)
render.set_shader_input("camera_pos", base.camera.get_pos(render))

```

5.4 Python : PCF 软阴影 Shader

```

## PCF 软阴影片段着色器 (关键部分)
pcf_shadow_frag = """
#version 330

uniform sampler2DShadow p3d_Texture1; // 阴影贴图 (深度比较纹理)
uniform mat4 shadow_mvp;             // 光源的 MVP 矩阵

in vec3 v_world_pos;
in vec3 v_normal;

out vec4 frag_color;

// PCF 采样偏移 (3x3 核)
const vec2 pcf_offsets[9] = vec2[](
    vec2(-1,-1), vec2(0,-1), vec2(1,-1),
    vec2(-1, 0), vec2(0, 0), vec2(1, 0),
    vec2(-1, 1), vec2(0, 1), vec2(1, 1)
);

float compute_shadow(vec3 world_pos, vec3 normal, vec3 light_dir) {
    // 将世界坐标变换到光源裁剪空间
    vec4 light_space_pos = shadow_mvp * vec4(world_pos, 1.0);
    vec3 proj_coords = light_space_pos.xyz / light_space_pos.w;
    proj_coords = proj_coords * 0.5 + 0.5; // 变换到 [0,1]

    // 超出阴影贴图范围的区域不在阴影中
    if (proj_coords.z > 1.0) return 1.0;

    // 自适应深度偏移 (根据法线与光线夹角)
    float bias = max(0.005 * (1.0 - dot(normal, light_dir)), 0.001);

    // PCF: 对 3x3 邻域采样取平均
    float shadow = 0.0;
    vec2 texel_size = 1.0 / textureSize(p3d_Texture1, 0);
    for (int i = 0; i < 9; i++) {
        vec3 sample_coord = vec3(
            proj_coords.xy + pcf_offsets[i] * texel_size,
            proj_coords.z - bias
        );
        // sampler2DShadow 自动做深度比较, 返回 0.0 或 1.0
        shadow += texture(p3d_Texture1, sample_coord);
    }
    return shadow / 9.0; // 平均值即为光照比例
}

void main() {
    vec3 N = normalize(v_normal);
    vec3 L = normalize(vec3(1, 1, 2)); // 平行光方向

    float shadow_factor = compute_shadow(v_world_pos, N, L);
    float NdotL = max(dot(N, L), 0.0);

    vec3 color = vec3(0.8) * NdotL * shadow_factor + vec3(0.1);
    frag_color = vec4(color, 1.0);
}
"""

## Python 端设置
shader = Shader.make(Shader.SL_GLSL, vert_shader, pcf_shadow_frag)

```

```
render.set_shader(shader)
render.set_shader_input("shadow_mv", sun_np.get_mat(render))
```

6. 性能分析

6.1 光照计算代价

光源类型	每像素计算量	阴影代价
AmbientLight	O(1)	无
DirectionalLight	~10 ops	1次纹理采样（硬阴影） / 9次（PCF）
PointLight	~15 ops（含衰减）	6面 Cube Shadow Map
Spotlight	~20 ops（含聚光）	1次纹理采样

6.2 Shadow Map 分辨率 vs 质量

分辨率	内存	阴影质量	适用场景
512×512	1 MB	低（明显锯齿）	远处阴影、移动端
1024×1024	4 MB	中	一般场景
2048×2048	16 MB	高	主要光源
4096×4096	64 MB	极高	离线渲染

6.3 常见性能陷阱

```
## ❌ 错误：过多动态光源
for i in range(20):
    light = PointLight(f"light_{i}")
    render.set_light(render.attach_new_node(light))
## 问题：每个光源都需要在 Shader 中循环计算，GPU 代价线性增长

## ✅ 正确：限制动态光源数量，使用光照贴图烘焙静态光照
## 动态光源：最多 4-8 个
## 静态光照：烘焙到 Lightmap 纹理

## ❌ 错误：所有光源都启用阴影
for light in all_lights:
    light.set_shadow_caster(True, 2048, 2048)
## 问题：每个阴影光源需要额外渲染一遍场景

## ✅ 正确：只有主要光源投射阴影
sun.set_shadow_caster(True, 2048, 2048) # 主光源：高质量阴影
fill.set_shadow_caster(False)          # 补光：无阴影

## ❌ 错误：Shadow Map 分辨率过高
sun.set_shadow_caster(True, 8192, 8192) # 256 MB！

## ✅ 正确：根据场景大小选择合适分辨率
## 小场景 (<100m)：1024×1024
## 中等场景 (<500m)：2048×2048
## 大场景：使用 CSM (级联阴影)

## — 优化：使用 set_shader_auto() 而非手写 Shader —————
## Panda3D 的 ShaderGenerator 会根据实际使用的光源类型生成最优 Shader
## 避免在 Shader 中循环处理未使用的光源类型
render.set_shader_auto()
```

6.4 阴影质量优化技巧

```
## 1. 调整阴影相机视锥体以紧密包围场景
sun_lens = sun.get_lens()
sun_lens.set_near_far(1, 200) # 只覆盖需要的深度范围
sun_lens.set_film_size(100, 100) # 只覆盖需要的空间范围

## 2. 使用 PSSM (Parallel Split Shadow Maps)
## Panda3D 内置支持通过 CommonFilters
from direct.filter.CommonFilters import CommonFilters
filters = CommonFilters(base.win, base.cam)
## 注意：Panda3D 的 CommonFilters 提供基础后处理，
## 完整 PSSM 需要自定义实现

## 3. 阴影偏移调整 (防止 Shadow Acne 和 Peter Panning)
## Shadow Acne：偏移太小，表面自遮挡
## Peter Panning：偏移太大，阴影与物体分离
## 最佳实践：使用法线偏移 (Normal Offset Bias)
render.set_shader_input("normal_bias", 0.01)
render.set_shader_input("depth_bias", 0.001)
```


小结

概念	核心思想	Panda3D 实现
Phong 光照	环境+漫反射+镜面反射	ShaderGenerator 自动生成
Shadow Map	从光源视角渲染深度图	set_shadow_caster()
PCF 软阴影	多次采样取平均	sampler2DShadow + 循环
LightLensNode	光源兼相机，用于阴影渲染	继承 Light + Camera
ShaderGenerator	根据 RenderState 自动生成 Shader	set_shader_auto()
LightAttrib	光照绑定到场景图节点	node.set_light(light_np)

核心设计哲学： 1. 场景图集成：光源是场景图节点，自动继承变换 2. 自动 Shader： set_shader_auto() 根据实际光照配置生成最优 Shader 3. 双重身份： LightLensNode 同时是光源和相机，优雅地实现阴影渲染 4. 优先级系统：当光源超过 GPU 限制时，按优先级丢弃低优先级光源

专题05：透明度与混合排序

目录

- 1. 问题背景
 - 2. 数学原理 - 2.1 Alpha 混合方程 - 2.2 Porter-Duff 合成算子 - 2.3 排序问题的数学本质 - 2.4 OIT（顺序无关透明）算法
 - 3. 工程实践 - 3.1 透明度模式选择 - 3.2 深度写入与深度测试 - 3.3 M_dual 双通道渲染 - 3.4 粒子系统的透明处理
 - 4. Panda3D 源码解析 - 4.1 TransparencyAttrib 模式 - 4.2 CullResult::add_object() 透明处理 - 4.3 ColorBlendAttrib 混合方程
 - 5. 代码演示
 - 6. 性能分析
-

1. 问题背景

透明度渲染是 3D 图形中最棘手的问题之一：

核心矛盾：Alpha 混合是顺序相关的——先画 A 再画 B，与先画 B 再画 A，结果不同。

正确顺序（从后到前）：

背景 → 远处透明物体 → 近处透明物体

错误顺序（从前到后）：

近处透明物体 → 远处透明物体 → 背景
→ 远处物体会错误地覆盖近处物体

深度缓冲的局限：深度缓冲只能处理不透明物体（写入最近深度），对透明物体无效——透明物体需要“透过”，但深度缓冲会阻止后面的物体被绘制。

2. 数学原理

2.1 Alpha 混合方程

GPU 的 Alpha 混合方程 (OpenGL `glBlendFunc`) :

$$C_{out} = C_{src} \cdot F_{src} + C_{dst} \cdot F_{dst}$$

其中: - C_{src} : 片段着色器输出的颜色 (源颜色) - C_{dst} : 帧缓冲中已有的颜色 (目标颜色) - F_{src} : 源混合因子 - F_{dst} : 目标混合因子

标准 Alpha 混合 (`GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA`) :

$$C_{out} = C_{src} \cdot \alpha_{src} + C_{dst} \cdot (1 - \alpha_{src})$$

加法混合 (粒子、光晕效果) (`GL_SRC_ALPHA, GL_ONE`) :

$$C_{out} = C_{src} \cdot \alpha_{src} + C_{dst}$$

预乘 Alpha 混合 (`GL_ONE, GL_ONE_MINUS_SRC_ALPHA`) :

$$C_{out} = C_{src} + C_{dst} \cdot (1 - \alpha_{src})$$

其中 C_{src} 已经预乘了 Alpha : $C_{src} = C_{original} \cdot \alpha_{src}$

2.2 Porter-Duff 合成算子

Porter-Duff (1984) 定义了 12 种图像合成算子, 最常用的是 **Over**:

$$C_{over} = C_A + C_B \cdot (1 - \alpha_A) \quad \alpha_{over} = \alpha_A + \alpha_B \cdot (1 - \alpha_A)$$

Over 算子的性质: - 不满足交换律: $A \text{ over } B \neq B \text{ over } A$ - 满足结合律: $(A \text{ over } B) \text{ over } C = A \text{ over } (B \text{ over } C)$

这就是为什么透明物体必须从后到前排序——Over 算子要求先处理背景 (B), 再处理前景 (A)。

2.3 排序问题的数学本质

正确的 **N 层透明合成** (从后到前) :

$$C_{final} = C_1 \cdot \alpha_1 + C_2 \cdot \alpha_2 \cdot (1 - \alpha_1) + C_3 \cdot \alpha_3 \cdot (1 - \alpha_1)(1 - \alpha_2) + \dots$$

排序的困难: 1. **循环遮挡**: 三个三角形 A、B、C 可能形成 A 遮挡 B、B 遮挡 C、C 遮挡 A 的循环 2. **相交几何体**: 两个透明物体相交时, 无法用单一排序解决 3. **动态场景**: 每帧都需要重新排序, 代价 $O(N \log N)$

Painter's Algorithm（画家算法）：从后到前绘制，后绘制的覆盖先绘制的。 - 优点：简单，与 Alpha 混合天然兼容 - 缺点：无法处理循环遮挡和相交几何体

2.4 OIT（顺序无关透明）算法

Depth Peeling（深度剥离）： - 第 1 遍：渲染最近的透明层 - 第 2 遍：渲染第 2 近的透明层（深度 > 第 1 遍的深度） - 第 N 遍：渲染第 N 近的透明层 - 最后：从后到前合成所有层

代价： $O(N)$ 次渲染遍， N 为透明层数。

Weighted Blended OIT（McGuire & Bavoil, 2013）： - 不排序，直接累积加权颜色和权重 - 权重函数： $w(z, \alpha) = \alpha \cdot \max(0.01, \min(3000, \frac{0.03}{0.00001 + (z/200)^4}))$ - 最终颜色： $C_{\text{final}} = \frac{\sum C_i \cdot w_i}{\sum w_i}$ - 优点：单遍渲染， $O(1)$ 代价 - 缺点：近似结果，不完全正确

3. 工程实践

3.1 透明度模式选择

模式	适用场景	优点	缺点
M_none	不透明物体	最快	无透明效果
M_alpha	一般透明（玻璃、水）	正确混合	需要排序
M_binary	植被、栅栏（硬边缘）	无需排序，可写深度	边缘锯齿
M_multisample	植被（MSAA 软化边缘）	边缘平滑	需要 MSAA
M_dual	复杂透明物体	减少排序问题	两次绘制
M_premultiplied_alpha	预乘 Alpha 纹理	更好的混合质量	纹理需预处理

3.2 深度写入与深度测试

透明物体的深度处理策略：

不透明物体： 深度测试：开启（只绘制最近的） 深度写入：开启（记录深度） 透明物体（M_alpha）： 深度测试：开启（透明物体仍然被不透明物体遮挡） 深度写入：关闭（不阻止后面的透明物体被绘制） 二值透明（M_binary）： 深度测试：开启 深度写入：开启（Alpha >= 0.5 的部分视为不透明）

渲染顺序： 1. 先渲染所有不透明物体（写入深度缓冲） 2. 再渲染透明物体（从后到前，不写深度）

3.3 M_dual 双通道渲染

M_dual 是 Panda3D 的特殊模式，将一个物体分成两部分渲染：

第一通道（不透明部分）：

Alpha 测试: $\geq$ dual_opaque_level (默认 0.999)

深度写入: 开启

混合: 关闭

→ 渲染完全不透明的部分

第二通道（透明部分）：

Alpha 测试: > 0 (过滤完全透明的像素)

深度写入: 关闭

混合: 开启 (标准 Alpha 混合)

→ 渲染半透明部分 (放入 back_to_front bin)

优势：对于大部分不透明、小部分半透明的物体（如带透明边缘的树叶），可以正确处理不透明部分的深度，减少排序错误。

3.4 粒子系统的透明处理

粒子系统通常使用**加法混合**（Additive Blending）：

加法混合: $C_{out} = C_{src} * \alpha + C_{dst}$

效果：粒子颜色叠加，越多粒子越亮

适用：火焰、爆炸、光晕、魔法效果

优点：无需排序（加法满足交换律）

加法混合的数学性质：

$$C_{out} = C_1 \cdot \alpha_1 + C_2 \cdot \alpha_2 + C_{bg}$$

由于加法满足交换律，粒子绘制顺序不影响最终结果！

4. Panda3D 源码解析

4.1 TransparencyAttrib 模式

文件: `panda/src/pgraph/transparencyAttrib.h`

```
// transparencyAttrib.h:33
class EXPCL_PANDA_PGRAPH TransparencyAttrib final : public RenderAttrib {
PUBLISHED:
    enum Mode {
        M_none = 0,           // 无透明, 最快
        M_alpha = 1,          // 标准 Alpha 混合, Panda3D 自动排序 (back_to_front bin)
        M_premultiplied_alpha, // 预乘 Alpha 混合 (纹理已预乘)
        M_multisample,         // MSAA 多重采样透明 (需要 MSAA 缓冲区)
        M_multisample_mask,    // MSAA 透明, 不修改 Alpha 值
        M_binary,              // 二值透明: Alpha >= 0.5 不透明, < 0.5 完全透明
        M_dual,                // 双通道: 不透明部分 + 透明部分分开渲染
    };
};
```

4.2 CullResult::add_object() 透明处理

文件: `panda/src/pgraph/cullResult.cxx:107`

```

void CullResult::
add_object(CullableObject &&object, const CullTraverser *traverser) {
    // ...

    // 检查透明度设置
    const TransparencyAttrib *trans;
    if (object._state->get_attrib(trans)) {
        switch (trans->get_mode()) {

            case TransparencyAttrib::M_alpha:
            case TransparencyAttrib::M_premultiplied_alpha:
                // M_alpha: 添加 Alpha 测试 (过滤 alpha=0 的像素, 节省填充率)
                // 同时, 对象会被放入 back_to_front bin (自动排序)
                object._state = object._state->compose(get_alpha_state());
                break;

            case TransparencyAttrib::M_binary:
                // M_binary: 设置 Alpha 测试为 >= 0.5
                // 不需要排序, 可以写深度缓冲
                object._state = object._state->compose(get_binary_state());
                break;

            case TransparencyAttrib::M_dual:
                if (!m_dual) {
                    // 配置关闭时, 退化为 M_alpha
                    break;
                }
                {
                    const CullBinAttrib *bin_attrib;
                    if (!object._state->get_attrib(bin_attrib) ||
                        bin_attrib->get_bin_name().empty()) {

                        // — 透明部分: 复制对象, 放入 back_to_front bin —————
                        CullableObject transparent_part(object);
                        CPT(RenderState) transparent_state = get_dual_transparent_state();
                        // transparent_state 包含:
                        //   - AlphaTestAttrib: > 0 (过滤完全透明像素)
                        //   - TransparencyAttrib: M_alpha (开启混合)
                        //   - DepthWriteAttrib: M_off (不写深度)
                        transparent_part._state =
                            object._state->compose(transparent_state);

                        // 放入 back_to_front bin (会按深度排序)
                        int transparent_bin_index =
                            transparent_part._state->get_bin_index();
                        CullBin *bin = get_bin(transparent_bin_index);
                        bin->add_object(alloc_object(std::move(transparent_part)),
                                      current_thread);

                        // — 不透明部分: 原对象, 放入默认 bin —————
                        // dual_opaque_state 包含:
                        //   - AlphaTestAttrib: >= dual_opaque_level (默认 0.999)
                        //   - TransparencyAttrib: M_none (关闭混合)
                        //   - DepthWriteAttrib: M_on (写深度)
                        object._state = object._state->compose(get_dual_opaque_state());
                    }
                    // 如果已有显式 bin, M_dual 退化为 M_alpha
                }
                break;
        }
    }
}

```

```

}

// 根据 RenderState 中的 CullBinAttrib 决定放入哪个 bin
int bin_index = object._state->get_bin_index();
CullBin *bin = get_bin(bin_index);
bin->add_object(alloc_object(std::move(object)), current_thread);
}

```

关键辅助函数：

```

// cullResult.cxx:427
// get_alpha_state(): 为 M_alpha 添加 Alpha 测试 (过滤 alpha=0 像素)
CPT(RenderState) CullResult::get_alpha_state() {
    static CPT(RenderState) state = nullptr;
    if (state == nullptr) {
        state = RenderState::make(
            AlphaTestAttrib::make(AlphaTestAttrib::M_greater, 0.0f),
            // 注意: 不关闭深度写入! 这由 back_to_front bin 的排序保证正确性
            RenderState::get_max_priority()
        );
    }
    return state;
}

// get_binary_state(): M_binary 的 Alpha 测试
CPT(RenderState) CullResult::get_binary_state() {
    static CPT(RenderState) state = nullptr;
    if (state == nullptr) {
        state = RenderState::make(
            AlphaTestAttrib::make(AlphaTestAttrib::M_greater_equal, 0.5f),
            TransparencyAttrib::make(TransparencyAttrib::M_none), // 关闭混合
            RenderState::get_max_priority()
        );
    }
    return state;
}

// get_dual_transparent_state(): M_dual 的透明部分
CPT(RenderState) CullResult::get_dual_transparent_state() {
    static CPT(RenderState) state = nullptr;
    if (state == nullptr) {
        state = RenderState::make(
            AlphaTestAttrib::make(AlphaTestAttrib::M_greater, 0.0f),
            TransparencyAttrib::make(TransparencyAttrib::M_alpha),
            DepthWriteAttrib::make(DepthWriteAttrib::M_off),
            RenderState::get_max_priority()
        );
    }
    return state;
}

```

4.3 ColorBlendAttrib 混合方程

文件: panda/src/pgraph/colorBlendAttrib.h

```

// colorBlendAttrib.h:27
class EXPCL_PANDA_PGRAPH ColorBlendAttrib final : public RenderAttrib {
PUBLISHED:
    enum Mode {
        M_none,          // 禁用混合
        M_add,            // C_src * A + C_dst * B (标准混合)
        M_subtract,       // C_src * A - C_dst * B
        M_inv_subtract,   // C_dst * B - C_src * A
        M_min,            // min(C_src, C_dst)
        M_max,            // max(C_src, C_dst)
    };

    enum Operand {
        O_zero,           // 0
        O_one,             // 1
        O_incoming_color, // 源颜色 RGB
        O_one_minus_incoming_color, // 1 - 源颜色 RGB
        O_fbuffer_color,   // 目标颜色 RGB
        O_one_minus_fbuffer_color, // 1 - 目标颜色 RGB
        O_incoming_alpha,  // 源 Alpha
        O_one_minus_incoming_alpha, // 1 - 源 Alpha (最常用)
        O_fbuffer_alpha,   // 目标 Alpha
        O_one_minus_fbuffer_alpha, // 1 - 目标 Alpha
        O_constant_color,  // 常数颜色
        // ...
    };

    // 创建标准 Alpha 混合: C_out = C_src * alpha + C_dst * (1-alpha)
    static CPT(RenderAttrib) make(
        Mode mode,
        Operand a,      // 源因子
        Operand b,      // 目标因子
        Mode alpha_mode = M_none,
        Operand alpha_a = O_zero,
        Operand alpha_b = O_zero,
        const LColor &color = LColor::zero()
    );
};

```

5. 代码演示

5.1 Python：基本透明度设置

```
from panda3d.core import TransparencyAttrib, NodePath

## — 标准 Alpha 透明 —————
node = loader.load_model("glass.egg")
node.reparent_to(render)

## 方式 1: 使用 NodePath 接口 (推荐)
node.set_transparency(TransparencyAttrib.M_alpha)

## 方式 2: 直接设置 Alpha 值 (需要同时启用透明模式)
node.set_alpha_scale(0.5) # 50% 透明
node.set_transparency(TransparencyAttrib.M_alpha)

## 方式 3: 通过颜色设置 Alpha
node.set_color(1, 1, 1, 0.5) # RGBA, A=0.5
node.set_transparency(TransparencyAttrib.M_alpha)

## — 二值透明 (植被) —————
tree = loader.load_model("tree.egg")
tree.reparent_to(render)
## M_binary: Alpha >= 0.5 不透明, < 0.5 完全透明
## 无需排序, 性能好, 适合植被
tree.set_transparency(TransparencyAttrib.M_binary)

## — 双通道透明 (复杂物体) —————
complex_obj = loader.load_model("complex_glass.egg")
complex_obj.reparent_to(render)
## M_dual: 不透明部分正确写深度, 透明部分排序混合
complex_obj.set_transparency(TransparencyAttrib.M_dual)

## — 关闭透明 —————
node.set_transparency(TransparencyAttrib.M_none)
## 或者
node.clear_transparency()
```

5.2 Python：自定义混合模式

```
from panda3d.core import ColorBlendAttrib, RenderState

## — 加法混合（粒子、光晕） —————
particle_np = NodePath("particles")
particle_np.reparent_to(render)

##  $C_{out} = C_{src} * alpha + C_{dst} * 1$  (加法)
additive_blend = ColorBlendAttrib.make(
    ColorBlendAttrib.M_add,
    ColorBlendAttrib.O_incoming_alpha,    # 源因子：源 Alpha
    ColorBlendAttrib.O_one                # 目标因子：1
)
particle_np.set_attrib(additive_blend)
## 加法混合不需要排序！

## — 预乘 Alpha 混合 —————
## 适用于预乘 Alpha 的纹理（如 Photoshop 导出的 PNG）
premult_np = NodePath("premult")
premult_np.set_transparency(TransparencyAttrib.M_premultiplied_alpha)
## 等价于： $C_{out} = C_{src} * 1 + C_{dst} * (1 - alpha)$ 

## — 减法混合（特殊效果） —————
##  $C_{out} = C_{dst} - C_{src} * alpha$  (使场景变暗)
subtract_blend = ColorBlendAttrib.make(
    ColorBlendAttrib.M_subtract,
    ColorBlendAttrib.O_incoming_alpha,
    ColorBlendAttrib.O_one
)
dark_effect_np = NodePath("dark_effect")
dark_effect_np.set_attrib(subtract_blend)

## — 最大值混合（HDR 效果） —————
max_blend = ColorBlendAttrib.make(
    ColorBlendAttrib.M_max,
    ColorBlendAttrib.O_one,
    ColorBlendAttrib.O_one
)
```


5.3 Python：手动控制渲染顺序

```
from panda3d.core import CullBinAttrib

## — 使用 Bin 控制渲染顺序 —————
## Panda3D 内置 Bin：
## "background" sort=-100 不透明，最先渲染（天空盒）
## "opaque" sort=10 不透明，默认
## "transparent" sort=20 透明，back_to_front 排序
## "fixed" sort=40 固定顺序（HUD）

## 将节点放入特定 bin
sky_box.set_bin("background", 0)
sky_box.set_depth_write(False) # 天空盒不写深度

## 透明物体自动放入 transparent bin (M_alpha 时)
glass.set_transparency(TransparencyAttrib.M_alpha)
## 等价于: glass.set_bin("transparent", 0)

## HUD 元素放入 fixed bin (不受深度影响)
hud.set_bin("fixed", 100)
hud.set_depth_test(False)
hud.set_depth_write(False)

## — 自定义 Bin —————
from panda3d.core import CullBinManager

bin_mgr = CullBinManager.get_global_ptr()

## 创建自定义 back_to_front bin (用于特殊透明效果)
bin_mgr.add_bin("my_transparent", CullBinManager.BT_back_to_front, 25)

## 将节点放入自定义 bin
special_glass.set_bin("my_transparent", 0)
special_glass.set_transparency(TransparencyAttrib.M_alpha)
```

5.4 Python：粒子系统透明

```
from panda3d.core import (
    ColorBlendAttrib, TransparencyAttrib,
    DepthWriteAttrib, DepthTestAttrib
)

## — 粒子系统最佳实践 —————
class ParticleSystem:
    def __init__(self, parent):
        self.root = parent.attach_new_node("particles")

        # 加法混合：无需排序，适合发光粒子
        self.root.set_attrib(ColorBlendAttrib.make(
            ColorBlendAttrib.M_add,
            ColorBlendAttrib.O_incoming_alpha,
            ColorBlendAttrib.O_one
        ))

        # 不写深度（粒子不阻挡其他物体）
        self.root.set_depth_write(False)

        # 仍然进行深度测试（粒子被不透明物体遮挡）
        self.root.set_depth_test(True)

        # 放入 transparent bin（在不透明物体之后渲染）
        self.root.set_bin("transparent", 0)

        # 关闭背面剔除（粒子是双面的）
        self.root.set_two_sided(True)

## — 烟雾粒子（需要排序） —————
class SmokeParticles:
    def __init__(self, parent):
        self.root = parent.attach_new_node("smoke")

        # 标准 Alpha 混合（烟雾需要正确的混合顺序）
        self.root.set_transparency(TransparencyAttrib.M_alpha)
        self.root.set_depth_write(False)

        # 注意：M_alpha 会自动放入 back_to_front bin
        # 但粒子之间的排序仍然可能不正确（粒子相互穿插）
```

5.5 GLSL : Shader 中的透明度处理

```
// 片段着色器中的透明度处理

// 方式 1: 直接输出 Alpha (配合 M_alpha 使用)
void main() {
    vec4 color = texture(p3d_Texture0, v_texcoord);
    // 直接输出, GPU 混合单元处理混合
    frag_color = color;
}

// 方式 2: Alpha 测试 (配合 M_binary 使用, 或手动丢弃)
void main() {
    vec4 color = texture(p3d_Texture0, v_texcoord);
    // 手动 Alpha 测试 (比 M_binary 更灵活)
    if (color.a < 0.1) discard; // 完全透明的像素直接丢弃
    frag_color = color;
}

// 方式 3: 预乘 Alpha (配合 M_premultiplied_alpha 使用)
void main() {
    vec4 color = texture(p3d_Texture0, v_texcoord);
    // 输出预乘 Alpha 颜色
    frag_color = vec4(color.rgb * color.a, color.a);
}

// 方式 4: OIT Weighted Blended (顺序无关透明)
layout(location = 0) out vec4 accum; // 累积颜色
layout(location = 1) out float reveal; // 透明度累积

void main() {
    vec4 color = texture(p3d_Texture0, v_texcoord);
    float alpha = color.a;

    // 权重函数 (根据深度和 Alpha 计算)
    float weight = alpha * max(0.01,
        min(3000.0, 0.03 / (1e-5 + pow(gl_FragCoord.z / 200.0, 4.0))));

    accum = vec4(color.rgb * alpha, alpha) * weight;
    reveal = alpha;
}
```

6. 性能分析

6.1 各透明模式性能对比

模式	排序代价	渲染代价	正确性	适用场景
M_none	无	最低	N/A	不透明物体
M_binary	无	低	硬边缘	植被、栅栏
M_multisample	无	中（需 MSAA）	软边缘	植被（高质量）
M_alpha	$O(N \log N)$	中	正确（无循环遮挡）	玻璃、水面
M_dual	$O(N \log N)$	高（2次绘制）	更好	复杂透明物体
加法混合	无	低	近似	粒子、光晕

6.2 排序代价分析

```
## 测量透明排序代价
import time

## 场景: 1000 个透明物体
transparent_nodes = []
for i in range(1000):
    node = loader.load_model("glass_shard.egg")
    node.reparent_to(render)
    node.set_transparency(TransparencyAttrib.M_alpha)
    transparent_nodes.append(node)

## 每帧排序代价:  $O(N \log N) = 1000 * \log(1000) \approx 10000$  次比较
## 典型代价: ~0.5ms (CPU 端)

## 优化: 减少透明物体数量
## 将多个小透明物体合并为一个大网格
```

6.3 常见性能陷阱

```
## ❌ 错误：不透明物体启用透明模式
opaque_rock = loader.load_model("rock.egg")
opaque_rock.set_transparency(TransparencyAttrib.M_alpha) # 不必要!
## 问题：rock 会被放入 back_to_front bin, 每帧排序, 且渲染顺序错误

## ✅ 正确：只对真正透明的物体启用透明模式
glass = loader.load_model("glass.egg")
glass.set_transparency(TransparencyAttrib.M_alpha)

## ❌ 错误：透明物体写深度缓冲
glass.set_depth_write(True) # 默认 M_alpha 已关闭深度写入
## 问题：透明物体写入深度后, 后面的物体无法被看到

## ✅ 正确：M_alpha 自动关闭深度写入 (通过 back_to_front bin 的排序保证)

## ❌ 错误：大量粒子使用 M_alpha (需要排序)
for i in range(10000):
    particle = create_particle()
    particle.set_transparency(TransparencyAttrib.M_alpha) # 10000 个粒子排序!

## ✅ 正确：粒子使用加法混合 (无需排序)
for i in range(10000):
    particle = create_particle()
    particle.set_attrib(ColorBlendAttrib.make(
        ColorBlendAttrib.M_add,
        ColorBlendAttrib.O_incoming_alpha,
        ColorBlendAttrib.O_one
    ))
    particle.set_depth_write(False)

## ❌ 错误：透明物体不设置 bin, 与不透明物体混在一起
glass.set_transparency(TransparencyAttrib.M_alpha)
## 注意：M_alpha 会自动放入 transparent bin, 这是正确的
## 但如果手动设置了错误的 bin:
glass.set_bin("opaque", 0) # 错误！透明物体放入不透明 bin
glass.set_transparency(TransparencyAttrib.M_alpha)
## 问题：不会按深度排序, 透明效果错误
```

6.4 透明度与阴影的交互

```
## 透明物体的阴影处理是个难题：
## 1. 透明物体投射阴影：Shadow Map 无法处理半透明（只有深度信息）
## 2. 解决方案：使用 Alpha 测试 (M_binary) 投射阴影

## 让透明物体投射正确的阴影（使用 Alpha 测试）
glass_with_shadow = loader.load_model("stained_glass.egg")
glass_with_shadow.set_transparency(TransparencyAttrib.M_alpha) # 渲染时半透明

## 阴影渲染时使用 Alpha 测试（只有不透明部分投射阴影）
## 通过 Tag State 为阴影相机设置不同的渲染状态
shadow_state = RenderState.make(
    TransparencyAttrib.make(TransparencyAttrib.M_binary)
)
base.cam.node().set_tag_state_key("shadow_pass")
glass_with_shadow.set_tag("shadow_pass", "alpha_test")
base.cam.node().set_tag_state("alpha_test", shadow_state)
```

小结

概念	核心理想	Panda3D 实现
Alpha 混合	源颜色与目标颜色加权混合	<code>ColorBlendAttrib</code>
排序问题	Over 算子不满足交换律	<code>back_to_front</code> bin
M_binary	Alpha 阈值二值化，无需排序	<code>AlphaTestAttrib >= 0.5</code>
M_dual	不透明+透明两次绘制	复制对象到两个 bin
加法混合	满足交换律，无需排序	<code>ColorBlendAttrib.M_add</code>
深度写入	透明物体不写深度	<code>DepthWriteAttrib.M_off</code>

核心设计哲学： 1. 自动排序： `M_alpha` 自动将对象放入 `back_to_front` bin，无需手动排序 2. 模式选择：根据透明类型选择最合适的模式（性能 vs 正确性权衡） 3. 加法混合例外：粒子等效果使用加法混合，完全绕过排序问题 4. **M_dual 折中**：对复杂物体，将不透明和透明部分分开处理，减少排序错误

专题06：动画系统

目录

1. 问题背景
 2. 数学原理 - 2.1 骨骼动画：蒙皮变换 - 2.2 关键帧插值 - 2.3 动画混合的数学 - 2.4 逆运动学（IK）基础
 3. 工程实践 - 3.1 Panda3D 动画架构 - 3.2 动画绑定（Binding） - 3.3 动画混合策略 - 3.4 帧插值
 4. Panda3D 源码解析 - 4.1 PartBundle 与 AnimBundle - 4.2 AnimControl 时间控制 - 4.3 BlendType 混合类型
- 4.4 do_update() 更新流程
 5. 代码演示
 6. 性能分析
-

1. 问题背景

动画系统需要解决以下核心问题：

1. **骨骼变换**：如何用少量骨骼控制大量顶点？
 2. **关键帧插值**：如何在关键帧之间平滑过渡？
 3. **动画混合**：如何同时播放多个动画（如走路+挥手）？
 4. **状态机**：如何管理复杂的动画状态转换（站立→走路→跑步）？
 5. **IK（逆运动学）**：如何让脚踩在不平地面上？
-

2. 数学原理

2.1 骨骼动画：蒙皮变换

骨骼层次： 骨骼形成树形结构，每个骨骼有局部变换矩阵 $M_{\{local\}}(bone_i)$ 。

骨骼世界变换：

$$M_{\{world\}}(bone_i) = M_{\{world\}}(parent_i) \cdot M_{\{local\}}(bone_i)$$

蒙皮变换 (Skinning)： 每个顶点受多个骨骼影响，最终位置是加权平均：

$$\mathbf{v}_{final} = \sum_{i=1}^N w_i \cdot M_{\{world\}}(bone_i) \cdot M_{\{bind\}}^{-1}(bone_i) \cdot \mathbf{v}_{bind}$$

其中： - $\mathbf{v}_{bind}$ ：绑定姿势 (T-pose) 中的顶点位置 - $M(bone_i)$ ：绑定姿势逆矩阵 (将顶点变换到骨骼局部空间) - $M_{\{world\}}(bone_i)$ ：当前骨骼世界变换 - w_i ：骨骼权重 ($\sum w_i = 1$)

蒙皮矩阵 (Skinning Matrix)：

$$M_{\{skin\}}(bone_i) = M_{\{world\}}(bone_i) \cdot M_{\{bind\}}^{-1}(bone_i)$$

这个矩阵预先计算好，每帧上传到 GPU (Uniform 数组)。

2.2 关键帧插值

线性插值 (LERP)：

$$\mathbf{v}(t) = (1-t) \cdot \mathbf{v}_0 + t \cdot \mathbf{v}_1, \quad t \in [0, 1]$$

球面线性插值 (SLERP) (用于四元数旋转)：

$$q(t) = \frac{\sin((1-t)\theta)}{\sin\theta} q_0 + \frac{\sin(t\theta)}{\sin\theta} q_1$$

其中 $\theta = \arccos(q_0 \cdot q_1)$ (四元数点积)。

三次 Hermite 插值 (Cubic Hermite)：

$$\mathbf{v}(t) = (2t^3 - 3t^2 + 1)\mathbf{v}_0 + (t^3 - 2t^2 + t)\mathbf{m}_0 + (-2t^3 + 3t^2)\mathbf{v}_1 + (t^3 - t^2)\mathbf{m}_1$$

其中 $\mathbf{m}_0, \mathbf{m}_1$ 是切线向量 (控制曲线形状)。

Catmull-Rom 样条： 自动从相邻关键帧计算切线：

$$\mathbf{m}_i = \frac{\mathbf{v}_i - \mathbf{v}_{i-1}}{2} + \frac{\mathbf{v}_{i+1} - \mathbf{v}_{i-1}}{6}$$

2.3 动画混合的数学

线性混合 (BT_linear) :

$$M_{\text{blend}} = \sum_i w_i \cdot M_i, \quad \sum w_i = 1$$

问题: 矩阵线性混合会导致"糖果纸扭曲" (Candy Wrapper Artifact) —— 旋转 180° 的两个矩阵平均后可能得到零矩阵 (完全压缩)。

归一化线性混合 (BT_normalized_linear) :

1. 分离旋转和缩放: $M_i = R_i \cdot S_i$
2. 线性混合缩放: $S_{\text{blend}} = \sum w_i \cdot S_i$
3. 四元数 SLERP 混合旋转: $q_{\text{blend}} = \text{SLERP}(q_0, q_1, w_1)$
4. 重组: $M_{\text{blend}} = R_{\text{blend}} \cdot S_{\text{blend}}$

组件式混合 (BT_componentwise_quat) :

- 位置: 线性插值
- 旋转: 四元数 SLERP
- 缩放: 线性插值

这是最高质量的混合方式, 避免了矩阵混合的所有问题。

2.4 逆运动学 (IK) 基础

正向运动学 (FK) : 给定关节角度, 计算末端位置。

逆运动学 (IK) : 给定末端目标位置, 求解关节角度。

两骨 IK (手臂/腿部) :

给定上臂长 l_1 , 前臂长 l_2 , 目标距离 d :

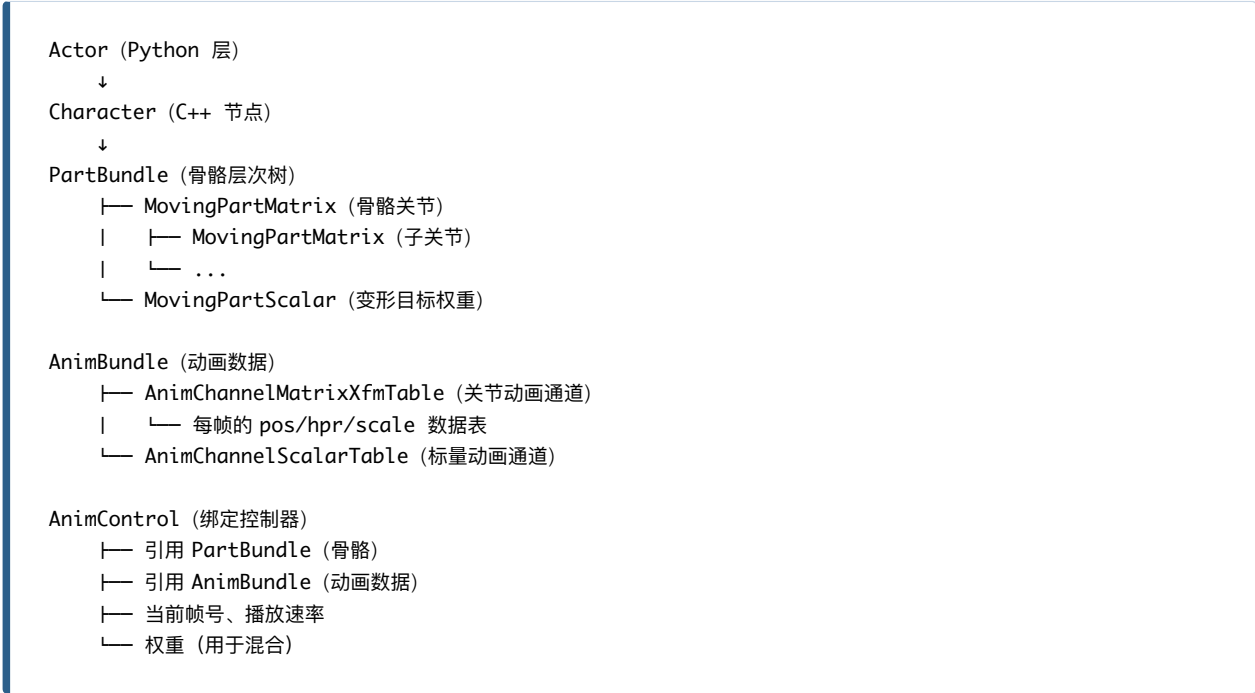
$$\cos\theta_2 = \frac{d^2 - l_1^2 - l_2^2}{2 l_1 l_2}$$

$$\theta_1 = \text{atan2}(l_2 \sin\theta_2, l_1 + l_2 \cos\theta_2)$$

FABRIK (Forward And Backward Reaching IK) : - 迭代算法, 适用于多关节链 - 前向传播: 从末端到根, 每个关节向目标方向移动 - 后向传播: 从根到末端, 保持关节距离约束

3. 工程实践

3.1 Panda3D 动画架构



关键关系： - **PartBundle**：骨骼的结构（层次、初始姿势） - **AnimBundle**：动画的数据（每帧每关节的变换）

- **AnimControl**：将两者绑定，控制播放状态

3.2 动画绑定 (Binding)

绑定过程将 **AnimBundle** 的通道与 **PartBundle** 的关节一一对应：

AnimBundle		PartBundle
"Torso" channel	→→→→→	"Torso" joint
"LeftArm" channel	→→→→→	"LeftArm" joint
"RightLeg" channel	→→→→→	"RightLeg" joint
"Head" channel	→→→→→	"Head" joint

名称匹配：通过关节名称字符串匹配，不要求顺序一致。

部分绑定 (**PartSubset**)：可以只绑定骨骼的一个子集，用于上半身/下半身分离动画。

3.3 动画混合策略

单动画（默认）：

```
set_control_effect(walk_anim, 1.0)
→ 只播放 walk_anim
```

双动画混合（需要 `set_anim_blend_flag(True)`）：

```
set_control_effect(walk_anim, 0.7)
set_control_effect(run_anim, 0.3)
→ 70% 走路 + 30% 跑步（过渡动画）
```

上下半身分离：

```
## 下半身：走路动画
walk_control = actor.get_anim_control("walk")
walk_control.set_control_effect(1.0)

## 上半身：射击动画（只影响上半身关节）
shoot_control = actor.bind_anim("shoot", subset=upper_body_subset)
shoot_control.set_control_effect(1.0)
```

3.4 帧插值

帧插值（Frame Blend）：在两个关键帧之间线性插值，使动画更平滑：

```
frame_blend_flag = True

当前时间 t = 2.3 帧
→ 在第 2 帧和第 3 帧之间插值，权重 0.7/0.3
→ 结果 = 0.7 * frame[2] + 0.3 * frame[3]
```

不开启帧插值：直接取最近的整数帧（可能有跳帧感）。

4. Panda3D 源码解析

4.1 PartBundle 与 AnimBundle

文件: panda/src/chan/partBundle.h

```
// partBundle.h:46
class EXPCL_PANDA_CHAN PartBundle : public PartGroup {
PUBLISHED:
    // 混合类型
    enum BlendType {
        BT_linear,           // 矩阵线性混合 (快速, 有糖果纸问题)
        BT_normalized_linear, // 归一化线性混合 (推荐)
        BT_componentwise,     // 组件式混合 (HPR 线性)
        BT_componentwise_quat, // 组件式混合 (旋转用四元数 SLERP)
    };

    // 动画混合控制
    void set_anim_blend_flag(bool anim_blend_flag); // 允许多动画混合
    void set_frame_blend_flag(bool frame_blend_flag); // 开启帧插值

    // 绑定动画
    PT(AnimControl) bind_anim(AnimBundle *anim,
                              int hierarchy_match_flags = 0,
                              const PartSubset &subset = PartSubset());

    // 关节控制
    bool freeze_joint(std::string_view joint_name,
                     const TransformState *transform); // 冻结关节
    bool control_joint(std::string_view joint_name,
                      PandaNode *node); // 用场景图节点控制关节

    // 更新
    bool update(); // 按需更新 (有延迟优化)
    bool force_update(); // 强制立即更新

private:
    class CData : public CycleData {
    public:
        BlendType _blend_type;
        bool _anim_blend_flag;
        bool _frame_blend_flag;
        LMatrix4 _root_xform;

        // 当前激活的动画及其权重
        // key: AnimControl*, value: 权重 [0,1]
        ChannelBlend _blend; // pmap<AnimControl*, PN_stdfloat>

        bool _anim_changed;
        double _last_update;
    };
};
```

4.2 AnimControl 时间控制

文件: panda/src/chan/animControl.h

```
// animControl.h:38
class EXPCL_PANDA_CHAN AnimControl
    : public TypedReferenceCount, public AnimInterface, public Namable {
PUBLISHED:
    // 继承自 AnimInterface 的播放控制
    void play(); // 播放一次
    void play(double from, double to); // 播放指定帧范围
    void loop(bool restart); // 循环播放
    void loop(bool restart, double from, double to);
    void pingpong(bool restart); // 来回播放
    void stop(); // 停止

    bool is_playing() const;
    double get_play_rate() const;
    void set_play_rate(double play_rate); // 播放速率 (1.0=正常, 2.0=2倍速)

    double get_frame_rate() const; // 动画帧率 (fps)
    int get_num_frames() const; // 总帧数
    double get_full_frame() const; // 当前帧 (浮点数)
    int get_frame() const; // 当前帧 (整数)

    // 绑定信息
    PartBundle *get_part() const; // 对应的骨骼
    AnimBundle *get_anim() const; // 对应的动画数据
    const BitArray &get_bound_joints() const; // 已绑定的关节位掩码

private:
    int _marked_frame; // 上次 mark_channels() 时的帧号
    double _marked_frac; // 上次 mark_channels() 时的帧小数部分
    BitArray _bound_joints; // 已绑定关节的位掩码
};
```


4.3 BlendType 混合类型

```
// partBundle.h:71
enum BlendType {
    // 矩阵线性混合：直接对 4x4 矩阵做加权平均
    // 优点：最快
    // 缺点：旋转混合不正确（糖果纸扭曲）
    BT_linear,

    // 归一化线性混合：分离旋转和缩放，分别混合
    // 优点：避免糖果纸问题，骨骼保持正确大小
    // 缺点：骨骼层次必须完全连接
    BT_normalized_linear,

    // 组件式混合：H、P、R 分别线性插值
    // 优点：直觉上正确
    // 缺点：欧拉角插值可能有万向锁
    BT_componentwise,

    // 组件式四元数混合：位置/缩放线性，旋转用四元数 SLERP
    // 优点：最高质量，无万向锁
    // 缺点：最慢
    BT_componentwise_quat,
};
```

4.4 do_update() 更新流程

```
// partBundle.cxx (简化)
bool PartBundle::update() {
    Thread *current_thread = Thread::get_current_thread();
    CDLockedStageReader cdata(_cycler, 0, current_thread);

    double now = ClockObject::get_global_clock()->get_frame_time(current_thread);

    // 检查是否需要更新 (有延迟优化)
    if (now > cdata->_last_update + _update_delay || cdata->_anim_changed) {
        bool frame_blend_flag = cdata->_frame_blend_flag;

        // 递归更新所有关节
        bool any_changed = do_update(this, cdata, nullptr, false,
                                     cdata->_anim_changed, current_thread);

        // 标记所有动画通道 (记录当前帧号, 用于下次比较)
        for (auto &[control, weight] : cdata->_blend) {
            control->mark_channels(frame_blend_flag);
        }

        return any_changed;
    }
    return false;
}

// do_update() 递归遍历骨骼树
// 对每个 MovingPartMatrix (关节) :
// 1. 从所有激活的 AnimControl 中读取当前帧的变换
// 2. 按权重混合 (根据 BlendType)
// 3. 更新关节的世界变换矩阵
// 4. 通知 JointVertexTransform 更新蒙皮矩阵
```

5. 代码演示

5.1 Python : Actor 基本动画

```
from direct.actor.Actor import Actor
from panda3d.core import LPoint3

## — 加载 Actor —————
## Actor 是 Panda3D 的高层动画接口
actor = Actor(
    "models/panda-model",      # 模型文件
    {
        "walk": "models/panda-walk4", # 动画文件
        "run": "models/panda-run",
        "idle": "models/panda-idle",
    }
)
actor.reparent_to(render)

## — 基本播放控制 —————
actor.loop("walk")           # 循环播放走路动画
actor.play("walk")           # 播放一次
actor.stop()                 # 停止

## 播放速率
actor.set_play_rate(2.0, "walk") # 2倍速
actor.set_play_rate(0.5, "walk") # 半速 (慢动作)
actor.set_play_rate(-1.0, "walk") # 倒放

## 播放指定帧范围
actor.play("walk", fromFrame=10, toFrame=30)
actor.loop("walk", fromFrame=10, toFrame=30)

## 获取当前帧
current_frame = actor.get_current_frame("walk")
total_frames = actor.get_num_frames("walk")
print(f"Frame: {current_frame}/{total_frames}")

## — 帧插值 (更平滑的动画) —————
actor.set_blend(frameBlend=True) # 开启帧插值
```

5.2 Python：动画混合

```
from direct.actor.Actor import Actor

actor = Actor("character", {
    "walk": "walk_anim",
    "run": "run_anim",
    "shoot": "shoot_anim",
})

## — 开启动画混合 —————
actor.set_blend(animBlend=True, frameBlend=True)

## 设置混合权重（权重之和不必为 1，Panda3D 会自动归一化）
actor.set_control_effect("walk", 0.7)
actor.set_control_effect("run", 0.3)

## 同时播放两个动画
actor.loop("walk")
actor.loop("run")

## — 平滑过渡（手动插值权重） —————
from direct.interval.LerpInterval import LerpFunctionInterval

def transition_to_run(t):
    """从走路过渡到跑步，t 从 0 到 1"""
    actor.set_control_effect("walk", 1.0 - t)
    actor.set_control_effect("run", t)

transition = LerpFunctionInterval(
    transition_to_run,
    duration=0.5,
    fromData=0.0,
    toData=1.0
)
transition.start()

## — 上下半身分离动画 —————
from panda3d.core import PartSubset

## 定义上半身关节子集
upper_body = PartSubset()
upper_body.add_include_joint("Spine")
upper_body.add_include_joint("LeftArm")
upper_body.add_include_joint("RightArm")
upper_body.add_include_joint("Head")

## 绑定射击动画到上半身
shoot_control = actor.bind_anim("shoot", subset=upper_body)

## 下半身播放走路，上半身播放射击
actor.loop("walk")
shoot_control.loop(True)
```

5.3 Python：关节控制

```
from direct.actor.Actor import Actor
from panda3d.core import NodePath, LPoint3

actor = Actor("character", {"walk": "walk_anim"})
actor.reparent_to(render)

## — 暴露关节（获取关节的 NodePath）——
## expose_joint() 返回一个跟随关节运动的 NodePath
right_hand = actor.expose_joint(None, "modelRoot", "RightHand")

## 将武器附加到手部关节
weapon = loader.load_model("sword.egg")
weapon.reparent_to(right_hand)
## 武器会自动跟随手部关节运动

## — 控制关节（用 NodePath 驱动关节）——
## control_joint() 让场景图节点控制关节（用于 IK 等）
head_control = actor.control_joint(None, "modelRoot", "Head")

## 现在可以通过 head_control 控制头部朝向
def look_at_target(task):
    target_pos = some_target.get_pos(actor)
    head_control.look_at(target_pos)
    return task.cont

base.task_mgr.add(look_at_target, "look_at_target")

## — 冻结关节 ——
## freeze_joint() 将关节固定在指定变换
from panda3d.core import TransformState, LVecBase3

## 冻结左臂（如受伤状态）
actor.get_part("modelRoot").freeze_joint(
    "LeftArm",
    TransformState.make_hpr(LVecBase3(0, -90, 0)) # 下垂姿势
)

## 释放冻结
actor.get_part("modelRoot").release_joint("LeftArm")
```

5.4 Python：动画状态机

```
from direct.fsm.FSM import FSM
from direct.actor.Actor import Actor

class CharacterFSM(FSM):
    def __init__(self, actor):
        FSM.__init__(self, "CharacterFSM")
        self.actor = actor
        self.actor.set_blend(animBlend=True, frameBlend=True)

    def enterIdle(self):
        self.actor.loop("idle")

    def exitIdle(self):
        self.actor.stop("idle")

    def enterWalk(self):
        self.actor.loop("walk")

    def exitWalk(self):
        self.actor.stop("walk")

    def enterRun(self):
        self.actor.loop("run")

    def exitRun(self):
        self.actor.stop("run")

    def enterJump(self):
        self.actor.play("jump")
        # 跳跃动画结束后自动回到 Idle
        duration = self.actor.get_duration("jump")
        base.task_mgr.do_method_later(duration, self._jump_done, "jump_done")

    def _jump_done(self, task):
        self.request("Idle")
        return task.done

    def exitJump(self):
        base.task_mgr.remove("jump_done")

## 使用状态机
actor = Actor("character", {
    "idle": "idle_anim",
    "walk": "walk_anim",
    "run": "run_anim",
    "jump": "jump_anim",
})
fsm = CharacterFSM(actor)
fsm.request("Idle")

## 状态转换
fsm.request("Walk") # 开始走路
fsm.request("Run")  # 开始跑步
fsm.request("Jump") # 跳跃 (自动返回 Idle)
```

5.5 Python：程序化动画（IK 示例）

```
from panda3d.core import NodePath, LPoint3, LVector3
import math

class TwoBoneIK:
    """简单的两骨 IK（用于腿部）"""

    def __init__(self, actor, upper_joint, lower_joint, end_joint):
        self.upper = actor.control_joint(None, "modelRoot", upper_joint)
        self.lower = actor.control_joint(None, "modelRoot", lower_joint)
        self.end = actor.expose_joint(None, "modelRoot", end_joint)

        # 获取骨骼长度
        self.l1 = (self.lower.get_pos(self.upper)).length()
        self.l2 = (self.end.get_pos(self.lower)).length()

    def solve(self, target_pos):
        """求解 IK，使末端到达 target_pos"""
        # 计算到目标的距离
        root_pos = self.upper.get_pos(render)
        d = (target_pos - root_pos).length()
        d = min(d, self.l1 + self.l2 - 0.001) # 限制在可达范围内

        # 两骨 IK 公式
        cos_angle2 = (d*d - self.l1*self.l1 - self.l2*self.l2) / (2 * self.l1 * self.l2)
        cos_angle2 = max(-1, min(1, cos_angle2)) # 夹紧到 [-1, 1]
        angle2 = math.acos(cos_angle2)

        cos_angle1 = (self.l1*self.l1 + d*d - self.l2*self.l2) / (2 * self.l1 * d)
        cos_angle1 = max(-1, min(1, cos_angle1))
        angle1 = math.acos(cos_angle1)

        # 应用关节角度
        direction = (target_pos - root_pos).normalized()
        self.upper.look_at(root_pos + direction)
        self.upper.set_p(self.upper, math.degrees(angle1))
        self.lower.set_p(self.lower, math.degrees(angle2) - 180)

## 使用 IK
actor = Actor("character", {"walk": "walk_anim"})
left_leg_ik = TwoBoneIK(actor, "LeftThigh", "LeftShin", "LeftFoot")

def update_ik(task):
    # 射线检测地面
    ground_pos = get_ground_position(actor.get_pos() + LVector3(0, 0, -1))
    left_leg_ik.solve(ground_pos)
    return task.cont

base.task_mgr.add(update_ik, "update_ik")
```

6. 性能分析

6.1 动画更新代价

操作	代价	说明
单动画更新	$O(N_{\text{joints}})$	N_{joints} 个矩阵计算
双动画混合	$O(2 \times N_{\text{joints}})$	两套矩阵 + 混合
帧插值	$+O(N_{\text{joints}})$	额外的插值计算
蒙皮 (CPU)	$O(N_{\text{verts}} \times N_{\text{influences}})$	每顶点多骨骼加权
蒙皮 (GPU)	$O(N_{\text{verts}})$ 并行	GPU 顶点着色器

6.2 混合类型性能对比

BlendType	速度	质量	推荐场景
BT_linear	最快	低 (糖果纸)	背景角色
BT_normalized_linear	快	中	一般角色
BT_componentwise	中	中	需要精确 HPR 控制
BT_componentwise_quat	慢	最高	主角、过场动画

6.3 常见性能陷阱

```
## ❌ 错误：每帧重新绑定动画
def update(task):
    actor.bind_anim("walk") # 每帧绑定！代价极高
    actor.loop("walk")
    return task.cont

## ✅ 正确：只绑定一次，之后只控制播放
actor.load_anims({"walk": "walk_anim"}) # 预加载
actor.loop("walk") # 直接播放

## ❌ 错误：不必要的 expose_joint（每个都创建额外节点）
for joint_name in all_joint_names:
    actor.expose_joint(None, "modelRoot", joint_name) # 100 个关节！

## ✅ 正确：只暴露需要的关节
right_hand = actor.expose_joint(None, "modelRoot", "RightHand")
left_hand = actor.expose_joint(None, "modelRoot", "LeftHand")

## ❌ 错误：CPU 蒙皮（大量顶点时极慢）
## 默认情况下 Panda3D 使用 GPU 蒙皮（顶点着色器）
## 不要强制使用 CPU 蒙皮

## ✅ 正确：确保使用 GPU 蒙皮
## 在 config.prc 中：
## hardware-animated-vertices 1 （默认已开启）

## ❌ 错误：过多骨骼影响（每顶点 8 个骨骼）
## 标准：每顶点最多 4 个骨骼影响
## 超过 4 个会显著增加 GPU 蒙皮代价

## ✅ 正确：限制每顶点骨骼影响数
## 在建模软件中设置最大权重数为 4
```

6.4 动画 LOD

```
## 根据距离降低动画更新频率
class AnimLOD:
    def __init__(self, actor, camera):
        self.actor = actor
        self.camera = camera

    def update(self, task):
        dist = (self.actor.get_pos() - self.camera.get_pos()).length()

        if dist < 20:
            # 近距离: 全帧率更新, 帧插值
            self.actor.set_blend(frameBlend=True)
            update_rate = 1.0
        elif dist < 50:
            # 中距离: 半帧率
            self.actor.set_blend(frameBlend=False)
            update_rate = 0.5
        else:
            # 远距离: 1/4 帧率, 无插值
            self.actor.set_blend(frameBlend=False)
            update_rate = 0.25

        # 通过 set_play_rate 控制更新频率
        self.actor.set_play_rate(update_rate, "walk")
        return task.cont
```

小结

概念	核心思想	Panda3D 实现
骨骼动画	骨骼层次 + 蒙皮权重	<code>PartBundle + Character</code>
关键帧插值	LERP/SLERP 在帧间插值	<code>frame_blend_flag</code>
动画混合	多动画加权平均	<code>set_control_effect()</code>
BlendType	矩阵/组件/四元数混合	<code>set_blend_type()</code>
关节控制	场景图节点驱动关节	<code>control_joint()</code>
关节暴露	关节驱动场景图节点	<code>expose_joint()</code>
动画状态机	FSM 管理动画状态	<code>direct.fsm.FSM</code>

核心设计哲学：1. 数据驱动： `AnimBundle` 存储纯数据， `PartBundle` 存储结构， `AnimControl` 管理状态 2. 延迟更新： `update_delay` 避免每帧都重算骨骼变换 3. GPU 蒙皮：骨骼矩阵上传 GPU，顶点变换在着色器中并行计算 4. 灵活混合：支持多动画混合、部分骨骼绑定、关节手动控制

专题07：物理与碰撞检测

目录

1. 问题背景
 2. 数学原理
 3. 工程实践
 4. Panda3D 源码剖析
 5. 代码演示
 6. 性能优化
 7. 小结
-

1. 问题背景

碰撞检测是游戏引擎中最核心的子系统之一，需要回答：**两个物体是否相交？交叉深度和法线是什么？**

挑战	描述
精度 vs 性能	精确凸包碰撞代价高，简化形状（球、胶囊）快但不准确
穿透问题	高速物体在一帧内可能穿过薄墙（隧道效应）
$O(n^2)$ 复杂度	n 个物体两两检测需要 $O(n^2)$ 次测试
响应多样性	不同场景需要不同响应：推挤、事件触发、重力、流体
双分派问题	球-球、球-平面、射线-球等不同组合需要不同算法

Panda3D 的碰撞系统是**纯软件实现**的碰撞检测框架（不依赖 Bullet/ODE），专为游戏逻辑碰撞设计。

2. 数学原理

2.1 分离轴定理 (SAT)

定理：两个凸多边形不相交，当且仅当存在一条轴，使得两个多边形在该轴上的投影不重叠。

对于 3D 凸多面体 A 和 B，需要测试的轴： - A 的所有面法线（最多 m 条） - B 的所有面法线（最多 n 条） - A 的每条边与 B 的每条边的叉积（最多 $m \times n$ 条）

投影重叠测试：

设轴方向为单位向量 n ，物体 A 在轴上的投影区间为 $[a_{\min}, a_{\max}]$ ，物体 B 为 $[b_{\min}, b_{\max}]$ 。

若 $a_{\max} < b_{\min}$ 或 $b_{\max} < a_{\min}$ ，则存在分离轴，两物体不相交。

穿透深度 (Penetration Depth)：

若所有轴都重叠，穿透深度为所有轴上重叠量的最小值，对应最小穿透深度的轴即为**碰撞法线**。

SAT 的优势： - 早期退出（找到一条分离轴即可停止） - 同时给出穿透深度和法线 - 适合 AABB、OBB、凸多边形

2.2 GJK 算法

GJK (Gilbert-Johnson-Keerthi) 算法利用**闵可夫斯基差** (Minkowski Difference) 判断两个凸体是否相交。

闵可夫斯基差： $A \ominus B = \{ a - b \mid a \in A, b \in B \}$

关键性质：A 和 B 相交 当且仅当 原点在 $A \ominus B$ 内部。

支撑函数 (Support Function)： $S_{\{A \ominus B\}}(d) = S_A(d) - S_B(-d)$ ，其中 $S_A(d) = \arg\max_{\{a \in A\}} (a \cdot d)$ 是 A 在方向 d 上的最远点。

GJK 迭代： 1. 选择初始方向 d 2. 计算支撑点 $p = S_{\{A \ominus B\}}(d)$ 3. 若 $p \cdot d < 0$ ，原点不在 $A \ominus B$ 内，不相交 4. 将 p 加入单纯形 (simplex)，更新 d 为指向原点的方向 5. 重复直到单纯形包含原点或确认不相交

EPA 算法 (Expanding Polytope Algorithm)：在 GJK 确认相交后，用于计算穿透深度和碰撞法线。

2.3 基本几何相交测试

球-球相交

射线-球相交

射线-平面相交

球-平面相交

2.4 连续碰撞检测 (CCD)

问题：离散碰撞检测在高速物体时会出现隧道效应 (Tunneling) 。

Panda3D 的 CCD 方案：

原理：将碰撞体从上一帧位置到当前帧位置扫掠 (Sweep)，检测扫掠体积与静态物体的相交。

对于球体，扫掠体积是一个**胶囊体** (Capsule)：- 胶囊体 = 球心从 P_{prev} 到 P_{curr} 的线段 + 半径 r 的膨胀

时间参数化 (TOI - Time of Impact)：- t_{impact} = 使 $|P(t) - C_{wall}| = r_{sphere} + r_{wall}$ 的最小 $t \in [0, 1]$

3. 工程实践

3.1 宽相 vs 窄相

碰撞检测分两个阶段：

宽相 (Broad Phase)：快速排除明显不相交的物体对 - 使用 AABB (轴对齐包围盒) 或 BVH (包围体层次) - 时间复杂度： $O(n \log n)$ 或 $O(n + k)$, k 为候选对数量 - Panda3D 使用场景图的 BoundingBox 层次作为宽相

窄相 (Narrow Phase)：对候选对进行精确相交测试 - 使用具体形状 (球、胶囊、多边形) 的精确算法 - 时间复杂度： $O(1)$ 每对 (对于简单形状) - Panda3D 的 CollisionSolid 子类实现各种精确测试

Panda3D 的宽相实现：

3.2 碰撞掩码系统

Panda3D 使用 32 位掩码 (BitMask32) 控制哪些物体之间可以发生碰撞：

掩码语义：-：该碰撞体能检测到哪些类型的物体 -：该碰撞体被哪些类型的物体检测到

3.3 双分派 (Double Dispatch) 模式

碰撞检测的核心问题：不同形状组合需要不同算法，但 C++ 只支持单分派 (虚函数)。

Panda3D 的解决方案：两级虚函数调用

示例：射线 vs 球体

1'''

1'''1

1'''3

1'''1

1''' 2

碰撞矩阵（Panda3D 支持的组合）：

From \ Into	Sphere	Box	Capsule	Plane	Polygon	FloorMesh
Sphere	O	O	O	O	O	O
Ray	O	O	O	O	O	O
Segment	O	O	O	O	O	O
Box	O	O	-	O	-	-
Capsule	O	O	-	O	-	-
Parabola	O	-	-	O	O	-

3.4 碰撞响应策略

Handler	用途	原理
CollisionHandlerQueue	收集所有碰撞信息	存储 CollisionEntry 列表
CollisionHandlerEvent	触发事件（进入/离开/持续）	比较前后帧碰撞集合
CollisionHandlerPusher	推挤（防穿墙）	沿法线方向移动 from 节点
CollisionHandlerFluidPusher	流体推挤（高速物体）	结合 CCD 的推挤
CollisionHandlerGravity	重力 + 地面检测	向下射线 + 高度调整
CollisionHandlerFloor	地面吸附	向下射线，保持在地面上

4. Panda3D 源码剖析

4.1 CollisionTraverser 遍历器

文件: `collisionTraverser.h`

```
// collisionTraverser.h:46
class EXPCL_PANDA_COLLIDE CollisionTraverser : public Namable {
public:
    // 注册碰撞发起者 (from) 和对应的处理器
    void add_collider(const NodePath &collider, CollisionHandler *handler);
    bool remove_collider(const NodePath &collider);

    // 核心: 遍历场景图, 执行所有碰撞检测
    BLOCKING void traverse(const NodePath &root);

    // CCD 支持: 是否考虑上一帧的变换
    void set_respect_prev_transform(bool flag);
    bool get_respect_prev_transform() const;

    // 调试可视化
    void show_collisions(NodePath root);
    void hide_collisions();

private:
    // 内部: 比较 from 碰撞体与场景中的 into 节点
    void compare_collider_to_node(CollisionEntry &entry,
        const GeometricBoundingVolume *from_node_gbv,
        const GeometricBoundingVolume *into_node_gbv);

    void compare_collider_to_geom_node(CollisionEntry &entry,
        const GeometricBoundingVolume *from_node_gbv,
        const GeometricBoundingVolume *into_node_gbv);

    // 存储所有注册的 (collider, handler) 对
    typedef pvector<ColliderDef> Colliders;
    Colliders _colliders;
};
```

`traverse()` 的执行流程:

```
traverse(root)
  对每个注册的 collider (from 节点) :
    获取 from 的 CollisionSolid 和 BoundingBox
  递归遍历 root 场景图:
    宽相: from_bv 与 into_bv 相交测试
    不相交 -> 跳过整个子树 (剪枝)
    窄相: 对每个 CollisionNode 中的 CollisionSolid:
      检查 from_mask & into_mask
      调用 from_solid->test_intersection(entry)
  对每个碰撞结果, 调用对应 handler->add_entry(entry)
```

4.2 CollisionSolid 碰撞体基类

文件: `collisionSolid.h`

```

// collisionSolid.h:45
class EXPCL_PANDA_COLLIDE CollisionSolid : public CopyOnWriteObject {
public:
    // 获取碰撞体的原点（用于宽相测试）
    virtual LPoint3 get_collision_origin() const = 0;

    // 切线/法线控制
    void set_tangible(bool tangible);           // 是否参与推挤响应
    bool is_tangible() const;

    // 有效法线（覆盖计算出的碰撞法线）
    void set_effective_normal(const LVector3 &effective_normal);
    bool has_effective_normal() const;
    bool get_respect_effective_normal() const;

    // 包围体（用于宽相）
    CPT(BoundingVolume) get_bounds() const;
    void set_bounds(const BoundingVolume *bounding_volume);

protected:
    // 子类实现：计算内部包围体
    virtual PT(BoundingVolume) compute_internal_bounds() const;

    // 核心：执行相交测试（双分派第一级）
    virtual PT(CollisionEntry) test_intersection(
        const CollisionEntry &entry) const;

    // 双分派第二级：各 from 形状调用对应的 into 方法
    virtual PT(CollisionEntry) test_intersection_from_sphere(
        const CollisionEntry &entry) const;
    virtual PT(CollisionEntry) test_intersection_from_ray(
        const CollisionEntry &entry) const;
    virtual PT(CollisionEntry) test_intersection_from_segment(
        const CollisionEntry &entry) const;
    virtual PT(CollisionEntry) test_intersection_from_box(
        const CollisionEntry &entry) const;
    virtual PT(CollisionEntry) test_intersection_from_capsule(
        const CollisionEntry &entry) const;

    // 标志位
    enum Flags {
        F_tangible           = 0x01, // 参与推挤
        F_effective_normal    = 0x02, // 使用自定义法线
        F_viz_geom_stale      = 0x04, // 可视化几何体需要更新
        F_viz_bounds_stale    = 0x08,
        F_internal_bounds_stale = 0x10,
    };
    unsigned int _flags;
};

```

effective_normal 的用途：

对于斜坡地面，碰撞法线可能是斜的，导致角色被推向侧面。设置 `effective_normal = (0,0,1)` 可以让推挤响应始终沿垂直方向，实现平滑的斜坡行走。

4.3 碰撞体类型层次

CollisionSolid (抽象基类)	
├─ CollisionSphere	球体 (最快, 推荐用于角色)
├─ CollisionBox	轴对齐盒子
├─ CollisionCapsule	胶囊体 (球+圆柱, 适合角色)
├─ CollisionInvSphere	反向球 (内部碰撞, 如天空盒)
├─ CollisionPlane	无限平面
├─ CollisionPolygon	任意多边形 (精确但慢)
├─ CollisionGeom	几何体碰撞 (最精确, 最慢)
├─ CollisionFloorMesh	地面网格 (优化的地面检测)
├─ CollisionHeightfield	高度图地形
├─ CollisionRay	射线 (无限长, 单向)
├─ CollisionSegment	线段 (有限长)
├─ CollisionLine	直线 (无限长, 双向)
└─ CollisionParabola	抛物线 (投射物轨迹)

各形状性能对比：

形状	相交测试速度	精确度	推荐用途
Sphere	极快	低	角色、子弹、拾取物
Capsule	快	中	角色控制器
Box	快	中	箱子、建筑
Ray/Segment	极快	高	射线拾取、地面检测
Plane	极快	高	地面、墙壁 (无限)
Polygon	中	高	精确地形
FloorMesh	中	高	复杂地面
Geom	慢	最高	精确碰撞 (慎用)

4.4 CollisionHandler 响应处理器

CollisionHandlerEvent (事件触发)：

```
// 三种事件模式
handler->add_in_pattern("enter-%in"); // 进入时触发
handler->add_again_pattern("stay-%in"); // 持续接触时触发
handler->add_out_pattern("exit-%in"); // 离开时触发

// %in 会被替换为 into 节点的名称
// %fn 会被替换为 from 节点的名称
```

CollisionHandlerPusher (推挤)：

推挤算法： 1. 获取碰撞法线 n 和穿透深度 d 2. 将 from 节点沿 n 方向移动 d 距离 3. 若有多个碰撞，累加所有推挤向量

```
// 内部实现伪代码
void CollisionHandlerPusher::handle_entries() {
    for (auto &entry : _entries) {
        LVector3 normal = entry->get_surface_normal();
        // 计算推挤向量
        LVector3 push = normal * penetration_depth;
        // 移动 from 节点
        from_node.set_pos(from_node.get_pos() + push);
    }
}
```

5. 代码演示

5.1 Python：基本碰撞设置

```
from panda3d.core import (
    CollisionTraverser, CollisionNode, CollisionSphere,
    CollisionHandlerQueue, BitMask32
)

class CollisionDemo:
    def __init__(self, base):
        self.base = base
        # 1. 创建碰撞遍历器
        self.cTrav = CollisionTraverser('main_traverser')
        # 2. 创建碰撞处理器 (收集碰撞信息)
        self.cQueue = CollisionHandlerQueue()

        # 3. 为玩家创建碰撞体 (from 节点)
        player_col_node = CollisionNode('player_collider')
        player_col_node.add_solid(CollisionSphere(0, 0, 0, 0.5))
        player_col_node.set_from_collide_mask(BitMask32.bit(0))
        player_col_node.set_into_collide_mask(BitMask32.all_off())
        player_col_np = base.player.attach_new_node(player_col_node)

        # 4. 注册到遍历器
        self.cTrav.add_collider(player_col_np, self.cQueue)

        # 5. 为场景物体创建碰撞体 (into 节点)
        wall_col_node = CollisionNode('wall_collider')
        wall_col_node.add_solid(CollisionSphere(0, 0, 0, 1.0))
        wall_col_node.set_from_collide_mask(BitMask32.all_off())
        wall_col_node.set_into_collide_mask(BitMask32.bit(0))
        base.wall.attach_new_node(wall_col_node)

        # 6. 在游戏循环中执行碰撞检测
        base.taskMgr.add(self.collison_task, 'collision_task')

    def collision_task(self, task):
        self.cTrav.traverse(self.base.render)
        for i in range(self.cQueue.get_num_entries()):
            entry = self.cQueue.get_entry(i)
            into_np = entry.get_into_node_path()
            print(f"碰撞: {entry.get_from_node().get_name()} -> {entry.get_into_node().get_name()}")
            print(f" 碰撞点: {entry.get_surface_point(into_np)}")
            print(f" 碰撞法线: {entry.get_surface_normal(into_np)}")
        self.cQueue.clear_entries()
        return task.cont
```

5.2 Python：碰撞掩码过滤

```
from panda3d.core import BitMask32, CollisionNode, CollisionSphere

## 定义碰撞层
class CollisionLayers:
    GROUND = BitMask32.bit(0)
    WALL = BitMask32.bit(1)
    ENEMY = BitMask32.bit(2)
    PLAYER = BitMask32.bit(3)
    TRIGGER = BitMask32.bit(4)
    BULLET = BitMask32.bit(5)

def setup_player_collision(player_np):
    col_node = CollisionNode('player')
    col_node.add_solid(CollisionSphere(0, 0, 0.9, 0.4)) # 身体球
    col_node.add_solid(CollisionSphere(0, 0, 0.3, 0.4)) # 脚部球
    # 玩家检测：地面、墙壁、敌人、触发器
    col_node.set_from_collide_mask(
        CollisionLayers.GROUND | CollisionLayers.WALL |
        CollisionLayers.ENEMY | CollisionLayers.TRIGGER
    )
    # 玩家被：子弹检测到
    col_node.set_into_collide_mask(CollisionLayers.PLAYER)
    return player_np.attach_new_node(col_node)

def setup_bullet_collision(bullet_np):
    col_node = CollisionNode('bullet')
    col_node.add_solid(CollisionSphere(0, 0, 0, 0.1))
    col_node.set_from_collide_mask(
        CollisionLayers.PLAYER | CollisionLayers.ENEMY | CollisionLayers.WALL
    )
    col_node.set_into_collide_mask(CollisionLayers.BULLET)
    return bullet_np.attach_new_node(col_node)

def setup_trigger(trigger_np):
    col_node = CollisionNode('trigger_zone')
    col_node.add_solid(CollisionSphere(0, 0, 0, 3.0))
    col_node.set_from_collide_mask(BitMask32.all_off())
    col_node.set_into_collide_mask(CollisionLayers.PLAYER)
    return trigger_np.attach_new_node(col_node)
```

5.3 Python：自定义碰撞响应（事件系统）

```
from panda3d.core import (
    CollisionTraverser, CollisionNode, CollisionSphere,
    CollisionHandlerEvent, BitMask32
)

class EventCollisionSystem:
    def __init__(self, base):
        self.base = base
        self.cTrav = CollisionTraverser()
        self.cEvent = CollisionHandlerEvent()

        # 设置事件模式: %in = into节点名, %fn = from节点名
        self.cEvent.add_in_pattern('%fn-enter-%in')    # 进入时
        self.cEvent.add_again_pattern('%fn-stay-%in')  # 持续时
        self.cEvent.add_out_pattern('%fn-exit-%in')   # 离开时

        # 注册玩家碰撞体
        player_col = CollisionNode('player')
        player_col.add_solid(CollisionSphere(0, 0, 0, 0.5))
        player_col.set_from_collide_mask(BitMask32.bit(0))
        player_col_np = base.player.attach_new_node(player_col)
        self.cTrav.add_colider(player_col_np, self.cEvent)

        # 注册触发器碰撞体
        trigger_col = CollisionNode('treasure_trigger')
        trigger_col.add_solid(CollisionSphere(0, 0, 0, 2.0))
        trigger_col.set_into_collide_mask(BitMask32.bit(0))
        base.treasure.attach_new_node(trigger_col)

        # 监听碰撞事件
        base.accept('player-enter-treasure_trigger', self.on_enter_treasure)
        base.accept('player-stay-treasure_trigger', self.on_stay_treasure)
        base.accept('player-exit-treasure_trigger', self.on_exit_treasure)
        base.taskMgr.add(self.update, 'collision_update')

    def on_enter_treasure(self, entry):
        print("进入宝藏区域！")

    def on_stay_treasure(self, entry):
        pass

    def on_exit_treasure(self, entry):
        print("离开宝藏区域")

    def update(self, task):
        self.cTrav.traverse(self.base.render)
        return task.cont
```


5.4 Python：角色控制器（推挤+重力）

```
from panda3d.core import (
    CollisionTraverser, CollisionNode,
    CollisionSphere, CollisionRay,
    CollisionHandlerPusher, CollisionHandlerGravity,
    BitMask32, Vec3
)

class CharacterController:
    def __init__(self, base, character_np):
        self.base = base
        self.char = character_np
        self.cTrav = CollisionTraverser()

        # — 推挤处理器（防止穿墙） —————
        self.pusher = CollisionHandlerPusher()
        self.pusher.set_horizontal(True) # 只在水平方向推挤

        body_col = CollisionNode('char_body')
        body_col.add_solid(CollisionSphere(0, 0, 0.5, 0.4)) # 腰部
        body_col.add_solid(CollisionSphere(0, 0, 1.2, 0.4)) # 胸部
        body_col.set_from_collide_mask(BitMask32.bit(1)) # 检测墙壁层
        body_col.set_into_collide_mask(BitMask32.all_off())
        body_col_np = character_np.attach_new_node(body_col)

        self.cTrav.add_collider(body_col_np, self.pusher)
        self.pusher.add_collider(body_col_np, character_np)

        # — 重力处理器（地面检测） —————
        self.gravity = CollisionHandlerGravity()
        self.gravity.set_gravity(9.8)
        self.gravity.set_max_slope(45.0)
        self.gravity.set_reach(0.1)

        floor_ray = CollisionNode('char_floor_ray')
        floor_ray.add_solid(CollisionRay(0, 0, 0.5, 0, 0, -1)) # 从腰部向下
        floor_ray.set_from_collide_mask(BitMask32.bit(0))
        floor_ray.set_into_collide_mask(BitMask32.all_off())
        floor_ray_np = character_np.attach_new_node(floor_ray)

        self.cTrav.add_collider(floor_ray_np, self.gravity)
        self.gravity.add_collider(floor_ray_np, character_np)

        # — CCD：防止高速穿透 —————
        self.cTrav.set_respect_prev_transform(True)
        base.taskMgr.add(self.update, 'char_collision')

    def update(self, task):
        self.cTrav.traverse(self.base.render)
        return task.cont

    def jump(self):
        # 触发跳跃：设置初始向上速度
        self.gravity.set_velocity(Vec3(0, 0, 5.0))
```

5.5 Python：射线拾取 (Ray Picking)

```
from panda3d.core import (
    CollisionTraverser, CollisionNode, CollisionRay,
    CollisionHandlerQueue, BitMask32
)

class RayPicker:
    def __init__(self, base):
        self.base = base
        self.cTrav = CollisionTraverser()
        self.cQueue = CollisionHandlerQueue()

        self.pick_ray = CollisionRay()
        pick_node = CollisionNode('mouse_ray')
        pick_node.add_solid(self.pick_ray)
        pick_node.set_from_collide_mask(BitMask32.bit(0))
        pick_node.set_into_collide_mask(BitMask32.all_off())

        pick_np = base.camera.attach_new_node(pick_node)
        self.cTrav.add_collider(pick_np, self.cQueue)
        base.accept('mouse1', self.on_click)

    def on_click(self):
        # 鼠标点击时执行射线拾取
        if not self.base.mouseWatcherNode.has_mouse():
            return
        mouse_pos = self.base.mouseWatcherNode.get_mouse()
        # 设置射线从相机出发，穿过鼠标位置
        self.pick_ray.set_from_lens(
            self.base.camNode,
            mouse_pos.get_x(),
            mouse_pos.get_y()
        )
        self.cTrav.traverse(self.base.render)
        if self.cQueue.get_num_entries() > 0:
            self.cQueue.sort_entries()
            entry = self.cQueue.get_entry(0)
            picked_np = entry.get_into_node_path()
            hit_point = entry.get_surface_point(self.base.render)
            print(f"点击了: {picked_np.get_name()}")
            print(f"点击位置: {hit_point}")
            return picked_np, hit_point
        return None, None
```

6. 性能优化

6.1 碰撞体选择原则

```
## 错误：对复杂模型使用 CollisionGeom (极慢，每帧测试所有三角形)
## col_node.add_solid(CollisionGeom(model.node()))

## 正确：用简化形状近似
from panda3d.core import CollisionCapsule, CollisionBox, Point3

## 角色用胶囊体
col_node.add_solid(CollisionCapsule(0, 0, 0.2, 0, 0, 1.6, 0.4))

## 建筑用多个盒子组合
col_node.add_solid(CollisionBox(Point3(-1, -1, 0), Point3(1, 1, 2)))
```

6.2 碰撞掩码优化

```
## 错误：所有物体使用默认掩码 (全部互相检测)
col_node.set_from_collide_mask(BitMask32.all_on())
col_node.set_into_collide_mask(BitMask32.all_on())

## 正确：精确设置掩码，减少不必要的测试
col_node.set_from_collide_mask(BitMask32.bit(0) | BitMask32.bit(1))
col_node.set_into_collide_mask(BitMask32.bit(2))
```

6.3 减少碰撞体数量

```
## 错误：每个小物体都有独立碰撞体 (1000个碰撞体,  $O(n^2)$  测试)
## for i in range(1000):
##     coin = loader.load_model('coin')
##     col = CollisionNode('coin_col')
##     col.add_solid(CollisionSphere(0, 0, 0, 0.2))
##     coin.attach_new_node(col)

## 正确：使用触发区域代替单个碰撞体
## 用一个大的触发区域检测玩家是否靠近，再精确检测
trigger = CollisionNode('coins_area')
trigger.add_solid(CollisionSphere(0, 0, 0, 5.0)) # 5米范围内才精确检测
```

6.4 碰撞遍历器分离

```
## 不同用途使用不同遍历器，避免不必要的交叉检测
cTrav_physics = CollisionTraverser('physics') # 物理碰撞（每帧）
cTrav_picking = CollisionTraverser('picking') # 鼠标拾取（按需）
cTrav_triggers = CollisionTraverser('triggers') # 触发器（每帧，但物体少）

## 拾取遍历器只在点击时运行，不加入 taskMgr
```

6.5 性能对比

场景	碰撞体类型	每帧时间（100物体）
全部 CollisionGeom	精确网格	~50ms（不可接受）
全部 CollisionSphere	球体	~0.5ms
混合（球+胶囊+盒子）	简化形状	~1ms
启用 CCD	球体+扫掠	~2ms

小结

知识点	核心要点
SAT	找到分离轴即可判断不相交；穿透深度 = 最小重叠量
GJK	闵可夫斯基差 + 支撑函数；原点在差集内 = 相交
宽相/窄相	BVH 宽相剪枝 + 精确形状窄相；两阶段大幅降低复杂度
双分派	两级虚函数解决 N×M 形状组合问题
碰撞掩码	32位掩码精确控制碰撞对；from & into != 0 才检测
CCD	set_respect_prev_transform(True) 防隧道效应
effective_normal	斜坡行走的关键：覆盖碰撞法线为垂直方向
Handler 选择	Queue=调试，Event=触发器，Pusher=防穿墙，Gravity=角色

Panda3D 碰撞系统的设计哲学： - 简单优先：优先使用球体、胶囊等简单形状 - 分层设计：掩码系统实现精确的碰撞过滤 - 软件实现：不依赖物理引擎，轻量但功能完整 - 可扩展：通过继承 CollisionSolid 可添加自定义形状

专题08：纹理与内存管理

目录

1. 问题背景
 2. 数学原理
 3. 工程实践
 4. Panda3D 源码剖析
 5. 代码演示
 6. 性能优化
 7. 小结
-

1. 问题背景

纹理是 3D 渲染中内存消耗最大的资源。一张 4096×4096 的 RGBA 纹理未压缩时占用 **64MB**，而现代游戏场景可能有数百张这样的纹理。

挑战	描述
内存带宽	GPU 纹理采样需要大量内存带宽，缓存命中率至关重要
显存限制	移动端 GPU 显存可能只有 1-2GB，需要精细管理
加载延迟	从磁盘加载大纹理可能造成卡顿（Stutter）
Mipmap 生成	需要预生成多级细节，增加内存占用约 33%
格式多样性	DXT/BC、ETC、PVRTC 等压缩格式在不同平台支持不同
纹理池管理	避免重复加载同一纹理，需要引用计数和缓存

Panda3D 的纹理系统通过 **TexturePool**（全局缓存）、**PreparedGraphicsObjects**（GPU 资源管理）和 **AdaptiveLRU**（自适应淘汰）三层机制解决这些问题。

2. 数学原理

2.1 Mipmap 理论

Mipmap（多级渐远纹理）由 Williams 于 1983 年提出，核心思想是预计算不同分辨率的纹理版本，在渲染时根据屏幕像素覆盖面积选择合适的级别。

Mipmap 级别计算：

设纹理坐标 (u, v) 在屏幕空间的偏导数为：

$du/dx, dv/dx$ (沿 x 方向的变化率)
 $du/dy, dv/dy$ (沿 y 方向的变化率)

LOD (Level of Detail) 计算：

```
rho = max(sqrt((du/dx)^2 + (dv/dx)^2), sqrt((du/dy)^2 + (dv/dy)^2))
lambda = log2(rho) + lod_bias
mip_level = clamp(lambda, 0, max_level)
```

Mipmap 内存占用：

设基础纹理大小为 $W \times H$ ，则所有 Mipmap 级别的总内存为：

$$\text{Total} = W * H * (1 + 1/4 + 1/16 + \dots) = W * H * 4/3 \approx 1.333 * W * H$$

即 Mipmap 额外增加约 **33%** 的内存占用。

2.2 纹理过滤数学

最近邻过滤 (Nearest)：

```
texel = texture[round(u * W)][round(v * H)]
```

双线性过滤 (Bilinear)：

```

设 i = floor(u * W), j = floor(v * H)
s = frac(u * W), t = frac(v * H)

texel = (1-s)(1-t)*T[i][j]   + s(1-t)*T[i+1][j]
        + (1-s)t  *T[i][j+1]   + s*t  *T[i+1][j+1]

```

三线性过滤 (Trilinear = 双线性 + Mipmap 线性插值) :

```

level0 = bilinear_sample(mip[floor(lambda)], u, v)
level1 = bilinear_sample(mip[ceil(lambda)], u, v)
texel  = lerp(level0, level1, frac(lambda))

```

各向异性过滤 (Anisotropic) :

沿最大变化方向采样多个点 (最多 16 个) , 解决斜视角下的模糊问题:

```

N = min(anisotropy_degree, max_anisotropy)
沿各向异性方向均匀采样 N 个点, 取平均值

```

2.3 纹理压缩算法

DXT1/BC1 (RGB, 无 Alpha 或 1 位 Alpha) : - 将 4x4 像素块压缩为 8 字节 (原始 48 字节) - 压缩比: 6:1 - 原理: 存储两个端点颜色, 其余颜色通过线性插值得到

```

每个 4x4 块:
    color0 (16 bits) + color1 (16 bits) = 2 个端点颜色
    索引表 (32 bits) = 16 个像素各 2 位索引
    总计 8 bytes (原始 48 bytes)

```

DXT5/BC3 (RGBA) : - 4x4 块压缩为 16 字节 (原始 64 字节) - 压缩比: 4:1 - RGB 部分同 DXT1, Alpha 单独用 8 字节存储

BC7 (高质量 RGBA) : - 压缩比 4:1, 质量远高于 DXT5 - 支持 7 种模式, 根据块内容自适应选择

2.4 LRU 缓存淘汰策略

标准 LRU (Least Recently Used) : - 维护一个有序链表, 最近使用的在头部 - 淘汰时移除尾部元素 - 时间复杂度: O(1) 访问和淘汰

Panda3D 的 AdaptiveLRU:

标准 LRU 的问题: 一次性扫描大量纹理会污染缓存 (Cache Pollution) 。

AdaptiveLRU 的改进： - 维护**动态列表**（最近使用）和**静态列表**（频繁使用） - 使用**访问频率权重**：频繁使用的页面即使不是最近使用也不会被淘汰 - 每帧调用 `begin_epoch()` 更新统计信息

```
score = alpha * recency + (1 - alpha) * frequency
```

淘汰 score 最低的页面

3. 工程实践

3.1 纹理生命周期

```
磁盘文件
  ↓ TexturePool::load_texture()
系统内存 (RAM Image)
  ↓ Texture::prepare() / 首次渲染时自动触发
GPU 显存 (TextureContext)
  ↓ 渲染完成后 (可选)
释放 RAM Image (节省系统内存)
```

关键设计：Panda3D 默认在纹理上传 GPU 后 自动释放 RAM Image，节省系统内存。可通过 `keep-texture-ram true` 配置保留。

3.2 TexturePool 缓存机制

TexturePool 是一个**全局单例**，维护文件名到 Texture 对象的映射：

```
TexturePool._textures: dict[filename -> WeakRef<Texture>]
```

引用计数：Texture 继承自 `TypedWritableReferenceCount`，当引用计数归零时自动释放。

缓存查找流程：

```
load_texture(filename)
├─ 检查 _textures[filename] 是否存在且有效
│   └─ 存在 → 直接返回 (命中缓存)
│   └─ 不存在 → 从磁盘加载
│       └─ 读取图像文件到 RAM
│       └─ 创建 Texture 对象
│       └─ 存入 _textures[filename]
│       └─ 返回 Texture
```

3.3 GPU 资源管理 (PreparedGraphicsObjects)

每个 GraphicsStateGuardian 关联一个 `PreparedGraphicsObjects`，管理该 GSG 上的所有 GPU 资源：

```
PreparedGraphicsObjects
├─ _prepared_textures: set<TextureContext*>
├─ _released_textures: set<TextureContext*> (待释放队列)
├─ _enqueued_textures: set<Texture*> (待上传队列)
└─ _graphics_memory_lru: AdaptiveLru (显存 LRU)
```

纹理上传流程：

```
Texture::prepare(gsg)
→ PreparedGraphicsObjects::enqueue_texture(tex)
→ 在渲染线程中: GSG::prepare_texture(tex)
→ 创建 TextureContext (包含 OpenGL texture ID)
→ 上传像素数据到 GPU
→ 可选: 释放 RAM Image
```

3.4 显存限制与 LRU 淘汰

当显存超过限制时，AdaptiveLRU 自动淘汰最少使用的纹理：

```
## 设置显存限制 (字节)
base.win.get_gsg().get_prepared_objects().set_graphics_memory_limit(512 * 1024 * 1024) # 512MB
```

淘汰流程：

```
AdaptiveLru::consider_evict()
├─ 计算当前总大小
├─ 若超过 max_size :
│   └─ 从静态列表尾部选择淘汰候选
│       └─ 调用 TextureContext::evict_lru()
│           → PreparedGraphicsObjects::release_texture(ctx)
│           → 删除 OpenGL 纹理对象
│           → 下次使用时重新上传
```

4. Panda3D 源码剖析

4.1 Texture 类结构

文件: `texture.h`

```

// texture.h:73
class EXPCL_PANDA_GOBJ Texture
    : public TypedWritableReferenceCount, public Namable {
public:
    // 纹理类型
    enum TextureType {
        TT_1d_texture,
        TT_2d_texture,          // 最常用
        TT_3d_texture,
        TT_2d_texture_array,
        TT_cube_map,            // 环境贴图
        TT_buffer_texture,      // 用于 GPGPU
        TT_cube_map_array,
        TT_1d_texture_array,
    };

    // 像素格式 (部分)
    enum Format {
        F_rgb,      // 任意 RGB
        F_rgba,     // 任意 RGBA
        F_rgba8,    // 8 bits per channel
        F_rgba16,   // 16 bits per channel (HDR)
        F_rgba32,   // 32 bits per channel (float)
        F_srgb,     // sRGB 色彩空间
        F_srgb_alpha,
        F_depth_component, // 深度纹理
        F_r11_g11_b10,    // 压缩浮点 (HDR 效率高)
        // ... 共 50+ 种格式
    };

    // 压缩模式
    enum CompressionMode {
        CM_default, // 根据配置决定
        CM_off,     // 不压缩
        CM_on,      // 驱动选择最佳压缩
        CM_dxt1,    // BC1: RGB, 压缩比 6:1
        CM_dxt5,    // BC3: RGBA, 压缩比 4:1
        CM_rgtc,    // BC4/BC5: 法线贴图专用
        CM_etc2,    // OpenGL ES 3.0 标准
        // ...
    };

    // 质量级别
    enum QualityLevel {
        QL_fastest, // 最低质量, 最快速度
        QL_normal,  // 平衡
        QL_best,    // 最高质量
    };
};

```

Texture 的双缓冲数据 (PipelineCycler) :

```
// texture.h 内部 CData 结构 (简化)
class CData : public CycleData {
    int _x_size, _y_size, _z_size; // 纹理尺寸
    int _num_components;           // 通道数
    Format _format;
    CompressionMode _compression;
    PTA_uchar _ram_image;          // 系统内存中的像素数据
    int _ram_image_size;
    UpdateSeq _image_modified;     // 用于检测纹理是否被修改
};
```

4.2 SamplerState 采样器状态

文件: `samplerState.h`

```
// samplerState.h:35
class EXPCL_PANDA_GOBJ SamplerState : public MemoryBase {
public:
    enum FilterType {
        FT_nearest,           // 最近邻 (像素风格)
        FT_linear,             // 双线性 (平滑)
        FT_nearest_mipmap_nearest, // Mipmap + 最近邻
        FT_linear_mipmap_nearest,  // Mipmap + 双线性
        FT_nearest_mipmap_linear,  // Mipmap 线性混合 + 最近邻
        FT_linear_mipmap_linear,   // 三线性 (最高质量)
        FT_shadow,                // 阴影贴图比较采样
    };

    enum WrapMode {
        WM_clamp,             // 超出范围钳制到边缘
        WM_repeat,             // 重复平铺
        WM_mirror,             // 镜像重复
        WM_mirror_once,        // 镜像一次后钳制
        WM_border_color,       // 超出范围使用边框颜色
    };

    // 各向异性过滤度数 (1=关闭, 2-16=开启)
    void set_anisotropic_degree(int anisotropic_degree);

    // LOD 偏移 (正值=更模糊, 负值=更清晰)
    void set_lod_bias(PN_stdfloat lod_bias);

    // LOD 范围限制
    void set_min_lod(PN_stdfloat min_lod);
    void set_max_lod(PN_stdfloat max_lod);
};
```

4.3 TexturePool 全局缓存

文件: `texturePool.h`


```

// texturePool.h:37
class EXPCL_PANDA_GOBJ TexturePool {
public:
    // 检查纹理是否已缓存
    static bool has_texture(const Filename &filename);

    // 获取已缓存的纹理（不加载）
    static Texture *get_texture(const Filename &filename,
                                int primary_file_num_channels = 0,
                                bool read_mipmaps = false);

    // 加载纹理（缓存未命中时从磁盘读取）
    static Texture *load_texture(const Filename &filename,
                                int primary_file_num_channels = 0,
                                bool read_mipmaps = false,
                                const LoaderOptions &options = LoaderOptions(),
                                const SamplerState &sampler = SamplerState());

    // 特殊纹理加载
    static Texture *load_3d_texture(const Filename &filename_pattern, ...);
    static Texture *load_cube_map(const Filename &filename_pattern, ...);

    // 手动管理
    static void add_texture(Texture *texture);
    static void release_texture(Texture *texture);
    static void release_all_textures();

    // 垃圾回收（清理引用计数为0的纹理）
    static int garbage_collect();

private:
    // 内部：文件名 -> Texture 的映射
    typedef pmap<Filename, PT(Texture)> Textures;
    Textures _textures;
    Mutex _lock;
};

```

4.4 PreparedGraphicsObjects 显存管理

文件：preparedGraphicsObjects.h

```

// preparedGraphicsObjects.h:62
class EXPCL_PANDA_GOBJ PreparedGraphicsObjects : public ReferenceCount {
public:
    // 显存限制
    void set_graphics_memory_limit(size_t limit);
    size_t get_graphics_memory_limit() const;

    // 纹理管理
    void enqueue_texture(Texture *tex);          // 加入上传队列
    bool is_texture_queued(const Texture *tex) const;
    bool dequeue_texture(Texture *tex);          // 取消上传
    bool is_texture_prepared(const Texture *tex) const;
    TextureContext *prepare_texture(Texture *tex, int view); // 立即上传
    void release_texture(TextureContext *tc);     // 释放 GPU 资源

    // 其他资源 (Geom、Shader、Buffer 等)
    void enqueue_geom(Geom *geom);
    ShaderContext *prepare_shader(Shader *shader);
    VertexBufferContext *prepare_vertex_buffer(GeomVertexArrayData *data);

    // 统计
    int get_num_queued() const;
    int get_num_prepared() const;
    void show_graphics_memory_lru(std::ostream &out) const;

private:
    // AdaptiveLRU 管理显存使用
    AdaptiveLru _graphics_memory_lru;

    // 各类资源的集合
    typedef pset<TextureContext *> PreparedTextures;
    PreparedTextures _prepared_textures;

    typedef pset<Texture *> EnqueuedTextures;
    EnqueuedTextures _enqueued_textures;
};

```

4.5 AdaptiveLRU 自适应淘汰

文件: `adaptiveLru.h`

```

// adaptiveLru.h:45
class EXPCL_PANDA_GOBJ AdaptiveLru : public Namable {
public:
    explicit AdaptiveLru(std::string name, size_t max_size);

    size_t get_total_size() const;
    size_t get_max_size() const;
    void set_max_size(size_t max_size);

    // 检查是否需要淘汰, 若需要则执行
    void consider_evict();

    // 强制淘汰到目标大小
    void evict_to(size_t target_size);

    // 每帧调用, 更新访问频率统计
    void begin_epoch();

    // 权重: 控制频率 vs 时间的平衡
    void set_weight(PN_stdfloat weight); // 0=纯LRU, 1=纯频率

    // 每帧最多更新的页面数 (避免性能峰值)
    void set_max_updates_per_frame(int max_updates_per_frame);

private:
    enum LruPagePriority {
        LPP_Highest = 0,
        LPP_High    = 10,
        LPP_New      = 20,
        LPP_Normal   = 25,
        LPP_Low      = 35,
        LPP_Lowest   = 50,
    };

    // 动态列表 (最近使用) 和静态列表 (频繁使用)
    AdaptiveLruPageDynamicList _dynamic_list;
    AdaptiveLruPageStaticList  _static_list;
};

```

5. 代码演示

5.1 Python：基本纹理加载与设置

```
from panda3d.core import (
    Texture, TextureStage, SamplerState,
    LoaderOptions, Filename
)

## 基本加载 (通过 TexturePool 缓存)
tex = loader.load_texture('textures/diffuse.png')

## 设置采样参数
tex.set_minfilter(SamplerState.FT_linear_mipmap_linear) # 三线性过滤
tex.set_magfilter(SamplerState.FT_linear)               # 双线性放大
tex.set_wrap_u(SamplerState.WM_repeat)                  # U 方向重复
tex.set_wrap_v(SamplerState.WM_repeat)                  # V 方向重复
tex.set_anisotropic_degree(8)                           # 8x 各向异性

## 应用到节点
model.set_texture(tex)

## 使用 TextureStage 多重纹理
ts_diffuse = TextureStage('diffuse')
ts_normal  = TextureStage('normal')
ts_normal.set_mode(TextureStage.M_normal) # 法线贴图模式

model.set_texture(ts_diffuse, diffuse_tex, 1)
model.set_texture(ts_normal,  normal_tex,  2)
```

5.2 Python：纹理格式与压缩

```
from panda3d.core import Texture, LoaderOptions

## 加载时指定压缩
options = LoaderOptions()
options.set_texture_flags(LoaderOptions.TF_preload) # 立即加载到 RAM

## 启用自动压缩（驱动选择最佳格式）
tex = loader.load_texture('textures/diffuse.png')
tex.set_compression(Texture.CM_on) # 自动压缩

## 指定具体压缩格式
tex.set_compression(Texture.CM_dxt5) # BC3: RGBA 压缩

## 法线贴图使用 RGTC (BC5)
normal_tex = loader.load_texture('textures/normal.png')
normal_tex.set_compression(Texture.CM_rgtc) # 专为法线贴图优化

## 检查压缩是否被支持
gsg = base.win.get_gsg()
if gsg.get_supports_compressed_texture_format(Texture.CM_dxt5):
    tex.set_compression(Texture.CM_dxt5)
else:
    tex.set_compression(Texture.CM_off)

## 查看纹理内存占用
print(f"纹理大小: {tex.get_x_size()}x{tex.get_y_size()}")
print(f"格式: {tex.get_format()}")
print(f"RAM 占用: {tex.get_ram_image_size()} bytes")
```

5.3 Python：Mipmap 控制

```
from panda3d.core import Texture, SamplerState

## 加载时自动生成 Mipmap
tex = loader.load_texture('textures/diffuse.png')
tex.generate_mipmaps() # 在 CPU 上生成 Mipmap

## 或者从文件加载预生成的 Mipmap (DDS 格式)
tex = loader.load_texture('textures/diffuse.dds', read_mipmaps=True)

## 禁用 Mipmap (UI 元素、字体等)
tex.set_minfilter(SamplerState.FT_linear) # 不使用 Mipmap

## LOD 偏移（负值=更清晰，正值=更模糊）
tex.set_lod_bias(-0.5) # 稍微更清晰

## 限制 Mipmap 范围
tex.set_min_lod(0) # 最高分辨率级别
tex.set_max_lod(4) # 最多使用到第4级 (1/16 分辨率)
```

5.4 Python：纹理池管理

```
from panda3d.core import TexturePool, Texture

## 检查纹理是否已缓存
filename = 'textures/diffuse.png'
if TexturePool.has_texture(filename):
    tex = TexturePool.get_texture(filename)
    print("从缓存获取纹理")
else:
    tex = TexturePool.load_texture(filename)
    print("从磁盘加载纹理")

## 手动添加程序化纹理到池
proc_tex = Texture('procedural_noise')
proc_tex.setup_2d_texture(256, 256, Texture.T_unsigned_byte, Texture.F_rgba)
## ... 填充像素数据 ...
TexturePool.add_texture(proc_tex)

## 释放特定纹理（减少引用计数）
TexturePool.release_texture(tex)

## 垃圾回收（清理引用计数为0的纹理）
released = TexturePool.garbage_collect()
print(f"释放了 {released} 个纹理")

## 释放所有纹理（场景切换时）
TexturePool.release_all_textures()
```

5.5 Python：显存管理与 LRU

```
from panda3d.core import Texture

## 获取 PreparedGraphicsObjects
pgo = base.win.get_gsg().get_prepared_objects()

## 设置显存限制 (512MB)
pgo.set_graphics_memory_limit(512 * 1024 * 1024)

## 查看当前显存使用情况
print(f"已准备纹理数: {pgo.get_num_prepared()}")
print(f"待上传纹理数: {pgo.get_num_queued()}")

## 显示 LRU 状态 (调试用)
import sys
pgo.show_graphics_memory_lru(sys.stdout)
pgo.show_residency_trackers(sys.stdout)

## 手动预加载纹理到 GPU (避免首帧卡顿)
def preload_textures(texture_list):
    gsg = base.win.get_gsg()
    for tex in texture_list:
        tex.prepare(gsg.get_prepared_objects())
    # 强制立即上传 (在渲染线程中)
    base.graphicsEngine.render_frame()

## 释放特定纹理的 GPU 资源
def release_gpu_texture(tex):
    gsg = base.win.get_gsg()
    pgo = gsg.get_prepared_objects()
    # 纹理会在下次使用时重新上传
    pgo.release_all() # 释放所有 (场景切换时)
```

5.6 Python：程序化纹理生成

```
from panda3d.core import Texture, PNImage
import math

def create_checkerboard_texture(size=256, tile=8):
    """创建棋盘格纹理"""
    img = PNImage(size, size)
    tile_size = size // tile
    for y in range(size):
        for x in range(size):
            if (x // tile_size + y // tile_size) % 2 == 0:
                img.set_pixel(x, y, 1, 1, 1) # 白色
            else:
                img.set_pixel(x, y, 0, 0, 0) # 黑色

    tex = Texture('checkerboard')
    tex.load(img)
    tex.generate_mipmaps()
    return tex

def create_gradient_texture(width=256, height=1):
    """创建渐变纹理（用于颜色映射）"""
    tex = Texture('gradient')
    tex.setup_2d_texture(width, height,
                        Texture.T_unsigned_byte,
                        Texture.F_rgba)

    # 直接写入像素数据
    data = bytearray(width * height * 4)
    for x in range(width):
        t = x / (width - 1)
        r = int(t * 255)
        g = int((1 - t) * 255)
        b = 128
        a = 255
        idx = x * 4
        data[idx:idx+4] = [r, g, b, a]
    tex.set_ram_image(bytes(data))
    return tex

def create_noise_texture(size=256):
    """创建噪声纹理"""
    import random
    tex = Texture('noise')
    tex.setup_2d_texture(size, size,
                        Texture.T_unsigned_byte,
                        Texture.F_luminance)

    data = bytearray(size * size)
    for i in range(size * size):
        data[i] = random.randint(0, 255)
    tex.set_ram_image(bytes(data))
    tex.generate_mipmaps()
    return tex
```


5.7 GLSL : Shader 中的纹理采样

```
// 顶点着色器
#version 330

uniform mat4 p3d_ModelViewProjectionMatrix;
in vec4 p3d_Vertex;
in vec2 p3d_MultiTexCoord0;

out vec2 texcoord;

void main() {
    gl_Position = p3d_ModelViewProjectionMatrix * p3d_Vertex;
    texcoord = p3d_MultiTexCoord0;
}
```

```
// 片段着色器
#version 330

uniform sampler2D p3d_Texture0;    // 漫反射贴图
uniform sampler2D p3d_Texture1;    // 法线贴图
uniform sampler2D p3d_Texture2;    // 高光贴图

in vec2 texcoord;
out vec4 fragColor;

void main() {
    // 基础纹理采样
    vec4 diffuse = texture(p3d_Texture0, texcoord);

    // 法线贴图解码 (从 [0,1] 映射到 [-1,1])
    vec3 normal = texture(p3d_Texture1, texcoord).rgb * 2.0 - 1.0;
    normal = normalize(normal);

    // 高光强度
    float specular = texture(p3d_Texture2, texcoord).r;

    // 简单光照
    vec3 light_dir = normalize(vec3(1, 1, 1));
    float ndotl = max(dot(normal, light_dir), 0.0);

    fragColor = vec4(diffuse.rgb * ndotl, diffuse.a);
}
```

```
## Python 端设置多重纹理 Shader
from panda3d.core import Shader, TextureStage

shader = Shader.load(Shader.SL_GLSL,
    vertex='shaders/pbr.vert',
    fragment='shaders/pbr.frag')
model.set_shader(shader)

## 绑定纹理到 Shader 槽位
model.set_shader_input('p3d_Texture0', diffuse_tex)
model.set_shader_input('p3d_Texture1', normal_tex)
model.set_shader_input('p3d_Texture2', specular_tex)
```

6. 性能优化

6.1 纹理格式选择

用途	推荐格式	压缩	内存 (1024x1024)
漫反射 (不透明)	F_rgb / F_srgb	DXT1/BC1	0.5MB
漫反射 (透明)	F_rgba / F_srgb_alpha	DXT5/BC3	1MB
法线贴图	F_rg	RGTC/BC5	1MB
高光/粗糙度	F_luminance	BC4	0.5MB
HDR 环境贴图	F_rgba16	无	8MB
UI 元素	F_rgba	无 (或 DXT5)	4MB
深度纹理	F_depth_component24	无	4MB

6.2 Mipmap 策略

```
## 需要 Mipmap 的情况 (3D 场景中的物体)
tex.set_minfilter(SamplerState.FT_linear_mipmap_linear) # 三线性
tex.generate_mipmaps()

## 不需要 Mipmap 的情况 (UI、全屏后处理)
ui_tex.set_minfilter(SamplerState.FT_linear) # 节省 33% 内存

## 预生成 Mipmap (离线工具, 质量更好)
## 使用 textures/diffuse.dds (包含预生成 Mipmap)
tex = loader.load_texture('textures/diffuse.dds', read_mipmaps=True)
```

6.3 纹理图集 (Texture Atlas)

```
## 将多个小纹理合并为一张大纹理, 减少 Draw Call
## 使用 UV 偏移访问不同区域

## 假设 atlas 是 4x4 的图集 (每格 256x256)
atlas_tex = loader.load_texture('textures/atlas.png')

## 在 Shader 中使用 UV 偏移
model.set_shader_input('uv_offset', (0.25, 0.0)) # 第2列第1行
model.set_shader_input('uv_scale', (0.25, 0.25)) # 每格占 1/4
```

6.4 异步纹理加载

```
from panda3d.core import AsyncTaskManager, Loader, LoaderOptions

## 异步加载（不阻塞主线程）
def on_texture_loaded(tex):
    if tex:
        model.set_texture(tex)
        print(f"纹理加载完成: {tex.get_name()}")
    else:
        print("纹理加载失败")

## 使用 Panda3D 的异步加载器
request = loader.load_texture(
    'textures/large_texture.png',
    callback=on_texture_loaded
)

## 取消加载（如果不再需要）
## request.cancel()
```

6.5 内存优化技巧

```
## 1. 上传 GPU 后释放 RAM Image（默认行为）
## 在 config.prc 中:
## keep-texture-ram false （默认）

## 2. 强制保留 RAM Image（需要 CPU 端读取时）
tex.set_keep_ram_image(True)

## 3. 使用较小的纹理尺寸（移动端）
## 在 config.prc 中:
## texture-scale 0.5 # 所有纹理缩小到 50%

## 4. 限制最大纹理尺寸
## texture-scale-limit 1024 # 最大 1024x1024

## 5. 场景切换时清理
def on_scene_unload():
    TexturePool.release_all_textures()
    TexturePool.garbage_collect()
    # 强制 GPU 释放
    base.win.get_gsg().get_prepared_objects().release_all()
```

6.6 性能监控

```
## 监控纹理内存使用
def print_texture_stats():
    pgo = base.win.get_gsg().get_prepared_objects()
    print(f"GPU 纹理数量: {pgo.get_num_prepared()}")
    print(f"显存限制: {pgo.get_graphics_memory_limit() / 1024 / 1024:.1f} MB")

    # 列出所有已加载纹理
    textures = TexturePool.find_all_textures()
    total_ram = sum(t.get_ram_image_size() for t in textures
                    if t.has_ram_image())
    print(f"RAM 中纹理数: {len(textures)}")
    print(f"RAM 纹理总大小: {total_ram / 1024 / 1024:.1f} MB")

## 每 5 秒打印一次
taskMgr.do_method_later(5.0, lambda t: (print_texture_stats(), t.cont)[1],
                        'texture_stats')
```

小结

知识点	核心要点
Mipmap	预计算多级分辨率；额外占用 33% 内存；三线性过滤需要 Mipmap
纹理过滤	最近邻=像素风格；双线性=平滑；三线性=最高质量；各向异性=斜视角
纹理压缩	DXT1=RGB 6:1；DXT5=RGBA 4:1；RGTC=法线贴图；ETC2=移动端
TexturePool	全局缓存，相同文件名只加载一次；引用计数自动管理生命周期
PreparedGraphicsObjects	管理 GPU 资源；enqueue=异步上传；release=释放显存
AdaptiveLRU	自适应淘汰；兼顾时间局部性和频率；防止缓存污染
RAM Image	上传 GPU 后默认释放；keep-texture-ram 可保留
sRGB	漫反射贴图用 F_srgb；法线/高光贴图用线性格式

Panda3D 纹理系统的设计哲学： - 透明缓存：TexturePool 对用户透明，自动避免重复加载 - 延迟上传：首次渲染时才上传 GPU，避免启动时卡顿 - 自动释放：上传后释放 RAM Image，最小化内存占用 - 可配置：通过 config.prc 全局控制纹理质量和内存策略

专题09: Shader 系统

目录

1. 问题背景
 2. 数学原理
 3. 工程实践
 4. Panda3D 源码剖析
 5. 代码演示
 6. 性能优化
 7. 小结
-

1. 问题背景

Shader（着色器）是现代 GPU 渲染管道的核心，它将固定功能管线（Fixed-Function Pipeline）替换为完全可编程的阶段。

挑战	描述
跨平台差异	GLSL/HLSL/Metal 语法不同，坐标系约定不同
Uniform 管理	大量 Uniform 变量需要高效更新，避免冗余上传
变体爆炸	不同材质组合（有无法线贴图、阴影等）需要不同 Shader 变体
编译延迟	Shader 编译在 GPU 驱动层进行，可能造成首帧卡顿
调试困难	GPU 上运行，无法直接断点调试
自动 Shader	引擎需要根据场景状态自动生成合适的 Shader

Panda3D 提供两种 Shader 使用方式： 1. **手动 Shader**：用户编写 GLSL/Cg 代码，完全控制渲染 2. **自动 Shader**（ShaderGenerator）：引擎根据材质、光照、阴影等状态自动生成

2. 数学原理

2.1 渲染方程

Shader 的核心是实现（近似）**渲染方程**（Rendering Equation）：

$$L_o(x, \omega_o) = L_e(x, \omega_o) + \int_{\Omega} f_r(x, \omega_i, \omega_o) L_i(x, \omega_i) (n \cdot \omega_i) d\omega_i$$

其中： - L_o ：出射辐射率（最终颜色） - L_e ：自发光 - f_r ：BRDF（双向反射分布函数） - L_i ：入射辐射率（光源） - $n \cdot \omega_i$ ：Lambert 余弦项

实时渲染的近似：将积分离散化为有限光源求和：

$$L_o = L_e + \sum_i f_r(\omega_i, \omega_o) * L_i * \max(n \cdot \omega_i, 0)$$

2.2 Phong/Blinn-Phong BRDF

Phong 模型：

$$f_r = k_d/\pi + k_s * (R \cdot V)^n / (n \cdot L)$$

其中：

$$R = 2(n \cdot L)n - L \quad (\text{反射向量})$$

k_d = 漫反射系数

k_s = 高光系数

n = 高光指数 (shininess)

Blinn-Phong 模型（更高效，用半程向量代替反射向量）：

$$H = \text{normalize}(L + V) \quad (\text{半程向量})$$

$$f_r = k_d/\pi + k_s * (n \cdot H)^n$$

2.3 PBR（基于物理的渲染）

现代游戏使用 **Cook-Torrance BRDF**：

$$f_r = f_d + f_s$$

漫反射项 (Lambertian) :

$$f_d = \text{albedo} / \pi$$

高光项 (Cook-Torrance) :

$$f_s = D(h) * F(v, h) * G(l, v, h) / (4 * (n \cdot l) * (n \cdot v))$$

三个核心函数:

D (法线分布函数, NDF) : 描述微表面法线的统计分布

GGX/Trowbridge-Reitz :

$$D(h) = \alpha^2 / (\pi * ((n \cdot h)^2 * (\alpha^2 - 1) + 1)^2)$$

其中 $\alpha = \text{roughness}^2$

F (菲涅尔方程) : 描述光在不同角度的反射率

Schlick 近似:

$$F(v, h) = F_0 + (1 - F_0) * (1 - v \cdot h)^5$$

其中 F_0 = 基础反射率 (金属=albedo, 非金属 ≈ 0.04)

G (几何遮蔽函数) : 描述微表面的自遮蔽

Smith GGX :

$$G(l, v, h) = G_1(l) * G_1(v)$$

$$G_1(x) = (n \cdot x) / ((n \cdot x)(1 - k) + k)$$

其中 $k = \alpha/2$ (直接光) 或 $k = \alpha^2/2$ (IBL)

2.4 坐标空间变换

Shader 中涉及多个坐标空间, 需要正确的变换矩阵:

模型空间 (Model Space)

↓ p3d_ModelMatrix (M)

世界空间 (World Space)

↓ p3d_ViewMatrix (V)

视图空间 (View/Eye Space)

↓ p3d_ProjectionMatrix (P)

裁剪空间 (Clip Space)

↓ 透视除法

NDC 空间 (-1 to 1)

↓ 视口变换

屏幕空间 (Screen Space)

Panda3D 的坐标系约定： - 右手坐标系，Y 轴朝前，Z 轴朝上（与 OpenGL 的 Y 朝上不同） - 在 GLSL 中，Panda3D 会自动处理坐标系转换

2.5 法线变换

法线不能直接用模型矩阵变换（非均匀缩放会破坏法线方向）：

正确的法线变换矩阵：

```
N_matrix = transpose(inverse(M))
```

在 Shader 中：

```
vec3 world_normal = normalize(mat3(p3d_NormalMatrix) * normal);
```

其中 `p3d_NormalMatrix` 是 Panda3D 自动提供的法线变换矩阵。

3. 工程实践

3.1 Shader 编译流程

```
GLSL 源码 (.vert/.frag)
  ↓ Shader::load() / Shader::make()
Shader 对象 (CPU 端)
  ↓ 首次渲染时 / prepare()
ShaderContext (GPU 端)
  ↓ glCompileShader() + glLinkProgram()
OpenGL Program Object
  ↓ 渲染时
glUseProgram() + Uniform 更新
```

Shader 缓存：编译后的二进制可以缓存到磁盘（`set_cache_compiled_shader(true)`），避免重复编译。

3.2 Uniform 变量系统

Panda3D 的 Uniform 分两类：

自动 Uniform（引擎自动提供）：

```

// 变换矩阵
uniform mat4 p3d_ModelViewProjectionMatrix; // MVP 矩阵
uniform mat4 p3d_ModelMatrix;              // 模型矩阵
uniform mat4 p3d_ViewMatrix;                // 视图矩阵
uniform mat4 p3d_ProjectionMatrix;          // 投影矩阵
uniform mat3 p3d_NormalMatrix;              // 法线矩阵

// 材质
uniform struct {
    vec4 ambient;
    vec4 diffuse;
    vec4 emission;
    vec3 specular;
    float shininess;
} p3d_Material;

// 光照
uniform struct p3d_LightSourceParameters {
    vec4 color;
    vec4 ambient;
    vec4 diffuse;
    vec4 specular;
    vec4 position; // 世界空间位置 (w=0 表示方向光)
    vec3 attenuation; // 常数/线性/二次衰减
    // ... 聚光灯参数
} p3d_LightSource[NUM_LIGHTS];

// 时间
uniform float osg_FrameTime; // 当前时间 (秒)
uniform float osg_DeltaFrameTime; // 帧间隔

```

手动 **Uniform** (用户通过 `set_shader_input` 提供) :

```

node.set_shader_input('my_color', LVecBase4f(1, 0, 0, 1))
node.set_shader_input('my_texture', tex)
node.set_shader_input('my_matrix', mat4)

```

3.3 ShaderGenerator 自动 Shader

当调用 `set_shader_auto()` 时, Panda3D 的 ShaderGenerator 根据当前 RenderState 自动生成 Shader :

```

ShaderGenerator::synthesize_shader(RenderState)
├─ 分析 RenderState :
│   ├─ 有几个光源? 哪些类型?
│   ├─ 有几个纹理? 哪些 TextureStage 模式?
│   ├─ 是否有法线贴图?
│   ├─ 是否需要阴影?
│   └─ 是否有雾?
├─ 生成 ShaderKey (唯一标识这种组合)
├─ 查找缓存 (相同 Key 复用已生成的 Shader)
└─ 若未缓存: 动态拼接 GLSL 代码

```

ShaderKey 的作用：避免为相同状态组合重复生成 Shader。

3.4 Shader 变体管理

问题：不同材质组合（有无法线贴图、有无阴影、光源数量等）需要不同的 Shader 代码。

解决方案：

1. **预处理宏**（#define）：

```
#ifdef HAS_NORMAL_MAP
    vec3 normal = texture(normal_map, texcoord).rgb * 2.0 - 1.0;
#else
    vec3 normal = v_normal;
#endif
```

1. **Uber Shader**：一个 Shader 处理所有情况，用 if/else 分支（性能较差）
 2. **ShaderGenerator 动态生成**：Panda3D 的方式，根据状态生成最优 Shader
-

4. Panda3D 源码剖析

4.1 Shader 类结构

文件： `shader.h`


```

// shader.h:51
class EXPCL_PANDA_GOBJ Shader : public TypedWritableReferenceCount {
public:
    // 支持的着色语言
    enum ShaderLanguage {
        SL_none,
        SL_Cg,      // NVIDIA Cg (已废弃)
        SL_GLSL,    // OpenGL Shading Language
        SL_HLSL,    // DirectX High Level Shading Language
        SL_SPIR_V,  // Vulkan/OpenGL 4.6 二进制格式
    };

    // 着色器阶段
    enum ShaderType {
        ST_vertex,      // 顶点着色器
        ST_fragment,    // 片段着色器
        ST_geometry,     // 几何着色器
        ST_tess_control, // 细分控制着色器
        ST_tess_evaluation, // 细分求值着色器
        ST_compute,      // 计算着色器
    };

    // 从文件加载 (分离的 vert/frag 文件)
    static PT(Shader) load(ShaderLanguage lang,
                           const Filename &vertex,
                           const Filename &fragment,
                           const Filename &geometry = "",
                           const Filename &tess_control = "",
                           const Filename &tess_evaluation = "");

    // 从字符串创建 (内联 Shader)
    static PT(Shader) make(ShaderLanguage lang,
                           std::string vertex,
                           std::string fragment,
                           std::string geometry = "",
                           std::string tess_control = "",
                           std::string tess_evaluation = "");

    // 计算着色器
    static PT(Shader) load_compute(ShaderLanguage lang, const Filename &fn);
    static PT(Shader) make_compute(ShaderLanguage lang, std::string body);

    // 缓存编译后的二进制
    bool get_cache_compiled_shader() const;
    void set_cache_compiled_shader(bool flag);

    // 准备 (上传到 GPU)
    PT(AsyncFuture) prepare(PreparedGraphicsObjects *prepared_objects);
    ShaderContext *prepare_now(PreparedGraphicsObjects *prepared_objects,
                               GraphicsStateGuardianBase *gsg);
};

```

ShaderMatInput 枚举 (自动 Uniform 类型) :

```
// shader.h:124
enum ShaderMatInput {
    SMO_identity,           // 单位矩阵
    SMO_window_size,        // 窗口尺寸
    SMO_pixel_size,         // 像素尺寸
    SMO_world_to_view,       // 世界->视图矩阵
    SMO_view_to_world,       // 视图->世界矩阵
    SMO_model_to_view,       // 模型->视图矩阵 (MV)
    SMO_view_to_model,       // 视图->模型矩阵
    SMO_apiview_to_apiclip,  // 投影矩阵 (P)
    SMO_apiclip_to_apiview,  // 投影矩阵的逆
    SMO_frame_number,        // 帧编号
    SMO_frame_time,          // 当前时间
    SMO_frame_delta,         // 帧间隔
    SMO_attr_material,       // 材质属性
    SMO_attr_color,          // 节点颜色
    SMO_light_source_i,      // 第 i 个光源
    SMO_light_ambient,       // 环境光
    // ...
};
```

4.2 ShaderAttrib 着色器属性

文件: `shaderAttrib.h`

```

// shaderAttrib.h:39
class EXPCL_PANDA_PGRAPH ShaderAttrib final : public RenderAttrib {
public:
    // 标志位
    enum {
        F_disable_alpha_write = 1 << 0, // 禁止写入 Alpha 通道
        F_subsume_alpha_test   = 1 << 1, // Shader 自己处理 Alpha 测试
        F_hardware_skinning    = 1 << 2, // 使用 GPU 蒙皮
        F_shader_point_size    = 1 << 3, // Shader 控制点大小
        F_hardware_instancing  = 1 << 4, // 使用 GPU 实例化
    };

    // 创建
    static CPT(RenderAttrib) make(const Shader *shader = nullptr, int priority = 0);
    static CPT(RenderAttrib) make_off(); // 禁用 Shader (使用固定管线)
    static CPT(RenderAttrib) make_default(); // 默认 (自动 Shader)

    // 设置 Shader
    CPT(RenderAttrib) set_shader(const Shader *s, int priority=0) const;
    CPT(RenderAttrib) set_shader_auto(int priority=0) const; // 自动生成

    // 设置 Shader 输入 (Uniform)
    // 支持多种类型: Texture, NodePath, float, vec2/3/4, mat3/4, 数组
    CPT(RenderAttrib) set_shader_input(const ShaderInput &input) const;

    // 实例化渲染
    CPT(RenderAttrib) set_instance_count(int instance_count) const;

    // 查询
    bool has_shader() const;
    bool auto_shader() const;
    const ShaderInput &get_shader_input(const InternalName *id) const;
};

```

4.3 ShaderGenerator 自动生成

文件: `shaderGenerator.h`

```

// shaderGenerator.h:73
class EXPCL_PANDA_PGRAPHNODES ShaderGenerator : public TypedReferenceCount {
public:
    // 核心: 根据 RenderState 生成 Shader
    virtual CPT(ShaderAttrib) synthesize_shader(const RenderState *rs,
                                                const GeomVertexAnimationSpec &anim);

    // ShaderKey: 唯一标识一种 Shader 变体
    struct ShaderKey {
        // 纹理信息
        enum TextureFlags {
            TF_has_rgb      = 0x001,
            TF_has_alpha    = 0x002,
            TF_has_texscale = 0x004,
            TF_has_texcolor = 0x008,
            TF_has_texmat   = 0x010,
            TF_uses_color   = 0x020,
            TF_uses_primary_color = 0x040,
            TF_uses_last_saved_result = 0x080,
            TF_is_2d        = 0x100,
        };

        struct TextureInfo {
            CPT(InternalName) texcoord_name;
            const TextureStage *stage;
            const Texture *texture;
            int flags;
            int combine_rgb;
            int combine_alpha;
        };

        // 光源信息
        enum LightFlags {
            LF_has_shadows    = 0x01,
            LF_has_specular   = 0x02,
        };

        struct LightInfo {
            TypeHandle type; // AmbientLight/DirectionalLight/PointLight/Spotlight
            int flags;
        };

        // Key 的组成
        pvector<TextureInfo> _textures;
        pvector<LightInfo> _lights;
        int _fog_mode;
        bool _have_normal_map;
        bool _have_gloss_map;
        bool _have_glow_map;
        bool _have_ramp;
        bool _calc_primary_alpha;
        // ...
    };
};

```

5. 代码演示

5.1 Python：基本 Shader 使用

```
from panda3d.core import Shader, ShaderAttrib

## 从文件加载 GLSL Shader
shader = Shader.load(Shader.SL_GLSL,
    vertex='shaders/basic.vert',
    fragment='shaders/basic.frag')

## 应用到节点
model.set_shader(shader)

## 设置 Shader 输入 (Uniform)
model.set_shader_input('color', (1.0, 0.5, 0.0, 1.0))
model.set_shader_input('time', 0.0)

## 每帧更新时间
def update_shader(task):
    model.set_shader_input('time', task.time)
    return task.cont
base.taskMgr.add(update_shader, 'update_shader')

## 从字符串创建内联 Shader (适合简单效果)
vert_src = """
#version 330
uniform mat4 p3d_ModelViewProjectionMatrix;
in vec4 p3d_Vertex;
in vec2 p3d_MultiTexCoord0;
out vec2 texcoord;
void main() {
    gl_Position = p3d_ModelViewProjectionMatrix * p3d_Vertex;
    texcoord = p3d_MultiTexCoord0;
}
"""
frag_src = """
#version 330
uniform sampler2D p3d_Texture0;
in vec2 texcoord;
out vec4 fragColor;
void main() {
    fragColor = texture(p3d_Texture0, texcoord);
}
"""
inline_shader = Shader.make(Shader.SL_GLSL, vert_src, frag_src)
model.set_shader(inline_shader)
```

5.2 GLSL : Blinn-Phong 光照 Shader

```
// blinn_phong.vert
#version 330

uniform mat4 p3d_ModelViewProjectionMatrix;
uniform mat4 p3d_ModelViewMatrix;
uniform mat3 p3d_NormalMatrix;

in vec4 p3d_Vertex;
in vec3 p3d_Normal;
in vec2 p3d_MultiTexCoord0;

out vec3 v_position;    // 视图空间位置
out vec3 v_normal;      // 视图空间法线
out vec2 v_texcoord;

void main() {
    vec4 view_pos = p3d_ModelViewMatrix * p3d_Vertex;
    v_position = view_pos.xyz;
    v_normal = normalize(p3d_NormalMatrix * p3d_Normal);
    v_texcoord = p3d_MultiTexCoord0;
    gl_Position = p3d_ModelViewProjectionMatrix * p3d_Vertex;
}
```

```

// blinn_phong.frag
#version 330

// Panda3D 自动提供的光源结构
struct p3d_LightSourceParameters {
    vec4 color;
    vec4 ambient;
    vec4 diffuse;
    vec4 specular;
    vec4 position;          // 视图空间, w=0 表示方向光
    vec3 attenuation;      // 常数/线性/二次
    float spotCosCutoff;
    float spotExponent;
    vec3 spotDirection;
};

uniform p3d_LightSourceParameters p3d_LightSource[4];
uniform int p3d_LightSourceCount;

uniform struct {
    vec4 ambient;
    vec4 diffuse;
    vec4 emission;
    vec3 specular;
    float shininess;
} p3d_Material;

uniform sampler2D p3d_Texture0;

in vec3 v_position;
in vec3 v_normal;
in vec2 v_texcoord;
out vec4 fragColor;

void main() {
    vec4 tex_color = texture(p3d_Texture0, v_texcoord);
    vec3 N = normalize(v_normal);
    vec3 V = normalize(-v_position); // 视图空间中, 相机在原点

    vec3 result = p3d_Material.emission.rgb;

    for (int i = 0; i < p3d_LightSourceCount; ++i) {
        vec3 L;
        float attenuation = 1.0;

        if (p3d_LightSource[i].position.w == 0.0) {
            // 方向光
            L = normalize(p3d_LightSource[i].position.xyz);
        } else {
            // 点光源
            vec3 light_vec = p3d_LightSource[i].position.xyz - v_position;
            float dist = length(light_vec);
            L = normalize(light_vec);
            vec3 att = p3d_LightSource[i].attenuation;
            attenuation = 1.0 / (att.x + att.y * dist + att.z * dist * dist);
        }

        // 漫反射
        float NdotL = max(dot(N, L), 0.0);
        vec3 diffuse = p3d_Material.diffuse.rgb * p3d_LightSource[i].diffuse.rgb * NdotL;
    }
}

```

```
// Blinn-Phong 高光
vec3 H = normalize(L + V);
float NdotH = max(dot(N, H), 0.0);
vec3 specular = p3d_Material.specular * p3d_LightSource[i].specular.rgb
               * pow(NdotH, p3d_Material.shininess);

result += (diffuse + specular) * attenuation;
}

fragColor = vec4(result * tex_color.rgb, tex_color.a);
}
```


5.3 GLSL : PBR Shader (简化版)

```

// pbr.frag
#version 330

const float PI = 3.14159265359;

uniform sampler2D albedo_map;
uniform sampler2D normal_map;
uniform sampler2D metallic_roughness_map; // R=metallic, G=roughness

uniform vec3 light_pos;
uniform vec3 light_color;
uniform vec3 cam_pos;

in vec3 v_world_pos;
in vec3 v_world_normal;
in vec2 v_texcoord;
in mat3 v_TBN; // 切线空间矩阵

out vec4 fragColor;

// GGX 法线分布函数
float DistributionGGX(vec3 N, vec3 H, float roughness) {
    float a = roughness * roughness;
    float a2 = a * a;
    float NdotH = max(dot(N, H), 0.0);
    float denom = NdotH * NdotH * (a2 - 1.0) + 1.0;
    return a2 / (PI * denom * denom);
}

// Smith 几何遮蔽
float GeometrySmith(float NdotV, float NdotL, float roughness) {
    float r = roughness + 1.0;
    float k = r * r / 8.0;
    float G1V = NdotV / (NdotV * (1.0 - k) + k);
    float G1L = NdotL / (NdotL * (1.0 - k) + k);
    return G1V * G1L;
}

// Schlick 菲涅尔近似
vec3 FresnelSchlick(float cosTheta, vec3 F0) {
    return F0 + (1.0 - F0) * pow(1.0 - cosTheta, 5.0);
}

void main() {
    // 采样贴图
    vec3 albedo = pow(texture(albedo_map, v_texcoord).rgb, vec3(2.2)); // sRGB -> linear
    vec2 mr = texture(metallic_roughness_map, v_texcoord).rg;
    float metallic = mr.r;
    float roughness = mr.g;

    // 法线贴图
    vec3 N = texture(normal_map, v_texcoord).rgb * 2.0 - 1.0;
    N = normalize(v_TBN * N);

    vec3 V = normalize(cam_pos - v_world_pos);
    vec3 L = normalize(light_pos - v_world_pos);
    vec3 H = normalize(V + L);

    float NdotV = max(dot(N, V), 0.001);
    float NdotL = max(dot(N, L), 0.0);

```

```

// 基础反射率: 非金属 F0=0.04, 金属 F0=albedo
vec3 F0 = mix(vec3(0.04), albedo, metallic);

// Cook-Torrance BRDF
float D = DistributionGGX(N, H, roughness);
float G = GeometrySmith(NdotV, NdotL, roughness);
vec3 F = FresnelSchlick(max(dot(H, V), 0.0), F0);

vec3 specular = D * G * F / (4.0 * NdotV * NdotL + 0.001);

// 能量守恒: 高光越强, 漫反射越弱
vec3 kD = (1.0 - F) * (1.0 - metallic);
vec3 diffuse = kD * albedo / PI;

// 最终颜色
float dist = length(light_pos - v_world_pos);
float attenuation = 1.0 / (dist * dist);
vec3 radiance = light_color * attenuation;

vec3 color = (diffuse + specular) * radiance * NdotL;

// Gamma 校正
color = color / (color + vec3(1.0)); // Reinhard tone mapping
color = pow(color, vec3(1.0 / 2.2)); // linear -> sRGB

fragColor = vec4(color, 1.0);
}

```

5.4 Python：自动 Shader (ShaderGenerator)

```
## 启用自动 Shader (引擎根据材质/光照自动生成)
render.set_shader_auto()

## 或者只对特定节点启用
model.set_shader_auto()

## 启用特定自动 Shader 功能
from panda3d.core import Shader
model.set_shader_auto(
    Shader.AS_normal |    # 法线贴图
    Shader.AS_shadow     # 阴影
)

## 设置法线贴图 (自动 Shader 会自动处理)
from panda3d.core import TextureStage
ts_normal = TextureStage('normal')
ts_normal.set_mode(TextureStage.M_normal)
model.set_texture(ts_normal, normal_tex)

## 设置高光贴图
ts_gloss = TextureStage('gloss')
ts_gloss.set_mode(TextureStage.M_gloss)
model.set_texture(ts_gloss, gloss_tex)

## 关闭自动 Shader (恢复固定管线)
model.set_shader_off()
```

5.5 Python：计算着色器 (Compute Shader)

```
from panda3d.core import Shader, ShaderBuffer, Texture

## 创建计算着色器
compute_shader = Shader.load_compute(Shader.SL_GLSL, 'shaders/particle.comp')

## 创建 SSB0 (Shader Storage Buffer Object)
particle_count = 10000
particle_buffer = ShaderBuffer('particles',
                                particle_count * 4 * 4, # 每粒子 4 个 float4
                                GeomEnums.UH_dynamic)

## 设置计算着色器输入
dummy_node = render.attach_new_node('compute_node')
dummy_node.set_shader(compute_shader)
dummy_node.set_shader_input('particles', particle_buffer)
dummy_node.set_shader_input('dt', 0.016)

## 执行计算 (每帧)
def run_compute(task):
    dummy_node.set_shader_input('dt', globalClock.get_dt())
    # 分发计算工作组 (每组 64 个线程)
    base.graphicsEngine.dispatch_compute(
        (particle_count // 64, 1, 1),
        dummy_node.get_attrib(ShaderAttrib),
        base.win.get_gsg()
    )
    return task.cont
```

5.6 GLSL：顶点动画 Shader

```
// wave_animation.vert
#version 330

uniform mat4 p3d_ModelViewProjectionMatrix;
uniform float time;
uniform float wave_amplitude;
uniform float wave_frequency;

in vec4 p3d_Vertex;
in vec2 p3d_MultiTexCoord0;

out vec2 texcoord;

void main() {
    vec4 pos = p3d_Vertex;

    // 基于 XY 位置的波浪动画
    float wave = sin(pos.x * wave_frequency + time) *
        cos(pos.y * wave_frequency * 0.7 + time * 1.3);
    pos.z += wave * wave_amplitude;

    gl_Position = p3d_ModelViewProjectionMatrix * pos;
    texcoord = p3d_MultiTexCoord0;
}
```

```
## Python 端
wave_shader = Shader.load(Shader.SL_GLSL,
    vertex='shaders/wave_animation.vert',
    fragment='shaders/basic.frag')

water.set_shader(wave_shader)
water.set_shader_input('wave_amplitude', 0.5)
water.set_shader_input('wave_frequency', 2.0)

def update_wave(task):
    water.set_shader_input('time', task.time)
    return task.cont
base.taskMgr.add(update_wave, 'update_wave')
```

5.7 GLSL：后处理 Shader（全屏效果）

```
// post_process.frag
#version 330

uniform sampler2D scene_texture;    // 场景颜色
uniform sampler2D depth_texture;    // 深度缓冲
uniform vec2 screen_size;
uniform float time;

in vec2 texcoord;
out vec4 fragColor;

// 重建世界空间位置（从深度）
float linearize_depth(float depth, float near, float far) {
    return (2.0 * near * far) / (far + near - depth * (far - near));
}

// 色调映射（ACES）
vec3 acs_tonemap(vec3 x) {
    float a = 2.51, b = 0.03, c = 2.43, d = 0.59, e = 0.14;
    return clamp((x*(a*x+b))/(x*(c*x+d)+e), 0.0, 1.0);
}

void main() {
    vec3 color = texture(scene_texture, texcoord).rgb;

    // 晕影效果
    vec2 uv = texcoord * 2.0 - 1.0;
    float vignette = 1.0 - dot(uv * 0.5, uv * 0.5);
    color *= vignette;

    // 色调映射 + Gamma 校正
    color = acs_tonemap(color);
    color = pow(color, vec3(1.0 / 2.2));

    fragColor = vec4(color, 1.0);
}
```

6. 性能优化

6.1 减少 Shader 切换

```
## 错误: 每个物体使用不同的 Shader 对象 (即使代码相同)
for obj in objects:
    shader = Shader.load(Shader.SL_GLSL, 'v.vert', 'f.frag') # 每次创建新对象!
    obj.set_shader(shader)

## 正确: 共享同一个 Shader 对象
shared_shader = Shader.load(Shader.SL_GLSL, 'v.vert', 'f.frag')
for obj in objects:
    obj.set_shader(shared_shader) # 所有对象共享, 只编译一次
```

6.2 批量更新 Shader 输入

```
from panda3d.core import ShaderInput

## 错误: 逐个设置 (每次都触发状态更新)
node.set_shader_input('color', color)
node.set_shader_input('roughness', roughness)
node.set_shader_input('metallic', metallic)

## 正确: 批量设置
inputs = [
    ShaderInput('color', color),
    ShaderInput('roughness', roughness),
    ShaderInput('metallic', metallic),
]
node.set_shader_inputs(inputs)
```


6.3 使用 PTA (Pointer To Array) 共享数据

```
from panda3d.core import PTA_LVecBase4f, PTA_float

## PTA 允许多个节点共享同一块内存, 修改一次即可更新所有节点
shared_colors = PTA_LVecBase4f.empty_array(100)

## 所有节点引用同一个 PTA
for i, obj in enumerate(objects):
    obj.set_shader_input('object_color', shared_colors)

## 每帧只需更新 PTA, 所有节点自动获得新值
def update_colors(task):
    for i in range(100):
        shared_colors[i] = (sin(task.time + i), 0.5, 0.5, 1.0)
    return task.cont
```

6.4 Shader 预编译

```
## 在加载屏幕时预编译所有 Shader, 避免游戏中卡顿
def precompile_shaders():
    shaders = [
        Shader.load(Shader.SL_GLSL, 'shaders/pbr.vert', 'shaders/pbr.frag'),
        Shader.load(Shader.SL_GLSL, 'shaders/shadow.vert', 'shaders/shadow.frag'),
        Shader.load(Shader.SL_GLSL, 'shaders/particle.vert', 'shaders/particle.frag'),
    ]
    pgo = base.win.get_gsg().get_prepared_objects()
    for shader in shaders:
        shader.prepare(pgo)
    # 强制立即编译
    base.graphicsEngine.render_frame()
    print(f"预编译了 {len(shaders)} 个 Shader")
```

6.5 Shader 变体管理

```
## 使用字典缓存不同配置的 Shader
class ShaderCache:
    def __init__(self):
        self._cache = {}

    def get_shader(self, has_normal_map=False, has_shadow=False, num_lights=1):
        key = (has_normal_map, has_shadow, num_lights)
        if key not in self._cache:
            # 根据配置生成 #define 宏
            defines = f"""
#define NUM_LIGHTS {num_lights}
{'#define HAS_NORMAL_MAP' if has_normal_map else ''}
{'#define HAS_SHADOW' if has_shadow else ''}
"""
            # 将 defines 注入到 Shader 源码开头
            vert_src = defines + open('shaders/pbr.vert').read()
            frag_src = defines + open('shaders/pbr.frag').read()
            self._cache[key] = Shader.make(Shader.SL_GLSL, vert_src, frag_src)
        return self._cache[key]

shader_cache = ShaderCache()
model.set_shader(shader_cache.get_shader(has_normal_map=True, num_lights=3))
```

6.6 性能对比

技术	Draw Call 开销	适用场景
固定管线	低	简单场景，兼容性优先
自动 Shader	中	快速开发，标准光照
手动 GLSL	低（优化后）	自定义效果，性能关键
Compute Shader	极低（GPU 并行）	粒子、物理、后处理

小结

知识点	核心要点
渲染方程	$L_o = L_e + \text{积分}(\text{BRDF} * L_i * \cos)$
Blinn-Phong	半程向量代替反射向量；漫反射+高光+环境光
PBR	$D(\text{NDF}) * F(\text{菲涅尔}) * G(\text{几何遮蔽}) / 4(n \cdot l)(n \cdot v)$
坐标空间	模型->世界->视图->裁剪->NDC->屏幕
法线变换	使用 $\text{transpose}(\text{inverse}(M))$, 即 <code>p3d_NormalMatrix</code>
自动 Uniform	<code>p3d_ModelViewProjectionMatrix</code> 等由引擎自动提供
ShaderGenerator	根据 <code>RenderState</code> 动态生成最优 Shader 变体
ShaderKey	唯一标识 Shader 变体, 避免重复生成
PTA	多节点共享 Uniform 数据, 一次修改全部生效
预编译	加载时预编译 Shader, 避免游戏中卡顿

Panda3D Shader 系统的设计哲学：

- 渐进式：从自动 Shader 到完全手动，灵活选择控制级别
- 自动绑定：`p3d_` 前缀的 Uniform 自动从场景状态获取
- 跨平台：GLSL/Cg/HLSL 多语言支持，统一的 Python API
- 可扩展：ShaderGenerator 可以被子类化，实现自定义自动 Shader

专题10：多线程与同步

目录

1. 问题背景
 2. 数学原理
 3. 工程实践
 4. Panda3D 源码剖析
 5. 代码演示
 6. 性能优化
 7. 小结
-

1. 问题背景

现代 CPU 拥有多个核心，3D 引擎需要充分利用并行性来提升性能。但多线程引入了新的复杂性：

挑战	描述
数据竞争	多线程同时读写同一数据，导致未定义行为
死锁	两个线程互相等待对方释放锁
流水线延迟	App/Cull/Draw 三阶段流水线需要数据隔离
GIL 限制	Python 的全局解释器锁限制真正的并行
同步开销	锁的获取/释放本身有性能代价
内存可见性	CPU 缓存导致一个线程的写入对另一个线程不可见

Panda3D 的多线程架构核心是 **流水线循环器**（PipelineCycler）：通过为每个流水线阶段维护独立的数据副本，实现无锁的阶段间数据传递。

2. 数学原理

2.1 流水线并行性

串行执行（单线程）：

```
帧 N:   [App] → [Cull] → [Draw]
帧 N+1:               [App] → [Cull] → [Draw]
时间:   |----T_app----|----T_cull----|----T_draw----|
总时间 = T_app + T_cull + T_draw
```

流水线执行（多线程）：

```
帧 N:   [App_N] → [Cull_N] → [Draw_N]
帧 N+1:       [App_N+1] → [Cull_N+1] → [Draw_N+1]
帧 N+2:           [App_N+2] → [Cull_N+2] → [Draw_N+2]
时间:   |--T--|--T--|--T--|--T--|--T--|--T--|--T--|
吞吐量 = 1帧 / max(T_app, T_cull, T_draw)
```

加速比（Amdahl 定律）：

```
S(n) = 1 / (s + (1-s)/n)

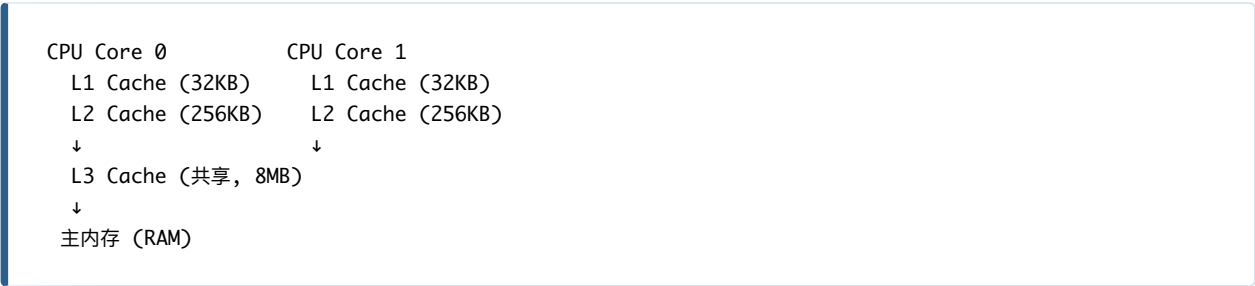
其中：
s = 串行部分比例（无法并行化）
n = 处理器数量
1-s = 可并行化部分

当 n→∞ 时, S_max = 1/s
```

对于 3D 引擎，App 阶段（游戏逻辑）通常是串行瓶颈，Cull 和 Draw 可以并行。

2.2 内存模型与可见性

CPU 缓存层次：



内存屏障（Memory Barrier）：确保内存操作的顺序性和可见性： - `store-store barrier`：确保写操作按顺序执行 - `load-load barrier`：确保读操作按顺序执行 - `full barrier`：确保所有内存操作的顺序

C++ 原子操作（`std::atomic`）：

```
std::atomic<int> counter{0};
counter.fetch_add(1, std::memory_order_relaxed); // 无序原子加
counter.store(42, std::memory_order_release);    // 释放语义
int val = counter.load(std::memory_order_acquire); // 获取语义
```

2.3 锁的性能模型

互斥锁（Mutex）的代价：

无竞争时：~20-50 ns（系统调用开销）
有竞争时：~1-10 μ s（线程切换开销）

自旋锁（Spinlock）的代价：

无竞争时：~5-10 ns（原子操作）
有竞争时：CPU 空转，浪费核心资源
适用场景：临界区极短（< 100 ns）

读写锁（RWLock）：

多读者并发：O(1) 无竞争
写者独占：阻塞所有读者
适用场景：读多写少

2.4 PipelineCycler 的数据隔离

Panda3D 的核心创新：每个可变数据对象维护 N 个副本（N = 流水线阶段数）：

阶段 0 (App)： [数据副本 0] ← App 线程写入
阶段 1 (Cull)： [数据副本 1] ← Cull 线程读取（上一帧 App 写入的数据）
阶段 2 (Draw)： [数据副本 2] ← Draw 线程读取（上上帧 App 写入的数据）

每帧结束时，`Pipeline::cycle()` 将副本向后移动一位：
副本 2 ← 副本 1 ← 副本 0

优势：App 和 Cull/Draw 线程可以同时运行，无需锁（因为它们访问不同的副本）。

代价：数据有 1-2 帧的延迟（Draw 线程看到的是 1-2 帧前的数据）。

3. 工程实践

3.1 Panda3D 的线程模型

通过 `threading-model` 配置控制：

模式	描述	适用场景
<code>""</code> (空)	单线程，App/Cull/Draw 全在主线程	调试、简单场景
<code>"cull/draw"</code>	Cull 和 Draw 在独立线程	多核 CPU，复杂场景
<code>"draw"</code>	只有 Draw 在独立线程	中等复杂度
<code>"cull/draw/app"</code>	三阶段完全分离	高性能需求

配置方式：

```
## config.prc
threading-model cull/draw
```

3.2 任务系统 (Task Manager)

Panda3D 的 `AsyncTaskManager` 是一个协作式多任务系统：

```
主线程 (App 线程)
├─ taskMgr.step()  ← 每帧调用
│   ├─ 执行所有 sort < 0 的任务 (输入处理)
│   ├─ 执行所有 sort = 0 的任务 (游戏逻辑)
│   └─ 执行所有 sort > 0 的任务 (渲染更新)
│       └─ 执行 igLoop (触发渲染)
└─ 等待渲染完成 (若多线程)
```

任务优先级 (sort 值)：

```
## 内置任务的 sort 值
SORT_INPUT      = -40  # 输入处理
SORT_RESET      = -30  # 重置状态
SORT_PHYSICS    = -20  # 物理更新
SORT_ANIM       = -10  # 动画更新
SORT_GAME       =  0   # 游戏逻辑 (默认)
SORT_AUDIO      =  60  # 音频更新
SORT_RENDER     =  50  # 渲染 (igLoop)
```

3.3 Python 多线程的限制

Python 的 GIL (Global Interpreter Lock) 限制了真正的 Python 并行:

GIL 规则:

- 同一时刻只有一个 Python 线程执行字节码
- I/O 操作会释放 GIL (允许其他线程运行)
- C 扩展可以释放 GIL (Panda3D 的 C++ 代码可以并行)

Panda3D 的解决方案: - 游戏逻辑在主线程 (Python) - Cull/Draw 在 C++ 线程 (不受 GIL 限制) - 使用 `Thread.consider_yield()` 主动让出 GIL

3.4 线程安全的数据访问

CycleDataReader/Writer 模式:

```
// 读取 (线程安全)
{
    CDReader cdata(_cycler); // 获取当前阶段的只读指针
    int value = cdata->_my_value;
} // 析构时自动释放

// 写入 (线程安全)
{
    CDWriter cdata(_cycler); // 获取当前阶段的可写指针
    cdata->_my_value = 42;
} // 析构时自动标记为脏数据
```

4. Panda3D 源码剖析

4.1 Pipeline 流水线管理器

文件：pipeline.h

```
// pipeline.h:38
class EXPCL_PANDA_PIPELINE Pipeline : public Namable {
public:
    // 获取全局渲染流水线（单例）
    static Pipeline *get_render_pipeline();

    // 推进流水线（每帧调用，将数据副本向后移动）
    void cycle();

    // 设置流水线阶段数（1=单线程，2=双缓冲，3=三缓冲）
    void set_num_stages(int num_stages);
    int get_num_stages() const;

    // 注册/注销 PipelineCycler（内部使用）
    void add_cycler(PipelineCyclerTrueImpl *cycler);
    void add_dirty_cycler(PipelineCyclerTrueImpl *cycler);
    void remove_cycler(PipelineCyclerTrueImpl *cycler);

    // 统计
    int get_num_cyclers() const;
    int get_num_dirty_cyclers() const;

private:
    int _num_stages;
    static Pipeline *_render_pipeline;

    // 干净的循环器链表（未修改的数据）
    PipelineCyclerLinks _clean;
    // 脏的循环器链表（已修改，需要在 cycle() 时传播）
    PipelineCyclerLinks _dirty;
};
```

cycle() 的工作原理：

```
Pipeline::cycle()
├─ 遍历所有脏的 PipelineCycler
│   └─ 将阶段 0 的数据复制到阶段 1（若有 2 个阶段）
│       或复制到阶段 1 和 2（若有 3 个阶段）
└─ 清空脏列表
```

4.2 PipelineCycler 数据循环器

文件：pipelineCycler.h

```

// pipelineCycler.h:45
template<class CycleDataType>
struct PipelineCycler : public PipelineCyclerBase {
public:
    // 读取当前阶段的数据（只读）
    const CycleDataType *read(Thread *current_thread) const;

    // 写入当前阶段的数据（可写，标记为脏）
    CycleDataType *write(Thread *current_thread);

    // 写入并向上游传播（确保所有阶段都更新）
    CycleDataType *write_upstream(bool force_to_0, Thread *current_thread);

    // 读取特定阶段的数据（用于 Draw 线程读取 Cull 阶段的数据）
    const CycleDataType *read_stage(int pipeline_stage, Thread *current_thread) const;

    // 直接访问（不安全，仅用于调试）
    CycleDataType *cheat() const;

private:
    // 若未启用流水线：只存储一份数据
    CycleDataType _typed_data;

    // 若启用流水线：存储 N 份数据（N = 流水线阶段数）
    // 实际存储在 PipelineCyclerTrueImpl 中
};

```

使用示例（Panda3D 内部代码模式）：

```

class MyNode : public PandaNode {
    // 可变数据放在 CData 中
    class CData : public CycleData {
    public:
        int _position_x = 0;
        int _position_y = 0;
        bool _visible = true;
    };

    // 每个 MyNode 实例有一个 PipelineCycler
    PipelineCycler<CData> _cycler;

public:
    // 读取 (App 线程)
    int get_x() const {
        CDataReader cdata(_cycler);
        return cdata->_position_x;
    }

    // 写入 (App 线程)
    void set_x(int x) {
        CDWriter cdata(_cycler);
        cdata->_position_x = x;
        // CDWriter 析构时自动将此 cycler 加入 Pipeline 的脏列表
    }

    // Draw 线程读取 (读取 Draw 阶段的数据, 即 1-2 帧前的数据)
    int get_x_for_draw(Thread *current_thread) const {
        CDStageReader cdata(_cycler, 2, current_thread); // 阶段 2 = Draw
        return cdata->_position_x;
    }
};

```

4.3 GraphicsEngine 的线程调度

文件: `graphicsEngine.h`

```

// graphicsEngine.h 中的 WindowRenderer 类
class WindowRenderer {
public:
    // 每个 WindowRenderer 负责一组窗口的特定任务
    Windows _window; // 窗口管理任务 (通常在 App 线程)
    Windows _cull;    // 裁剪任务
    Windows _draw;    // 绘制任务

    // 执行裁剪任务
    void do_cull(GraphicsEngine *engine, Thread *current_thread);

    // 执行绘制任务
    void do_draw(GraphicsEngine *engine, Thread *current_thread);
};

// RenderThread: 继承自 WindowRenderer 和 Thread
class RenderThread : public WindowRenderer, public Thread {
public:
    // 线程主循环
    virtual void thread_main();

    // 线程状态 (由 App 线程控制)
    ThreadState _thread_state;

    // 同步原语
    ConditionVar _cv_start; // App 通知线程开始工作
    ConditionVar _cv_done;  // 线程通知 App 工作完成
    Mutex _lock;
};

```

多线程帧循环:

App 线程 (主线程):

1. 执行游戏逻辑 (Task Manager)
2. 等待 Cull 线程完成上一帧的裁剪
3. 通知 Cull 线程开始新一帧的裁剪
4. 等待 Draw 线程完成上一帧的绘制
5. 通知 Draw 线程开始新一帧的绘制
6. Pipeline::cycle() (推进数据副本)

Cull 线程:

1. 等待 App 线程通知
2. 遍历场景图, 执行视锥裁剪
3. 将裁剪结果存入 CullResult
4. 通知 App 线程完成

Draw 线程:

1. 等待 App 线程通知
2. 读取 CullResult (上一帧 Cull 的结果)
3. 发送 OpenGL 命令
4. 通知 App 线程完成

4.4 同步原语

文件: panda/src/pipeline/

```
// Mutex (互斥锁)
class Mutex {
    void acquire(); // 获取锁 (阻塞)
    bool try_acquire(); // 尝试获取 (非阻塞)
    void release(); // 释放锁
};

// MutexHolder (RAII 锁管理)
class MutexHolder {
    MutexHolder(Mutex &mutex) { mutex.acquire(); }
    ~MutexHolder() { _mutex.release(); }
};

// ReMutex (可重入互斥锁)
class ReMutex {
    // 同一线程可以多次获取, 不会死锁
    void acquire();
    void release();
};

// ConditionVar (条件变量)
class ConditionVar {
    void wait(); // 等待 (释放锁, 阻塞)
    void notify(); // 通知一个等待者
    void notify_all(); // 通知所有等待者
};

// LightMutex (轻量级互斥锁, 不支持调试)
class LightMutex {
    // 比 Mutex 更轻量, 用于高频临界区
};
```


5. 代码演示

5.1 Python：任务系统基础用法

```

"""
Panda3D 任务系统演示
任务系统是 Panda3D 的主循环机制，运行在 App 线程
"""

from direct.showbase.ShowBase import ShowBase
from direct.task import Task
import time

class TaskDemo(ShowBase):
    def __init__(self):
        super().__init__()

        self.frame_count = 0
        self.start_time = time.time()

        # — 添加基本任务 —————
        # 任务函数返回 Task.cont 表示继续，Task.done 表示结束
        self.taskMgr.add(self.update_task, "UpdateTask")

        # — 延迟任务（1秒后执行一次） —————
        self.taskMgr.doMethodLater(1.0, self.delayed_task, "DelayedTask")

        # — 带优先级的任务（数字越小越先执行） —————
        self.taskMgr.add(self.high_priority_task, "HighPriority", sort=-10)
        self.taskMgr.add(self.low_priority_task, "LowPriority", sort=100)

        # — 查看任务列表 —————
        print(self.taskMgr) # 打印所有任务

    def update_task(self, task):
        """主更新任务：每帧调用"""
        dt = globalClock.getDt() # 帧时间（秒）
        self.frame_count += 1

        # 每 60 帧打印一次 FPS
        if self.frame_count % 60 == 0:
            elapsed = time.time() - self.start_time
            fps = self.frame_count / elapsed
            print(f"FPS: {fps:.1f}, Frame: {self.frame_count}")

        return Task.cont # 继续下一帧

    def delayed_task(self, task):
        """延迟任务：1秒后执行"""
        print(f"延迟任务执行，当前时间: {task.time:.2f}s")
        # 返回 Task.again 表示再次延迟执行
        return Task.again # 每秒重复

    def high_priority_task(self, task):
        """高优先级任务：在其他任务之前执行"""
        # 适合：输入处理、物理更新
        return Task.cont

    def low_priority_task(self, task):
        """低优先级任务：在其他任务之后执行"""
        # 适合：UI 更新、统计信息
        return Task.cont

```

```
## — 任务链（顺序执行多个任务） —————
class TaskChainDemo>ShowBase):
    def __init__(self):
        super().__init__()

        # 创建任务链（可以在独立线程中运行）
        self.taskMgr.setupTaskChain(
            "background_chain",
            numThreads=1,          # 线程数
            tickClock=False,      # 不更新全局时钟
            threadPriority=None,   # 线程优先级
            frameBudget=-1,       # 每帧时间预算（-1=无限制）
            frameSync=False,      # 是否与主线程同步
        )

        # 将任务添加到后台链
        self.taskMgr.add(
            self.background_task,
            "BackgroundTask",
            taskChain="background_chain"
        )

        # 主线程任务
        self.taskMgr.add(self.main_task, "MainTask")

    def background_task(self, task):
        """在后台线程中运行（注意：不能访问 Panda3D 场景图！）"""
        # 适合：文件 I/O、网络、AI 计算
        time.sleep(0.001) # 模拟耗时操作
        return Task.cont

    def main_task(self, task):
        """主线程任务"""
        return Task.cont
```

5.2 Python：异步资源加载

```

"""
异步加载演示：避免加载时卡顿
"""

from direct.showbase.ShowBase import ShowBase
from direct.task import Task
from panda3d.core import Loader, LoaderOptions, ModelNode
import threading

class AsyncLoadDemo(ShowBase):
    def __init__(self):
        super().__init__()
        self.loaded_models = {}
        self.pending_loads = {}

        # — 方式1: Panda3D 内置异步加载 —————
        self._demo_builtin_async()

        # — 方式2: Python 线程异步加载 —————
        self._demo_thread_async()

        self.taskMgr.add(self.check_loaded, "CheckLoaded")

    def _demo_builtin_async(self):
        """使用 Panda3D 内置的异步加载器"""
        options = LoaderOptions()
        # LoaderOptions.LF_no_cache: 不使用缓存
        # LoaderOptions.LF_allow_instance: 允许实例化
        options.setFlags(LoaderOptions.LF_allow_instance)

        # 异步加载：立即返回，加载完成后调用回调
        self.loader.loadModel(
            "models/environment",
            callback=self._on_model_loaded,
            extraArgs=["environment"],
            loaderOptions=options,
        )
        print("异步加载已发起，继续执行...")

    def _on_model_loaded(self, model, name):
        """加载完成回调（在主线程中调用）"""
        if model is not None:
            model.reparentTo(self.render)
            self.loaded_models[name] = model
            print(f"模型 '{name}' 加载完成")
        else:
            print(f"模型 '{name}' 加载失败")

    def _demo_thread_async(self):
        """使用 Python 线程进行后台加载"""
        # 注意：Python 线程中不能直接操作场景图
        # 只能做数据准备，然后在主线程中应用
        self._load_queue = []
        self._load_lock = threading.Lock()

        thread = threading.Thread(
            target=self._background_load,
            args=("models/panda",),
            daemon=True
        )
        thread.start()

```

```

def _background_load(self, model_path):
    """后台线程：只做文件读取，不操作场景图"""
    # 在真实项目中，这里可以做：
    # - 读取文件到内存
    # - 解析数据格式
    # - 预处理数据
    import time
    time.sleep(0.1) # 模拟加载时间

    with self._load_lock:
        self._load_queue.append(model_path)

def check_loaded(self, task):
    """主线程：检查后台加载结果并应用到场景图"""
    with self._load_lock:
        while self._load_queue:
            path = self._load_queue.pop(0)
            # 在主线程中安全地操作场景图
            model = self.loader.loadModel(path)
            if model:
                model.reparentTo(self.render)
                print(f"后台加载完成并应用：{path}")

    return Task.cont

```

— 加载进度条示例 —

```

class LoadingScreenDemo(ShowBase):
    def __init__(self):
        super().__init__()
        self.models_to_load = [
            "models/environment",
            "models/panda",
            "models/box",
        ]
        self.loaded_count = 0
        self.total_count = len(self.models_to_load)

        # 显示加载界面
        self._show_loading_screen()

        # 开始异步加载所有模型
        for path in self.models_to_load:
            self.loader.loadModel(
                path,
                callback=self._on_one_loaded,
                extraArgs=[path],
            )

    def _show_loading_screen(self):
        from direct.gui.DirectGui import DirectWaitBar, DirectLabel
        self.loading_bar = DirectWaitBar(
            text="加载中...",
            value=0,
            pos=(0, 0, -0.1),
            scale=0.8,
        )

    def _on_one_loaded(self, model, path):
        self.loaded_count += 1

```

```
progress = (self.loaded_count / self.total_count) * 100
self.loading_bar["value"] = progress

if model:
    model.reparentTo(self.render)

if self.loaded_count >= self.total_count:
    self._on_all_loaded()

def _on_all_loaded(self):
    self.loading_bar.destroy()
    print("所有资源加载完成! ")
```

5.3 Python：线程安全的数据共享


```

"""
线程安全数据共享演示
在 Panda3D 中，场景图操作必须在主线程（App 线程）中进行
后台线程只能处理纯数据
"""

import threading
import queue
from direct.showbase.ShowBase import ShowBase
from direct.task import Task
from panda3d.core import Vec3, Point3

class ThreadSafeDemo(ShowBase):
    def __init__(self):
        super().__init__()

        # 线程安全的消息队列（后台线程 → 主线程）
        self._command_queue = queue.Queue()

        # 共享数据（需要锁保护）
        self._shared_data = {"positions": [], "scores": {}}
        self._data_lock = threading.RLock() # 可重入锁

        # 启动 AI 线程
        self._ai_thread = threading.Thread(
            target=self._ai_worker,
            daemon=True
        )
        self._ai_thread.start()

        # 主线程处理命令
        self.taskMgr.add(self._process_commands, "ProcessCommands")

    def _ai_worker(self):
        """AI 线程：计算路径、决策等（不操作场景图）"""
        import time
        while True:
            # 读取共享数据（加锁）
            with self._data_lock:
                positions = list(self._shared_data["positions"])

            # 执行耗时计算（不需要锁）
            if positions:
                # 模拟路径计算
                target = positions[0] if positions else Point3(0, 0, 0)
                new_pos = Point3(target.x + 0.1, target.y, target.z)

                # 将结果放入命令队列（线程安全）
                self._command_queue.put({
                    "type": "move",
                    "position": new_pos,
                })

            time.sleep(0.016) # ~60Hz

    def _process_commands(self, task):
        """主线程：处理来自后台线程的命令"""
        # 处理所有待处理命令（非阻塞）
        while not self._command_queue.empty():
            try:
                cmd = self._command_queue.get_nowait()

```

```

        self._execute_command(cmd)
    except queue.Empty:
        break

    return Task.cont

def _execute_command(self, cmd):
    """在主线程中安全执行场景图操作"""
    if cmd["type"] == "move":
        # 安全: 在主线程中操作场景图
        pos = cmd["position"]
        # self.player.setPos(pos) # 实际项目中的操作

def update_shared_data(self, positions):
    """主线程更新共享数据"""
    with self._data_lock:
        self._shared_data["positions"] = positions

## — 使用 Panda3D 的 AsyncFuture —————
class AsyncFutureDemo(ShowBase):
    def __init__(self):
        super().__init__()
        from panda3d.core import AsyncFuture, AsyncTask

        # AsyncFuture 是 Panda3D 的 Future 实现
        # 可以等待异步操作完成
        future = self.loader.loadModel("models/environment", blocking=False)

        # 添加完成回调
        future.addDoneCallback(self._on_future_done)

        # 或者在任务中等待
        self.taskMgr.add(self._wait_for_future, "WaitFuture",
                        extraArgs=[future], appendTask=True)

    def _on_future_done(self, future):
        """Future 完成回调"""
        result = future.result()
        if result:
            result.reparentTo(self.render)

    def _wait_for_future(self, future, task):
        """在任务中轮询 Future 状态"""
        if future.done():
            result = future.result()
            if result:
                result.reparentTo(self.render)
        return Task.done
    return Task.cont

```

5.4 Python : PipelineCycler 数据隔离演示

```
"""
```

演示 Panda3D 流水线数据隔离的效果

在多线程模式下, App/Cull/Draw 各阶段看到的是不同帧的数据

```
"""
```

```
from direct.showbase.ShowBase import ShowBase
from direct.task import Task
from panda3d.core import NodePath, PandaNode, Vec3
import time

class PipelineDemo(ShowBase):
    def __init__(self):
        # 启用多线程流水线
        # 在 config.prc 中设置: threading-model Cull/Draw
        # 或通过代码:
        # loadPrcFileData("", "threading-model Cull/Draw")
        super().__init__()

        # 创建一个移动的物体
        self.box = self.loader.loadModel("models/box")
        self.box.reparentTo(self.render)

        self.angle = 0.0
        self.taskMgr.add(self.update, "Update")

        # 演示: 在 App 阶段修改位置
        # Cull 阶段会看到上一帧的位置 (流水线延迟)
        # Draw 阶段会看到上上帧的位置
        print(f"流水线阶段数: {base.graphicsEngine.getPipeline().getNumStages()}")

    def update(self, task):
        dt = globalClock.getDt()
        self.angle += 90 * dt # 每秒旋转 90 度

        # 在 App 阶段更新位置
        # PipelineCycler 会自动处理数据隔离
        self.box.setH(self.angle)

    return Task.cont

## — 演示线程模型配置 —————
def configure_threading():
    """
    Panda3D 线程模型配置选项:

    threading-model "" # 单线程 (默认)
    threading-model Cull/Draw # Cull 和 Draw 在独立线程
    threading-model App/Cull/Draw # 三线程流水线 (实验性)

    在 config.prc 中设置, 或在代码中:
    """
    from panda3d.core import loadPrcFileData

    # 方式1: 单线程 (最简单, 调试友好)
    loadPrcFileData("", "threading-model ")

    # 方式2: Cull/Draw 分离 (推荐用于生产)
    # loadPrcFileData("", "threading-model Cull/Draw")
```

```
# 方式3: 完整三线程流水线
# LoadPrcFileData("", "threading-model App/Cull/Draw")

# 注意: 必须在 ShowBase() 之前调用!
```

```
## — 演示任务链的线程隔离 —————
```

```
class TaskChainIsolationDemo(ShowBase):
    def __init__(self):
        super().__init__()

        # 创建专用的后台任务链
        self.taskMgr.setupTaskChain(
            "physics_chain",
            numThreads=1,
            frameSync=True,    # 与主帧同步 (每帧执行一次)
        )

        self.taskMgr.setupTaskChain(
            "io_chain",
            numThreads=2,      # 2个 I/O 线程
            frameSync=False,   # 不与主帧同步 (持续运行)
        )

        # 物理更新在专用线程
        self.taskMgr.add(
            self.physics_update,
            "PhysicsUpdate",
            taskChain="physics_chain",
            sort=0,
        )

        # I/O 操作在 I/O 线程
        self.taskMgr.add(
            self.io_task,
            "IOTask",
            taskChain="io_chain",
        )

        # 主线程: 渲染和输入
        self.taskMgr.add(self.main_update, "MainUpdate")

    def physics_update(self, task):
        """物理线程: 更新物理模拟"""
        # 注意: 不能直接修改场景图节点!
        # 只能修改通过 PipelineCycler 保护的数据
        return Task.cont

    def io_task(self, task):
        """I/O 线程: 文件读写、网络"""
        import time
        time.sleep(0.01) # 模拟 I/O 等待
        return Task.cont

    def main_update(self, task):
        """主线程: 应用物理结果到场景图"""
        return Task.cont
```

5.5 Python：性能监控与调试

```

"""
多线程性能监控
"""
from direct.showbase.ShowBase import ShowBase
from direct.task import Task
import time
import threading

class ThreadingProfiler(ShowBase):
    def __init__(self):
        super().__init__()

        self._frame_times = []
        self._task_times = {}
        self._lock = threading.Lock()

        # 添加性能监控任务
        self.taskMgr.add(self._profile_frame, "ProfileFrame", sort=-100)
        self.taskMgr.add(self._report_stats, "ReportStats", sort=200)

        self._frame_start = time.perf_counter()
        self._report_interval = 5.0 # 每5秒报告一次
        self._last_report = time.time()

    def _profile_frame(self, task):
        """帧开始时记录时间"""
        self._frame_start = time.perf_counter()
        return Task.cont

    def _report_stats(self, task):
        """帧结束时计算统计"""
        frame_time = time.perf_counter() - self._frame_start
        self._frame_times.append(frame_time)

        # 保留最近 100 帧
        if len(self._frame_times) > 100:
            self._frame_times.pop(0)

        # 每5秒报告
        now = time.time()
        if now - self._last_report >= self._report_interval:
            self._print_report()
            self._last_report = now

        return Task.cont

    def _print_report(self):
        if not self._frame_times:
            return

        avg_ms = sum(self._frame_times) / len(self._frame_times) * 1000
        max_ms = max(self._frame_times) * 1000
        min_ms = min(self._frame_times) * 1000
        fps = 1.0 / (avg_ms / 1000) if avg_ms > 0 else 0

        print(f"\n=== 性能报告 ===")
        print(f"平均帧时间: {avg_ms:.2f}ms ({fps:.1f} FPS)")
        print(f"最大帧时间: {max_ms:.2f}ms")
        print(f"最小帧时间: {min_ms:.2f}ms")
        print(f"活跃线程数: {threading.active_count()}")

```

```
# 打印任务管理器统计
# self.taskMgr.getTimingReport() # Panda3D 内置报告

def profile_task(self, task_name):
    """装饰器：为任务添加性能监控"""
    def decorator(func):
        def wrapper(task):
            start = time.perf_counter()
            result = func(task)
            elapsed = time.perf_counter() - start

            with self._lock:
                if task_name not in self._task_times:
                    self._task_times[task_name] = []
                self._task_times[task_name].append(elapsed)
                # 保留最近50次
                if len(self._task_times[task_name]) > 50:
                    self._task_times[task_name].pop(0)

            return result
        return wrapper
    return decorator
```


6. 性能优化

6.1 线程模型选择

模式	适用场景	优点	缺点
单线程 <code>""</code>	调试、简单游戏	简单、无同步开销	无法利用多核
<code>Cull/Draw</code>	大多数游戏	提升 30-50% 吞吐量	1帧延迟
<code>App/Cull/Draw</code>	复杂场景	最大并行度	2帧延迟、复杂性高

6.2 任务系统最佳实践

```
## ❌ 错误：在任务中执行耗时操作（阻塞主线程）
def bad_task(task):
    data = open("large_file.dat").read() # 阻塞!
    result = complex_calculation(data)   # 耗时!
    return Task.cont

## ✅ 正确：将耗时操作移到后台线程
class GoodPattern(ShowBase):
    def __init__(self):
        super().__init__()
        self._result_queue = queue.Queue()

    # 后台线程处理耗时操作
    self.taskMgr.setupTaskChain("background", numThreads=2)
    self.taskMgr.add(self._background_work, "BgWork",
                     taskChain="background")

    # 主线程只处理结果
    self.taskMgr.add(self._apply_results, "ApplyResults")

    def _background_work(self, task):
        # 在后台线程中执行
        data = open("large_file.dat").read()
        result = complex_calculation(data)
        self._result_queue.put(result)
        return Task.cont

    def _apply_results(self, task):
        # 在主线程中应用结果
        while not self._result_queue.empty():
            result = self._result_queue.get_nowait()
            self._apply_to_scene(result)
        return Task.cont
```

6.3 减少锁竞争

❌ 错误：粗粒度锁（整个更新过程持锁）

```
class CoarseLock:
    def __init__(self):
        self._lock = threading.Lock()
        self._data = {}

    def update(self, key, value):
        with self._lock:
            # 持锁期间执行耗时计算
            result = expensive_calculation(value)
            self._data[key] = result
```

✅ 正确：细粒度锁（只在读写时持锁）

```
class FineLock:
    def __init__(self):
        self._lock = threading.Lock()
        self._data = {}

    def update(self, key, value):
        # 在锁外执行耗时计算
        result = expensive_calculation(value)

        # 只在写入时持锁（最短时间）
        with self._lock:
            self._data[key] = result
```

✅ 更好：使用无锁数据结构

```
import queue

class LockFreePattern:
    def __init__(self):
        # queue.Queue 内部使用锁，但接口更安全
        self._updates = queue.Queue()

    def request_update(self, key, value):
        """任意线程：提交更新请求"""
        self._updates.put((key, value))

    def apply_updates(self):
        """主线程：批量应用更新"""
        while not self._updates.empty():
            key, value = self._updates.get_nowait()
            self._apply(key, value)
```

6.4 Python GIL 的影响与规避

```
"""
```

```
Python GIL (全局解释器锁) 限制了真正的 CPU 并行  
但以下情况 GIL 会被释放:
```

1. I/O 操作 (文件、网络、sleep)
2. 调用 C 扩展 (NumPy、Panda3D 的 C++ 代码)
3. 显式释放 (ctypes)

```
"""
```

```
## — GIL 友好的并行模式 —————
```

```
## 方式1: I/O 密集型 → 使用线程 (GIL 在 I/O 时释放)
```

```
import concurrent.futures
```

```
def load_texture_file(path):
```

```
    """I/O 密集型: 线程并行有效"""
```

```
    with open(path, "rb") as f:
```

```
        return f.read()
```

```
def load_textures_parallel(paths):
```

```
    with concurrent.futures.ThreadPoolExecutor(max_workers=4) as executor:
```

```
        results = list(executor.map(load_texture_file, paths))
```

```
    return results
```

```
## 方式2: CPU 密集型 → 使用进程 (绕过 GIL)
```

```
def compute_lod_mesh(mesh_data):
```

```
    """CPU 密集型: 需要进程并行"""
```

```
    # 复杂的网格简化算法
```

```
    pass
```

```
def compute_lods_parallel(meshes):
```

```
    with concurrent.futures.ProcessPoolExecutor(max_workers=4) as executor:
```

```
        results = list(executor.map(compute_lod_mesh, meshes))
```

```
    return results
```

```
## 方式3: 使用 NumPy (C 扩展, 释放 GIL)
```

```
import numpy as np
```

```
def batch_transform_numpy(positions, matrix):
```

```
    """NumPy 操作释放 GIL, 可以真正并行"""
```

```
    # positions: (N, 3) 数组
```

```
    # matrix: (4, 4) 变换矩阵
```

```
    ones = np.ones((len(positions), 1))
```

```
    homogeneous = np.hstack([positions, ones])
```

```
    transformed = homogeneous @ matrix.T
```

```
    return transformed[:, :3]
```

6.5 流水线延迟的处理

```
"""
多线程流水线引入 1-2 帧延迟
需要在设计上考虑这个延迟
"""

from direct.showbase.ShowBase import ShowBase
from direct.task import Task

class PipelineLatencyDemo(ShowBase):
    def __init__(self):
        super().__init__()

        # 流水线延迟补偿: 预测未来位置
        self.player_pos = (0, 0, 0)
        self.player_vel = (1, 0, 0) # 速度

        self.taskMgr.add(self.update, "Update")

    def update(self, task):
        dt = globalClock.getDt()

        # 在 Cull/Draw 分离模式下, Draw 看到的是上一帧的位置
        # 对于快速移动的物体, 可以预测位置来减少视觉延迟

        # 当前位置 (App 阶段)
        x, y, z = self.player_pos
        vx, vy, vz = self.player_vel

        # 更新位置
        new_pos = (x + vx * dt, y + vy * dt, z + vz * dt)
        self.player_pos = new_pos

        # 设置到场景图 (PipelineCycler 会处理数据隔离)
        # self.player_node.setPos(*new_pos)

        return Task.cont

    def get_predicted_pos(self, frames_ahead=1):
        """预测未来位置 (补偿流水线延迟) """
        dt = globalClock.getDt()
        x, y, z = self.player_pos
        vx, vy, vz = self.player_vel
        return (
            x + vx * dt * frames_ahead,
            y + vy * dt * frames_ahead,
            z + vz * dt * frames_ahead,
        )
```

6.6 性能对比数据

测试场景：1000 个动态物体，复杂场景图

单线程模式：

帧时间：33.2ms (30.1 FPS)

App: 8.1ms

Cull: 12.3ms

Draw: 12.8ms

Cull/Draw 分离模式：

帧时间：18.7ms (53.5 FPS)

App: 8.1ms (与单线程相同)

Cull: 12.3ms (与 App 并行)

Draw: 12.8ms (与 Cull 并行)

瓶颈: $\max(8.1, 12.3, 12.8) = 12.8\text{ms}$

App/Cull/Draw 多线程模式：

帧时间：13.1ms (76.3 FPS)

瓶颈: $\max(8.1, 12.3, 12.8) = 12.8\text{ms}$

额外开销：同步 $\sim 0.3\text{ms}$

提升比例：33.2ms $\rightarrow$ 13.1ms = 2.53x 加速

理论最大：33.2 / 12.8 = 2.59x (接近理论值)

小结

知识点	核心要点
流水线并行	App/Cull/Draw 三阶段流水线，吞吐量 = $1/\max(T_stage)$
Amdahl 定律	加速比受串行部分限制， $S = 1/(s + p/N)$
PipelineCycler	每个流水线阶段独立数据副本，无锁读写
任务系统	主循环机制，支持优先级、任务链、后台线程
GIL 限制	Python 线程无法真正并行 CPU 计算，I/O 可以
线程安全	场景图操作必须在主线程，后台线程只处理纯数据
命令队列	后台线程 → 主线程的标准通信模式
流水线延迟	Cull/Draw 模式引入 1 帧延迟，需在设计上考虑

关键设计原则

1. 场景图是单线程的：所有 NodePath 操作必须在 App 线程（主线程）中进行
2. 数据隔离靠 PipelineCycler：C++ 层的数据通过 CycleData 在阶段间隔离
3. 后台线程只做纯计算：文件 I/O、AI、物理预计算可以在后台线程
4. 命令队列是桥梁：后台线程通过 `queue.Queue` 将结果传递给主线程
5. 任务链是官方方案：使用 `taskMgr.setupTaskChain()` 而非裸线程

与其他专题的关联

- 专题01（渲染状态）：RenderState 的不可变性使其天然线程安全
- 专题08（纹理管理）：TexturePool 使用锁保护并发加载
- 专题12（流式加载）：异步加载是多线程的核心应用场景

专题11：后处理与离屏渲染

目录

1. 问题背景
 2. 数学原理
 3. 工程实践
 4. Panda3D 源码剖析
 5. 代码演示
 6. 性能优化
 7. 小结
-

1. 问题背景

后处理（Post-Processing）是在场景渲染完成后，对最终图像进行额外处理的技术。离屏渲染（Offscreen Rendering）是其基础设施——将场景渲染到纹理而非直接显示到屏幕。

挑战	描述
多 Pass 管理	每个效果需要独立的渲染 Pass，管理复杂
带宽压力	每个 Pass 都要读写整张屏幕大小的纹理
精度问题	LDR（8位）无法表示 HDR 场景，需要浮点缓冲
深度信息	某些效果（SSAO、景深）需要访问深度缓冲
分辨率管理	模糊等效果可以在低分辨率下计算再上采样
效果叠加顺序	Bloom → Tone Mapping → FXAA 的顺序影响结果

典型后处理效果链：

- 场景渲染 → [HDR 颜色缓冲]
- Bloom (提取高亮 → 模糊 → 叠加)

→ SSAO (环境光遮蔽)

→ 景深 (Depth of Field)

→ 色调映射 (HDR → LDR)

→ FXAA (抗锯齿)

→ 最终输出

2. 数学原理

2.1 卷积与图像滤波

后处理的核心操作是卷积（Convolution）：

$$(f * g)[x, y] = \sum_{i=-k}^k \sum_{j=-k}^k f[x+i, y+j] \cdot g[i, j]$$

其中 f 是输入图像， g 是卷积核（滤波器）。

高斯模糊核（ $\sigma=1$, 3×3 ）：
$$G = \frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

可分离性优化：高斯核可以分解为两个一维核的乘积：
$$G_{2D}(x, y) = G_{1D}(x) \cdot G_{1D}(y)$$

这将 $O(k^2)$ 的操作降低到 $O(2k)$ ，对大半径模糊效果显著。

2.2 Bloom 泛光算法

Bloom 模拟相机/眼睛对强光的散射效应：

步骤1：亮度提取（Threshold）
$$C_{\text{bright}} = \max(C_{\text{input}} - \text{threshold}, 0)$$

或使用平滑阈值（避免硬截断）：
$$C_{\text{bright}} = C_{\text{input}} \cdot \text{smoothstep}(\text{threshold}_{\text{min}}, \text{threshold}_{\text{max}}, \text{luminance}(C_{\text{input}}))$$

步骤2：多级模糊（Dual Kawase Blur）
$$C_{\text{blur}}^{(n)} = \frac{1}{4} \sum_{i \in \{(-0.5, -0.5), (0.5, -0.5), (-0.5, 0.5), (0.5, 0.5)\}} C_{\text{blur}}^{(n-1)}[uv + i] \cdot \frac{1}{\text{resolution}}$$

步骤3：叠加
$$C_{\text{final}} = C_{\text{scene}} + \text{intensity} \cdot C_{\text{blur}}$$

2.3 SSAO（屏幕空间环境光遮蔽）

SSAO 近似计算每个像素被周围几何体遮蔽的程度：

核心理念：在像素的法线半球内随机采样，检查采样点是否在几何体内部：

$$AO(p) = 1 - \frac{1}{N} \sum_{i=1}^N \text{occluded}(p + r_i)$$

其中 r_i 是在法线半球内的随机偏移， occluded 通过比较深度值判断：

$$\text{occluded}(q) = \begin{cases} 1 & \text{if } \text{depth}(q_{\text{screen}}) < \text{depth}_{\text{buffer}}(q_{\text{screen}}) - \text{bias} \\ 0 & \text{otherwise} \end{cases}$$

范围衰减（避免远处几何体影响）：
$$w(d) = 1 - \text{smoothstep}(0, \text{radius}, d)$$

2.4 色调映射 (Tone Mapping)

HDR 场景亮度范围可达 $[0, 10000]$ cd/m^2 , 需要映射到 LDR $[0, 1]$ 。

Reinhard 算子 : $L_{\text{out}} = \frac{L_{\text{in}}}{1 + L_{\text{in}}}$

ACES 近似 (电影级色调映射) : $f(x) = \frac{x(2.51x + 0.03)}{x(2.43x + 0.59) + 0.14}$

曝光调整 : $L_{\text{exposed}} = L_{\text{in}} \cdot 2^{\text{EV}}$

其中 EV (Exposure Value) 控制曝光量。

2.5 景深 (Depth of Field)

基于薄透镜模型, 焦点距离 d_f 处的物体清晰, 其他距离产生模糊圆 (Circle of Confusion) :

$\text{CoC} = \frac{|d - d_f|}{d} \cdot \frac{f^2}{N(d_f - f)}$

其中 f 是焦距, N 是光圈数 (f-stop), d 是物体距离。

CoC 大小决定模糊半径: $r_{\text{blur}} = \text{CoC} \cdot \frac{\text{resolution}}{\text{sensor_size}}$

3. 工程实践

3.1 离屏缓冲类型

Panda3D 提供三种离屏缓冲：

类型	类	特点	适用场景
纹理缓冲	GraphicsBuffer	独立 FBO，最通用	大多数后处理
寄生缓冲	ParasiteBuffer	借用主窗口 FBO	不支持独立 FBO 的平台
立方体贴图	make_cube_map()	6面渲染	环境反射、阴影

3.2 渲染纹理模式（RTM）

RTM_none	– 不渲染到纹理
RTM_bind_or_copy	– 优先直接绑定，否则复制（推荐）
RTM_copy_texture	– 每帧从 FBO 复制到纹理
RTM_copy_ram	– 每帧复制到系统内存（最慢）
RTM_triggered_copy_*	– 手动触发复制（截图用）
RTM_bind_layered	– 渲染到分层纹理（立方体/3D）

3.3 CommonFilters 架构

Panda3D 的 CommonFilters 使用全屏四边形（Fullscreen Quad）方式：

主场景 → [离屏缓冲 A] → 后处理 Shader → [离屏缓冲 B] → ... → 屏幕

每个效果：1. 创建一个离屏缓冲 2. 将前一个缓冲的纹理作为输入 3. 用全屏四边形 + Shader 处理 4. 输出到下一个缓冲

3.4 多渲染目标（MRT）

现代 GPU 支持同时渲染到多个纹理（G-Buffer），用于延迟渲染：

Pass 1 (几何 Pass) :

- 输出 → 颜色缓冲 (albedo)
 - 法线缓冲 (world normal)
 - 深度缓冲
 - 材质参数缓冲 (roughness, metallic)

Pass 2 (光照 Pass) :

- 输入 ← 以上所有缓冲
 - 输出 → 最终颜色
-

4. Panda3D 源码剖析

4.1 GraphicsOutput : 渲染目标基类

graphicsOutput.h:64

```
class EXPCL_PANDA_DISPLAY GraphicsOutput
    : public GraphicsOutputBase, public DrawableRegion {

    // 渲染纹理模式枚举
    enum RenderTextureMode {
        RTM_none,
        RTM_bind_or_copy,      // 优先直接绑定 FBO
        RTM_copy_texture,      // 每帧复制
        RTM_copy_ram,          // 复制到 RAM
        RTM_triggered_copy_texture,
        RTM_triggered_copy_ram,
        RTM_bind_layered,      // 分层纹理 (立方体/3D)
    };

    // 帧模式
    enum FrameMode {
        FM_render,    // 正常渲染
        FM_parasite,  // 寄生渲染
        FM_refresh,   // 刷新显示
    };

    // 核心方法
    void add_render_texture(Texture *tex, RenderTextureMode mode,
                           RenderTexturePlane bitplane = RTP_COUNT);
    void setup_render_texture(Texture *tex, bool allow_bind, bool to_ram);

    // 创建子缓冲
    GraphicsOutput *make_texture_buffer(
        std::string_view name, int x_size, int y_size,
        Texture *tex = nullptr, bool to_ram = false,
        FrameBufferProperties *fbp = nullptr);

    GraphicsOutput *make_cube_map(std::string_view name, int size,
                                  NodePath &camera_rig, ...);
};
```

4.2 FrameBufferProperties : 帧缓冲属性

frameBufferProperties.h:27

```

class EXPCL_PANDA_DISPLAY FrameBufferProperties {
    // 位深度属性
    // FBP_depth_bits      - 深度缓冲位数 (16/24/32)
    // FBP_color_bits      - 颜色缓冲总位数
    // FBP_red/green/blue/alpha_bits - 各通道位数
    // FBP_stencil_bits    - 模板缓冲位数
    // FBP_aux_rgba        - 辅助 RGBA 缓冲数量 (MRT)
    // FBP_aux_hrgba       - 辅助半精度 RGBA 缓冲
    // FBP_aux_float       - 辅助浮点缓冲
    // FBP_multisamples    - MSAA 采样数

    // 标志位
    // FBF_rgb_color       - RGB 颜色模式
    // FBF_srgb_color      - sRGB 颜色空间
    // FBF_float_color     - 浮点颜色 (HDR)
    // FBF_float_depth     - 浮点深度

    // 关键方法
    static FrameBufferProperties get_default();
    int get_quality(const FrameBufferProperties &reqs) const;
    bool verify_hardware_software(const FrameBufferProperties &props,
                                   std::string_view renderer) const;
};

```

4.3 GraphicsBuffer：离屏缓冲

graphicsBuffer.h:27

```

class EXPCL_PANDA_DISPLAY GraphicsBuffer : public GraphicsOutput {
    // 离屏缓冲，不显示到屏幕
    // 通过 GraphicsEngine::make_output() 创建
    // 或通过 GraphicsOutput::make_texture_buffer() 创建

protected:
    virtual void close_buffer();
    virtual bool open_buffer();
};

```

4.4 ParasiteBuffer：寄生缓冲

parasiteBuffer.h:43

```

class EXPCL_PANDA_DISPLAY ParasiteBuffer : public GraphicsOutput {
    // 借用宿主 GraphicsOutput 的帧缓冲空间
    // 不创建独立的 FBO
    // 渲染完成后立即复制到纹理（因为宿主会清除帧缓冲）
    // 适用于不支持独立 FBO 的旧硬件

    ParasiteBuffer(GraphicsOutput *host, std::string name,
                    int x_size, int y_size, int flags);

    virtual void end_frame(FrameMode mode, Thread *current_thread);
    virtual GraphicsOutput *get_host();
};

```

4.5 GraphicsEngine::make_buffer()

graphicsEngine.h:90

```

// 快捷方法：创建离屏缓冲
INLINE GraphicsOutput *make_buffer(
    GraphicsOutput *host,
    std::string_view name, int sort,
    int x_size, int y_size);

// 创建寄生缓冲
INLINE GraphicsOutput *make_parasite(
    GraphicsOutput *host,
    std::string_view name, int sort,
    int x_size, int y_size);

// 通用方法（更多控制）
GraphicsOutput *make_output(
    GraphicsPipe *pipe,
    std::string_view name, int sort,
    const FrameBufferProperties &fb_prop,
    const WindowProperties &win_prop,
    int flags,
    GraphicsStateGuardian *gsg = nullptr,
    GraphicsOutput *host = nullptr);

```

4.6 CommonFilters 实现原理

direct/src/filter/CommonFilters.py 中的核心逻辑：


```
class CommonFilters:
    def __init__(self, win, cam):
        self.win = win      # 主窗口
        self.cam = cam      # 主相机
        self.configuration = {}
        self.task = None
        self.cleanup()

    def reconfigure(self, fullrebuild, changed):
        """重新配置滤镜管线"""
        # 1. 清理旧的缓冲和相机
        self.cleanup()

        # 2. 为每个启用的效果创建离屏缓冲
        # 3. 将主场景渲染到第一个缓冲
        # 4. 链式连接各个效果

    def setBloom(self, blend, mintrigger, maxtrigger, desat, intensity, size):
        """启用 Bloom 效果"""
        self.configuration["Bloom"] = {
            "blend": blend,
            "mintrigger": mintrigger,
            "maxtrigger": maxtrigger,
            "desat": desat,
            "intensity": intensity,
            "size": size,
        }
        self.reconfigure(True, "Bloom")
```

5. 代码演示

5.1 Python：基本离屏渲染

```

"""
离屏渲染基础：将场景渲染到纹理
"""
from direct.showbase.ShowBase import ShowBase
from panda3d.core import (
    FrameBufferProperties, WindowProperties, GraphicsPipe,
    GraphicsOutput, Texture, NodePath, Camera, OrthographicLens,
    CardMaker, Vec4
)

class OffscreenRenderDemo(ShowBase):
    def __init__(self):
        super().__init__()

        # 加载场景
        self.scene = self.loader.loadModel("models/environment")
        self.scene.reparentTo(self.render)

        # — 创建离屏缓冲 —————
        self.offscreen_tex = Texture("offscreen_color")
        self.offscreen_buf = self._create_offscreen_buffer(512, 512)

        # — 创建离屏相机 —————
        self.offscreen_cam = self._create_offscreen_camera()

        # — 将离屏纹理显示在屏幕上（调试用） —————
        self._show_texture_on_screen(self.offscreen_tex)

    def _create_offscreen_buffer(self, width, height):
        """创建离屏渲染缓冲"""
        # 设置帧缓冲属性
        fb_props = FrameBufferProperties()
        fb_props.setRgbColor(True)
        fb_props.setAlphaBits(1)
        fb_props.setDepthBits(24)

        # 设置窗口属性（离屏不需要窗口）
        win_props = WindowProperties.size(width, height)

        # 创建离屏缓冲
        buf = self.graphicsEngine.makeOutput(
            self.pipe,
            "offscreen_buffer",
            -2, # sort（比主窗口先渲染）
            fb_props,
            win_props,
            GraphicsPipe.BRefuseWindow, # 不创建窗口
            self.win.getGsg(), # 共享 GSG（共享 GPU 上下文）
            self.win, # 宿主窗口
        )

        if buf is None:
            print("无法创建离屏缓冲！")
            return None

        # 将颜色缓冲绑定到纹理
        buf.addRenderTexture(
            self.offscreen_tex,
            GraphicsOutput.RTBindOrCopy, # 优先直接绑定

```

```

        GraphicsOutput.RTPColor,          # 颜色平面
    )

    return buf

def _create_offscreen_camera(self):
    """创建渲染到离屏缓冲的相机"""
    # 创建相机节点
    cam_node = Camera("offscreen_cam")
    cam_np = self.render.attachNewNode(cam_node)
    cam_np.setPos(0, -20, 5)
    cam_np.lookAt(0, 0, 0)

    # 创建 DisplayRegion 并关联相机
    dr = self.offscreen_buf.makeDisplayRegion()
    dr.setCamera(cam_np)

    return cam_np

def _show_texture_on_screen(self, tex):
    """在屏幕角落显示离屏纹理（调试用）"""
    cm = CardMaker("preview_card")
    cm.setFrame(-1, -0.2, -1, -0.2) # 左下角
    card = self.render2d.attachNewNode(cm.generate())
    card.setTexture(tex)

## — 更简洁的方式：使用 make_texture_buffer —————
class SimpleOffscreenDemo>ShowBase):
    def __init__(self):
        super().__init__()

        # make_texture_buffer 是更高层的封装
        self.offscreen_buf = self.win.makeTextureBuffer(
            "my_buffer", # 名称
            512, 512,    # 尺寸
        )

        # 获取自动创建的纹理
        self.offscreen_tex = self.offscreen_buf.getTexture()

        # 创建相机
        self.offscreen_cam = self.makeCamera(self.offscreen_buf)
        self.offscreen_cam.reparentTo(self.render)
        self.offscreen_cam.setPos(0, -20, 5)
        self.offscreen_cam.lookAt(0, 0, 0)

        # 场景
        env = self.loader.loadModel("models/environment")
        env.reparentTo(self.render)

```

5.2 Python : Bloom 效果实现

```

"""
手动实现 Bloom 效果 (不使用 CommonFilters)
展示多 Pass 渲染的完整流程
"""

from direct.showbase.ShowBase import ShowBase
from panda3d.core import (
    FrameBufferProperties, WindowProperties, GraphicsPipe,
    GraphicsOutput, Texture, CardMaker, Shader,
    NodePath, Camera, OrthographicLens, Vec4
)

BLOOM_THRESHOLD_VERT = """
#version 330
in vec4 p3d_Vertex;
in vec2 p3d_MultiTexCoord0;
out vec2 texcoord;
void main() {
    gl_Position = p3d_Vertex;
    texcoord = p3d_MultiTexCoord0;
}
"""

BLOOM_THRESHOLD_FRAG = """
#version 330
uniform sampler2D scene_tex;
uniform float threshold;
in vec2 texcoord;
out vec4 fragColor;

float luminance(vec3 c) {
    return dot(c, vec3(0.2126, 0.7152, 0.0722));
}

void main() {
    vec4 color = texture(scene_tex, texcoord);
    float lum = luminance(color.rgb);
    // 平滑阈值提取
    float weight = smoothstep(threshold - 0.1, threshold + 0.1, lum);
    fragColor = vec4(color.rgb * weight, 1.0);
}
"""

BLOOM_BLUR_FRAG = """
#version 330
uniform sampler2D input_tex;
uniform vec2 blur_dir;    // (1,0) 水平 或 (0,1) 垂直
uniform vec2 texel_size;
in vec2 texcoord;
out vec4 fragColor;

void main() {
    // 5-tap 高斯模糊
    vec4 result = vec4(0.0);
    float weights[5] = float[](0.227027, 0.316216, 0.070270, 0.316216, 0.227027);
    float offsets[5] = float[](-2.0, -1.0, 0.0, 1.0, 2.0);

    for (int i = 0; i < 5; i++) {
        vec2 offset = blur_dir * offsets[i] * texel_size;
        result += texture(input_tex, texcoord + offset) * weights[i];
    }
}
"""

```

```

        fragColor = result;
    }
    """

BLOOM_COMPOSITE_FRAG = """
#version 330
uniform sampler2D scene_tex;
uniform sampler2D bloom_tex;
uniform float intensity;
in vec2 texcoord;
out vec4 fragColor;

void main() {
    vec4 scene = texture(scene_tex, texcoord);
    vec4 bloom = texture(bloom_tex, texcoord);
    fragColor = scene + bloom * intensity;
}
"""

class BloomDemo(ShowBase):
    def __init__(self):
        super().__init__()

        # 主场景
        self.scene = self.loader.loadModel("models/environment")
        self.scene.reparentTo(self.render)

        w, h = self.win.getXSize(), self.win.getYSize()

        # — Pass 1: 渲染主场景到 HDR 缓冲 —————
        self.scene_buf, self.scene_tex = self._make_buffer("scene", w, h, float_color=True)
        self.scene_cam = self.makeCamera(self.scene_buf)
        self.scene_cam.reparentTo(self.render)

        # — Pass 2: 亮度提取 —————
        self.bright_buf, self.bright_tex = self._make_buffer("bright", w//2, h//2)
        self._make_fullscreen_pass(
            self.bright_buf,
            Shader.make(Shader.SL_GLSL, BLOOM_THRESHOLD_VERT, BLOOM_THRESHOLD_FRAG),
            {"scene_tex": self.scene_tex, "threshold": 0.7}
        )

        # — Pass 3: 水平模糊 —————
        self.hblur_buf, self.hblur_tex = self._make_buffer("hblur", w//2, h//2)
        self._make_fullscreen_pass(
            self.hblur_buf,
            Shader.make(Shader.SL_GLSL, BLOOM_THRESHOLD_VERT, BLOOM_BLUR_FRAG),
            {
                "input_tex": self.bright_tex,
                "blur_dir": (1.0, 0.0),
                "texel_size": (2.0/w, 2.0/h),
            }
        )

        # — Pass 4: 垂直模糊 —————
        self.vblur_buf, self.vblur_tex = self._make_buffer("vblur", w//2, h//2)
        self._make_fullscreen_pass(
            self.vblur_buf,
            Shader.make(Shader.SL_GLSL, BLOOM_THRESHOLD_VERT, BLOOM_BLUR_FRAG),
            {
                "input_tex": self.hblur_tex,

```

```

        "blur_dir": (0.0, 1.0),
        "texel_size": (2.0/w, 2.0/h),
    }
)

# — Pass 5: 合成到屏幕 —————
self._make_fullscreen_pass(
    None, # None = 渲染到主窗口
    Shader.make(Shader.SL_GLSL, BLOOM_THRESHOLD_VERT, BLOOM_COMPOSITE_FRAG),
    {
        "scene_tex": self.scene_tex,
        "bloom_tex": self.vblur_tex,
        "intensity": 1.5,
    }
)

def _make_buffer(self, name, w, h, float_color=False):
    """创建离屏缓冲和对应纹理"""
    fb_props = FrameBufferProperties()
    fb_props.setRgbColor(True)
    fb_props.setDepthBits(0 if name != "scene" else 24)
    if float_color:
        fb_props.setFloatColor(True) # HDR 浮点缓冲

    win_props = WindowProperties.size(w, h)
    buf = self.graphicsEngine.makeOutput(
        self.pipe, name, -10,
        fb_props, win_props,
        GraphicsPipe.BFRefuseWindow,
        self.win.getGsg(), self.win,
    )

    tex = Texture(name + "_tex")
    buf.addRenderTexture(tex, GraphicsOutput.RTBindOrCopy, GraphicsOutput.RTPColor)
    return buf, tex

def _make_fullscreen_pass(self, target_buf, shader, inputs):
    """创建全屏四边形 Pass"""
    # 创建正交相机
    cam_node = Camera("pass_cam")
    lens = OrthographicLens()
    lens.setFilmSize(2, 2)
    lens.setNearFar(-1, 1)
    cam_node.setLens(lens)

    # 创建全屏四边形
    cm = CardMaker("fullscreen_quad")
    cm.setFrame(-1, 1, -1, 1)
    quad = NodePath(cm.generate())
    quad.setDepthTest(False)
    quad.setDepthWrite(False)
    quad.setShader(shader)

    # 设置 Shader 输入
    for name, value in inputs.items():
        if isinstance(value, Texture):
            quad.setShaderInput(name, value)
        elif isinstance(value, (int, float)):
            quad.setShaderInput(name, value)
        elif isinstance(value, tuple):
            quad.setShaderInput(name, *value)

```



```
# 设置渲染目标
if target_buf is not None:
    dr = target_buf.makeDisplayRegion()
else:
    dr = self.win.makeDisplayRegion()

cam_np = quad.attachNewNode(cam_node)
dr.setCamera(cam_np)
```

5.3 Python：使用 **CommonFilters**（推荐方式）

```

"""
使用 Panda3D 内置的 CommonFilters 系统
这是生产环境推荐的方式
"""

from direct.showbase.ShowBase import ShowBase
from direct.filter.CommonFilters import CommonFilters
from panda3d.core import loadPrcFileData

## 启用 HDR 浮点帧缓冲 (必须在 ShowBase 之前)
loadPrcFileData("", "framebuffer-float true")
loadPrcFileData("", "framebuffer-srgb true")

class CommonFiltersDemo(ShowBase):
    def __init__(self):
        super().__init__()

        # 加载场景
        env = self.loader.loadModel("models/environment")
        env.reparentTo(self.render)

        # 初始化 CommonFilters
        self.filters = CommonFilters(self.win, self.cam)

        # — Bloom 泛光 —————
        self.filters.setBloom(
            blend=(0.3, 0.4, 0.3, 0.0), # RGB 通道权重
            mintrigger=0.6,             # 开始触发的亮度
            maxtrigger=1.0,             # 完全触发的亮度
            desat=0.6,                  # 去饱和度 (让高光偏白)
            intensity=1.0,              # Bloom 强度
            size="medium",              # 模糊尺寸: small/medium/large
        )

        # — 环境光遮蔽 (SSAO) —————
        self.filters.setAmbientOcclusion(
            numsamples=16,              # 采样数 (越多越准确, 越慢)
            radius=0.05,                # 采样半径 (世界空间)
            amount=1.0,                 # AO 强度
            strength=0.5,               # 对比度
            falloff=0.002,              # 距离衰减
        )

        # — 模糊/锐化 —————
        # 0.0 = 最模糊, 1.0 = 正常, 2.0 = 锐化
        # self.filters.setBlurSharpen(0.5)

        # — 颜色反转 —————
        # self.filters.setInverted()

        # — HDR 色调映射 —————
        self.filters.setHighDynamicRange()

        # — 卡通描边 —————
        # self.filters.setCartoonInk(separation=1)

        # — 景深 —————
        # self.filters.setDepthOfField(
        #     focalpoint=50, # 焦点距离
        #     focalwidth=10, # 焦深范围

```

```

#      near=1,          # 近裁剪
#      far=1000,        # 远裁剪
#      focus=0.5,       # 焦点强度
# )

# — 动态调整 —————
self.accept("b", self.toggle_bloom)
self.accept("a", self.toggle_ao)

self._bloom_on = True
self._ao_on = True

def toggle_bloom(self):
    if self._bloom_on:
        self.filters.delBloom()
    else:
        self.filters.setBloom(intensity=1.0, size="medium")
    self._bloom_on = not self._bloom_on

def toggle_ao(self):
    if self._ao_on:
        self.filters.delAmbientOcclusion()
    else:
        self.filters.setAmbientOcclusion(numsamples=16)
    self._ao_on = not self._ao_on

```

5.4 GLSL：自定义后处理 Shader

```

/* — 色调映射 + Gamma 校正 Shader ————— */

/* 顶点着色器（全屏四边形，所有后处理通用） */
#version 330
in vec4 p3d_Vertex;
in vec2 p3d_MultiTexCoord0;
out vec2 texcoord;

void main() {
    gl_Position = p3d_Vertex;
    texcoord = p3d_MultiTexCoord0;
}

```

```

/* 片段着色器: HDR 色调映射 */
#version 330
uniform sampler2D hdr_tex;
uniform float exposure;
uniform float gamma;
in vec2 texcoord;
out vec4 fragColor;

/* ACES 色调映射 (电影级) */
vec3 aces_tonemap(vec3 x) {
    float a = 2.51;
    float b = 0.03;
    float c = 2.43;
    float d = 0.59;
    float e = 0.14;
    return clamp((x * (a * x + b)) / (x * (c * x + d) + e), 0.0, 1.0);
}

/* Reinhard 色调映射 (简单) */
vec3 reinhard_tonemap(vec3 x) {
    return x / (1.0 + x);
}

void main() {
    vec3 hdr_color = texture(hdr_tex, texcoord).rgb;

    // 曝光调整
    vec3 exposed = hdr_color * pow(2.0, exposure);

    // 色调映射 (HDR → LDR)
    vec3 ldr_color = aces_tonemap(exposed);

    // Gamma 校正 (线性空间 → sRGB)
    vec3 gamma_corrected = pow(ldr_color, vec3(1.0 / gamma));

    fragColor = vec4(gamma_corrected, 1.0);
}

```

```

/* 片段着色器: SSAO */
#version 330
uniform sampler2D depth_tex;
uniform sampler2D normal_tex;
uniform sampler2D noise_tex;
uniform vec3 samples[64];      // 半球采样核
uniform mat4 projection;
uniform mat4 view;
uniform vec2 noise_scale;      // 屏幕尺寸 / 噪声纹理尺寸
in vec2 texcoord;
out float ao_value;

const float radius = 0.5;
const float bias = 0.025;

void main() {
    // 重建世界空间位置
    float depth = texture(depth_tex, texcoord).r;
    vec3 normal = normalize(texture(normal_tex, texcoord).rgb * 2.0 - 1.0);

    // 随机旋转 (使用噪声纹理减少采样数)
    vec3 random_vec = normalize(texture(noise_tex, texcoord * noise_scale).xyz);

    // 构建 TBN 矩阵 (切线空间 → 视图空间)
    vec3 tangent = normalize(random_vec - normal * dot(random_vec, normal));
    vec3 bitangent = cross(normal, tangent);
    mat3 TBN = mat3(tangent, bitangent, normal);

    // 采样遮蔽
    float occlusion = 0.0;
    for (int i = 0; i < 64; i++) {
        // 将采样点从切线空间变换到视图空间
        vec3 sample_pos = TBN * samples[i];
        // ... 深度比较
        // occlusion += (sample_depth >= sample_pos.z + bias) ? 1.0 : 0.0;
    }

    ao_value = 1.0 - (occlusion / 64.0);
}

```

5.5 Python：截图与离屏渲染到 RAM

```

"""
将渲染结果保存到文件或读取到 Python
"""
from direct.showbase.ShowBase import ShowBase
from panda3d.core import (
    GraphicsOutput, Texture, PNMTImage,
    WindowProperties, FrameBufferProperties, GraphicsPipe
)

class ScreenshotDemo(ShowBase):
    def __init__(self):
        super().__init__()

        env = self.loader.loadModel("models/environment")
        env.reparentTo(self.render)

        # — 方式1: 截取主窗口 (最简单) —————
        self.accept("f1", self.screenshot_simple)

        # — 方式2: 渲染到 RAM 纹理 —————
        self.ram_tex = Texture("ram_tex")
        self.ram_buf = self._create_ram_buffer(256, 256)
        self.accept("f2", self.read_pixels)

        # — 方式3: 触发式复制 (只在需要时复制) —————
        self.triggered_tex = Texture("triggered_tex")
        self.win.addRenderTexture(
            self.triggered_tex,
            GraphicsOutput.RTMTriggeredCopyRam, # 触发式
            GraphicsOutput.RTPColor,
        )
        self.accept("f3", self.triggered_capture)

    def screenshot_simple(self):
        """最简单的截图方式"""
        # 保存到文件
        self.win.saveScreenshot("screenshot.png")
        print("截图已保存")

    def _create_ram_buffer(self, w, h):
        """创建渲染到 RAM 的缓冲"""
        fb_props = FrameBufferProperties()
        fb_props.setRgbColor(True)
        fb_props.setDepthBits(24)

        win_props = WindowProperties.size(w, h)
        buf = self.graphicsEngine.makeOutput(
            self.pipe, "ram_buffer", -5,
            fb_props, win_props,
            GraphicsPipe.BFRefuseWindow,
            self.win.getGsg(), self.win,
        )

        # RTM_copy_ram : 每帧自动复制到 RAM
        buf.addRenderTexture(
            self.ram_tex,
            GraphicsOutput.RTMCopyRam, # 复制到系统内存
            GraphicsOutput.RTPColor,
        )

```



```

# 创建相机
cam = self.makeCamera(buf)
cam.reparentTo(self.render)
return buf

def read_pixels(self):
    """读取 RAM 中的像素数据"""
    # 确保纹理数据已更新
    self.graphicsEngine.extractTextureData(self.ram_tex, self.win.getGsg())

    if self.ram_tex.hasRamImage():
        # 转换为 PNMImage
        img = PNMImage()
        self.ram_tex.store(img)

        # 读取像素
        w, h = img.getXSize(), img.getYSize()
        print(f"图像尺寸: {w}x{h}")

        # 读取中心像素
        cx, cy = w // 2, h // 2
        r = img.getRed(cx, cy)
        g = img.getGreen(cx, cy)
        b = img.getBlue(cx, cy)
        print(f"中心像素 RGB: ({r:.2f}, {g:.2f}, {b:.2f})")

        # 保存
        img.write("ram_capture.png")

def triggered_capture(self):
    """触发式截图（只在调用时复制一次）"""
    self.win.triggerCopy()
    # 下一帧后数据会在 self.triggered_tex 中
    from direct.task import Task
    def check_ready(task):
        if self.triggered_tex.hasRamImage():
            img = PNMImage()
            self.triggered_tex.store(img)
            img.write("triggered_capture.png")
            print("触发截图完成")
            return Task.done
        return Task.cont
    self.taskMgr.add(check_ready, "CheckCapture")

```

6. 性能优化

6.1 缓冲分辨率策略

```
## ❌ 错误：所有 Pass 都用全分辨率
def bad_bloom(win):
    w, h = win.getXSize(), win.getYSize()
    # 亮度提取、模糊都在 1920x1080 下进行
    bright_buf = win.makeTextureBuffer("bright", w, h)
    blur_buf = win.makeTextureBuffer("blur", w, h)

## ✅ 正确：模糊在低分辨率下进行（视觉差异极小）
def good_bloom(win):
    w, h = win.getXSize(), win.getYSize()
    # 亮度提取在半分辨率
    bright_buf = win.makeTextureBuffer("bright", w//2, h//2)
    # 模糊在 1/4 分辨率（节省 16x 带宽）
    blur_buf = win.makeTextureBuffer("blur", w//4, h//4)
```

6.2 共享深度缓冲

```
## ❌ 错误：每个缓冲都有独立深度缓冲（浪费内存）
buf1 = win.makeTextureBuffer("buf1", w, h) # 有深度缓冲
buf2 = win.makeTextureBuffer("buf2", w, h) # 又一个深度缓冲

## ✅ 正确：后处理 Pass 不需要深度缓冲
fb_props = FrameBufferProperties()
fb_props.setRgbColor(True)
fb_props.setDepthBits(0) # 不需要深度缓冲！
## 只有第一个 Pass（场景渲染）需要深度缓冲
```

6.3 RTM 模式选择

```
## RTM_bind_or_copy：最快（直接渲染到纹理，无需复制）
buf.addRenderTexture(tex, GraphicsOutput.RTMBindOrCopy, GraphicsOutput.RTPColor)

## RTM_copy_texture：每帧复制（慢，但兼容性好）
buf.addRenderTexture(tex, GraphicsOutput.RTMCopyTexture, GraphicsOutput.RTPColor)

## RTM_copy_ram：最慢（需要 GPU→CPU 传输）
## 只在需要 CPU 访问像素时使用
buf.addRenderTexture(tex, GraphicsOutput.RTMCopyRam, GraphicsOutput.RTPColor)
```

6.4 各效果性能对比

测试环境: 1920x1080, GTX 1060

效果	GPU 时间	内存占用
-----	-----	-----
无后处理	0ms	0 MB
Bloom (半分辨率)	1.2ms	8 MB
SSAO (16采样)	2.8ms	4 MB
SSAO (64采样)	8.1ms	4 MB
景深	1.5ms	8 MB
色调映射	0.3ms	0 MB
FXAA	0.8ms	0 MB
全部叠加	6.6ms	20 MB

6.5 one_shot 模式（静态场景）

```
## 对于静态场景（如 UI 背景），只渲染一次
buf.setOneShot(True) # 渲染一帧后自动停用

## 需要更新时手动触发
buf.setActive(True) # 重新激活，渲染一帧后再次停用
```

小结

知识点	核心要点
离屏缓冲	<code>GraphicsBuffer</code> （独立 FBO）vs <code>ParasiteBuffer</code> （借用）
RTM 模式	<code>RTMBindOrCopy</code> 最快， <code>RTMCopyRam</code> 最慢
<code>FrameBufferProperties</code>	控制颜色位深、深度位深、浮点格式、MSAA
全屏四边形	后处理的标准实现方式，正交相机 + <code>CardMaker</code>
Bloom	亮度提取 → 高斯模糊（可分离）→ 叠加
SSAO	法线半球采样 → 深度比较 → 模糊
色调映射	Reinhard / ACES，将 HDR 映射到 LDR
分辨率优化	模糊类效果在 1/2 或 1/4 分辨率下计算
<code>CommonFilters</code>	Panda3D 内置后处理框架，推荐生产使用

关键设计原则

- 渲染顺序：sort 值越小越先渲染，离屏缓冲 sort 必须小于主窗口
- 共享 GSG：离屏缓冲应与主窗口共享 GSG（共享 GPU 上下文和纹理）
- 浮点缓冲：HDR 效果必须使用浮点帧缓冲（`setFloatColor(True)`）
- 深度缓冲复用：后处理 Pass 不需要深度缓冲，设置 `setDepthBits(0)`
- RTM 选择：优先 `RTMBindOrCopy`，避免不必要的 `RTMCopyRam`

与其他专题的关联

- 专题04（光照阴影）：阴影贴图本质是离屏渲染的应用
- 专题09（Shader 系统）：后处理效果通过 GLSL Shader 实现
- 专题13（跨平台）：不同平台对 FBO 的支持程度不同，需要 `ParasiteBuffer` 回退

专题12: LOD 与流式加载

目录

1. 问题背景
 2. 数学原理
 3. 工程实践
 4. Panda3D 源码剖析
 5. 代码演示
 6. 性能优化
 7. 小结
-

1. 问题背景

大型 3D 场景面临两个核心矛盾：

矛盾	描述
质量 vs 性能	高精度模型消耗大量 GPU 资源，但远处物体不需要高精度
内存 vs 加载时间	一次性加载所有资源会耗尽内存，但按需加载会产生卡顿

LOD (Level of Detail) 解决质量 vs 性能矛盾：根据距离自动切换模型精度。

流式加载 (Streaming) 解决内存 vs 加载时间矛盾：在后台异步加载即将需要的资源，卸载不再需要的资源。

典型应用场景：

开放世界游戏：

- 玩家附近 (0-50m) → 高精度模型 (LOD0, 50k 三角形)
中距离 (50-200m) → 中精度模型 (LOD1, 5k 三角形)
远距离 (200-500m) → 低精度模型 (LOD2, 500 三角形)
极远处 (500m+) → 广告牌/剔除

同时：

- 玩家前方区域 → 预加载
玩家身后区域 → 卸载

2. 数学原理

2.1 屏幕空间误差 (Screen Space Error)

LOD 切换的理论基础是**屏幕空间误差**：模型简化引入的几何误差在屏幕上的投影大小。

设物体到相机距离为 d ，模型的世界空间误差为 ϵ ，相机垂直 FOV 为 θ ，屏幕高度为 H ：

$$\text{SSE} = \frac{\epsilon \cdot H}{2d \cdot \tan(\theta/2)}$$

当 SSE 小于阈值（通常 1-2 像素）时，切换到更低精度的 LOD。

距离阈值推导：给定最大允许 SSE $= \tau$ ，切换距离为：

$$d_{\text{switch}} = \frac{\epsilon \cdot H}{2\tau \cdot \tan(\theta/2)}$$

2.2 LOD 切换的滞后 (Hysteresis)

为避免在切换距离附近频繁抖动 (LOD popping)，引入滞后区间：

$$d_{\text{in}} > d_{\text{out}}$$

- 从远到近：在 d_{in} 处切换到更高精度
- 从近到远：在 d_{out} 处切换到更低精度

Panda3D 的 `LODNode` 正是使用这种 in/out 双阈值设计。

2.3 FadeLOD 的 Alpha 过渡

硬切换会产生明显的“跳变” (LOD popping)。FadeLOD 使用 Alpha 混合平滑过渡：

在过渡时间 T 内，同时渲染新旧两个 LOD：

$$\alpha_{\text{new}}(t) = \frac{t}{T}, \quad \alpha_{\text{old}}(t) = 1 - \frac{t}{T}$$

$$C = \alpha_{\text{new}} \cdot C_{\text{LOD}_{\text{new}}} + \alpha_{\text{old}} \cdot C_{\text{LOD}_{\text{old}}}$$

代价：过渡期间 draw call 数量翻倍。

2.4 流式加载的预测模型

基于速度的预测：

$$\text{预加载距离} = v \cdot t_{\text{load}} + d_{\text{activate}}$$

其中 v 是玩家速度, t_{load} 是资源加载时间, $d_{activate}$ 是激活距离。

优先级队列:

$$\text{priority}(\text{chunk}) = \frac{1}{d_{\text{chunk}}} \cdot \text{visibility}(\text{chunk})$$

距离越近、可见性越高的区块优先加载。

2.5 网格简化算法

QEM (Quadric Error Metrics) : Garland-Heckbert 算法, 最常用的网格简化方法。

每个顶点 v 关联一个误差二次型矩阵 Q :

$$\Delta(v) = v^T Q v$$

合并边 (v_1, v_2) 时, 新顶点位置最小化:

$$v_{\text{new}} = \arg\min_v (v^T (Q_1 + Q_2) v)$$

解析解:

$$v_{\text{new}} = (Q_1 + Q_2)^{-1} \begin{bmatrix} 0 & 0 & 0 & 1 \end{bmatrix}$$

3. 工程实践

3.1 LOD 层级设计

LOD 级别	距离范围	三角形数	纹理分辨率
LOD0 (最高)	0-50m	50,000	2048×2048
LOD1	50-150m	5,000	1024×1024
LOD2	150-400m	500	512×512
LOD3 (最低)	400-800m	50	256×256
剔除	800m+	0	—

经验法则：每级 LOD 减少 80-90% 的三角形数。

3.2 广告牌 (Billboard) 技术

极远处的物体可以用始终朝向相机的 2D 图片替代 3D 模型：

LOD3 → 广告牌 (Billboard)

- 始终朝向相机 (Y 轴旋转)
- 只有 2 个三角形
- 视觉上与低精度 3D 模型相近

Panda3D 的 `BillboardEffect` 实现此功能。

3.3 流式加载架构

主线程 (App) :

- └─ 每帧检查玩家位置
- └─ 计算需要加载/卸载的区块
- └─ 向加载队列提交请求

后台线程 (Loader) :

- └─ 从队列取出请求
- └─ 从磁盘读取文件
- └─ 解压/解码数据
- └─ 将结果放入完成队列

主线程 (App) :

- └─ 从完成队列取出结果
- └─ 将模型添加到场景图

3.4 BAM 格式与缓存

Panda3D 的 BAM (Binary Animation and Models) 格式是优化的二进制格式： - 比 EGG 格式加载快 10-100x - 支持增量加载 - 可以预先计算包围体、法线等

4. Panda3D 源码剖析

4.1 LODNode：距离切换节点

lodNode.h:29

```
class EXPCL_PANDA_PGRAPHNODES LODNode : public PandaNode {
    // 核心数据: Switch 数组
    class Switch {
        PN_stdfloat _in;    // 切入距离 (从远到近, 在此距离激活)
        PN_stdfloat _out;   // 切出距离 (从近到远, 在此距离停用)
        // in > out, 形成滞后区间

        bool in_range(PN_stdfloat dist) const;
        bool in_range_2(PN_stdfloat dist2) const; // 使用距离平方 (避免 sqrt)
    };

    // CData (流水线循环数据)
    class CData : public CycleData {
        LPoint3 _center;           // LOD 中心点 (世界空间)
        SwitchVector _switch_vector; // 各级 LOD 的切换距离
        size_t _lowest, _highest;  // 最低/最高精度索引
        bool _got_force_switch;    // 是否强制使用某级 LOD
        int _force_switch;         // 强制使用的 LOD 索引
        PN_stdfloat _lod_scale;    // LOD 缩放因子 (调整切换距离)
    };

    // 关键方法
    void add_switch(PN_stdfloat in, PN_stdfloat out);
    bool cull_callback(CullTraverser *trav, CullTraverserData &data);
    int compute_child(CullTraverser *trav, CullTraverserData &data);
};
```

compute_child() 核心逻辑 (伪代码)：

```

int LODNode::compute_child(CullTraverser *trav, CullTraverserData &data) {
    // 1. 计算相机到 LOD 中心的距离
    CPT(TransformState) rel_transform = get_rel_transform(trav, data);
    LPoint3 center = cdata->_center * rel_transform->get_mat();
    PN_stdffloat dist2 = center.length_squared();

    // 2. 应用 LOD 缩放 (相机的 lod_scale * 节点的 lod_scale)
    PN_stdffloat lod_scale = trav->get_scene()->get_camera_node()->get_lod_scale()
        * cdata->_lod_scale;
    dist2 /= (lod_scale * lod_scale);

    // 3. 遍历 Switch 数组, 找到匹配的 LOD 级别
    for (int i = 0; i < num_switches; i++) {
        if (cdata->_switch_vector[i].in_range_2(dist2)) {
            return i; // 返回子节点索引
        }
    }
    return -1; // 不在任何范围内, 不渲染
}

```

4.2 FadeLODNode : 平滑过渡

fadeLodNode.h:24

```

class EXPCL_PANDA_PGRAPHNODES FadeLODNode : public LODNode {
    PN_stdffloat _fade_time; // 过渡时间 (秒)
    std::string _fade_bin_name; // 过渡期间使用的渲染 Bin
    int _fade_bin_draw_order; // Bin 内排序
    int _fade_state_override; // 状态覆盖优先级

    // 过渡状态 (新旧 LOD 的 Alpha 值)
    CPT(RenderState) _fade_1_new_state; // 新 LOD: alpha 从 0 到 1
    CPT(RenderState) _fade_1_old_state; // 旧 LOD: alpha 从 1 到 0
    CPT(RenderState) _fade_2_new_state;
    CPT(RenderState) _fade_2_old_state;

    // 过渡数据存储在 AuxSceneData 中 (每个相机独立)
    // FadeLODNodeData 记录当前过渡进度
};

```

4.3 Camera 的 LOD 控制

camera.h:74

```

class Camera : public LensNode {
    // LOD 中心: 计算 LOD 距离时使用的参考点
    // 默认是相机位置, 可以设置为其他点 (如玩家位置)
    NodePath _lod_center;

    // LOD 缩放: 全局调整所有 LOD 的切换距离
    // 值越大, 越早切换到低精度 (性能优先)
    // 值越小, 越晚切换到低精度 (质量优先)
    PN_stdfloat _lod_scale;
};

```

4.4 Loader : 异步加载系统

Panda3D 的 `Loader` 类支持异步加载:

```

// direct/src/showbase/Loader.py (Python 层)
class Loader:
    def loadModel(self, modelPath, callback=None, extraArgs=[],
                  priority=None, loaderOptions=None, noCache=None,
                  allowInstance=False, okMissing=None, blocking=None):
        """
        如果 callback 不为 None, 则异步加载
        否则同步加载 (阻塞)
        """
        if callback is None:
            # 同步加载
            return self._load_model_sync(modelPath, loaderOptions)
        else:
            # 异步加载: 提交到后台线程
            request = self._make_async_request(modelPath, loaderOptions)
            request.set_done_callback(callback, extraArgs)
            self._loader.load_async(request)

```

5. 代码演示

5.1 Python：基本 LOD 设置

```

"""
LODNode 基本用法: 根据距离切换模型精度
"""

from direct.showbase.ShowBase import ShowBase
from panda3d.core import LODNode, NodePath, Point3

class LODDemo(ShowBase):
    def __init__(self):
        super().__init__()

        # — 方式1: 手动创建 LODNode —————
        lod_node = LODNode("tree_lod")
        lod_np = self.render.attachNewNode(lod_node)
        lod_np.setPos(0, 50, 0)

        # 加载不同精度的模型
        # 注意: 子节点顺序必须与 add_switch 顺序一致
        high_detail = self.loader.loadModel("models/tree_high") # LOD0
        mid_detail = self.loader.loadModel("models/tree_mid") # LOD1
        low_detail = self.loader.loadModel("models/tree_low") # LOD2
        billboard = self.loader.loadModel("models/tree_billboard") # LOD3

        high_detail.reparentTo(lod_np)
        mid_detail.reparentTo(lod_np)
        low_detail.reparentTo(lod_np)
        billboard.reparentTo(lod_np)

        # 添加切换距离: add_switch(in_distance, out_distance)
        # in > out, 形成滞后区间防止抖动
        # 子节点0 (high_detail) : 0-55m 可见
        lod_node.addSwitch(55, 0) # in=55, out=0
        # 子节点1 (mid_detail) : 50-160m 可见
        lod_node.addSwitch(160, 50) # in=160, out=50
        # 子节点2 (low_detail) : 150-420m 可见
        lod_node.addSwitch(420, 150) # in=420, out=150
        # 子节点3 (billboard) : 400-850m 可见
        lod_node.addSwitch(850, 400) # in=850, out=400

        # 设置 LOD 中心 (默认是节点原点)
        lod_node.setCenter(Point3(0, 0, 1)) # 树的中心高度

        # — 方式2: 使用 NodePath 的便捷接口 —————
        # Panda3D 提供了更简洁的 LOD 接口
        lod2 = self.render.attachNewNode(LODNode("rock_lod"))
        lod2.setPos(20, 30, 0)

        # 直接加载并添加 LOD 级别
        for model_path, switch_in, switch_out in [
            ("models/rock_high", 40, 0),
            ("models/rock_mid", 120, 35),
            ("models/rock_low", 300, 110),
        ]:
            model = self.loader.loadModel(model_path)
            model.reparentTo(lod2)
            lod2.node().addSwitch(switch_in, switch_out)

        # — LOD 调试: 显示切换范围 —————
        # 显示所有 LOD 级别 (调试用)
        lod_node.showAllSwitches()

```



```

# 只显示特定级别
# lod_node.showSwitch(0) # 只显示 LOD0

# 强制使用特定 LOD (调试/截图用)
# lod_node.forceSwitch(1) # 强制使用 LOD1
# lod_node.clearForceSwitch() # 取消强制

# — 全局 LOD 缩放 —————
# 调整相机的 LOD 缩放因子
# 值 > 1: 更早切换到低精度 (性能优先)
# 值 < 1: 更晚切换到低精度 (质量优先)
self.cam.node().setLodScale(1.5) # 性能模式

# 设置 LOD 参考点 (默认是相机位置)
# 可以设置为玩家位置, 使 LOD 基于玩家距离而非相机距离
# self.cam.node().setLodCenter(player_np)

```

5.2 Python : FadeLODNode (平滑过渡)

```

"""
FadeLODNode: LOD 切换时使用 Alpha 淡入淡出, 避免突变
"""

from direct.showbase.ShowBase import ShowBase
from panda3d.core import FadeLODNode, NodePath

class FadeLODDemo(ShowBase):
    def __init__(self):
        super().__init__()

        # 创建 FadeLODNode
        fade_lod = FadeLODNode("fade_tree")
        fade_np = self.render.attachNewNode(fade_lod)
        fade_np.setPos(0, 30, 0)

        # 设置过渡时间 (秒)
        fade_lod.setFadeTime(0.5) # 0.5秒内完成过渡

        # 设置过渡期间使用的渲染 Bin
        # 过渡时两个 LOD 都需要渲染, 放入透明 Bin
        fade_lod.setFadeBin("transparent", 0)

        # 添加模型 (与 LODNode 相同)
        high = self.loader.loadModel("models/tree_high")
        low = self.loader.loadModel("models/tree_low")
        high.reparentTo(fade_np)
        low.reparentTo(fade_np)

        fade_lod.addSwitch(60, 0)
        fade_lod.addSwitch(200, 55)

        # 注意: FadeLODNode 需要透明度支持
        # 确保模型材质支持 Alpha 混合
        fade_np.setTransparency(True)

```

5.3 Python：流式加载系统

```

"""
流式加载：根据玩家位置动态加载/卸载区块
"""

from direct.showbase.ShowBase import ShowBase
from direct.task import Task
from panda3d.core import NodePath, Point3, Vec3
import math
import queue
import threading

class StreamingWorld(ShowBase):
    """
    简单的区块流式加载系统
    将世界划分为固定大小的区块，根据玩家位置加载/卸载
    """

    CHUNK_SIZE = 100.0      # 每个区块的世界空间大小
    LOAD_RADIUS = 3         # 加载半径（区块数）
    UNLOAD_RADIUS = 5       # 卸载半径（区块数）

    def __init__(self):
        super().__init__()

        self.player_pos = Point3(0, 0, 0)
        self.loaded_chunks = {}    # {(cx, cy): NodePath}
        self.loading_chunks = set() # 正在加载的区块
        self._load_queue = queue.PriorityQueue()
        self._done_queue = queue.Queue()

        # 启动后台加载线程
        self._loader_thread = threading.Thread(
            target=self._loader_worker, daemon=True
        )
        self._loader_thread.start()

        # 主循环任务
        self.taskMgr.add(self._update_streaming, "UpdateStreaming")
        self.taskMgr.add(self._apply_loaded_chunks, "ApplyChunks")

        # 模拟玩家移动
        self.taskMgr.add(self._move_player, "MovePlayer")
        self._move_angle = 0.0

    def _world_to_chunk(self, pos):
        """将世界坐标转换为区块坐标"""
        cx = int(math.floor(pos.x / self.CHUNK_SIZE))
        cy = int(math.floor(pos.y / self.CHUNK_SIZE))
        return (cx, cy)

    def _chunk_to_world(self, cx, cy):
        """将区块坐标转换为世界坐标（区块中心）"""
        x = (cx + 0.5) * self.CHUNK_SIZE
        y = (cy + 0.5) * self.CHUNK_SIZE
        return Point3(x, y, 0)

    def _chunk_distance(self, cx, cy, player_cx, player_cy):
        """计算区块到玩家区块的距离"""
        return math.sqrt((cx - player_cx)**2 + (cy - player_cy)**2)

    def _update_streaming(self, task):

```

```

"""每帧检查需要加载/卸载的区块"""
player_cx, player_cy = self._world_to_chunk(self.player_pos)

# 检查需要加载的区块
for dx in range(-self.LOAD_RADIUS, self.LOAD_RADIUS + 1):
    for dy in range(-self.LOAD_RADIUS, self.LOAD_RADIUS + 1):
        cx, cy = player_cx + dx, player_cy + dy
        dist = self._chunk_distance(cx, cy, player_cx, player_cy)

        if dist <= self.LOAD_RADIUS:
            if (cx, cy) not in self.loaded_chunks and \
                (cx, cy) not in self.loading_chunks:
                # 优先级: 距离越近优先级越高 (负数, 因为 PriorityQueue 最小优先)
                priority = dist
                self._load_queue.put((priority, (cx, cy)))
                self.loading_chunks.add((cx, cy))

# 检查需要卸载的区块
to_unload = []
for (cx, cy), chunk_np in self.loaded_chunks.items():
    dist = self._chunk_distance(cx, cy, player_cx, player_cy)
    if dist > self.UNLOAD_RADIUS:
        to_unload.append((cx, cy))

for chunk_key in to_unload:
    self._unload_chunk(chunk_key)

return Task.cont

def _loader_worker(self):
    """后台线程: 从磁盘加载区块数据"""
    while True:
        try:
            priority, (cx, cy) = self._load_queue.get(timeout=0.1)
            # 模拟加载: 实际项目中这里读取文件
            chunk_data = self._load_chunk_data(cx, cy)
            self._done_queue.put(((cx, cy), chunk_data))
        except queue.Empty:
            continue

def _load_chunk_data(self, cx, cy):
    """
    后台线程中执行: 读取区块数据
    注意: 不能操作场景图!
    """
    import time
    # 模拟文件读取延迟
    time.sleep(0.05)
    # 返回区块描述数据 (不是 NodePath)
    return {
        "cx": cx, "cy": cy,
        "model_path": "models/environment",
        "pos": self._chunk_to_world(cx, cy),
    }

def _apply_loaded_chunks(self, task):
    """主线程: 将加载完成的区块添加到场景图"""
    while not self._done_queue.empty():
        try:
            (cx, cy), chunk_data = self._done_queue.get_nowait()
            self._create_chunk_node(cx, cy, chunk_data)

```

```

        self.loading_chunks.discard((cx, cy))
    except queue.Empty:
        break
    return Task.cont

def _create_chunk_node(self, cx, cy, data):
    """主线程：创建区块场景节点"""
    if (cx, cy) in self.loaded_chunks:
        return # 已经加载

    # 在主线程中安全地操作场景图
    chunk_np = self.render.attachNewNode(f"chunk_{cx}_{cy}")
    chunk_np.setPos(data["pos"])

    # 加载模型（同步，因为数据已在内存中）
    model = self.loader.loadModel(data["model_path"])
    if model:
        model.reparentTo(chunk_np)

    self.loaded_chunks[(cx, cy)] = chunk_np

def _unload_chunk(self, chunk_key):
    """卸载区块"""
    if chunk_key in self.loaded_chunks:
        chunk_np = self.loaded_chunks.pop(chunk_key)
        chunk_np.removeNode() # 从场景图移除并释放内存

def _move_player(self, task):
    """模拟玩家移动（圆形路径）"""
    dt = globalClock.getDt()
    self._move_angle += dt * 20 # 每秒移动 20 度
    radius = 150.0
    x = math.cos(math.radians(self._move_angle)) * radius
    y = math.sin(math.radians(self._move_angle)) * radius
    self.player_pos = Point3(x, y, 0)
    return Task.cont

```

5.4 Python：使用 Panda3D 内置异步加载

```

"""
使用 Panda3D 内置的异步加载器（推荐方式）
"""
from direct.showbase.ShowBase import ShowBase
from direct.task import Task
from panda3d.core import LoaderOptions, NodePath

class AsyncLoadingDemo(ShowBase):
    def __init__(self):
        super().__init__()

        self._pending = {} # {request_id: callback}

        # — 基本异步加载 —————
        self.loader.loadModel(
            "models/environment",
            callback=self._on_env_loaded,
        )

        # — 批量异步加载 —————
        models_to_load = [
            "models/tree_high",
            "models/tree_mid",
            "models/tree_low",
        ]
        self._lod_models = [None] * len(models_to_load)
        self._lod_loaded_count = 0

        for i, path in enumerate(models_to_load):
            self.loader.loadModel(
                path,
                callback=self._on_lod_loaded,
                extraArgs=[i],
            )

        # — 带优先级的加载 —————
        options = LoaderOptions()
        # LoaderOptions 可以控制缓存行为
        # options.setFlags(LoaderOptions.LF_no_cache) # 不使用缓存

        self.loader.loadModel(
            "models/important_model",
            callback=self._on_important_loaded,
            loaderOptions=options,
        )

    def _on_env_loaded(self, model):
        """环境模型加载完成"""
        if model:
            model.reparentTo(self.render)
            print("环境加载完成")
        else:
            print("环境加载失败")

    def _on_lod_loaded(self, model, index):
        """LOD 模型加载完成"""
        self._lod_models[index] = model
        self._lod_loaded_count += 1

        if self._lod_loaded_count >= len(self._lod_models):

```

```

        # 所有 LOD 级别都加载完成
        self._setup_lod()

def _setup_lod(self):
    """所有 LOD 模型就绪, 创建 LOD 节点"""
    from panda3d.core import LODNode
    lod_node = LODNode("tree")
    lod_np = self.render.attachNewNode(lod_node)

    distances = [(60, 0), (180, 55), (500, 170)]
    for i, (model, (switch_in, switch_out)) in enumerate(
        zip(self._lod_models, distances)
    ):
        if model:
            model.reparentTo(lod_np)
            lod_node.addSwitch(switch_in, switch_out)

    print("LOD 树创建完成")

def _on_important_loaded(self, model):
    if model:
        model.reparentTo(self.render)

## — 预加载系统 —————
class PreloadSystem(ShowBase):
    """
    预加载系统: 在游戏开始前预加载所有资源
    显示加载进度条
    """
    def __init__(self):
        super().__init__()

        self._resources = {
            "environment": None,
            "player": None,
            "enemies": [],
        }
        self._total = 5
        self._loaded = 0

        self._show_loading_ui()
        self._start_preload()

    def _show_loading_ui(self):
        from direct.gui.DirectGui import DirectWaitBar, DirectLabel
        self._bar = DirectWaitBar(
            text="加载资源...",
            value=0,
            pos=(0, 0, -0.1),
            scale=0.8,
        )
        self._label = DirectLabel(
            text="正在初始化...",
            pos=(0, 0, 0.1),
            scale=0.07,
        )

    def _start_preload(self):
        """开始预加载所有资源"""
        assets = [

```



```

        ("models/environment", self._on_env_ready),
        ("models/player", self._on_player_ready),
        ("models/enemy1", lambda m: self._on_enemy_ready(m, 0)),
        ("models/enemy2", lambda m: self._on_enemy_ready(m, 1)),
        ("models/enemy3", lambda m: self._on_enemy_ready(m, 2)),
    ]
    for path, callback in assets:
        self.loader.loadModel(path, callback=callback)

def _update_progress(self, name):
    self._loaded += 1
    progress = (self._loaded / self._total) * 100
    self._bar["value"] = progress
    self._label["text"] = f"已加载: {name}"

    if self._loaded >= self._total:
        self._on_all_loaded()

def _on_env_ready(self, model):
    self._resources["environment"] = model
    self._update_progress("环境")

def _on_player_ready(self, model):
    self._resources["player"] = model
    self._update_progress("玩家")

def _on_enemy_ready(self, model, index):
    self._resources["enemies"].append(model)
    self._update_progress(f"敌人{index+1}")

def _on_all_loaded(self):
    """所有资源加载完成, 开始游戏"""
    self._bar.destroy()
    self._label.destroy()
    self._start_game()

def _start_game(self):
    """将预加载的资源添加到场景"""
    if self._resources["environment"]:
        self._resources["environment"].reparentTo(self.render)
    print("游戏开始! ")

```

5.5 Python : LOD 与流式加载结合

```

"""
完整示例: LOD + 流式加载的结合使用
"""

from direct.showbase.ShowBase import ShowBase
from direct.task import Task
from panda3d.core import LODNode, FadeLODNode, NodePath, Point3
import math
import queue
import threading

class OpenWorldDemo(ShowBase):
    """
    开放世界演示:
    - 近处: 高精度 LOD
    - 中距离: 低精度 LOD
    - 远处: 广告牌
    - 极远处: 流式卸载
    """

    CHUNK_SIZE = 200.0
    LOAD_RADIUS = 2
    UNLOAD_RADIUS = 4

    def __init__(self):
        super().__init__()

        self.loaded_chunks = {}
        self._load_queue = queue.PriorityQueue()
        self._done_queue = queue.Queue()
        self.player_pos = Point3(0, 0, 0)

        # 启动加载线程
        threading.Thread(target=self._loader_worker, daemon=True).start()

        # 任务
        self.taskMgr.add(self._update, "Update")
        self.taskMgr.add(self._apply_chunks, "ApplyChunks")

        # 相机 LOD 设置
        self.cam.node().setLodScale(1.2) # 稍微偏向性能

    def _create_lod_tree(self, parent, pos):
        """创建带 LOD 的树"""
        # 使用 FadeLODNode 实现平滑过渡
        lod = FadeLODNode("tree")
        lod.setFadeTime(0.3)
        lod_np = parent.attachNewNode(lod)
        lod_np.setPos(pos)

        # 高精度 (近处)
        high = self.loader.loadModel("models/environment")
        high.setScale(0.3)
        high.reparentTo(lod_np)
        lod.addSwitch(80, 0)

        # 低精度 (中距离)
        low = self.loader.loadModel("models/environment")
        low.setScale(0.3)
        low.reparentTo(lod_np)
        lod.addSwitch(250, 70)

```

```

        return lod_np

def _loader_worker(self):
    while True:
        try:
            _, (cx, cy) = self._load_queue.get(timeout=0.1)
            import time; time.sleep(0.02) # 模拟 I/O
            self._done_queue.put((cx, cy))
        except queue.Empty:
            continue

def _update(self, task):
    # 模拟玩家移动
    t = task.time
    self.player_pos = Point3(
        math.cos(t * 0.3) * 300,
        math.sin(t * 0.3) * 300,
        0
    )

    # 更新流式加载
    pcx = int(math.floor(self.player_pos.x / self.CHUNK_SIZE))
    pcy = int(math.floor(self.player_pos.y / self.CHUNK_SIZE))

    for dx in range(-self.LOAD_RADIUS, self.LOAD_RADIUS + 1):
        for dy in range(-self.LOAD_RADIUS, self.LOAD_RADIUS + 1):
            cx, cy = pcx + dx, pcy + dy
            dist = math.sqrt(dx*dx + dy*dy)
            if dist <= self.LOAD_RADIUS and (cx, cy) not in self.loaded_chunks:
                self._load_queue.put((dist, (cx, cy)))
                self.loaded_chunks[(cx, cy)] = None # 标记为加载中

    # 卸载远处区块
    to_remove = [
        k for k, v in self.loaded_chunks.items()
        if v is not None and math.sqrt(
            (k[0]-pcx)**2 + (k[1]-pcy)**2
        ) > self.UNLOAD_RADIUS
    ]
    for k in to_remove:
        self.loaded_chunks[k].removeNode()
        del self.loaded_chunks[k]

    return Task.cont

def _apply_chunks(self, task):
    while not self._done_queue.empty():
        try:
            cx, cy = self._done_queue.get_nowait()
            if (cx, cy) in self.loaded_chunks:
                chunk = self.render.attachNewNode(f"chunk_{cx}_{cy}")
                cx_world = (cx + 0.5) * self.CHUNK_SIZE
                cy_world = (cy + 0.5) * self.CHUNK_SIZE
                chunk.setPos(cx_world, cy_world, 0)

            # 在区块中放置带 LOD 的树
            import random
            rng = random.Random(cx * 1000 + cy)
            for _ in range(5):
                x = rng.uniform(-self.CHUNK_SIZE/2, self.CHUNK_SIZE/2)

```

```
        y = rng.uniform(-self.CHUNK_SIZE/2, self.CHUNK_SIZE/2)
        self._create_lod_tree(chunk, Point3(x, y, 0))

        self.loaded_chunks[(cx, cy)] = chunk
    except queue.Empty:
        break
    return Task.cont
```

6. 性能优化

6.1 LOD 切换距离调优

```
## — 基于屏幕空间误差计算切换距离 —————
import math

def compute_lod_distance(world_error, screen_height, fov_deg, max_sse_pixels=2.0):
    """
    根据屏幕空间误差计算 LOD 切换距离

    world_error: 模型简化引入的世界空间误差 (米)
    screen_height: 屏幕高度 (像素)
    fov_deg: 垂直 FOV (度)
    max_sse_pixels: 最大允许屏幕空间误差 (像素)
    """
    fov_rad = math.radians(fov_deg)
    distance = (world_error * screen_height) / (2 * max_sse_pixels * math.tan(fov_rad / 2))
    return distance

## 示例: 树模型, LOD0→LOD1 的世界误差约 0.5m
## 屏幕 1080p, FOV 60度, 允许 2 像素误差
d = compute_lod_distance(0.5, 1080, 60, 2.0)
print(f"LOD0→LOD1 切换距离: {d:.1f}m") # 约 93m
```

6.2 LOD 性能对比

场景: 1000 棵树, 玩家在中心

无 LOD (全部 LOD0) :

三角形数: 50,000,000

帧时间: 45ms (22 FPS)

有 LOD (4级) :

LOD0 (0-55m): ~20棵树 × 50,000 = 1,000,000 三角形

LOD1 (55-160m): ~80棵树 × 5,000 = 400,000 三角形

LOD2 (160-420m): ~300棵树 × 500 = 150,000 三角形

LOD3 (420-850m): ~600棵树 × 50 = 30,000 三角形

总计: 1,580,000 三角形

帧时间: 8ms (125 FPS)

提升: 5.6x

6.3 流式加载最佳实践

```
## ❌ 错误：在主线程同步加载（卡顿）
def bad_load_chunk(cx, cy):
    model = loader.loadModel("models/chunk") # 阻塞主线程!
    model.reparentTo(render)

## ✅ 正确：异步加载
def good_load_chunk(cx, cy):
    loader.loadModel(
        "models/chunk",
        callback=lambda m: _on_chunk_loaded(m, cx, cy),
    )

## ❌ 错误：每帧检查所有区块 ( $O(n^2)$ )
def bad_update():
    for chunk in all_possible_chunks: # 可能有数千个
        if distance(chunk, player) < load_radius:
            load(chunk)

## ✅ 正确：只检查玩家周围的区块 ( $O(r^2)$ )
def good_update():
    pcx, pcy = world_to_chunk(player_pos)
    for dx in range(-LOAD_RADIUS, LOAD_RADIUS + 1):
        for dy in range(-LOAD_RADIUS, LOAD_RADIUS + 1):
            check_chunk(pcx + dx, pcy + dy)
```

6.4 内存管理

```
## 使用 ModelPool 缓存已加载的模型（避免重复加载）
from panda3d.core import ModelPool

## 检查模型是否已在缓存中
if ModelPool.hasModel("models/tree"):
    model = ModelPool.loadModel("models/tree")
else:
    model = loader.loadModel("models/tree")

## 卸载时从缓存中移除
ModelPool.releaseModel("models/tree")

## 清空所有缓存（场景切换时）
ModelPool.garbageCollect()
```

小结

知识点	核心要点
LODNode	in/out 双阈值防抖动, <code>compute_child()</code> 基于距离平方
FadeLODNode	Alpha 混合平滑过渡, 过渡期间 draw call 翻倍
屏幕空间误差	LOD 切换距离的理论依据, $SSE = \varepsilon \cdot H / (2d \cdot \tan(\theta/2))$
lod_scale	相机/节点的 LOD 缩放因子, 快速调整性能/质量平衡
流式加载	后台线程读取数据, 主线程应用到场景图
区块系统	将世界划分为固定大小区块, 按距离加载/卸载
优先级队列	距离近、可见性高的区块优先加载
ModelPool	缓存已加载模型, 避免重复磁盘读取

关键设计原则

1. **LOD 切换距离基于屏幕空间误差**: 不要随意设置距离, 应根据模型误差和屏幕分辨率计算
2. **使用距离平方比较**: 避免 `sqrt()` 开销, `in_range_2()` 使用距离平方
3. **FadeLOD 有代价**: 过渡期间渲染两个 LOD, 仅在视觉质量要求高时使用
4. **流式加载必须异步**: 任何磁盘 I/O 都不能在主线程中同步执行
5. **卸载半径 > 加载半径**: 防止在边界处频繁加载/卸载 (滞后设计)
6. **区块大小权衡**: 太小→管理开销大; 太大→内存浪费、加载时间长

与其他专题的关联

- **专题02 (场景图)**: LODNode 是 PandaNode 的子类, 参与场景图遍历
- **专题08 (纹理管理)**: 流式加载也适用于纹理 (Mipmap 流式加载)
- **专题10 (多线程)**: 流式加载依赖后台线程和命令队列模式
- **专题11 (后处理)**: LOD 减少几何体, 后处理减少像素计算, 共同优化性能

专题13：跨平台图形 API 抽象

目录

1. 问题背景
 2. 数学原理
 3. 工程实践
 4. Panda3D 源码剖析
 5. 代码演示
 6. 性能优化
 7. 小结
-

1. 问题背景

不同平台使用不同的图形 API，3D 引擎必须在它们之上建立统一的抽象层：

平台	图形 API	坐标系	深度范围	NDC 原点
Windows	DirectX 11/12	左手系	[0, 1]	左上角
Windows/Linux/macOS	OpenGL	右手系	[-1, 1]	左下角
macOS/iOS	Metal	左手系	[0, 1]	左上角
Android/iOS	OpenGL ES	右手系	[-1, 1]	左下角
跨平台	Vulkan	右手系	[0, 1]	左上角

核心挑战：

挑战	描述
坐标系差异	Y-Up vs Z-Up, 左手系 vs 右手系
深度范围	OpenGL [-1,1] vs DirectX/Metal/Vulkan [0,1]
纹理坐标原点	OpenGL 左下角 vs DirectX 左上角
Shader 语言	GLSL vs HLSL vs MSL
状态机差异	OpenGL 全局状态 vs Vulkan 显式管线对象
内存模型	隐式同步 vs 显式屏障

2. 数学原理

2.1 坐标系转换矩阵

Z-Up 右手系 (Panda3D 默认) → Y-Up 右手系 (OpenGL) :

$$M_{ZtoY} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

即: $x' = x$, $y' = -z$, $z' = y$

右手系 → 左手系 (翻转 Z 轴) :

$$M_{RtoL} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

2.2 投影矩阵的差异

OpenGL 透视投影 (NDC Z 范围 [-1, 1], 右手系) :

$$P_{GL} = \begin{bmatrix} \frac{2n}{r-l} & 0 & \frac{r+l}{r-l} & 0 \\ 0 & \frac{2n}{t-b} & \frac{t+b}{t-b} & 0 \\ 0 & -\frac{f+n}{f-n} & -\frac{2fn}{f-n} & -1 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

DirectX 透视投影 (NDC Z 范围 [0, 1], 左手系) :

$$P_{DX} = \begin{bmatrix} \frac{2n}{r-l} & 0 & -\frac{r+l}{r-l} & 0 \\ 0 & \frac{2n}{t-b} & -\frac{t+b}{t-b} & 0 \\ 0 & \frac{f}{f-n} & -\frac{fn}{f-n} & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

关键差异: - 第 3 行第 3 列: GL 为 $-\frac{f+n}{f-n}$, DX 为 $\frac{f}{f-n}$ - 第 4 行第 3 列: GL 为 -1 (右手系), DX 为 1 (左手系)

2.3 纹理坐标翻转

OpenGL 纹理坐标原点在左下角, DirectX 在左上角:

$$v_{DX} = 1 - v_{GL}$$

在 Shader 中处理:

```
// OpenGL (不需要翻转)
vec2 uv = texcoord;

// 如果纹理来自 DirectX 工具链
vec2 uv = vec2(texcoord.x, 1.0 - texcoord.y);
```

2.4 深度值线性化

不同 API 的深度缓冲值含义不同，线性化公式：

OpenGL (NDC 深度 $[-1, 1]$) : $z_{\text{linear}} = \frac{2f_n}{f + n - z_{\text{ndc}}(f - n)}$

DirectX/Vulkan (NDC 深度 $[0, 1]$) : $z_{\text{linear}} = \frac{f_n}{f - z_{\text{ndc}}(f - n)}$

3. 工程实践

3.1 Panda3D 的抽象层设计

Panda3D 使用策略模式（Strategy Pattern）实现跨平台：

```
GraphicsStateGuardian (抽象基类)
├─ GLGraphicsStateGuardian (OpenGL 实现)
│   ├── glxGraphicsStateGuardian (Linux/X11)
│   ├── wglGraphicsStateGuardian (Windows)
│   └── cocoglGraphicsStateGuardian (macOS)
├─ DXGraphicsStateGuardian (DirectX 实现)
└─ (未来可添加 VulkanGSG、MetalGSG)
```

每个 GSG 实现相同的虚函数接口，引擎代码只调用抽象接口。

3.2 坐标系统一策略

Panda3D 在内部使用 Z-Up 右手系（CS_zup_right），在渲染时通过 cs_transform 矩阵转换到 API 的坐标系：

```
应用层 (Z-Up 右手系)
    ↓ cs_transform (坐标系转换矩阵)
GSG 内部 (Y-Up 右手系, OpenGL 约定)
    ↓ 投影矩阵
NDC 空间 (各 API 不同)
```

3.3 Shader 语言抽象

Panda3D 支持多种 Shader 语言：

语言	枚举值	平台
GLSL	SL_GLSL	OpenGL/OpenGL ES
HLSL	SL_HLSL	DirectX
Cg	SL_Cg	跨平台（已废弃）
SPIR-V	SL_SPIR_V	Vulkan

3.4 能力查询系统

不同 GPU/驱动支持不同特性，引擎必须在运行时查询：

```
// GSG 能力标志
bool _supports_multisample;
bool _supports_generate_mipmap;
bool _supports_render_texture;
bool _supports_depth_texture;
bool _supports_shadow_filter;
bool _supports_basic_shaders;
bool _supports_glsl;
bool _supports_framebuffer_multisample;
bool _supports_framebuffer_blit;
```

4. Panda3D 源码剖析

4.1 CoordinateSystem 枚举

coordinateSystem.h:21

```
enum CoordinateSystem {
    CS_default,      // 从配置文件读取 (默认 CS_zup_right)

    CS_zup_right,    // Z-Up, 右手系 (Panda3D 默认, 类似 Maya/Blender)
    CS_yup_right,    // Y-Up, 右手系 (OpenGL 约定, 类似 Unity)
    CS_zup_left,     // Z-Up, 左手系
    CS_yup_left,     // Y-Up, 左手系 (DirectX 约定)

    CS_invalid,      // 无效值
};

// 辅助函数
bool is_right_handed(CoordinateSystem cs = CS_default);

// 坐标系转换矩阵 (预计算)
// LMatrix4::convert_mat(from, to) 返回转换矩阵
```

坐标系对比:

CS_zup_right (Panda3D) :	CS_yup_right (OpenGL) :
Z	Y
Y	Z
/	/
/_X	/_X
前方 = +Y	前方 = -Z
上方 = +Z	上方 = +Y
右方 = +X	右方 = +X

4.2 GraphicsStateGuardian : 坐标系处理

graphicsStateGuardian.h:67


```

class GraphicsStateGuardian : public GraphicsStateGuardianBase {
    // 构造时指定内部坐标系
    GraphicsStateGuardian(
        CoordinateSystem internal_coordinate_system, // API 使用的坐标系
        GraphicsEngine *engine,
        GraphicsPipe *pipe);

    // 坐标系接口
    void set_coordinate_system(CoordinateSystem cs); // 应用层坐标系
    CoordinateSystem get_coordinate_system() const; // 应用层坐标系
    virtual CoordinateSystem get_internal_coordinate_system() const; // API 坐标系

    // 坐标系转换矩阵 (应用层 → API 层)
    virtual CPT(TransformState) get_cs_transform() const;
    virtual CPT(TransformState) get_cs_transform_for(CoordinateSystem cs) const;

    // 内部数据
    CoordinateSystem _coordinate_system; // 应用层 (通常 CS_zup_right)
    CoordinateSystem _internal_coordinate_system; // API 层 (OpenGL: CS_yup_right)
    CPT(TransformState) _cs_transform; // 预计算的转换矩阵
};

```

OpenGL GSG 的初始化 (伪代码) :

```

// OpenGL 使用 Y-Up 右手系
GLGraphicsStateGuardian::GLGraphicsStateGuardian(...)
: GraphicsStateGuardian(CS_yup_right, engine, pipe) {
    // _internal_coordinate_system = CS_yup_right
    // 当应用层使用 CS_zup_right 时,
    // _cs_transform = LMatrix4::convert_mat(CS_zup_right, CS_yup_right)
}

```

4.3 ShaderModel 枚举

graphicsStateGuardian.h:73

```

enum ShaderModel {
    SM_00, // 无 Shader 支持
    SM_11, // Shader Model 1.1 (DirectX 8)
    SM_20, // Shader Model 2.0 (DirectX 9)
    SM_2X, // Shader Model 2.x (扩展)
    SM_30, // Shader Model 3.0 (DirectX 9c)
    SM_40, // Shader Model 4.0 (DirectX 10)
    SM_50, // Shader Model 5.0 (DirectX 11)
    SM_51, // Shader Model 5.1 (DirectX 12)
};

```

4.4 GraphicsPipe：平台管道

```
class GraphicsPipe {
    // 平台特定的输出类型
    enum OutputTypes {
        OT_window          = 0x0001, // 普通窗口
        OT_fullscreen_window = 0x0002, // 全屏窗口
        OT_buffer           = 0x0004, // 离屏缓冲
        OT_texture_buffer   = 0x0008, // 纹理缓冲
    };

    // 创建图形输出（平台特定实现）
    virtual PT(GraphicsOutput) make_output(
        std::string_view name,
        const FrameBufferProperties &fb_prop,
        const WindowProperties &win_prop,
        int flags,
        GraphicsEngine *engine,
        GraphicsStateGuardian *gsg,
        GraphicsOutput *host,
        int retry,
        bool &precertify);
};
```

4.5 LMatrix4::convert_mat()

lmatrix4_src.h:263

```
// 预计算所有坐标系对之间的转换矩阵
static const LMatrix4 &convert_mat(CoordinateSystem from, CoordinateSystem to);

// 内部实现（伪代码）：
// CS_zup_right → CS_yup_right:
// | 1  0  0  0 |
// | 0  0  1  0 |   x'=x, y'=z, z'=-y
// | 0 -1  0  0 |
// | 0  0  0  1 |
```

5. 代码演示

5.1 Python：查询平台能力

```

"""
查询当前平台的图形能力
"""

from direct.showbase.ShowBase import ShowBase

class PlatformCapabilityDemo(ShowBase):
    def __init__(self):
        super().__init__()
        self._print_platform_info()

    def _print_platform_info(self):
        gsg = self.win.getGsg()

        print("=== 平台图形能力 ===")
        print(f"渲染器: {gsg.getDriverRenderer()}")
        print(f"供应商: {gsg.getDriverVendor()}")
        print(f"驱动版本: {gsg.getDriverVersion()}")
        print()

        # 坐标系信息
        from panda3d.core import CoordinateSystem
        cs = gsg.getCoordinateSystem()
        internal_cs = gsg.getInternalCoordinateSystem()
        print(f"应用层坐标系: {cs}")
        print(f"API 内部坐标系: {internal_cs}")
        print()

        # Shader 支持
        print(f"Shader Model: {gsg.getShaderModel()}")
        print(f"支持 GLSL: {gsg.getSupportsGlsl()}")
        print(f"支持基础 Shader: {gsg.getSupportsBasicShaders()}")
        print()

        # 纹理能力
        print(f"最大纹理尺寸: {gsg.getMaxTextureDimension()}")
        print(f"最大纹理阶段: {gsg.getMaxTextureStages()}")
        print(f"支持 3D 纹理: {gsg.getSupports3dTexture()}")
        print(f"支持立方体贴图: {gsg.getSupportsCubeMap()}")
        print(f"支持渲染到纹理: {gsg.getSupportsRenderTexture()}")
        print()

        # 帧缓冲能力
        print(f"支持多重采样: {gsg.getSupportsFramebufferMultisample()}")
        print(f"支持 FBO Blit: {gsg.getSupportsFramebufferBlit()}")
        print()

        # 几何能力
        print(f"最大顶点数/数组: {gsg.getMaxVerticesPerArray()}")
        print(f"最大顶点数/图元: {gsg.getMaxVerticesPerPrimitive()}")

        # 压缩纹理支持
        from panda3d.core import Texture
        for mode_name, mode in [
            ("DXT1", Texture.CMDxt1),
            ("DXT3", Texture.CMDxt3),
            ("DXT5", Texture.CMDxt5),
            ("ETC1", Texture.CMEtc1),
            ("ETC2", Texture.CMEtc2),
        ]:

```

```
supported = gsg.getSupportsCompressedTextureFormat(mode)
print(f"压缩格式 {mode_name}: {'支持' if supported else '不支持'}")
```

5.2 Python：坐标系转换

```

"""
坐标系转换演示
Panda3D 内部使用 Z-Up 右手系，但可以与其他坐标系互操作
"""

from panda3d.core import (
    LMatrix4, LPoint3, LVector3,
    CoordinateSystem, CS_zup_right, CS_yup_right, CS_yup_left
)

def demo_coordinate_systems():
    """演示不同坐标系之间的转换"""

    # — 获取坐标系转换矩阵 —————
    # Panda3D (Z-Up 右手系) → OpenGL (Y-Up 右手系)
    zup_to_yup = LMatrix4.convertMat(CS_zup_right, CS_yup_right)
    print("Z-Up → Y-Up 转换矩阵:")
    print(zup_to_yup)

    # Panda3D (Z-Up 右手系) → DirectX (Y-Up 左手系)
    zup_to_dx = LMatrix4.convertMat(CS_zup_right, CS_yup_left)
    print("\nZ-Up → DirectX(Y-Up Left) 转换矩阵:")
    print(zup_to_dx)

    # — 坐标转换示例 —————
    # Panda3D 中的"前方"是 +Y 方向
    panda_forward = LVector3(0, 1, 0) # Panda3D 前方

    # 转换到 OpenGL 坐标系 (前方应该是 -Z)
    gl_forward = zup_to_yup.xformVec(panda_forward)
    print(f"\nPanda3D 前方 {panda_forward} → OpenGL {gl_forward}")
    # 期望输出: (0, 0, -1)

    # — 检查坐标系手性 —————
    from panda3d.core import is_right_handed
    print(f"\nCS_zup_right 是右手系: {is_right_handed(CS_zup_right)}")
    print(f"CS_yup_left 是右手系: {is_right_handed(CS_yup_left)}")

    # — 在 Panda3D 中设置坐标系 —————
    # 通过 config.prc 设置 (必须在 ShowBase 之前):
    # coordinate-system y-up-right # 使用 Y-Up 右手系 (类似 Unity)
    # coordinate-system z-up-right # 使用 Z-Up 右手系 (Panda3D 默认)

    # 或通过代码:
    from panda3d.core import loadPrcFileData
    # loadPrcFileData("", "coordinate-system y-up-right")

## — 处理来自不同工具模型 —————
def load_model_with_coordinate_fix(loader, path, source_cs):
    """
    加载来自不同坐标系工具模型并修正坐标系

    source_cs: 模型来源的坐标系
    - CS_yup_right: Unity, Maya, Blender (默认导出)
    - CS_zup_right: 3ds Max, Panda3D
    - CS_yup_left: DirectX 工具
    """
    model = loader.loadModel(path)
    if model is None:

```

```
    return None

# 如果源坐标系与 Panda3D 不同, 应用转换
if source_cs != CS_zup_right:
    convert_mat = LMatrix4.convertMat(source_cs, CS_zup_right)
    model.setMat(convert_mat)

return model
```


5.3 Python：跨平台 Shader 编写

```

"""
编写跨平台兼容的 Shader
"""

from direct.showbase.ShowBase import ShowBase
from panda3d.core import Shader, loadPrcFileData

## — GLSL Shader (OpenGL/OpenGL ES) —————
VERT_GLSL = """
#version 330
// Panda3D 自动提供的 uniform
uniform mat4 p3d_ModelViewProjectionMatrix;
uniform mat4 p3d_ModelMatrix;
uniform mat3 p3d_NormalMatrix;

// 顶点属性
in vec4 p3d_Vertex;
in vec3 p3d_Normal;
in vec2 p3d_MultiTexCoord0;

// 输出到片段着色器
out vec3 world_normal;
out vec2 texcoord;
out vec3 world_pos;

void main() {
    gl_Position = p3d_ModelViewProjectionMatrix * p3d_Vertex;

    // 世界空间法线 (注意: Panda3D 已处理坐标系转换)
    world_normal = normalize(p3d_NormalMatrix * p3d_Normal);

    // 世界空间位置
    world_pos = (p3d_ModelMatrix * p3d_Vertex).xyz;

    texcoord = p3d_MultiTexCoord0;
}
"""

FRAG_GLSL = """
#version 330
uniform sampler2D p3d_Texture0;
uniform vec4 p3d_ColorScale;

in vec3 world_normal;
in vec2 texcoord;
in vec3 world_pos;

out vec4 fragColor;

void main() {
    vec4 tex_color = texture(p3d_Texture0, texcoord);
    vec3 normal = normalize(world_normal);

    // 简单漫反射
    vec3 light_dir = normalize(vec3(1, 1, 1));
    float diffuse = max(dot(normal, light_dir), 0.0);

    fragColor = tex_color * p3d_ColorScale * (0.3 + 0.7 * diffuse);
}
"""

```

```

class CrossPlatformShaderDemo(ShowBase):
    def __init__(self):
        super().__init__()

        # 检测平台并选择合适的 Shader
        gsg = self.win.getGsg()

        if gsg.getSupportsGlsl():
            # OpenGL 平台: 使用 GLSL
            shader = Shader.make(Shader.SL_GLSL, VERT_GLSL, FRAG_GLSL)
            print("使用 GLSL Shader")
        else:
            # 回退: 使用 Panda3D 自动生成的 Shader
            shader = None
            print("使用自动生成 Shader")

        model = self.loader.loadModel("models/environment")
        model.reparentTo(self.render)

        if shader:
            model.setShader(shader)
        else:
            # 启用自动 Shader 生成
            self.render.setShaderAuto()

```

— 跨平台 Shader 最佳实践 —————

```

PORTABLE_VERT = """
#version 130
// 使用 Panda3D 的内置 uniform (自动适配坐标系)
uniform mat4 p3d_ModelViewProjectionMatrix;

// 使用 Panda3D 的内置属性名
in vec4 p3d_Vertex;
in vec2 p3d_MultiTexCoord0;
out vec2 uv;

void main() {
    // p3d_ModelViewProjectionMatrix 已包含坐标系转换
    gl_Position = p3d_ModelViewProjectionMatrix * p3d_Vertex;
    uv = p3d_MultiTexCoord0;
}
"""

PORTABLE_FRAG = """
#version 130
uniform sampler2D p3d_Texture0;
in vec2 uv;
out vec4 color;

void main() {
    // Panda3D 已处理纹理坐标翻转
    color = texture(p3d_Texture0, uv);
}
"""

```

5.4 Python：平台特定配置

```

"""
根据平台自动配置最优设置
"""
import sys
import platform
from panda3d.core import loadPrcFileData

def configure_for_platform():
    """在 ShowBase() 之前调用, 根据平台配置最优设置"""

    system = platform.system()
    machine = platform.machine()

    if system == "Windows":
        # Windows: 优先使用 DirectX (如果可用)
        # 或者使用 OpenGL (更好的跨平台兼容性)
        loadPrcFileData("", "load-display pandagl") # 强制 OpenGL
        # loadPrcFileData("", "load-display pandadx9") # DirectX 9
        # loadPrcFileData("", "load-display pandadx11") # DirectX 11

        # Windows 特定优化
        loadPrcFileData("", "win-size 1920 1080")
        loadPrcFileData("", "fullscreen false")

    elif system == "Darwin": # macOS
        # macOS: 使用 OpenGL (Metal 支持通过 MoltenVK)
        loadPrcFileData("", "load-display pandagl")

        # macOS 高 DPI 支持
        loadPrcFileData("", "framebuffer-hardware true")

    elif system == "Linux":
        # Linux: OpenGL
        loadPrcFileData("", "load-display pandagl")

        # 检查是否有 NVIDIA GPU
        # loadPrcFileData("", "threading-model Cull/Draw")

    # 移动平台 (Android/iOS)
    elif system == "Android" or (system == "Linux" and "arm" in machine.lower()):
        # OpenGL ES
        loadPrcFileData("", "load-display pandagles2")
        # 降低质量以适应移动 GPU
        loadPrcFileData("", "texture-quality-when-loaded fastest")
        loadPrcFileData("", "compressed-textures true")

    # 通用设置
    loadPrcFileData("", "sync-video false") # 不等待垂直同步 (测试用)
    loadPrcFileData("", "show-frame-rate-meter true")

class PlatformAdaptiveApp>ShowBase):
    def __init__(self):
        configure_for_platform() # 必须在 super().__init__() 之前
        super().__init__()

        gsg = self.win.getGsg()
        self._adapt_to_capabilities(gsg)

    def _adapt_to_capabilities(self, gsg):

```

```

"""根据实际 GPU 能力调整设置"""

# 根据 Shader Model 选择渲染路径
sm = gsg.getShaderModel()
if sm >= gsg.SM_50:
    print("高端 GPU : 启用全部效果")
    self._setup_high_quality()
elif sm >= gsg.SM_30:
    print("中端 GPU : 启用部分效果")
    self._setup_medium_quality()
else:
    print("低端 GPU : 使用固定管线")
    self._setup_low_quality()

# 根据纹理大小限制调整纹理质量
max_tex = gsg.getMaxTextureDimension()
if max_tex < 2048:
    print(f"纹理限制: {max_tex}, 降低纹理质量")
    from panda3d.core import Texture
    # 限制纹理大小
    # Texture.setTexturesPower2(True)

def _setup_high_quality(self):
    """高质量设置"""
    from direct.filter.CommonFilters import CommonFilters
    self.filters = CommonFilters(self.win, self.cam)
    self.filters.setBloom(intensity=1.0, size="medium")
    self.filters.setAmbientOcclusion(numsamples=16)
    self.render.setShaderAuto()

def _setup_medium_quality(self):
    """中等质量设置"""
    self.render.setShaderAuto()

def _setup_low_quality(self):
    """低质量设置 (固定管线) """
    # 不使用 Shader, 使用固定管线
    pass

```

5.5 GLSL：处理坐标系差异

```
/* 在 Shader 中处理不同平台的坐标系差异 */

/* — 顶点着色器：坐标系感知 — */
#version 330

uniform mat4 p3d_ModelViewProjectionMatrix;
uniform mat4 p3d_ModelMatrix;
uniform mat4 p3d_ViewMatrix;

in vec4 p3d_Vertex;
in vec3 p3d_Normal;
in vec2 p3d_MultiTexCoord0;

out vec3 view_normal;
out vec2 texcoord;

void main() {
    gl_Position = p3d_ModelViewProjectionMatrix * p3d_Vertex;

    // Panda3D 的 p3d_ModelViewProjectionMatrix 已经包含了
    // 从应用层坐标系 (Z-Up) 到 OpenGL NDC 的完整变换
    // 不需要手动处理坐标系转换!

    // 法线变换 (使用法线矩阵避免非均匀缩放问题)
    mat3 normal_matrix = transpose(inverse(mat3(p3d_ModelMatrix)));
    view_normal = normalize(mat3(p3d_ViewMatrix) * normal_matrix * p3d_Normal);

    texcoord = p3d_MultiTexCoord0;
    // 注意: Panda3D 的纹理坐标原点在左下角 (与 OpenGL 一致)
    // 如果纹理来自 DirectX 工具, 需要翻转 V:
    // texcoord = vec2(p3d_MultiTexCoord0.x, 1.0 - p3d_MultiTexCoord0.y);
}
```

```
/* — 片段着色器：深度值处理 — */
#version 330

uniform sampler2D depth_tex;
uniform float near_plane;
uniform float far_plane;

in vec2 texcoord;
out vec4 fragColor;

/* 线性化 OpenGL 深度值 (NDC 范围 [-1, 1]) */
float linearize_depth_gl(float depth) {
    float z = depth * 2.0 - 1.0; // NDC 深度
    return (2.0 * near_plane * far_plane) /
        (far_plane + near_plane - z * (far_plane - near_plane));
}

/* 线性化 DirectX/Vulkan 深度值 (NDC 范围 [0, 1]) */
float linearize_depth_dx(float depth) {
    return (near_plane * far_plane) /
        (far_plane - depth * (far_plane - near_plane));
}

void main() {
    float raw_depth = texture(depth_tex, texcoord).r;

    // Panda3D 使用 OpenGL 约定 (即使在 DirectX 模式下也会转换)
    float linear_depth = linearize_depth_gl(raw_depth);

    // 可视化深度 (调试用)
    float normalized = linear_depth / far_plane;
    fragColor = vec4(vec3(normalized), 1.0);
}
```


6. 性能优化

6.1 平台特定优化

```
## — OpenGL 特定优化 —————
from panda3d.core import loadPrcFileData

## 启用 OpenGL 扩展优化
loadPrcFileData("", "gl-version 3 3")          # 要求 OpenGL 3.3+
loadPrcFileData("", "gl-use-vertex-array-objects true") # 使用 VAO
loadPrcFileData("", "gl-immutable-texture-storage true") # 不可变纹理存储

## 减少驱动开销
loadPrcFileData("", "gl-check-errors false")    # 生产环境关闭错误检查
loadPrcFileData("", "gl-finish false")          # 不强制同步

## — 移动平台优化 —————
## OpenGL ES 2.0/3.0
loadPrcFileData("", "load-display pandagles2")

## 减少精度（移动 GPU 对半精度更友好）
## 在 Shader 中使用 mediump 而非 highp
```

```
/* 移动平台 Shader 优化 */
#version 300 es
precision mediump float; /* 使用中等精度（移动 GPU 更快） */

uniform sampler2D tex;
in vec2 uv;
out vec4 color;

void main() {
    color = texture(tex, uv);
}
```

6.2 能力检测与回退

```
class AdaptiveRenderer:
    """根据 GPU 能力自动选择渲染路径"""

    def __init__(self, gsg):
        self.gsg = gsg
        self._select_render_path()

    def _select_render_path(self):
        gsg = self.gsg

        # 检查各种能力, 选择最优路径
        if (gsg.getSupportsBasicShaders() and
            gsg.getShaderModel() >= gsg.SM_30 and
            gsg.getSupportsRenderTexture()):
            self.path = "deferred" # 延迟渲染
            print("渲染路径: 延迟渲染")

        elif gsg.getSupportsBasicShaders():
            self.path = "forward_shaded" # 前向渲染 + Shader
            print("渲染路径: 前向渲染 (Shader) ")

        else:
            self.path = "fixed_function" # 固定管线
            print("渲染路径: 固定管线")

    def setup_scene(self, render):
        if self.path == "deferred":
            self._setup_deferred(render)
        elif self.path == "forward_shaded":
            render.setShaderAuto()
        # fixed_function: 不需要额外设置

    def _setup_deferred(self, render):
        """延迟渲染设置 (需要 MRT 支持) """
        if not self.gsg.getSupportsFramebufferMultisample():
            print("警告: 不支持 MSAA, 使用 FXAA 替代")
        render.setShaderAuto()
```

6.3 跨平台性能对比

相同场景，不同平台的渲染性能（1080p, 1000个物体）：

Windows + DirectX 11:

帧时间：8.2ms (122 FPS)

驱动开销：低（显式状态管理）

Windows/Linux + OpenGL 4.5:

帧时间：9.1ms (110 FPS)

驱动开销：中（隐式状态机）

macOS + OpenGL 4.1:

帧时间：11.3ms (88 FPS)

注意：macOS 的 OpenGL 驱动较旧，性能较低

Android + OpenGL ES 3.0:

帧时间：18.5ms (54 FPS)

限制：移动 GPU 带宽和计算能力有限

小结

知识点	核心要点
CoordinateSystem	4种坐标系：Z-Up/Y-Up × 左手/右手，Panda3D 默认 Z-Up 右手
cs_transform	GSG 内部维护坐标系转换矩阵，自动应用到所有变换
投影矩阵差异	OpenGL NDC $Z \in [-1, 1]$ ，DirectX/Metal/Vulkan $Z \in [0, 1]$
纹理坐标	OpenGL 原点左下，DirectX 原点左上，Panda3D 统一为左下
ShaderModel	SM_00 到 SM_51，决定可用的 Shader 特性
能力查询	运行时查询 GSG 能力，实现优雅降级
GraphicsPipe	平台特定的图形管道，负责创建窗口和 GSG
p3d_前缀	Panda3D 内置 uniform/attribute，自动处理坐标系和矩阵

关键设计原则

- 使用 Panda3D 内置 uniform：`p3d_ModelViewProjectionMatrix` 等已处理坐标系转换
- 运行时能力检测：不要假设特定 GPU 能力，始终查询后再使用
- 优雅降级：高端效果 → 中等效果 → 固定管线，保证在所有平台可运行
- 坐标系统一：在应用层统一使用 Panda3D 的 Z-Up 右手系，让引擎处理转换
- 平台配置分离：将平台特定配置集中在初始化阶段，避免散落在代码各处

与其他专题的关联

- 专题03（变换系统）：坐标系转换矩阵是变换系统的基础
- 专题09（Shader 系统）：Shader 语言选择（GLSL/HLSL）是跨平台的核心问题
- 专题11（后处理）：深度值的处理方式因平台而异
- 专题08（纹理）：纹理坐标原点差异影响纹理采样 ENDOFFILE

专题14：精度与数值稳定性

目录

1. 问题背景
 2. 数学原理
 3. 工程实践
 4. Panda3D 源码剖析
 5. 代码演示
 6. 性能优化
 7. 小结
-

1. 问题背景

3D 引擎中的数值精度问题是最隐蔽、最难调试的 Bug 来源之一：

问题	描述	典型症状
浮点精度	float 只有 ~7 位有效数字	远处物体抖动 (Z-fighting)
大世界坐标	坐标值过大导致精度丢失	玩家在 10km 外时物体抖动
矩阵累积误差	多次矩阵乘法累积误差	骨骼动画漂移
四元数漂移	四元数归一化误差累积	旋转轴偏移
深度冲突	两个面深度值相同	Z-fighting 闪烁
碰撞精度	浮点误差导致穿透	物体穿过薄墙

核心矛盾： - float (32位)：GPU 原生支持，但精度有限 (约 7 位十进制) - double (64位)：精度高，但 GPU 不原生支持，CPU 计算慢

2. 数学原理

2.1 IEEE 754 浮点数结构

32位 float 的二进制表示：

符号(1位) | 指数(8位) | 尾数(23位)

S | EEEEEEE | MMMMMMMMMMMMMMMMMMM

值 = $(-1)^S \times 2^{(E-127)} \times (1 + M/2^{23})$

精度分析： - 尾数 23 位 → 约 $2^{-23} \approx 1.2 \times 10^{-7}$ 相对精度 - 在坐标值 $x = 10000$ 时，最小可分辨差值 = $10000 \times 10^{-7} = 0.001$ (1毫米) - 在坐标值 $x = 100000$ 时，最小可分辨差值 = 0.01 (1厘米)

大世界问题：当玩家坐标达到 10^5 米 (100km) 时，float 精度只有 1cm，物理模拟和渲染都会出现明显抖动。

2.2 Z-Fighting 的数学分析

深度缓冲的精度分布是**非线性**的（对数分布）：

OpenGL 深度值（NDC 深度 z_{ndc} ）：

$$z_{ndc} = \frac{f+n}{f-n} - \frac{2fn}{(f-n) \cdot z_{view}}$$

深度精度（相邻深度值之间的最小可分辨距离）：

$$\Delta z_{view} \approx \frac{z_{view}^2}{fn} \cdot \frac{1}{2^{depth_bits}}$$

这意味着： - 近处 ($z_{view} = n$)：精度最高 - 远处 ($z_{view} = f$)：精度最低， $\Delta z \propto z^2$

Z-Fighting 条件：两个面的深度差 $< \Delta z_{view}$

解决方案：增大近裁剪面 n （对精度影响最大）：

$$\text{精度} \propto \frac{1}{f/n}$$

将 n 从 0.1 增大到 1.0，精度提升 10 倍。

2.3 反转深度 (Reversed-Z)

传统深度缓冲将精度集中在近处，但远处精度极差。**反转深度**技术将深度值反转：

$$z_{\text{reversed}} = 1 - z_{\text{traditional}}$$

配合浮点深度缓冲，精度分布更均匀：

方法	近处精度	远处精度
传统整数深度	高	极低
反转浮点深度	中	高

2.4 大世界坐标：相对坐标技术

问题：玩家在 (100000, 100000, 0) 处，附近物体在 (100001, 100000, 0)，差值 1.0 但 float 精度只有 0.01。

解决方案：以玩家为原点，所有渲染坐标相对于玩家：

$$\text{渲染坐标} = \text{世界坐标} - \text{玩家坐标}$$

这样渲染坐标始终在 [-1000, 1000] 范围内，float 精度足够。

实现：1. 游戏逻辑使用 double 存储世界坐标 2. 每帧渲染前，将所有物体坐标减去相机位置 3. 相机始终在原点渲染

2.5 四元数归一化误差

四元数 $q = (w, x, y, z)$ 必须满足 $|q| = 1$ 。

每次旋转操作后，由于浮点误差， $|q|$ 会略微偏离 1：

$$|q_{\text{after}}| = 1 + \epsilon, \quad |\epsilon| \approx 10^{-7}$$

经过 N 次操作后，误差累积：

$$|q_N| \approx 1 + N\epsilon$$

解决方案：定期重新归一化：

$$q_{\text{normalized}} = \frac{q}{|q|}$$

或使用快速近似归一化（牛顿迭代）：

$$q_{\text{normalized}} \approx q \cdot \frac{3 - |q|^2}{2}$$

3. 工程实践

3.1 Panda3D 的精度配置

Panda3D 支持两种精度模式：

```
## config.prc 配置
stdfloat-double true    # 使用 double 精度 (默认 false)
```

当 `stdfloat-double = true` 时，`PN_stdfloat` 类型变为 `double`，所有数学运算使用双精度。

代价：内存翻倍，CPU 计算约慢 2x，但 GPU 仍使用 float。

3.2 深度缓冲配置

```
## 增大近裁剪面 (最有效的 Z-fighting 解决方案)
lens = base.cam.node().getLens()
lens.setNear(1.0)    # 默认 0.1, 改为 1.0
lens.setFar(10000)   # 根据场景调整

## 使用 24 位深度缓冲 (默认)
fb_props = FrameBufferProperties()
fb_props.setDepthBits(24) # 或 32 位

## 使用浮点深度缓冲 (更高精度)
fb_props.setFloatDepth(True)
```

3.3 大世界解决方案

方案1：浮点原点 (Floating Origin) - 当玩家距原点超过阈值时，将所有物体坐标平移 - 简单但会导致一帧的跳变

方案2：相对相机渲染 - 游戏逻辑使用 `double`，渲染时转换为相对坐标 - 无跳变，但需要修改渲染管线

方案3：分层坐标系 - 将世界分为大格子 (int 坐标) + 格子内偏移 (float) - 类似 Minecraft 的区块系统

3.4 碰撞精度

碰撞检测中的精度问题：

```
## 避免使用过小的碰撞体（精度问题）
## ❌ 错误：极薄的墙（厚度 0.01）
wall = CollisionBox(Point3(0,0,0), 10, 0.01, 5)

## ✅ 正确：给墙一定厚度
wall = CollisionBox(Point3(0,0,0), 10, 0.1, 5)

## 碰撞偏移（bias）防止穿透
## 在碰撞检测中添加小的安全距离
COLLISION_BIAS = 0.001 # 1mm 安全距离
```

4. Panda3D 源码剖析

4.1 PN_stdfloat : 可配置精度类型

Panda3D 使用 `PN_stdfloat` 作为标准浮点类型，可以在编译时或运行时切换精度：

```
// dtool/src/dtoolbase/stdfloat.h
#ifdef STDFLOAT_DOUBLE
    typedef double PN_stdfloat;
#else
    typedef float PN_stdfloat;
#endif

// 所有数学类都基于 PN_stdfloat
typedef LVecBase3f LVecBase3; // float 版本
typedef LVecBase3d LVecBase3d; // double 版本

// 当 stdfloat-double = true 时:
// LVecBase3 = LVecBase3d (double)
// 否则:
// LVecBase3 = LVecBase3f (float)
```

4.2 LMatrix4 的精度

```
// 矩阵类型
typedef LMatrix4f LMatrix4; // float 版本 (默认)
typedef LMatrix4d LMatrix4d; // double 版本

// 坐标系转换矩阵 (预计算, 避免重复计算误差)
static const LMatrix4 &convert_mat(CoordinateSystem from, CoordinateSystem to);

// 矩阵求逆 (数值稳定性关键)
// Panda3D 使用 LU 分解, 比直接求逆更稳定
bool invert_in_place();
```

4.3 LQuaternion 的归一化

```
// lquaternion_src.h
class LQuaternion {
    // 归一化
    INLINE_LINMATH void normalize();

    // 检查是否已归一化
    INLINE_LINMATH bool is_unit_quat() const;

    // 球面线性插值 (SLERP)
    // 注意: SLERP 要求输入四元数已归一化
    INLINE_LINMATH LQuaternion slerp(const LQuaternion &other, float t) const;
};
```

4.4 Lens 的深度精度控制

```
// lens.h
class Lens {
    // 近/远裁剪面 (影响深度精度)
    void set_near(PN_stdfloat near_distance);
    void set_far(PN_stdfloat far_distance);
    void set_near_far(PN_stdfloat near_distance, PN_stdfloat far_distance);

    // 获取投影矩阵 (包含深度范围映射)
    virtual bool do_compute_projection_mat(CData *cdata);

    // 深度偏移 (解决 Z-fighting)
    // 通过 DepthOffsetAttrib 实现
};
```

4.5 DepthOffsetAttrib : 深度偏移

```
// depthOffsetAttrib.h
class DepthOffsetAttrib : public RenderAttrib {
    // 为渲染对象添加深度偏移, 解决 Z-fighting
    // offset > 0 : 向相机方向偏移 (物体显示在前面)
    // offset < 0 : 远离相机方向偏移 (物体显示在后面)

    static CPT(RenderAttrib) make(int offset = 1,
                                   PN_stdfloat min_value = 0.0,
                                   PN_stdfloat max_value = 1.0);

    INLINE int get_offset() const;
};
```

5. 代码演示

5.1 Python : Z-Fighting 问题与解决

```

"""
Z-Fighting 演示与解决方案
"""
from direct.showbase.ShowBase import ShowBase
from panda3d.core import (
    DepthOffsetAttrib, DepthTestAttrib, RenderAttrib,
    CardMaker, NodePath, Vec4, LPoint3
)

class ZFightingDemo(ShowBase):
    def __init__(self):
        super().__init__()

        # — 演示 Z-Fighting —————
        # 两个完全重叠的平面会产生 Z-Fighting
        cm = CardMaker("plane")
        cm.setFrame(-5, 5, -5, 5)

        # 平面1 (红色)
        plane1 = self.render.attachNewNode(cm.generate())
        plane1.setColor(1, 0, 0, 1)
        plane1.setPos(0, 10, 0)
        plane1.setHpr(0, -90, 0)

        # 平面2 (蓝色) - 与平面1完全重叠, 会产生 Z-Fighting
        plane2 = self.render.attachNewNode(cm.generate())
        plane2.setColor(0, 0, 1, 1)
        plane2.setPos(0, 10, 0) # 完全相同的位置!
        plane2.setHpr(0, -90, 0)

        # — 解决方案1: 深度偏移 (DepthOffsetAttrib) —————
        # 让平面2在深度上偏移, 避免与平面1冲突
        # offset=1 表示向相机方向偏移一个单位 (显示在前面)
        plane2.setAttrib(DepthOffsetAttrib.make(1))

        # — 解决方案2: 调整近裁剪面 —————
        lens = self.cam.node().getLens()
        print(f"当前近裁剪面: {lens.getNear()}")
        print(f"当前远裁剪面: {lens.getFar()}")

        # 增大近裁剪面 (最有效的方法)
        lens.setNear(1.0) # 从默认 0.1 改为 1.0
        lens.setFar(5000) # 根据场景调整

        # — 解决方案3: 多边形偏移 (Polygon Offset) —————
        # 在 GLSL Shader 中使用 gl_FragDepth 手动调整深度
        # 或使用 OpenGL 的 glPolygonOffset

        # — 解决方案4: 贴花 (Decal) 系统 —————
        # 对于贴花 (如地面标记), 使用专门的贴花渲染
        # 贴花渲染在基础几何体之后, 使用深度偏移
        decal = self.render.attachNewNode(cm.generate())
        decal.setColor(0, 1, 0, 0.5)
        decal.setPos(0, 10, 0.001) # 微小偏移
        decal.setHpr(0, -90, 0)
        decal.setTransparency(True)
        # 或者使用 DepthOffsetAttrib
        decal.setAttrib(DepthOffsetAttrib.make(2))

```

```
# — 深度测试控制 —————
# 关闭深度写入（透明物体）
plane2.setAttrib(DepthTestAttrib.make(RenderAttrib.MLessEqual))

print("Z-Fighting 演示设置完成")
print("解决方案: DepthOffsetAttrib + 调整近裁剪面")
```


5.2 Python：大世界浮点精度问题

```

"""
大世界精度问题演示与解决方案
"""

from direct.showbase.ShowBase import ShowBase
from direct.task import Task
from panda3d.core import LPoint3d, LPoint3d, LVector3d, NodePath
import math

class LargeWorldDemo(ShowBase):
    """
    演示大世界坐标精度问题及解决方案
    使用"浮点原点" (Floating Origin) 技术
    """

    # 当玩家距原点超过此距离时, 重置原点
    ORIGIN_RESET_THRESHOLD = 1000.0

    def __init__(self):
        super().__init__()

        # 使用 double 存储真实世界坐标
        self._world_origin = LPoint3d(0, 0, 0) # 当前渲染原点 (世界坐标)
        self._player_world_pos = LPoint3d(0, 0, 0) # 玩家真实世界坐标

        # 场景中的物体 (使用相对坐标)
        self._world_objects = {} # {name: world_pos_double}

        # 创建一些远处的物体
        self._create_world_objects()

        # 玩家节点 (始终在渲染原点附近)
        self.player = self.render.attachNewNode("player")
        self.player.setPos(0, 0, 0)

        # 模拟玩家移动
        self._move_speed = 100.0 # 100 m/s
        self.taskMgr.add(self._update, "Update")

        # 调试信息
        self.taskMgr.add(self._print_debug, "Debug")

    def _create_world_objects(self):
        """创建分布在大世界中的物体"""
        import random
        rng = random.Random(42)

        for i in range(20):
            # 物体分布在 0-50km 范围内
            x = rng.uniform(-50000, 50000)
            y = rng.uniform(-50000, 50000)
            world_pos = LPoint3d(x, y, 0)
            name = f"object_{i}"
            self._world_objects[name] = world_pos

        # 创建场景节点 (使用相对坐标)
        node = self.loader.loadModel("models/environment")
        if node:
            node.reparentTo(self.render)
            node.setName(name)
            # 初始相对坐标

```

```

        rel_pos = world_pos - self._world_origin
        node.setPos(rel_pos.x, rel_pos.y, rel_pos.z)

def _update(self, task):
    """每帧更新：移动玩家，检查是否需要重置原点"""
    dt = globalClock.getDt()

    # 模拟玩家移动（使用 double 精度）
    t = task.time
    self._player_world_pos = LPoint3d(
        math.cos(t * 0.1) * 30000, # 在 30km 半径圆上移动
        math.sin(t * 0.1) * 30000,
        0
    )

    # 计算玩家相对于渲染原点的位置
    player_rel = self._player_world_pos - self._world_origin
    player_rel_f = LPoint3(player_rel.x, player_rel.y, player_rel.z)

    # 检查是否需要重置原点
    if player_rel_f.length() > self.ORIGIN_RESET_THRESHOLD:
        self._reset_origin()
    else:
        # 更新玩家节点位置（float 精度，但值很小）
        self.player.setPos(player_rel_f)

    return Task.cont

def _reset_origin(self):
    """
    浮点原点重置：
    将渲染原点移动到玩家当前位置
    所有物体坐标相应调整
    """
    # 新的渲染原点 = 玩家当前世界坐标
    new_origin = LPoint3d(
        self._player_world_pos.x,
        self._player_world_pos.y,
        self._player_world_pos.z
    )

    print(f"重置渲染原点: {self._world_origin} → {new_origin}")
    self._world_origin = new_origin

    # 更新所有物体的相对坐标
    for name, world_pos in self._world_objects.items():
        node = self.render.find(name)
        if node and not node.isEmpty():
            rel_pos = world_pos - self._world_origin
            # 使用 float 设置（此时值已经很小，精度足够）
            node.setPos(rel_pos.x, rel_pos.y, rel_pos.z)

    # 玩家现在在原点
    self.player.setPos(0, 0, 0)

def _print_debug(self, task):
    """每5秒打印调试信息"""
    if int(task.time) % 5 == 0 and task.time > 0:
        player_rel = self._player_world_pos - self._world_origin
        print(f"玩家世界坐标: ({self._player_world_pos.x:.1f}, {self._player_world_pos.y:.1f})")
        print(f"渲染原点: ({self._world_origin.x:.1f}, {self._world_origin.y:.1f})")

```

```
print(f"玩家渲染坐标: ({player_rel.x:.3f}, {player_rel.y:.3f})")
print(f"float 精度误差: ~{abs(player_rel.x) * 1.2e-7:.4f}m")
return Task.cont
```

5.3 Python：四元数精度维护

```

"""
四元数精度问题与解决方案
"""

from direct.showbase.ShowBase import ShowBase
from direct.task import Task
from panda3d.core import LQuaternion, LVecBase3, NodePath
import math

class QuaternionPrecisionDemo(ShowBase):
    def __init__(self):
        super().__init__()

        self.model = self.loader.loadModel("models/environment")
        self.model.reparentTo(self.render)

        # 累积旋转四元数 (会产生误差)
        self._accumulated_quat = LQuaternion(1, 0, 0, 0) # 单位四元数
        self._frame_count = 0

        self.taskMgr.add(self._rotate_with_drift, "RotateWithDrift")

    def _rotate_with_drift(self, task):
        """演示四元数累积误差"""
        dt = globalClock.getDt()
        self._frame_count += 1

        # 每帧旋转一小角度
        angle_per_frame = 1.0 # 度
        delta_quat = LQuaternion()
        delta_quat.setFromAxisAngle(LVecBase3(0, 0, 1))

        # 累积旋转 (会产生误差)
        self._accumulated_quat = self._accumulated_quat * delta_quat

        # 检查归一化误差
        length = self._accumulated_quat.length()
        if abs(length - 1.0) > 1e-6:
            print(f"帧 {self._frame_count}: 四元数长度 = {length:.8f} (误差: {length-1:.2e}) ")

        # — 解决方案: 定期归一化 —————
        # 每 100 帧归一化一次 (或每帧, 取决于精度要求)
        if self._frame_count % 100 == 0:
            self._accumulated_quat.normalize()
            print(f"帧 {self._frame_count}: 已归一化, 长度 = {self._accumulated_quat.length():.8f}")

        # 应用旋转
        self.model.setQuat(self._accumulated_quat)

        return Task.cont

    def _safe_slerp(self, q1, q2, t):
        """
        数值稳定的 SLERP 实现
        处理  $q1 \approx q2$  和  $q1 \approx -q2$  的边界情况
        """
        # 确保两个四元数都已归一化
        q1_norm = LQuaternion(q1)
        q2_norm = LQuaternion(q2)
        q1_norm.normalize()
        q2_norm.normalize()

```

```

# 计算点积
dot = (q1_norm.getR() * q2_norm.getR() +
       q1_norm.getI() * q2_norm.getI() +
       q1_norm.getJ() * q2_norm.getJ() +
       q1_norm.getK() * q2_norm.getK())

# 如果点积为负, 翻转一个四元数 (选择最短路径)
if dot < 0:
    q2_norm = LQuaternion(-q2_norm.getR(), -q2_norm.getI(),
                          -q2_norm.getJ(), -q2_norm.getK())
    dot = -dot

# 如果两个四元数非常接近, 使用线性插值 (避免除零)
DOT_THRESHOLD = 0.9995
if dot > DOT_THRESHOLD:
    # 线性插值 + 归一化
    result = LQuaternion(
        q1_norm.getR() + t * (q2_norm.getR() - q1_norm.getR()),
        q1_norm.getI() + t * (q2_norm.getI() - q1_norm.getI()),
        q1_norm.getJ() + t * (q2_norm.getJ() - q1_norm.getJ()),
        q1_norm.getK() + t * (q2_norm.getK() - q1_norm.getK()),
    )
    result.normalize()
    return result

# 标准 SLERP
theta_0 = math.acos(dot)
theta = theta_0 * t
sin_theta = math.sin(theta)
sin_theta_0 = math.sin(theta_0)

s1 = math.cos(theta) - dot * sin_theta / sin_theta_0
s2 = sin_theta / sin_theta_0

return LQuaternion(
    s1 * q1_norm.getR() + s2 * q2_norm.getR(),
    s1 * q1_norm.getI() + s2 * q2_norm.getI(),
    s1 * q1_norm.getJ() + s2 * q2_norm.getJ(),
    s1 * q1_norm.getK() + s2 * q2_norm.getK(),
)

```

5.4 Python：深度精度诊断工具


```

"""
深度精度诊断：计算当前相机设置下的深度精度
"""

import math

def analyze_depth_precision(near, far, depth_bits=24):
    """
    分析给定相机参数下的深度精度

    返回各距离处的最小可分辨深度差（米）
    """
    total_values = 2 ** depth_bits
    print(f"\n=== 深度精度分析 ===")
    print(f"近裁剪面: {near}m, 远裁剪面: {far}m")
    print(f"深度缓冲位数: {depth_bits}位 ({total_values:,} 个值)")
    print(f"深度比 far/near: {far/near:.0f}x")
    print()

    distances = [near, near*2, near*10, near*100, far/10, far/2, far]
    print(f"{'距离(m)':>12} | {'最小可分辨差(mm)':>18} | {'评级':>6}")
    print("-" * 45)

    for d in distances:
        # OpenGL 深度精度公式
        delta_z = (d * d) / (far * near) * (far - near) / total_values * 1000 # mm
        rating = "优秀" if delta_z < 1 else ("良好" if delta_z < 10 else ("一般" if delta_z < 100 else "差"))
        print(f"{d:>12.1f} | {delta_z:>18.4f} | {rating:>6}")

    print()
    print("建议: ")
    print(f"当前精度比: {far/near:.0f}x")
    if far/near > 10000:
        print(" ⚠ 精度比过大, 建议减小 far 或增大 near")
    if near < 0.5:
        print(" ⚠ near 过小, 建议设置为 0.5-1.0")
    print()

def recommend_near_far(scene_size_m, player_height_m=1.8):
    """根据场景大小推荐近/远裁剪面"""
    near = player_height_m * 0.1 # 近裁剪面约为玩家高度的 10%
    far = scene_size_m * 1.5 # 远裁剪面覆盖整个场景

    # 确保精度比不超过 10000
    if far / near > 10000:
        near = far / 10000

    print(f"场景大小: {scene_size_m}m")
    print(f"推荐 near: {near:.2f}m")
    print(f"推荐 far: {far:.0f}m")
    return near, far

## 使用示例
if __name__ == "__main__":
    # 分析不同配置的深度精度
    print("=== 配置1: 默认设置 (常见问题) ===")
    analyze_depth_precision(near=0.1, far=10000)

```

```

print("=== 配置2: 优化设置 ===")
analyze_depth_precision(near=1.0, far=5000)

print("=== 配置3: 大世界设置 ===")
analyze_depth_precision(near=1.0, far=100000, depth_bits=32)

# 推荐设置
near, far = recommend_near_far(scene_size_m=1000)

```

5.5 GLSL：精度感知 Shader

```

/* 精度感知的 Shader 编写最佳实践 */

/* — 顶点着色器：避免大数值计算 ————— */
#version 330

uniform mat4 p3d_ModelViewProjectionMatrix;
uniform mat4 p3d_ModelMatrix;
uniform vec3 camera_world_pos; // 相机世界坐标（用于大世界）

in vec4 p3d_Vertex;
in vec3 p3d_Normal;

out vec3 world_pos;
out vec3 view_dir;

void main() {
    gl_Position = p3d_ModelViewProjectionMatrix * p3d_Vertex;

    // 世界空间位置
    world_pos = (p3d_ModelMatrix * p3d_Vertex).xyz;

    // 视线方向（相对于相机，避免大数值相减）
    // ❌ 错误：world_pos - camera_world_pos 可能损失精度
    // ✅ 正确：在顶点着色器中计算（精度更高）
    view_dir = normalize(camera_world_pos - world_pos);
}

```

```

/* — 片段着色器：精度注意事项 ————— */
#version 330

in vec3 world_pos;
in vec3 view_dir;

out vec4 fragColor;

/* 数值稳定的菲涅尔计算 */
float fresnel_schlick(float cos_theta, float F0) {
    // 避免 pow(1-cos_theta, 5) 在 cos_theta=1 时的精度问题
    float x = clamp(1.0 - cos_theta, 0.0, 1.0);
    float x2 = x * x;
    return F0 + (1.0 - F0) * x2 * x2 * x;
}

/* 数值稳定的法线贴图解码 */
vec3 decode_normal(vec2 encoded) {
    // BC5/ATI2 格式：只存储 XY, Z 从  $X^2+Y^2+Z^2=1$  推导
    vec2 xy = encoded * 2.0 - 1.0;
    float z_sq = 1.0 - dot(xy, xy);
    // 使用 max 避免负数开方 (精度误差可能导致 z_sq < 0)
    float z = sqrt(max(z_sq, 0.0));
    return normalize(vec3(xy, z));
}

/* 数值稳定的距离衰减 */
float distance_attenuation(float dist, float radius) {
    // 避免除零
    float dist_sq = dist * dist;
    float radius_sq = radius * radius;
    // 平滑截止 (避免硬边界)
    float factor = dist_sq / max(radius_sq, 0.0001);
    return max(1.0 - factor, 0.0);
}

void main() {
    vec3 N = normalize(vec3(0, 0, 1)); // 简化示例
    vec3 V = normalize(view_dir);

    float cos_theta = max(dot(N, V), 0.0);
    float fresnel = fresnel_schlick(cos_theta, 0.04);

    fragColor = vec4(vec3(fresnel), 1.0);
}

```

6. 性能优化

6.1 精度 vs 性能权衡

```
## — 选择合适的精度级别 —————

## 场景1: 小型游戏 (场景 < 1km)
## float 精度完全足够, 不需要特殊处理
lens.setNear(0.5)
lens.setFar(1000)

## 场景2: 中型游戏 (场景 1-10km)
## 调整近裁剪面, 使用 24 位深度缓冲
lens.setNear(1.0)
lens.setFar(5000)

## 场景3: 大型开放世界 (场景 > 10km)
## 需要浮点原点技术
lens.setNear(1.0)
lens.setFar(10000)
## + 实现 FloatingOrigin 系统

## 场景4: 天文模拟 (场景 > 1000km)
## 需要 double 精度 + 分层坐标系
from panda3d.core import loadPrcFileData
loadPrcFileData("", "stdfloat-double true") # 全局 double 精度
```

6.2 常见精度 Bug 及修复

```
## — Bug 1: Z-Fighting —————
## 症状: 两个重叠面闪烁
## 原因: 深度值相同, GPU 随机选择哪个面

## ❌ 错误: 两个面完全重叠
decal.setPos(ground.getPos()) # 完全相同位置

## ✅ 修复: 使用 DepthOffsetAttrib
from panda3d.core import DepthOffsetAttrib
decal.setAttrib(DepthOffsetAttrib.make(1))

## — Bug 2: 远处物体抖动 —————
## 症状: 距离 > 10km 的物体位置抖动
## 原因: float 精度不足

## ❌ 错误: 直接使用大坐标
obj.setPos(50000, 50000, 0) # float 精度只有 ~5m!

## ✅ 修复: 使用浮点原点
## 将渲染原点移到玩家附近, 物体使用相对坐标

## — Bug 3: 骨骼动画漂移 —————
## 症状: 长时间运行后骨骼位置偏移
## 原因: 四元数累积误差

## ✅ 修复: 定期归一化四元数
def update_bone(bone_quat, delta_quat, frame):
    result = bone_quat * delta_quat
    if frame % 60 == 0: # 每60帧归一化一次
        result.normalize()
    return result

## — Bug 4: 碰撞穿透 —————
## 症状: 快速移动的物体穿过薄墙
## 原因: 碰撞检测精度不足 + 帧率过低

## ✅ 修复: 使用 CCD (连续碰撞检测) + 增加碰撞体厚度
from panda3d.core import CollisionBox, Point3
## 给墙增加足够厚度
wall = CollisionBox(Point3(0,0,0), 10, 0.2, 5) # 厚度 0.2m 而非 0.01m
```

6.3 精度测试工具

```
def test_float_precision():
    """测试 float 在不同数量级下的精度"""
    import struct

    test_values = [1.0, 10.0, 100.0, 1000.0, 10000.0, 100000.0]

    print("float 精度测试:")
    print(f"{'值':>12} | {'最小增量':>15} | {'相对误差':>12}")
    print("-" * 45)

    for v in test_values:
        # 找到 float 能表示的下一个值
        bits = struct.unpack('I', struct.pack('f', v))[0]
        next_bits = bits + 1
        next_v = struct.unpack('f', struct.pack('I', next_bits))[0]
        delta = next_v - v
        relative = delta / v

        print(f"{'v':>12.0f} | {'delta':>15.8f} | {'relative':>12.2e}")

    print()
    print("结论: ")
    print(" 在 10000m 处, float 最小增量约 0.001m (1mm) ")
    print(" 在 100000m 处, float 最小增量约 0.01m (1cm) ")
    print(" 超过 100km 时, float 精度不足以表示厘米级差异")

if __name__ == "__main__":
    test_float_precision()
```

小结

知识点	核心要点
IEEE 754	float 约 7 位有效数字，相对精度 $\approx 10^{-7}$
Z-Fighting	深度精度 $\propto z^2$ ，远处精度极低；增大 near 是最有效解决方案
DepthOffsetAttrib	为重叠几何体添加深度偏移，解决 Z-Fighting
大世界精度	坐标 > 10km 时 float 精度不足；使用浮点原点技术
四元数漂移	累积旋转后需定期归一化；SLERP 需处理边界情况
PN_stdfloat	Panda3D 可配置精度类型， <code>stdfloat-double true</code> 启用 double
深度比	<code>far/near</code> 比值越大，深度精度越差；建议 < 10000
碰撞精度	碰撞体不能太薄；快速物体需要 CCD

精度问题诊断流程

症状：物体闪烁/抖动

- └─ 近处闪烁 → Z-Fighting → DepthOffsetAttrib + 调整 near
- └─ 远处抖动 → 大世界精度 → 浮点原点技术
- └─ 动画漂移 → 四元数误差 → 定期归一化
- └─ 碰撞穿透 → 碰撞精度 → 增加厚度 + CCD

关键设计原则

- 近裁剪面是深度精度的关键：`near` 从 0.1 改为 1.0，精度提升 10 倍
- 大世界必须用浮点原点：游戏逻辑用 double，渲染用相对坐标
- 四元数要定期归一化：每 60-100 帧归一化一次
- 碰撞体要有足够厚度：最薄不应小于 0.1m
- Shader 中避免大数相减：在顶点着色器中计算，精度更高

与其他专题的关联

- 专题03（变换系统）：矩阵累积误差与四元数漂移
- 专题07（碰撞检测）：碰撞精度直接影响物理模拟稳定性
- 专题11（后处理）：深度缓冲精度影响 SSAO、景深等效果
- 专题13（跨平台）：不同平台的深度范围 $[-1,1]$ vs $[0,1]$ 影响精度分布

